МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский Нижегородский государственный университет им. Н.И. Лобачевского»

А.А. Яковлева, А.В. Кудрин

Программная среда LabView в научных исследованиях

Учебно-методическое пособие по курсу

«Программная среда LabView в научных исследованиях»

Рекомендовано методической комиссией физического факультета
для студентов ННГУ,
обучающихся по направлению подготовки
11.04.04 «Электроника и наноэлектроника» и

Нижний Новгород
2025

УДК 004.4'234+001.895(075.8)

ББК 32.972.39+72.5я73

Я 47

Я 47 Яковлева, А. А. Программная среда LabView в научных исследованиях: учебно-метод. пособие / А. А. Яковлева, А. В. Кудрин. - Нижний Новгород: Изд-во ННГУ, 2025. - 202 с. - Текст: электронный.

Рецензент: д.ф.-м.н., в.н.с. НИФТИ ННГУ В.Н. Трушин

В работе прорабатывается и закрепляется материал соответствующего раздела курса лекций «Программная среда LabView в научных исследованиях»: описаны основы графического языка программирования G, особенности программирования в среде LabView, взаимодействие с аппаратными средствами. Учебно-методическое пособие предназначено для студентов, обучающихся по направлению подготовки 11.04.04 «Электроника и наноэлектроника».

Ответственный за выпуск:

председатель методической комиссии

физического факультета ННГУ, к.ф.-м.н., доцент А.А. Перов

УДК 004.4'234+001.895(075.8)

ББК 32.972.39+72.5я73

© Нижегородский государственный университет

им. Н.И. Лобачевского, 2025

Оглавление

1. Среда LabView 6
Основные понятия и определения 6
Лицевая панель (Front Panel) 7
Основные элементы лицевой панели. 8
Block Diagram 9
Терминалы 9
Узлы 11
Проводники данных 12
Первая ознакомительная программа 13
Потоки данных 23
2. Меню среды LabVIEW 24
Плавающие палитры 33
Палитры Элементы управления и Функции 33
Палитра Функций 33
Палитра Элементов управления 36
Редактирование плавающих палитр 38
Закрепление палитры 38
Скрыть подпалитру 40
Отображение палитр 42
Поиск элемента или функции 44
Палитра инструментов 44
Линейка инструментов. 48
3. Оформление программ LabVIEW 49
Отладка программы 53
Подсветка при выполнении программы 55
Пошаговый режим отладки 60
Составная арифметика 63
Циклы 66

Локальные переменные 73
Вложенные циклы 76
Массивы 79
Полиморфизм 90
Функции для работы со временем 93
Счётчик времени (*Tick Count*) (мс) 94
Функция задержка (*Wait*) (мс) 100
Задержка до следующего кратного интервала (*Wait Until Next ms Multiple*) 103
Сдвиговый регистр 105
Сохранение данных 112
Структура последовательности 116
Структура выбора. Ветвление 121
Структура события 126
4. Графики и кластеры (расширенные функции) 138
Осциллограммы 138
Waveform Graph 152
Кластеры 159
Двухкоординатные графики 172
Строковый тип данных 182
5. Взаимодействие среды LabView с приборами 190
Использование звуковой карты 190
Использование COM порта 194
Использование USB порта 199
Литература 202

Введение

Учебное пособие направлено на освоение программной среды LabView для автоматизации экспериментальных и технологических установок. Работа ученого-экспериментатора непосредственно связана с проведением экспериментов для получения данных о свойствах объекта исследования и обработкой полученных данных. В экспериментальной работе могу использоваться готовые (поставляемые в виде конечного устройства или комплекса приборов) экспериментальные установки, позволяющие проводить эксперимент максимально автоматизировано. Примером таких установок являются спектрофотометры, рентгеновские дифрактометры, характериографы, микроскопы различного вида. Однако, в лабораториях значительный обьем экспериментов проводится на установках, собранных непосредственно экспериментаторами. Современные измерительные приборы, различные источники и другое оборудование (например, оптические подвижки с электроприводом) имеют стандартные аппаратные интерфейсы для связи с компьютером. Это позволяет объединить различные приборы в один аппаратно-программный комплекс, управляемый программой, созданной под конкретную экспериментальную задачу. Программной среда LabView позволяет относительно просто управлять практически любым оборудованием, имеющим интерфейс связи с компьютером (или какой-либо низкоуровневый аппаратный интерфейс, даже аналоговый). Одной из ключевых особенностей LabView является графический язык программирования G, который, как показывает практика, легко осваивается при наличии базовых знаний по программированию.

1. Среда LabView

Основные понятия и определения

Для начала откроем программу и рассмотрим, как выглядит её диалоговое окно и что в нём содержится (рис.1). После открытия окна мы видим по центру два больших раздела с кнопками **Create Project** и **Open Existing**.

- Кнопка **Open Existing** позволяет быстро открыть недавно сохранённую программу, но так как никакой программы на данный момент мы не создавали, то эта кнопка нам не понадобится.
- Кнопка **Create Project** позволяет создать отдельный проект, с помощью которого в дальнейшем мы сможем создавать собственные приложения с возможностью их установки на любой персональный компьютер, даже если на нём отсутствует среда LabView. Тема создания проекта довольно таки сложная и требует достаточно хорошего понимания базовых принципов и особенностей работы в среде программирования.

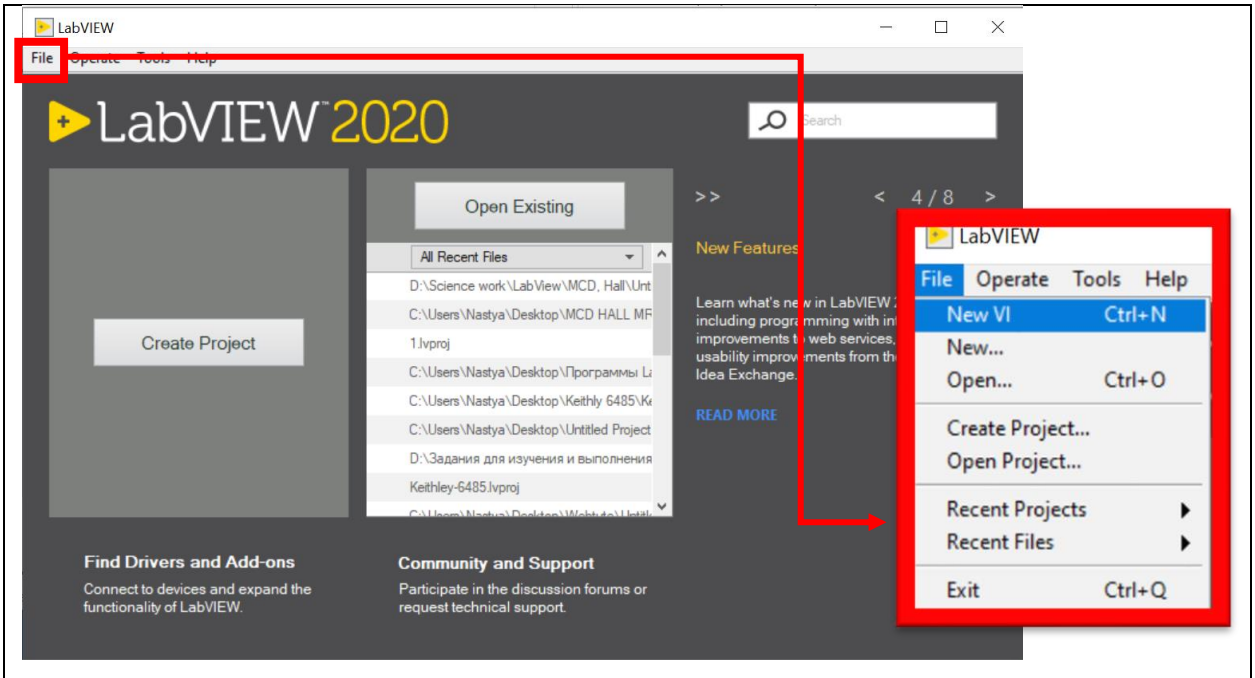


Рисунок 1. Вид стартового диалогового окна (вставка – меню File).

В верхней части окна вы увидите 4 возможные опции: **File, Operate, Tools, Help**. Чтобы начать работу нам понадобиться создать новый файл, для

этого надо нажать **File →New VI**. Возникает вопрос что же такое **VI.** В переводе с английского **VI (Virtual Instrument**) – это виртуальный прибор. По своей сути в среде программирования LabView мы с вами создаём прибор, который, в какой-то степени, является аналогом реальных измерительных приборов, используемых при проведении эксперимента. Виртуальный прибор (ВП), как и любой реальный измерительный прибор, может выполнять определённый набор действий, который был в него заложен. В реальном мире у каждого измерительного прибора есть панель управления (например, кнопка запуска/остановки измерений), которая позволяет задать определённые параметры приборов, которые требуется для проведения эксперимента. В ВП также есть возможность управления и изменения параметров, которые вы создали в процессе создания программы ВП. Поэтому виртуальный прибор является аналогом реального измерительного прибора.

Итак, мы создали новый виртуальный прибор (**New VI)** и теперь перед нами раскрываются два окна: лицевая (передняя) панель (**Front Panel**) и окно редактирования диаграмм (**Block Diagram**) – блок диаграмма (рис.2). На передней панели (Front Panel) размещаются элементы интерфейса программы, а кодовая панель (Block Diagram) содержит программный код. Переключение между этими основными окнами (панелями) осуществляется через пункт Windows в основной (верхней) инструментальной панели или комбинацией клавиш Ctrl+E (рисунок 2). Рассмотрим более подробно каждую из них.

Лицевая панель (Front Panel)

Лицевая панель – это окно серого цвета в клеточку, с помощью которого мы можем взаимодействовать с виртуальным прибором (программой), управлять им, а также видеть результаты выполнения программы.

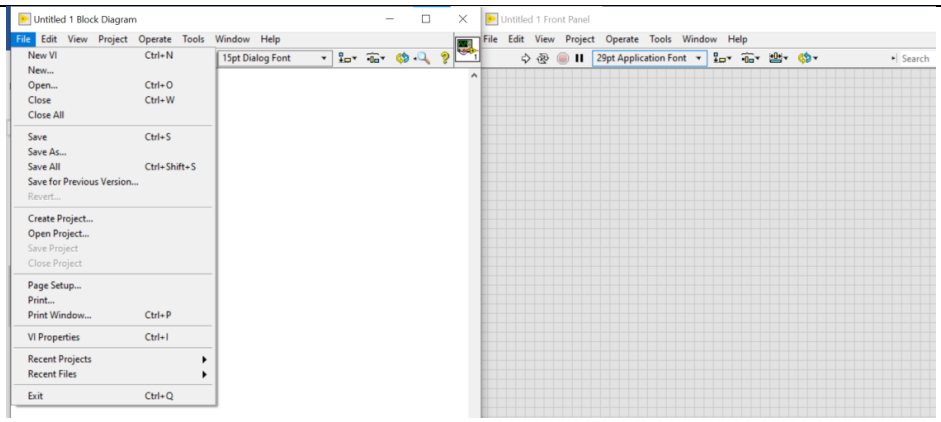


Рисунок 2. Вид окна блок-диаграммы и окна лицевой панели.

Основные элементы лицевой панели.

Основными элементами лицевой панели являются **элементы управления (Control)** и **индикаторы (Indicator)**. Элементы управления позволяют пользователю вводить данные, отправлять полученные данные в программный код для их дальнейшей обработки. Индикаторы показывают пользователю выходные данные, которые являются результатом выполнения программы. Для наилучшего понимания рассмотрим пример, представленный на рисунке 3. Представим, что синяя шкала с делениями – это градусник, с помощью которого мы можем задавать температуру путём перемещения синего ползунка. То есть эта шкала является элементов управления, с помощью которого мы вводим определённые данные – в данном случае значение температуры. При этом, двигая синий ползунок, мы можем изменять вводимые нами входные данные. Теперь, чтобы узнать точное значение указанной температуры, установим на переднюю панель окошечко, которое будет отображать установленное значение температуры (пока что не будем вдаваться в то как устроен код программы). Тогда установленное окошечко с отображаемыми данными является индикатором, который показывает нам выходные данные – температуру. При этом устанавливая другое значение температуры с помощью синего ползунка в шкале (изменяя входные данные) значение температуры в окошечке также будет изменяться (изменение выходных данных). Размещение элементов

управления и индикаторов на лицевой панели будет рассмотрено немного позже.

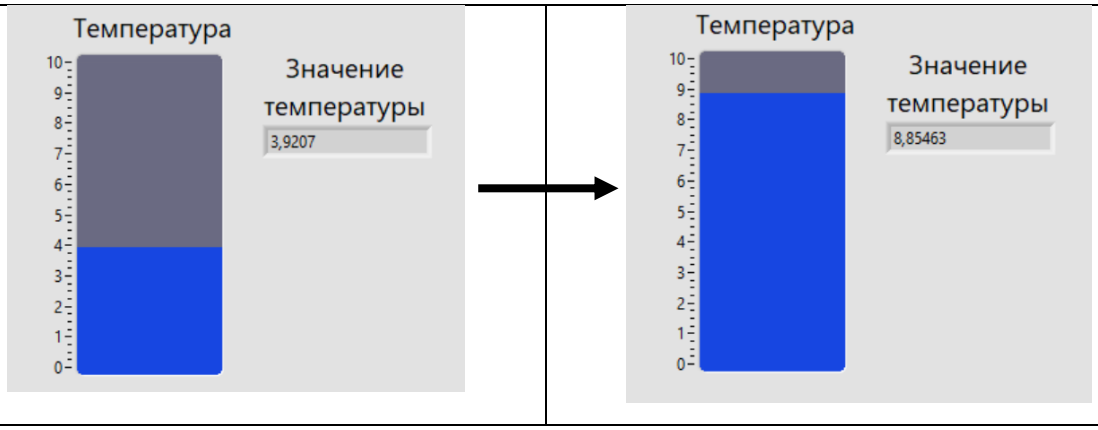


Рисунок 3. Отображение взаимосвязи элементов управления и индикаторов.

Block Diagram

Блок-диаграмма представляет собой белое окно, в котором мы можем размещать различные элементы, позволяющие нам написать код программы. В обычном программировании весь код программы выглядит как текстовые команды, написанные в определённом порядке. В среде программирования LabView код программы состоит из графических элементов, которые выполняют определённую функцию и соединены между собой проводниками. Такой вид кода называется графический код. Основными компонентами блок диаграммы являются: терминалы (terminals), узлы (nodes) и проводники данных (wires). Рассмотрим подробнее каждый из них.

Терминалы

Когда на лицевой панели нашей программы мы располагаем какой-либо элемент управления или индикатор, в блок диаграмме автоматически появляется иконка (терминал), которая соответствует элементу, установленному на лицевой панели. На пример, когда на лицевой панели мы расположили шкалу (как из примера выше), то на блок-диаграмме появится иконка (терминал), которая соответствует этой шкале (рис.4). Если с блок-

диаграммы удалить эту иконку, то с лицевой панели шкала пропадёт и наоборот, если с лицевой панели удалить шкалу, то на блок-диаграмме она также будет удалена.

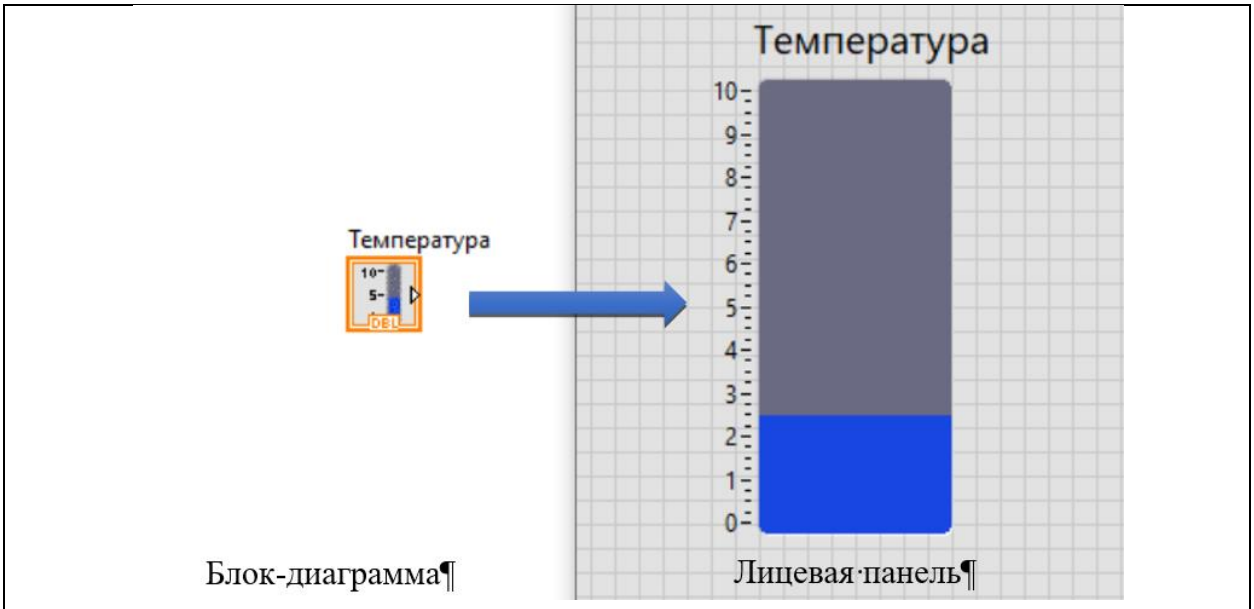


Рисунок 4. Отображение терминала элемента управления (Slide Control) и его вид на лицевой панели.

Для лучшего понимания что такое терминалы (иконки) и зачем они нужны в блок-диаграмме проведём аналогию с обычным измерительным прибором. Допустим мы хотим измерить значение напряжения, которое будет показывать резистор при пропускании через него тока 1 мкА. Для этого нам нужно на лицевой панели реального прибора задать значение тока 1 мкА, то есть ввести какое-то значение тока. Далее мы запускаем измерение напряжения и заданное нами значение тока поступает с лицевой панели в схему прибора, которая выполняет с заданным значением определённые действия (в данном случае пропускает ток через резистор и измеряет напряжение). После того, как схема прибора выполнит определённые действия и завершит измерения, на лицевой панели прибора появится измеренное значение напряжения. То есть выходные данные (напряжение), полученные после завершения работы схемы прибора, поступят на лицевую панель и покажут измеренное значение (отображение выходных данных).

Теперь опишем весь этот процесс относительно среды LabView. Задать значение тока в среде LabView мы можем с помощью элемента управления, который поместим на лицевую панель (аналогия задания тока на реальном измерительном приборе), при этом в блок диаграмме также появится терминал. Когда мы запустим виртуальный прибор (запустим измерение напряжения) код программы, который написан в блок-диаграмме, начнёт выполнять определённые действия для того, чтобы получить требуемое значение напряжения (аналогично схеме реального прибора). При этом, чтобы программа смогла рассчитать напряжение, в код должны поступать значения тока, которые мы ввели на лицевой панели. Значение тока, установленное на лицевой панели, передаётся в терминал, расположенный на блок-диаграмме, что позволяет получить требуемые значения тока, для выполнения кода программы. Для того, чтобы вывести полученное значение напряжения из кода программы, также понадобится установить терминал, который будет передавать выходные данные (данные после завершения работы кода) из блок-диаграммы в лицевую панель (аналогично отображению выходных данных на лицевой панели реального измерительного прибора). В качестве такого терминала используются индикаторы, которые устанавливаются на лицевой панели. То есть терминалы – это элементы, которые передают данные, с которыми взаимодействует человек, в код программы и обратно из кода программы. Иными словами, терминалы можно считать источниками входных и выходных данных.

Узлы

Узлами данных в среде LabView называет любые элементы, которые выполняют какое-либо действие в ходе выполнения программного кода. Узлы – это обобщающее название всех элементов, которые есть в обычном языке программирования: операторы, функции и подпрограммы.

Проводники данных

Проводниками данных называют провода, с помощью которых можно связывать терминалы и узлы в коде программы. Иными словами, проводники данных – это провода по которым передаются данные от одной части кода программы к другой части кода программы для их последовательной обработки. Важно запомнить, что к одному проводнику нельзя подсоединить несколько узлов, несколько терминалов с входными данными или несколько терминалов с выходными данными, иначе провод будет считаться повреждённым (broken) (рис.5(a)). Если к проводнику не присоединён ни один узел или терминал, провод также будет считаться повреждённым (рис.5(b)).

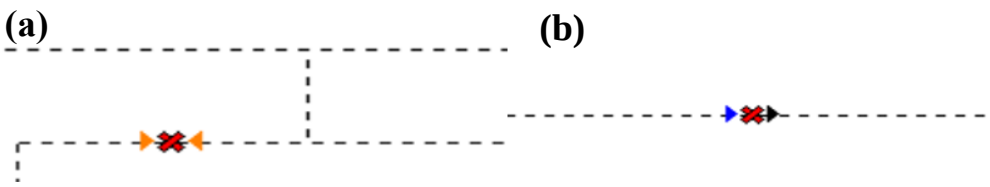


Рисунок 5. a) Вид повреждённого провода при соединении нескольких узлов. b) Вид повреждённого провода в отсутствии соединения проводника с терминалом или узлом.

В любой среде программирования существуют различные типы данных и LabView не является исключением. Тип данных в LabView определяется цветом проводника данных (провод), каждому типу данных соответствует определённый цвет и стиль провода. На рисунке 6 приведены основные типы данных проводников и их стили. Более подробное описание типов данных будет рассмотрено позже.

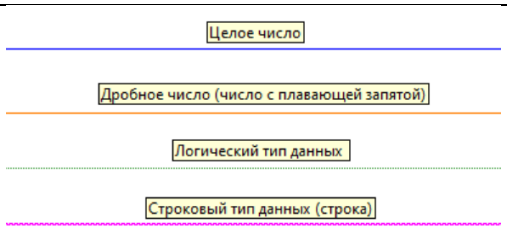


Рисунок 6. Изображение проводников данных различного типа.

Первая ознакомительная программа

Теперь, когда мы знаем, как создавать новые программы и частично разобрались в интерфейсе среды LabView, напишем простую программу для наилучшего понимания разобранного материала. Задача: написать программу, которая будет складывать два числа и умножать их на третье число. При этом важно, что три числа будут вводиться с лицевой панели, а также на лицевой панели будет отображаться результат вычислений.

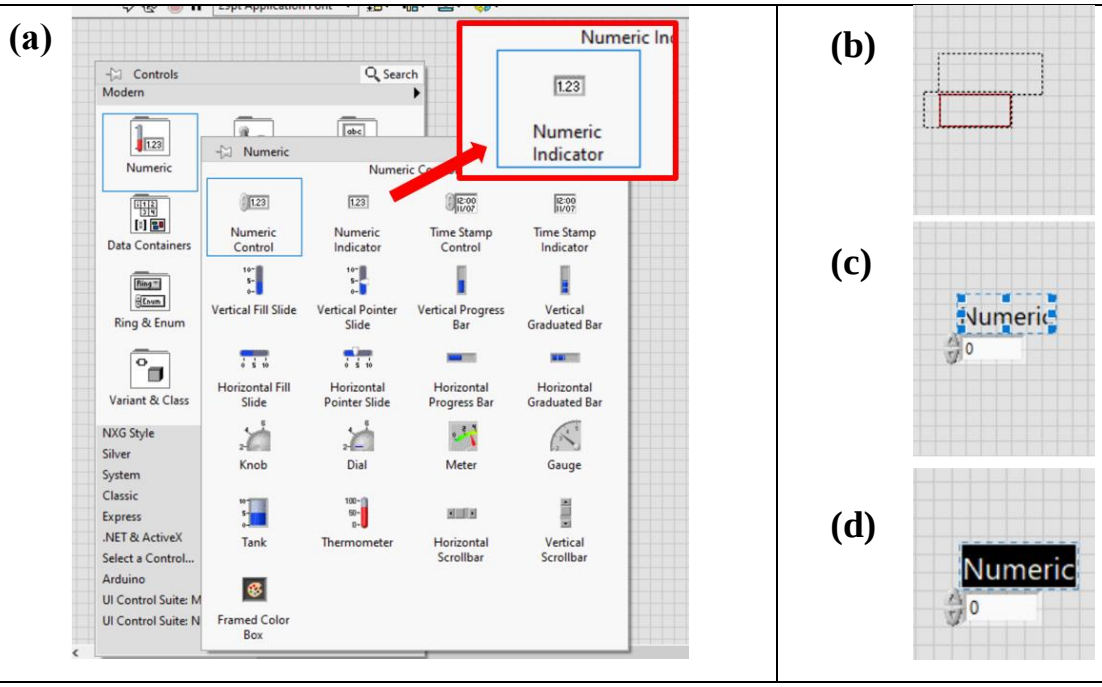


Рисунок 7. a) Элемент Numeric Control в разделе Modern палитры управления на лицевой панели (вставка – элемент управления Numeric Indicator). b) Вид элемента Numeric Control до его размещения на лицевой панели. c) Элемент Numeric Control с выделенным названием. d) Изменение названия элемента управления Numeric Control.

1. Чтобы написать программу нам понадобится добавить на лицевую панель 3 элемента управления, с помощью которых мы будем задавать числа для вычислений, и 1 индикатор, в котором будет отображаться результат наших вычислений. Для этого переключимся на окно Front Panel (лицевая панель) и щёлкнем правой кнопкой мыши.

Чтобы поместить элементы на лицевую панель, щёлкните по элементу в палитре, палитра исчезнет, а элемент, на который вы щёлкнули, будет привязан к курсору мыши двигаясь вместе с ним (рис. 7(b)). Выберите подходящее место на лицевой панели и щёлкните левой кнопкой мыши, тогда выбранный элемент отвяжется от мыши и останется на том месте, где вы щёлкнули мышью. Проделайте так ещё три раза, поместив на лицевую панель ещё 2 элемента управления и 1 индикатор. В результате мы получим вид лицевой панели, представленный на рисунке 8.

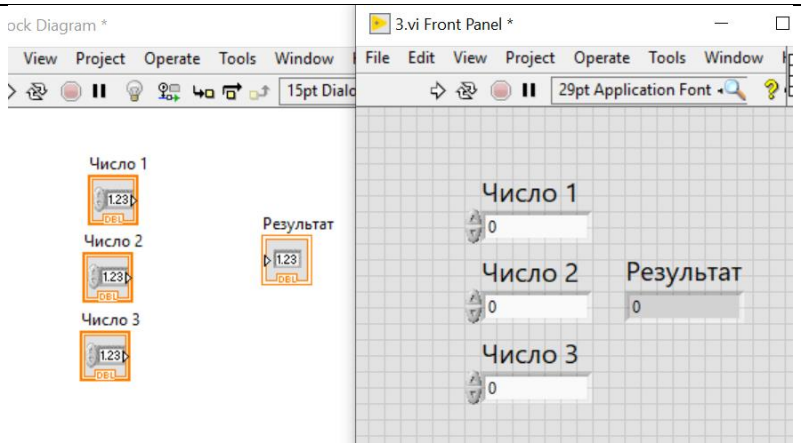


Рисунок 8. Вид элементов управления и индикаторов на блок-диаграмме и лицевой панели, требуемых для создания графического программного кода.

Чтобы переименовать элементы щёлкните курсором по названию элемента (название должно выделится в рамочку (рис. 7(c))), потом двойным щелчком левой кнопки мыши щёлкните по выделенному полю, оно станет чёрным (рис. 7(d)). Когда оно станет чёрным можно начинать писать новое имя элементу.

2. После того, как все требуемые для программы элементы были размещены на лицевой панели, нам нужно создать графический код на блок-

диаграмме. На данный момент наша блок-диаграмма содержит только терминалы, от созданных элементов управления и индикатора (см. рис. 8).

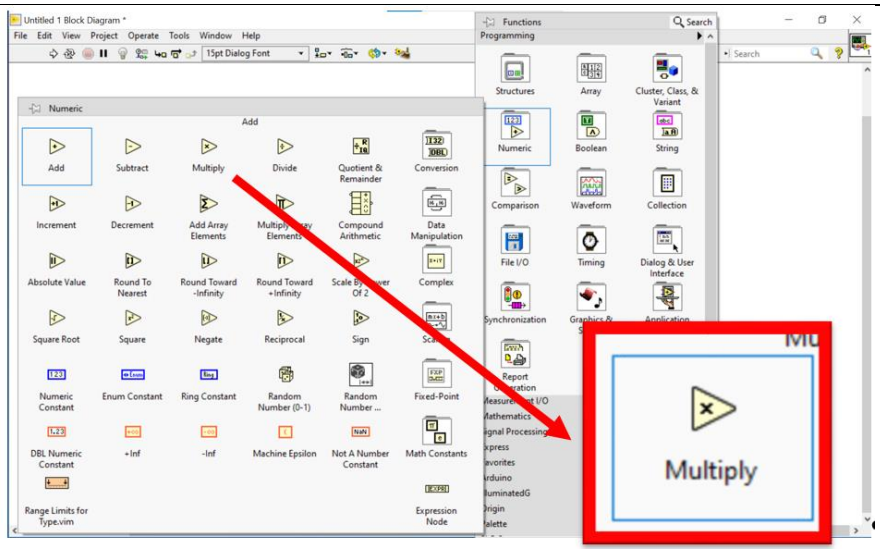


Рисунок 9. Функция Add (Сложения) в подпалитре Numeric из раздела Programming в палитре функций на блок-диаграмме (вставка – Функция Multiply (Умножения)).

Чтобы открыть палитру с элементами для блок-диаграммы, щёлкните правой кнопкой мыши. Сейчас нам понадобится раздел в палитре под названием Numeric. Чтобы посчитать сумму элементов нужно поместить на блок-диаграмму элемент с названием Add (сумма), для умножения – элемент Multiply (см. рис.9). Размещение элементов на блок-диаграмме аналогично описанному выше размещению элементов на лицевой панели. Теперь нам надо соединить узлы с помощью проводников данных. Выберем два элемента, которые нужно соединить, например, элемент с названием число 1 присоединить к элементу суммы (add). Чтобы это сделать наведём курсор мыши на правую грань терминала Число 1, при наведении на терминал курсор мыши должен сменится на курсор в виде катушки и появляется мигающая точка, которая является точкой присоединения проводника данных (рис.10(а)). Когда появляется курсор в виде катушки нужно щёлкнуть левой кнопкой мыши на мигающей точке для присоединения проводника. Движении курсором в виде катушки будет протягивать проводник по блок-

диаграмме, который выглядит как пунктирная линия (рис.10(b)). Теперь наводим курсор на элемент сложения и левой кнопкой мыши щёлкаем на верхней мигающей точке для присоединения проводника к элементу (рис.10(c)). После этого проводник данных станет сплошным и курсор катушки сменится на обычный курсор мыши, при этом число 1 и элемент суммы станут соединены между собой проводником (рис.10(d)).

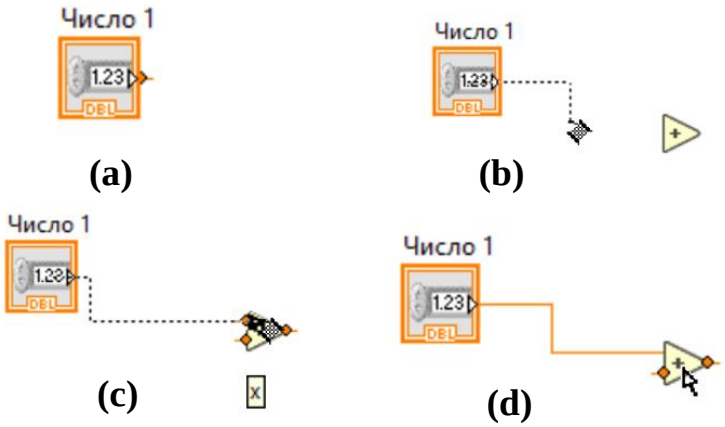


Рисунок 10. a) Вид терминала элемента управления Число 1 в коне блок-диаграммы. b) Присоединение проводника данных к терминалу элемента управления Число 1. c) Подведение проводника данных от терминала Число 1 к входному терминалу функции суммы. d) Соединение проводником данных терминалы Число 1 и функции Add.

Другие элементы нужно аналогичным образом соединить так, как показано на рисунке 11(a). После того как мы соединили все элементы нам нужно задать наши числа, с которыми мы хотим произвести сложение и умножение. Для этого перейдём на лицевую панель и введём значения 3 чисел. Теперь мы можем запускать программу. Для запуска программы нажмите на белую стрелочку, расположенную в левом верхнем углу (рис.11(b)). После того как программа закончит выполняться в индикаторе Результат появится результат вычислений. Если поменяем числа на другие и снова запустим программу, то увидим, что в индикаторе результат ответ изменился.

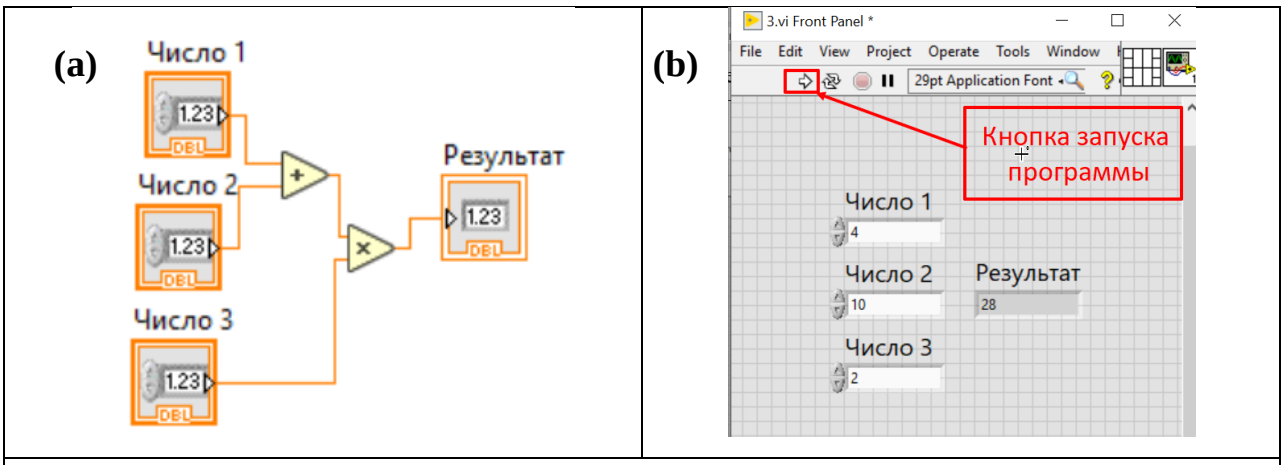
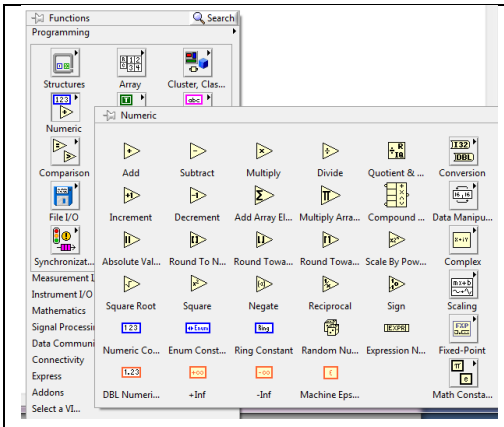


Рисунок 11. a) Графический код программы для осуществления арифметических действий над числами. b) Кнопка запуска программы в окне лицевой панели и отображение полученного результата.

Рассмотрим дополнительные особенности работы с данными числового типа. Одной из основных особенностей среды LabView является использование графического языка программирования (язык G). Графическое программирование обладает повышенной наглядностью и облегчает создание программ для пользователей, не имеющих значительного опыта в программировании. Все элементы программного кода могут быть представлены в виде значков, однако существуют также инструменты, позволяющие представить часть программного кода в традиционном строковом виде. Для начала создадим числовую константу. Для этого в окне программного кода нажатием правой кнопки мыши вызовем палитру с основными инструментами и в ней в разделе Programming наведем курсор на подраздел (пункт) Numeric. После этого раскроется вложенная палитра с набором функций (значков) для работы с данными числового типа (рисунок 12(a)). Выберем элемент Numeric Constant и поместим его на кодовую панель. В кодовой панели появится синий прямоугольник (рисунок 12(b)), в который можно вписать значение числовой константы. Если подвести курсор к правой границы значка, то курсор примет вид катушки и справа на значке появится залитый кружок (рисунок 12(c)). Этот кружок называется терминалом и через этот терминал производится вывод значения константы.

Выведем значение константы на переднюю панель. Для этого нажмем на передней панели правую кнопку мыши, вызовем основную палитру передней панели и наведем курсор на пункт Numeric. Появится палитра пункта Numeric (рисунок 13(a)). Выберем элемент Numeric Indicator и поместим его на кодовую панель.



a)



b)



c)

Рисунок 12. a) Элементы вложенной палитры Numeric в основной палитре Programming кодовой панели. b) Значок числовой константы в кодовой панели. c) Значок числовой константы с подведенным курсором к выходному терминалу константы.

На передней панели появится элемент отображения (индикатор) Numeric Indicator данных числового типа (рисунок 13(b)). При этом в кодовой панели появится значок Numeric этого элемента отображения (рисунок 13(c)). С левой стороны индикатора Numeric в кодовой панели имеется входной терминал. В кодовой панели курсором соединим выходной терминал числовой константы со входным терминалом индикатора Numeric. Появится проводник синего цвета, соединяющий эти элементы (рисунок 13(d)).

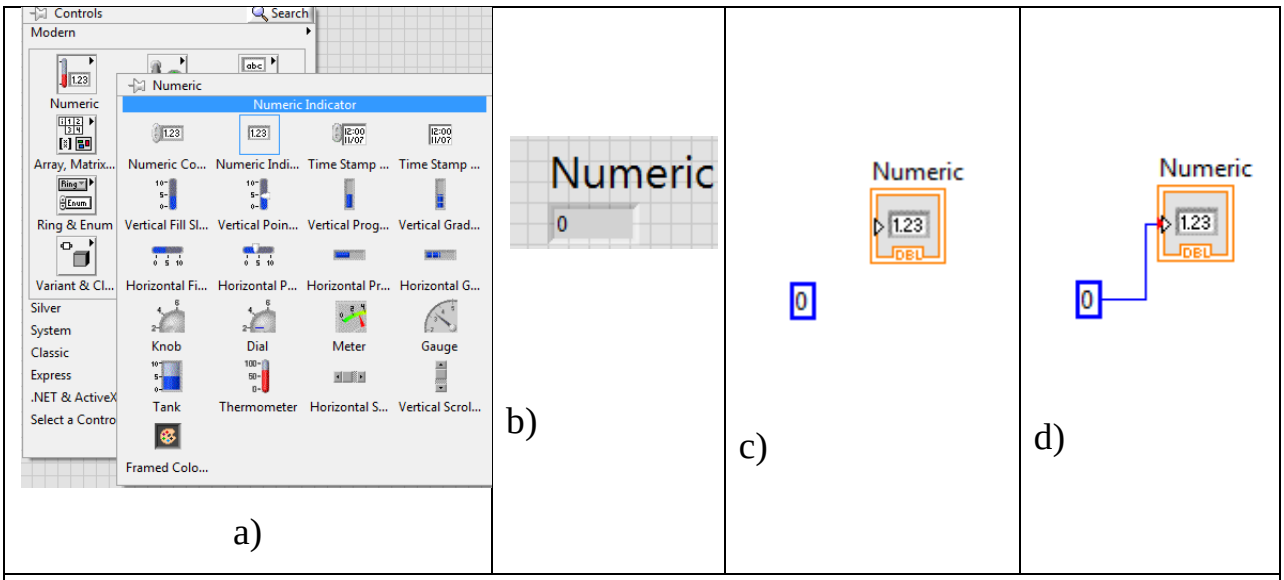


Рисунок 13. a) Элементы вложенной палитры Numeric в основной палитре передней панели. b) Индикатор отображения числовых данных. c) Значок числовой константы и значок числового индикатора в кодовой панели. d) Значок числовой константы и значок числового индикатора, соединенные проводником числовых данных.

Сейчас можно запустить созданную простейшую программу. Для этого необходимо нажать кнопку запуска (Run), расположенной на верхней инструментальной панели в виде бело стрелки (или комбинацией клавиш Ctrl+R) (рисунок 14(a)). Перед запуском программы введем в значок числовой константы в кодовой панели любое целое число (например, число 10, рисунок 14(b)). После выполнения программы значение числовой константы появится в индикаторе Numeric Indicator на передней панели. Следует обратить внимание, что если ввести не целочисленную константу, то значок константы и проводник в индикатор сменят цвет с синего на желтый (рисунок 14(c)). Это связано с тем, что данные различного типа в LabView имеют свой цвет. В частности, целочисленные данные имеют синий цвет, а данные с плавающей точкой желтый цвет. Тип данных, с которыми работает данный элемент можно посмотреть (и изменить) в пункте Properties в контекстном меню для данного элемента, вызываемого нажатием правой кнопки мыши на элемент (рисунок 15(a)).

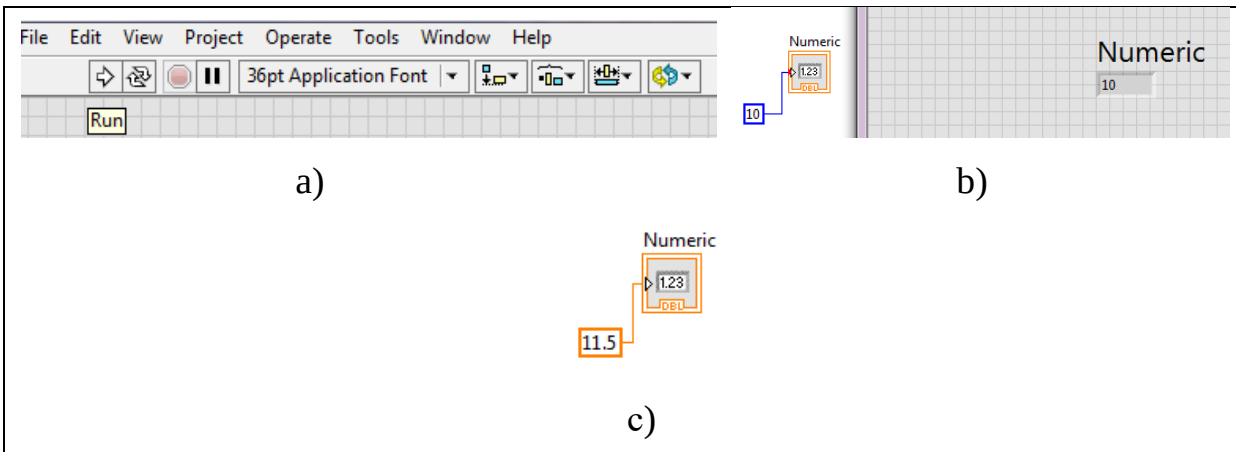
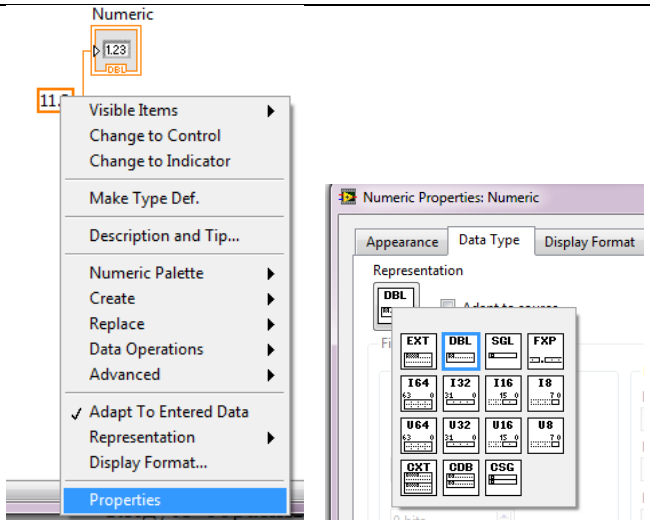


Рисунок 14. a) Запуск программы нажатием кнопки Run. b) Программа по передачи числовой константы и результат ее работы. c) Передача числовой константы с плавающей точкой.

После выбора пункта Properties, появляется дополнительное окно с радом вкладок. На вкладке Data Type можно изменить тип данных нажав на значок под надписью Representation (рисунок 15(b)). Разместим на передней панели элемент ввода данных Numeric Control. Для этого в пункте Numeric в основной палитре на правой панели (появляется при нажатии правой кнопки мыши на панель, рисунок 13(a)) выберем элемент Numeric Control и поместим его на переднюю панель (рисунок 16). Появится элемент внешне похожий на Numeric Indicator, однако он имеет белый фон в области отображения чисел и в него можно ввести число. В кодовой панели элемент Numeric Control имеет выходной терминал справа, что отличает его от элемента Numeric Indicator с входным терминалом слева. После соединения элементов можно по запуску программы передавать число из элемента ввода в элемент вывода.



a) b)

Рисунок 15. a) Контекстное меню для элемента в кодовой панели. b) Вкладка с типом данных для данного элемента.

Теперь умея вводить числа можно заняться математическими действиями над ними. Добавим на переднюю панель второй элемент Numeric Control. Отметим, что подпись элементы можно менять. Добавим в кодовой панели действие деления Divide, извлечение квадратного корня Square Root, умножения Multiply и генератор случайных чисел Random Number (0-1). Эти элементы находятся в пункте Numeric основной палитры инструментов в кодовой панели. Соединим эти элементы в любой последовательности так, чтобы использовались два вводимых в Numeric Control числа и результат подавался на Numeric Indicator (рисунок 16(b)). После запуска программы получим результат математических действий с введенными числами. Отметим, что для задания математических функций с одним потом числовых данных может использоваться функция Expression Node, располагающаяся в палитре Numeric кодовой панели. У функции Expression Node есть одни входной терминал, куда подается числовая величина, область задания функции и выходной терминал с результатом.

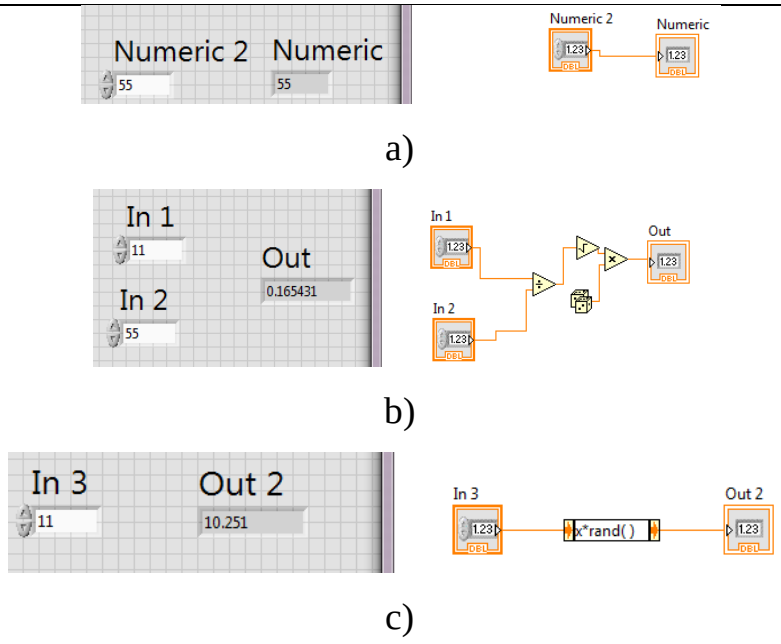


Рисунок 16. a) Элемент ввода Numeric Control и элемент вывода Numeric Indicator на передней и кодовой панели. b) Программа по выполнению математических действий с двумя вводимыми числами. c) Использование функции Expression Node.

В примере на рисунке 16(c) представлено действие по умножению подаваемой числовой константы на случайное число. При этом генератор случайных чисел представлен командой rand(), а не графическим значком. В контекстном меню (появляется по нажатию правой кнопки) для функций в кодовой панели есть команда Help при нажатии на которую появляется окно справки с описанием данной функции. Использование Help является обязательным при освоении LabView. В частности, в справке для функции Expression Node можно узнать структур команды внутри функции их синтаксис (syntax) и форму написания математических функций (functions) (рис.17).

Функция Expression Node работает с одним входным потоком числовых данных (с одной переменной). Для многих потоков (переменных) можно использовать функцию Formula Node, располагающуюся в пункте Structures основной палитры. О функции Formula Node будет написано позднее.

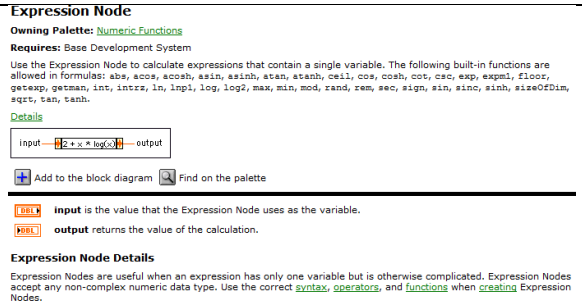


Рисунок 17. Справка Help, вызываемая в контекстном меню функции Expression Node.

Потоки данных

Так как LabView является графическим языком программирования (язык G), то код программы выполняется не так как в традиционных языках программирования.

Принцип, который управляет выполнением программы LabView, называется **потоком данных** (dataflow). Код узла выполняется только при условии, что на все входные терминалы поступили данные. После того как узел выполнит определённые действия, заложенные в нём, данные покинут узел через выходной терминал узла и по проводникам данным поступят на входные терминалы следующих узлов на блок-диаграмме. Разберём на примере созданной нами задачи. У нас есть три числа, первые два числа складываются, после чего умножаются на третье число. Мы запустили программу, одновременно все три числа начинают передаваться на узлы, однако число 1 и число 2 передаются на узел суммы, а число три на узел произведения. Узел суммы выполняет сложение чисел 1 и 2, но узел произведения не может произвести умножение суммы 1 и 2 чисел на число 3, так как сумма числа 1 и числа 2 ещё не сосчитана. То есть на узле произведения нет всех требуемых входных данных, поэтому перемножение не выполняется, и программа ждёт, пока узел суммы сосчитает сумму двух чисел и передаст полученное значение на узел произведения. В этом и заключается принцип выполнения программы с помощью потока данных.

2. Меню среды LabVIEW

Теперь рассмотрим горизонтальное меню расположенное в верхней части окна как лицевой панели, так и блок диаграммы. В меню содержится восемь пунктов: File (файл), Edit (редактирование), View(вид), Operate (операции), Project (проект), Tools (инструменты), Window (окно)и Help (помощь). Опишем подробнее каждый пункт этого меню, кроме пункта Project, он будет описан в процессе создания проектов в среде LabView.

В пункте **File** (файл), содержатся функции и операции, которые требуются для открытия, создания, записи файла.

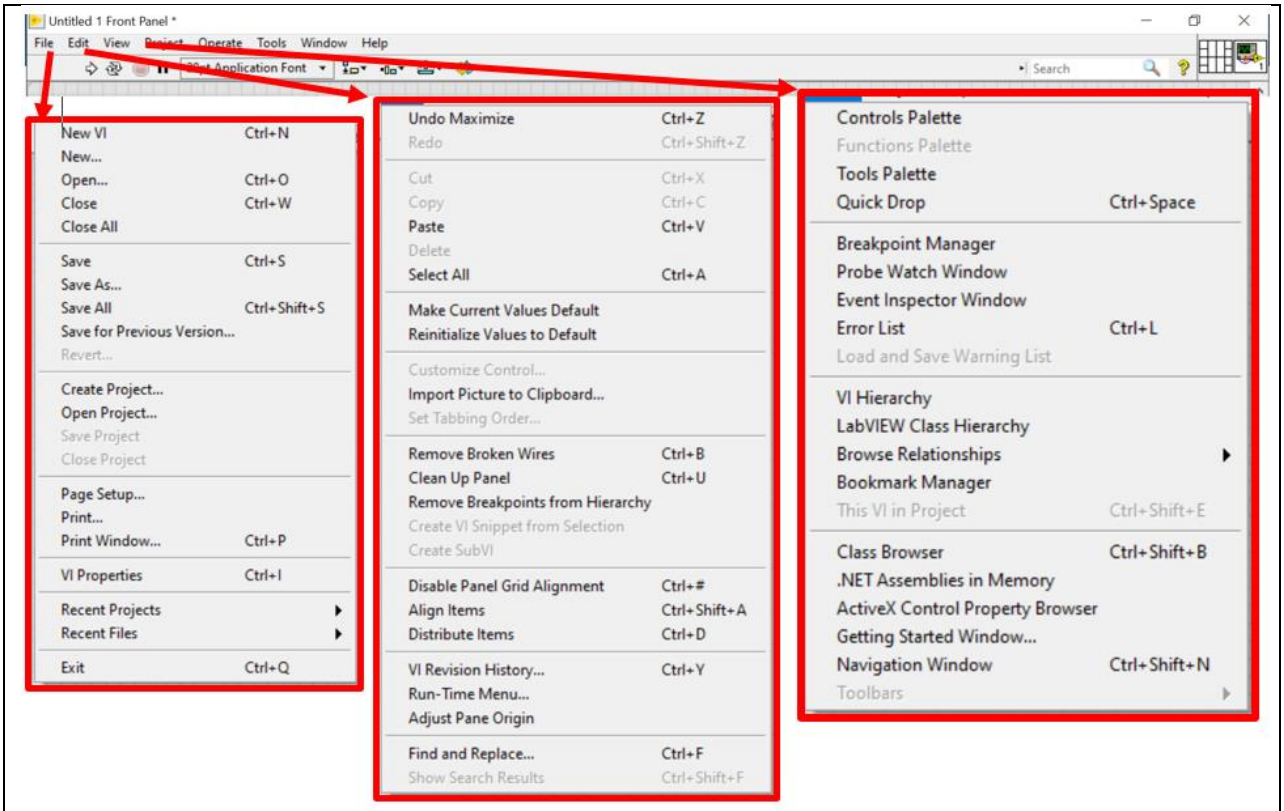


Рисунок 18. Интерфейсное окно среды программирования LabView, отображение содержимое меню File, Edit, View, содержащиеся на верхней интерфейсной панели.

Перечислим их:

New VI–создание нового виртуального инструмента;

New…–создание нового компонента:

• виртуального инструмента VI,

• глобальной переменной,

• элемента управления/индикатора,

• шаблона VI,

• шаблона глобальной переменной,

• шаблона элемента управления/индикатора,

• полиморфного VI,

• RunTime меню,

Open…–открывает существующий VI;

Close–закрывает активный VI;

Close All–закрывает все отрытые приложения;

Save–сохраняет текущий виртуальный инструмент надиске;

Save As…–сохраняет VI с новым именем;

Sav eAll–сохраняет все открытые приложения;

Save for Previous Version…–сохраняет программу для предыдущих версий LabView;

Revert…–возвращает VI к последней сохраненной версии;

Create Project **-** –создание нового проекта;

Open Project…–открывает существующий проект;

Save Project–сохраняет текущий проект на диске;

Close Project–закрывает активный проект;

Page Setup…–установка опций печати на принтер;

Print…– печать VI на принтер с возможностью выбора содержимого;

Print Window…–печать содержимого окна на принтер;

VI Properties…–свойства VI;

Recently Projects–содержит список последних открываемых десяти проектов;

Recently Files–содержит список последних открываемых десяти файлов;

Exit–завершение работы с LabView.

Следующим пунктом в меню является пункт **Edit**, в котором содержаться операции и функции для изменения объектов на интерфейсной

панели или в окне редактирования диаграмм, а также для вставки, копирования и удаления объектов.

Undo…– отмена действия;

Redo…– "возврат" отмененного действия;

Cut– удаление выбранного объекта и копирование его в буфер обмена;

Copy– копирование выделенного объекта в буфер обмена;

Paste– вставка объекта из буфера обмена;

Delete– удаление выбранного объекта;

Select All – выделяет все объекты, расположенные в окне;

Make current valued fault – устанавливает текущие значения элементов управления или индикаторов, как значение по умолчанию (к примеру, если установить в элементе управления значение, то при каждом открытии программы элемент управления будет иметь установленное ранее значение);

Reinitialize Values to Default–устанавливает все элементы управления и индикаторы в значения по умолчанию;

Customize Control… –открывает окно редактирования элементов управления и индикаторов;

Import Picture from Clipboard– копирует выбранный рисунок в буфер обмена для дальнейшей вставки;

Set Tabbing Order…– установка последовательности активизации объектов интерфейсной панели во время выполнения программы, при использовании клавиши Tab;

Remove Broken Wires– удаляет незадействованные или неправильные связи в окне редактирования диаграмм;

Clean up Panel (Diagram) – убирает пространство между элементами на лицевой панели или блок диаграмме;

Create SubVI– создание подпрограммы из выбранных объектов в окне редактирования диаграмм;

Disable (Enable) Panel (Diagram) Grid Alignment– добавляет или убирает сетку в окне лицевой панели или блок-диаграммы;

Align Items–выравнивает выделенные элементы по вертикали;

Distribute Items–устанавливает одинаковое расстояние между выделенными элементами по вертикали;

RunTime Menu…– редактирование меню;

Меню **View** содержит команды, которые позволяют находить элементы управления, палитры и всё, что связано с видом виртуального прибора:

Controls Palette – открывает окна с палитрой элементов;

Functions Palette – открывает окна с палитрой функций;

Tools Palette – открывает окна с палитрой инструментов;

Quick Drop– открывает диалоговое окно, с помощью которого можно быстро найти объект, палитры или элемент проекта по названию, а затем добавить объект на блок-схему или в окно передней панели.

Breakpoint Manager–инструментарий для управления точками останова программы при отладке.

Probe Watch Window –открывает диалоговое окно “Окно наблюдения за зондом (пробником)”. Что это такое и для чего используется будет рассмотрено ниже;

Error List–открывает лист ошибок, которые нужно устранить для корректной работы программы;

VI Hierarchy–открывает окно с иерархией виртуального прибора (порядок вызываемых функций);

LabVIEW Class Hierarchy –открывает окно с иерархиями классов;

Browse Relationships–просматривает связи элементов с библиотеками, подприборами и т.д…;

This VI in Project–добавляет этот VI (виртуального прибор) в проект;

Getting Started Window… – открывает стартовое окно при запуске программы LabVIEW;

Navigation Window– открывает окно навигации по лицевой панели или блок-диаграмме;

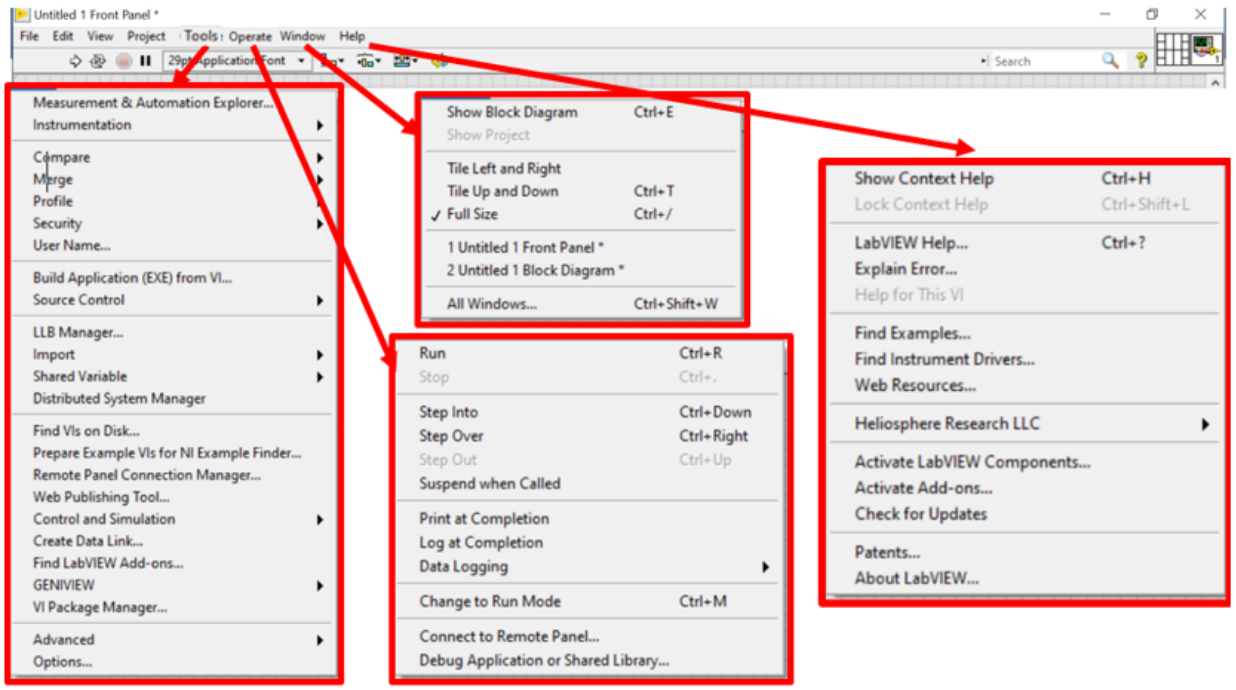


Рисунок 19. Интерфейсное окно среды программирования LabView, отображение содержимое меню Operate, Tools, Window, Help, содержащиеся на верхней интерфейсной панели.

Команды, связанные с запуском виртуального инструмента на исполнение, содержатся в меню управления **Operate:**

Run–запускает программу на выполнение;

Stop– прекращает выполнение программы;

Step Into – делает шаг вперед при пошаговой отладке программы;

Step Over– делает шаг вперед через один узел при пошаговой отладке программы;

Step Out– останавливает отладку программы;

Suspend when Called–пауза, когда вызываетсяVI;

Print at Completion– печать интерфейсной панели VI по окончанию его работы;

Log at Completion–регистрация данных по завершению работы программы;

Data Logging–настройки регистрации данных;

Change to Run Mode–переключатель между режимами выполнения программы и редактирования.

Меню **Tools** содержит пункты для управления и настройки вашего проекта и виртуального прибора**:**

Measurement & Automation Explorer…–открывает доступ к обозревателю измерений и автоматизации (Measurement & Automation Explorer), который вы можете использовать для настройки приборов и оборудования для сбора данных, подключенных к вашей системе;

Instrumentation–предоставляет доступ к инструментам для поиска или создания драйверов инструментов;

Compare–сравнение виртуальных инструментов, их иерархии, сравнение файлов. Этот пункт доступен только в системе профессионального развития;

Merge– обеспечивает доступ к функциям слияния. Этот пункт доступен только в системе профессионального развития;

Profile– обеспечивает доступ к функциям профилирования (помогает проанализировать код программы, улучшить его работоспособность и оптимизировать скорость выполнения программы). ***Profile*** включает в себя следующие элементы:

- ***Performance and Memory*** (производительность и память) – отображает окно профиля производительности и памяти, которое вы можете использовать для получения и отображения данных о времени выполнения и использовании памяти VIs.
- ***Show Buffer Allocations*** (показать распределение буфера)–отображает окно, которое вы можете использовать для определения конкретных областей на блок-схеме, где LabVIEW выделяет память.
- ***VI Metrics***–отображает окно, которое вы можете использовать для получения приблизительной оценки сложности приложения.
- ***Find Parallelizable Loops*** (найти распараллеливаемые циклы) – отображает окно результатов поиска распараллеливаемых циклов, которое вы можете

использовать для анализа VI на наличие циклов, которые вы можете распараллелить.

Security (безопасность)–обеспечивает доступ к функциям безопасности. ***Security*** включает в себя следующие элементы:

- ***Login*** (вход)–отображает диалоговое окно входа в систему NI Security, которое вы можете использовать для входа в LabView.
- ***Change Password*** (изменить пароль)– отображает диалоговое окно, которое вы можете использовать для изменения пароля текущего пользователя в домене.
- ***Logout*** (выход)– выход из системы выводит текущего пользователя из LabVIEW.
- ***Domain Account Manager*** (диспетчер учетных записей домена)–получает доступ к диспетчеру учетных записей домена, который вы можете использовать для создания и уничтожения локального домена, экспорта и импорта локального домена, применения политик домена и управления учетными записями пользователей и групп в доменах локально или удаленно.

User Name–отображает диалоговое окно входа пользователя, которое вы можете использовать для установки или изменения имени пользователя LabVIEW.

Source Control Accesses (система управления версиями) – получает доступ к операциям управления версиями. Этот пункт доступен только в системе профессионального развития.

Find VIson Disk–отображает окно, которое можно использовать для поиска VI по имени файла в каталоге.

Prepare Example VIs for NI Example Finder (подготовка примера Vis для***NI Example Finder*** (смотреть в меню HELP ниже) –отображает диалоговое окно, которое вы можете использовать для подготовки созданного вами примера VIs для отображения в NI Example Finder.

Web Publishing Tool (инструмент веб-публикации) –отображает диалоговое окно, которое можно использовать для создания HTML-документов и встраивания изображений передней панели VI.

Find Lab VIEW Add-ons (поиск дополнений LabVIEW) – запускает сеть инструментов (дополнений) LabVIEWраспространяемую в сети LabView Tools по адресу *ni.com/labview-tools-network*. Если у вас установлено ***VI Package Manager*** (VIPM) то нужно нажать на кнопку ***VI Package Manager*** (расположенную чуть ниже в меню), которое открывает программу со всеми возможными дополнениями для LabVIEW.

Advanced–расширенный доступ к расширенным функциям LabView. Расширенный включает в себя следующие элементы:

- ***Mass Compile*** (массовая компиляция) – отображает диалоговое окно массовой компиляции, которое вы можете использовать для компиляции или перекомпиляции каталога VIs или LLB. Вы также можете использовать метод массовой компиляции для загрузки и компиляции всех VIS в каталоге программным путем.
- ***Clear Compiled Object Cache*** (очистить кэш скомпилированных объектов)– отображает диалоговое окно, которое можно использовать для очистки скомпилированного кода либо из кэша объектов пользователя, либо из кэша объектов Application Builder. Вы также можете использовать методы Clear Compiled Object Cache и Clear Application Builder Cache, чтобы программно очистить все файлы, хранящиеся в этих объектных кэшах.
- ***Edit Error Codes*** (редактирование кодов ошибок)–диалоговое окно редактора кодов ошибок, которое можно использовать для создания или изменения текстового файла с кодом ошибки.
- ***Edit Palette Set*** (редактировать набор палитр)–диалоговое окно, которое можно использовать для редактирования набора палитр.
- ***Create or Edit Express VI*** (создать или отредактировать Express VI)–позволяет создавать, редактировать, переименовывать, перемещать или проверять Express VI.

- ***Export Strings*** (экспорт строк)–позволяет экспортировать строки VI для локализации VI.
- ***Import Strings*** (импорт строк)–позволяет импортировать строки VI для локализации VI.

Options (параметры)– позволяет использовать для настройки среды LabView, а также внешнего вида и поведения приложений LabView.

Меню **Window** содержит пункты для настройки внешнего вида текущих окон.

Show Front Panel/Show Block Diagram (показать переднюю панель / показать блок-диаграмму)–переключение между окнами передней панели и блок-диаграммы текущего VI.

Show Project (показать проект)–отображает окно обозревателя проектов для проекта, содержащего текущий VI.

Tile Left and Right–упорядочивает открытые окна слева направо.

Tile Up and Down–упорядочивает открытые окна сверху вниз.

Full Size–расширяет текущее окно, заполняя весь экран.

All Windows–открывает диалоговое окно, с помощью которого можно управлять всеми открытыми окнами виртуальных приборов.

Меню **HELP** содержит пункты для объяснения и определения функций LabView и других компонентов, предоставления полной документации по LabView и доступа к технической поддержке National Instruments.

Show Context Help (показать контекстную справку)–отображает или скрывает окно контекстной справки.

Lock Context Help (блокировка контекстной справки)–блокирует или разблокирует текущее содержимое окна контекстной справки.

LabVIEW Help (справка LabView)–открывает файл справочного материала для получения информации о палитрах, меню, инструментах, виртуальных приборов и функциях LabView. Справка LabView также содержит пошаговые инструкции по использованию функций LabView.

Explain Error (объяснение ошибки)–обеспечивает доступ к полной справочной информации по любым текущим ошибкам в VI.

Help for This VI (справка для этого VI)–напрямую получает доступ к полной справочной информации VI из справки LabView для текущего VI.

Find Examples (поиск примеров)–выполняет поиск по сотням примеров VIS. Вы можете изменить пример в соответствии с вашим приложением или скопировать и вставить один или несколько примеров в свой собственный VI.

Find Instrument Drivers (поиск драйверов приборов) – позволяет найти и установить драйверы приборов LabView Plug and Play.

About LabVIEW (информация LabView) – позволяет получить доступ к общей информации о вашей текущей установке LabView, включая номер версии и серийный номер.

Плавающие палитры

Палитры Элементы управления и Функции

Среда LabView состоит из двух окон (лицевая панель и блок–диаграмма), которые выполняют различные функции и предназначения в создании программы, поэтому для каждого окна существует своя палитра. Для лицевой панели – палитра элементов управления, для блок-диаграммы – палитра функций. В целом палитра инструментов представляет собой меню, которое состоит из структурированного набор различных элементов, которые объединены в группы. В зависимости от того, в каком окне вы находитесь, вам будет доступна панель, которая связанна с этим окном, то есть выбор палитры происходит автоматически и вам не требуется переключаться между двумя возможными палитрами. Рассмотрим более подробно каждую из этих палитр.

Палитра Функций

Палитра функций (Functions Palette) – палитра, которая содержит необходимые функции для работы с различными типами и структурами

данных, позволяющие написать графический код программы любой сложности на блок–диаграмме. Открыть эту палитру можно двумя способами. Первый способ – вызывать палитру из основного меню Window. Для этого надо нажать ***Window*** > ***Show Function Palette***. Второй способ – щёлкнуть правой кнопкой мыши по полю открытого окна (в данном случае по полю на блок–диаграмме), после чего откроется меню, вид которого представлен на рисунке 20(а). Все функции палитры разделены на группы – подпалитры, которые объединяют все элементы и функции предназначенные для определенной области использования (к примеру, в подпалитре Matematics, собранны функции, которые позволяют осуществлять какие–либо математический действия). При этом элементы и функции в каждой подпалитре также объединены в более узкие группы. Опишем более подробно имеющиеся подпалитры.

Подпалитра ***Programming*** содержит в себе функции и группы функций, которые необходимы для написания графического кода программы. Иными словами, это базовые функции, которые необходимы для программирования в среде LabView. Подпалитра ***Programming*** содержит такие группы функций как:

- ***Structures***– основные структуры: циклы, последовательности…
- ***Array***– функции для работы с массивами данных;
- ***Cluster, Class, Variant***–функции для создания кластеров и классов LabView и управления ими, преобразования данных LabView в формат, которым вы можете манипулировать независимо от типа данных;
- ***Numeric***– функции арифметических операций и вычислений;
- ***Boolean***– функции для работы с данными булевого типа;
- ***String***– функции для работы с данными строкового типа;
- ***Comparison***– функции сравнения;
- ***Waveform***– функции для работы с сигналами (создание сигнала, сложение волн, запись/чтение на диск,с диска, и т.д.);

- ***Collection***– функции сбора для манипулирования данными сбора.
- ***File I/O (I –input, O–output)***– функции для работы с файлами (чтение, запись, удаление и т.д.);
- ***Timing***– функции определения системного времени, даты, задержки и др.;
- ***Dialog & Users Interface***–функции диалогового окна и пользовательского интерфейса для создания диалоговых окон, предлагающих пользователям инструкции.;
- ***Sinchronization***– функции управления потоком данных, которые позволяют создавать уведомления, очереди, события и т.д.;
- ***Graphics & Sound***– функции для работы с графическими файлами и со звуком;
- ***Application Control***– функции управления приложением;
- ***Report Generation***– функции для создания отчетов, как в текстовом формате, так и формате html, вывод на печать;

Подпалитра ***Mathematics***– функции высшей математики (задачи линейной алгебры, матрицы, функции аппроксимации и т.д.);

Подпалитра ***Signal Processing***–функции для обработки сигналов для генерации сигналов, цифровой фильтрации, отображения данных в окне и анализа спектра.

Подпалитра ***Express*** –подпалитра экспресс-ВП и функциями для создания общих задач измерения.

Подпалитра ***Data Communication***– функции передачи данных для обмена данными между приложениями.

Подпалитра ***Instrument I/O***– набор функций для работы со стандартными измерительными устройствами;

Подпалитра ***User Libraries***–палитра пользовательских библиотек для добавления VIs в палитру функций. По умолчанию палитра пользовательских библиотек не содержит никаких объектов.

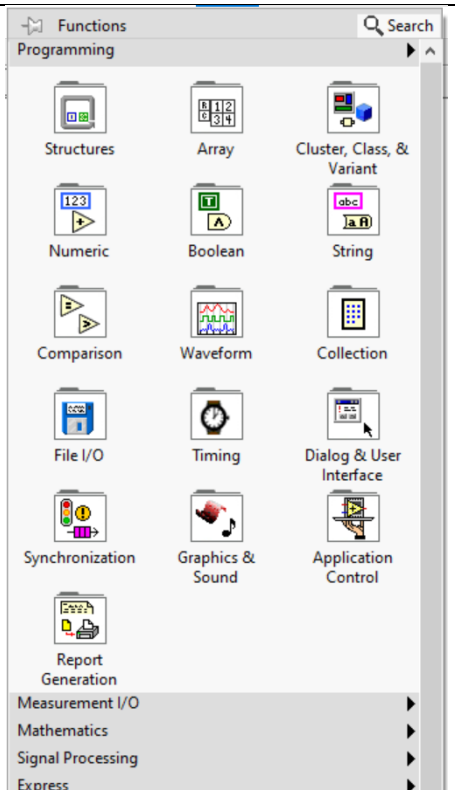
Также в палитре функций можно найти строчку в списке с названием ***Select a VI***, которая позволяет размещать элементы, которые не

отображаются на палитре функций. При нажатии на кнопку перед вами откроется содержимое вашего компьютера, в котором вам нужно выбрать виртуальный прибор, который требуется разместить на блок-диаграмме (более подробно рассмотрим эту функцию ниже).

Палитра Элементов управления

Палитра элементов управления (Controls) – палитра, которая содержит необходимые элементы управления и индикаторы для работы с различными типами, позволяющие осуществлять управление программой. Открыть эту палитру можно двумя способами. Первый способ – вызывать палитру из основного меню Window. Для этого надо нажать ***Window*** > ***Show Controls Palette***. Второй способ – щёлкнуть правой кнопкой мыши по полю открытого окна (в данном случае по полю на лицевой панели), после чего откроется меню, вид которого представлен на рисунке 20(b).

(a)



(b)

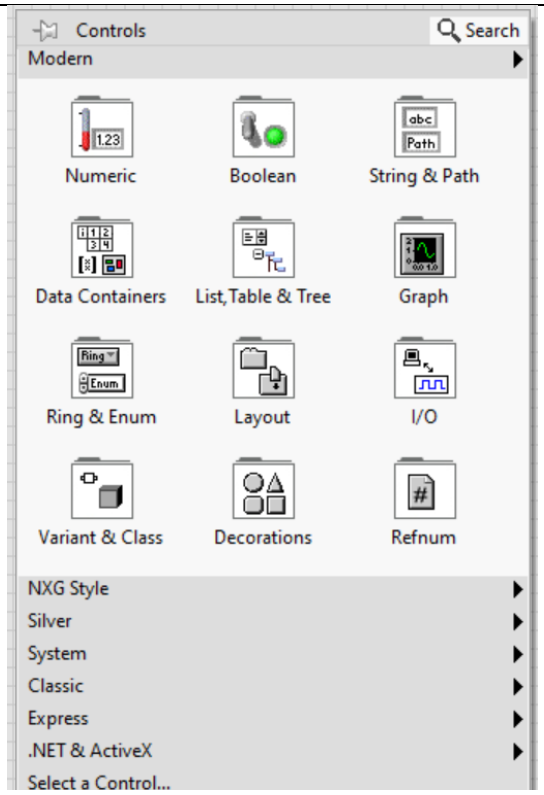


Рисунок 20. a) Палитра функций Function Palette, расположенная в окне блок–диаграммы. b) Палитра элементов управления Controls Palette в окне лицевой панели.

Подпалитра ***Modern*** содержит элементы управления и индикаторы, распределённые по группам, перечисленным ниже:

- ***Numeric***– элементы управления и индикаторы числового типа;
- ***Boolean***– элементы управления и индикаторы булевого типа;
- ***String & Path***– элементы управления и индикаторы строкового типа;
- ***Data Containers***– элементы управления и индикаторы массивов и кластеров различных типов данных;
- ***List, Table & Tree***– элементы управления и индикаторы списков и таблиц;
- ***Graph***– индикаторы для различных типов графиков;
- ***Ring & Enum***– элементы управления и индикаторы для создания выпадающих меню и перечней;
- ***Layout***–элементы для группировки элементов управления и индикаторов или для отображения передней панели другого VI на передней панели текущего VI.
- ***I/O***– графические объекты для конфигурирования и работы NI–аппаратуры;
- ***Variant & Class***–элементы управления и индикаторы вариантов, а также элементы управления и индикаторы классов.
- ***Decorations***– графические элементы оформления для графической группировки или разделения объектов передней панели, не влияя на функциональность VI. Украшения включают в себя такие объекты, как прямоугольники, линии или стрелки.;
- ***Refnum***– управление свойствами приложения;

Подпалитра ***System*** – содержит все те же самые элементы, что и в подпалитре ***Modern***, которые имеют системное оформление элементов и индикаторов.

Подпалитра ***Classic***–подпалитра, в которой собраны графические объекты предыдущей версии LabVIEW;

Подпалитра ***Addons***–подпалитра, которая содержит добавленные в программу пользователем элементы управления и индикаторы.

Подпалитра ***Select a Control***– выбор и установка элемента управления или индикатора, который не входит в стандартный набор Панели Управления;

Справа от названия каждой подпалитры можно увидеть треугольник, который при нажатии на него левой кнопкой мыши разворачивает подпалитру и показывает всё её содержимое (рис.20). Если повторно нажать на треугольник, то палитра скроет своё содержимое и останется только название подпалитры. Нажатием на треугольник можно просматривать содержимое любой подпалитры. При открытии окна палитры у вас автоматически будет отображаться раскрытое содержимое подпалитры Modern (если открыта лицевая панель, Programming – если открыта блок–диаграмма).

Редактирование плавающих палитр

Закрепление палитры

Когда мы выбираем элемент в палитре, который хотим поместить на блок-диаграмму или на лицевую панель, палитра закрывается, а выбранный элемент прикрепляется к курсору мышки до момента его размещения в отрытом окне. Для того, чтобы разместить ещё элементы в окне, нам снова потребуется открыть палитру инструментов и выбрать элемент, после чего окно палитры снова исчезнет. Так будет происходить каждый раз, когда нам потребуется разместить и найти какой–либо элемент. Многократное открывание палитры инструментов далеко не всегда удобно при написании графического кода программы. На этот случай в среде LabView предусмотрена возможность закрепления окна палитры, после которого палитра не будет исчезать после каждого выбора элемента. Палитра исчезнет только после того, как пользователь сам решить её убрать нажатием на крестик, расположенный в правом верхнем углу. Чтобы закрепить палитры на рабочем окне, нажмите на булавку в левом верхнем углу (рис.21(а)). Изначально булавка расположена горизонтально – палитра не закреплена, после нажатия на булавку её расположение сменится на вертикальное –

палитра закреплена. Чтобы открепить палитру требуется также нажать на булавку или просто закрыть её нажав на крестик (рис.21(b) и рис.21(c)).

Иногда может потребоваться закрепить не всю палитру элементов, а только её часть, которая используется наиболее часто. Такая возможность тоже есть. Для того, чтобы закрепить нужную группу элементов, зайдите в подпалитру и откройте в ней группу элементов, которую вы хотите закрепить.

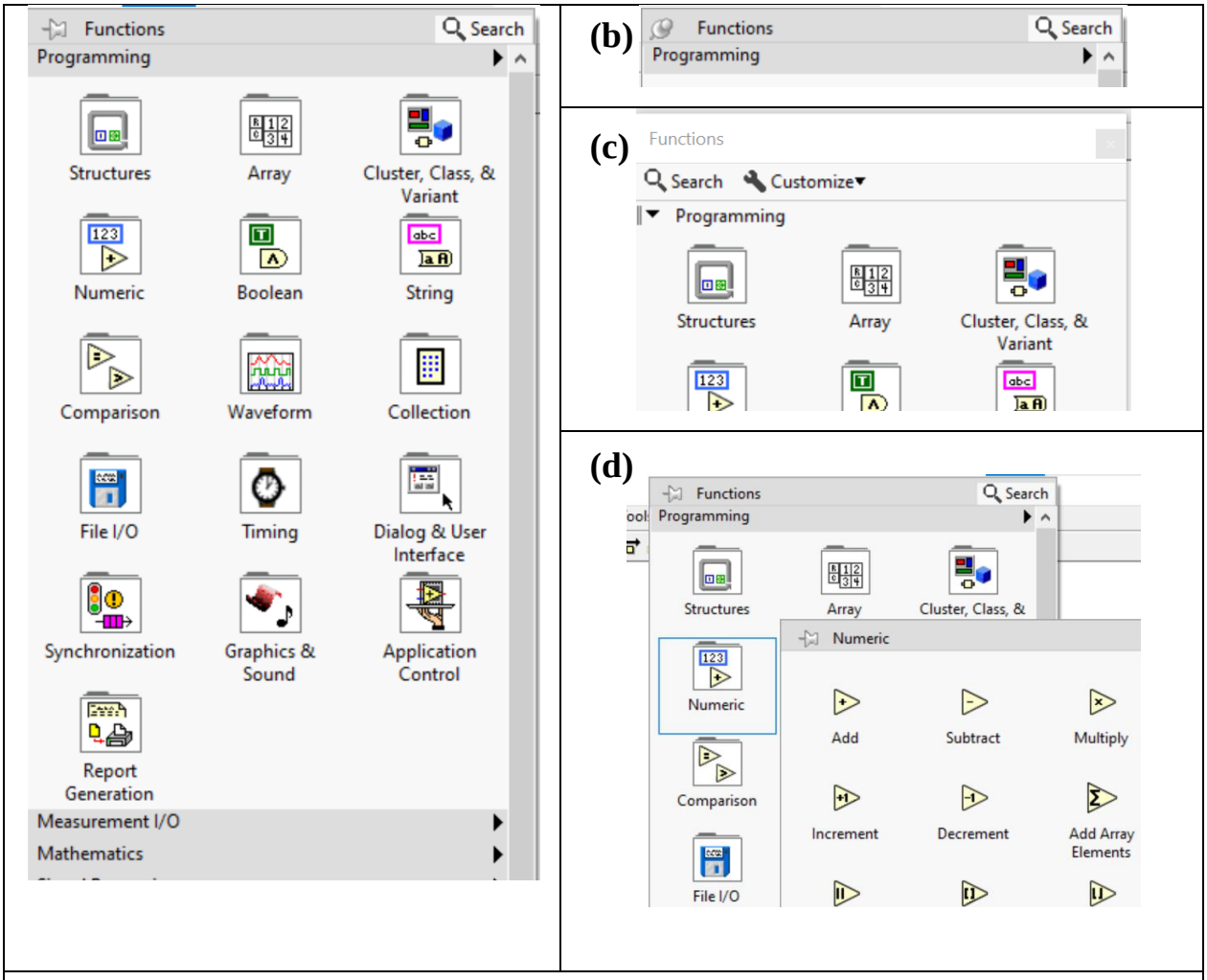


Рисунок 21. a) Вид булавки до закрепления палитры функций Function Palette в окне блок–диаграммы. b) Вид булавки после закрепления палитры функций Function Palette в окне блок–диаграммы. c) Вид кнопки в виде крестика для открепления палитры от диалогового окна. d) Вид булавки в подпалитре для её закрепления в окне блок–диаграммы.

В открывшемся окне группы элементов в левом верхнем углу, также, как и в самой палитре, расположена булавка, которая закрепит на рабочем окне только выбранную вами группу элементов, а не всю палитру инструментов (рис. 21(d)). К примеру, эта функция будет удобна в случае, когда нужно составить код, который будет производить расчёты сложного и длинного выражения, состоящего из большого количества математических операций. В этом случае группу функций, содержащую математические операции, удобно закрепить в рабочем окне, чтобы не открывать многократно палитру функций для написания кода.

Скрыть подпалитру

Подпалитры, которые вы используете можно скрыть из окна палитры, чтобы они не занимали место и упрощали поиск требуемых элементов и функций. Чтобы скрыть подпалитру нужно сначала закрепить всю палитру инструментов. После этого в верхней части палитры вы увидите две кнопки: ***Search*** и ***Customize***. Чтобы редактировать подпалитры нужно нажать на кнопку ***Customize*** (самая крайняя с права кнопка) (рис.22(a)). После нажатия перед вами появится выпадающее меню, в котором нужно выбрать раздел ***Change Visible Palettes*** (изменить видимые подпалитры) (рис.22(b)). Перед вами откроется окно, в котором вы увидите все подпалитры (рис.22(c)).

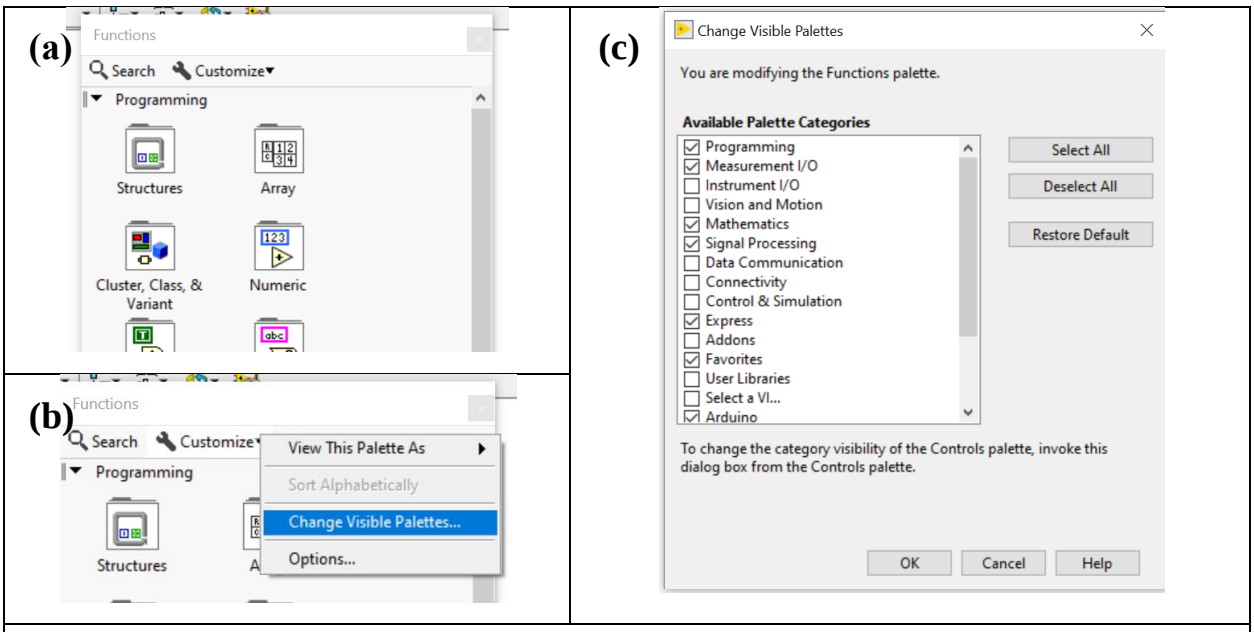


Рисунок 22. a) Кнопка редактирования подпалитр Customize в палитре функций Function Palette. b) Контекстное меню кнопки редактирования подпалитр Customize. c) Диалоговое окно для редактирования отображения подпалитр в палитре функций Function Palette.

Около каждой подпалитры стоит галочка, если убрать это галочку щёлкнув на название подпалитры, то эта подпалитра пропадёт из окна палитры и не будет там отображаться (то есть подпалитра будет скрыта). В правом верхнем углу диалогового окна вы увидите кнопку ***Select All*** (выбрать всё), нажатие на которую сделает все подпалитры видимыми и вам не придётся ставить галочку около названия каждой подпалитры. Ниже кнопки ***Select All*** расположена кнопка ***Deselect All***, которая делает все палитры невидимыми. Для сохранения ваших настроек подпалитр нажмите на кнопку ОК, расположенную внизу окна.

Отображение палитр

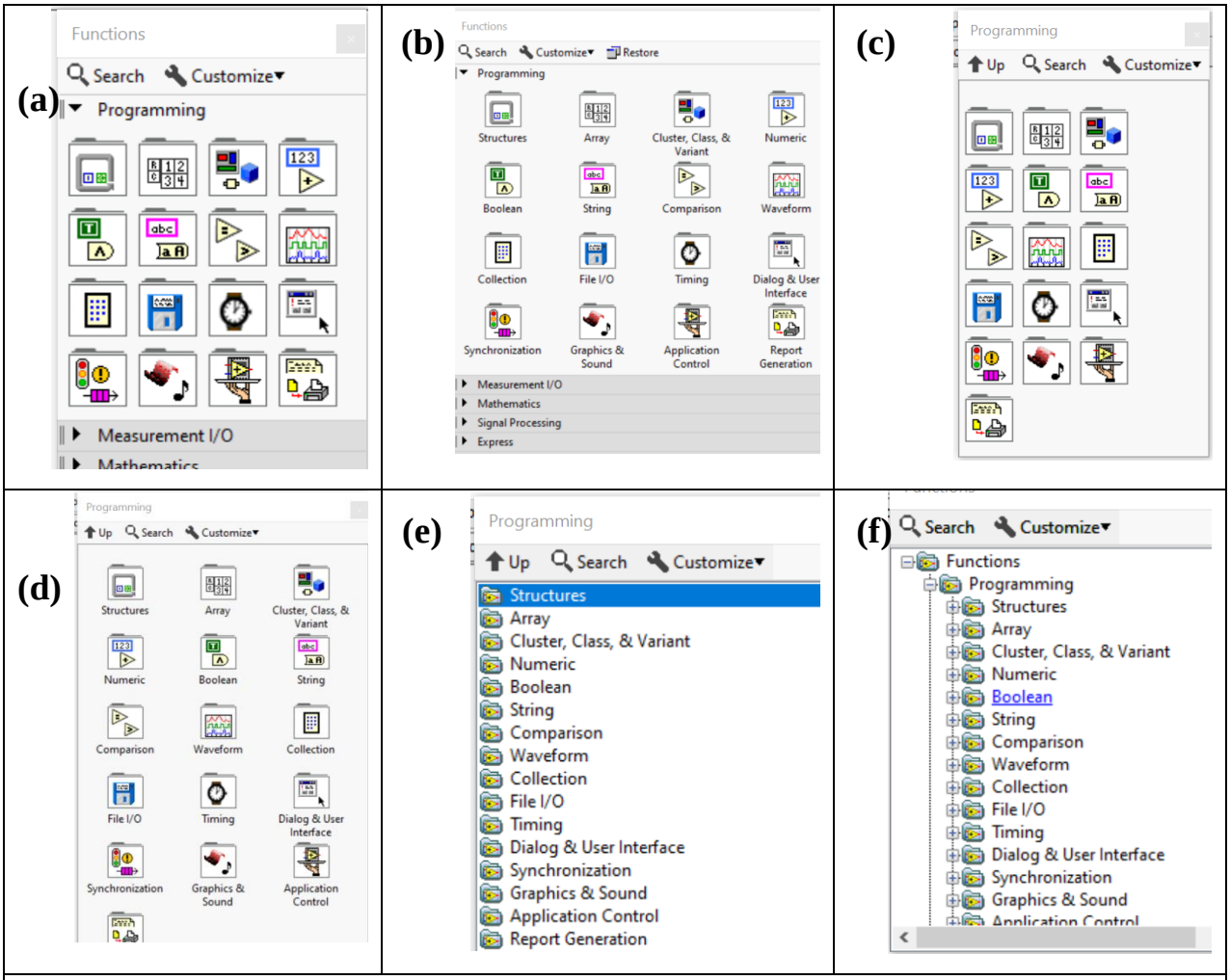


Рисунок 23. a) Стандартный **Category (Standart)** вид отображения палитры. b) Стандартный **Category (Icons and Text)** вид отображения палитры. c) Стандартный **Icons** вид отображения палитры. d) Стандартный **Icons and Text** вид отображения палитры. e) Стандартный **Text** вид отображения палитры. f) Стандартный **Tree** вид отображения палитры.

Также при нажатии на кнопку ***View*** можно увидеть подменю с названием ***View this palette as***, которое позволяет настраивать отображение объектов палитры.

Всего возможно шесть вариантов отображения палитры, перечислим их и сделаем краткое описание:

1. **Category (Standart)** – Категории, стандартный вид, который представляет собой набор упорядоченных папок с изображениями. При этом наборы папок относятся к определённой подпалитре.
2. **Category (Icons and Text)** – Иконки и текст. Этот вариант отображения похож на предыдущий. Отличие этих двух вариантов заключается в отображающемся названии папки, которое расположено прямо под иконкой.
3. **Icons** – Иконки. Это отображение представляет собой набор папок с изображениями (иконки), которые занимают всю палитру. Иконки также расположены в определённой подпалитре, но в данном виде отображения весь список подпалитр не видно. При выборе конретной подпалитры название не отображается в окне палитры, отображается только её содержимое. Поэтому для выбора другой подпалитры требуется выйти из содержимого текущей подпалитры и перейти к списку подпалитр. Чтобы это сделать нужно нажать на чёрную стрелочку вверх, расположенную в левой верхней части окна.
4. **Icons and Text** – Иконки и текст. Этот вид отображения помимо иконок, описанных в предыдущем пункте, содержит названия подпалитры. Добавление названий иконок упрощает поиск какого-либо элемента из палитры.
5. **Text**– текст. Этот вариант представляет собой папки с названиями подпалитр, при нажатии на которые открывается содержимое выбранной подпалитры. Чтобы увидеть содержимое другой подпалитры нам нужно вернуться к изначальному списку подпалитр. Чтобы вернуться назад надо нажать на стрелочку вверх, которая расположена в верхней части окна палитры. По своей сути текстовый вид отображения напоминает привычный нам вид отображения папок в персональном компьютере, когда при нажатии на какую-то папку нам открывается её содержимое, а нажатие стрелочки назад возвращает нас к предыдущей папке.
6. **Tree**– Дерево. В этом отображении также используется только текстовые обозначения элементов палитры. Отличие от предыдущего вида отображения

заключается в том, что содержимое какой-либо подпалитры можно увидеть путём её разворота (нажать на квадратик с плюсом около названия подпалитры), при это название самой подпалитры и других подпалитр не исчезнет. Чтобы свернуть содержимое выбранной подпалитры нужно нажать на квадратик с минусом около подпалитры.

Поиск элемента или функции

Элементов и функций в палитрах содержится большое количество и не всегда можно запомнить их месторасположение, поэтому не редко возникает вопрос где же найти нужный элемент, который когда-то уже использовал, но забыл группу и подпалитру, в которой он находится. На этот случай в подпалитре есть поиск (Search), который обозначен значком лупы в верхней части палитры (рис.24(b)). Чтобы найти элемент нужно нажать на поиск и начать вводить название нужного вам элемента (названия вводятся на английском языке) (рис.24(a)), поиск предложит вам подходящие варианты(рис.24(d)). В целом принцип работы поиска происходит аналогично поиску в любом браузере интернета.

Палитра инструментов

Также в среде LabView есть палитра инструментов (***Tools Palette***), которая содержит различные виды курсоров мыши, каждый из которых выполняет определённую функцию (рис.24(c)). Каждый вид курсора выполняет функцию, которая направлена на создание, редактирование или отладку графического кода программа (ниже будет подробный разбор отладки программы). При выборе определенного инструмента значок курсора мыши приобретает форму этого инструмента. Всего в палитре инструментов содержится 11 инструментов: управление (***Operate Value***, "палец"), перемещение (***Position / Size / Select***, "стрелка"), редактирование текста (***Edit Text***, "буква"), соединение (***Connect Wire***, "катушка"), контекстное меню объекта (***Object Shortcut Menu***), быстрая прокрутка окна

(***Scroll Window***), контрольная точка (***Breakpoint***), установка пробника (***Probe Data***, "пробник (зонд)"), получить цвет (***Get Color***, "пипетка"), установить цвет (***Set Color***,"кисть"), Автоматический выбор инструмента (***Automatic Tool Selection***) (таблица 1).

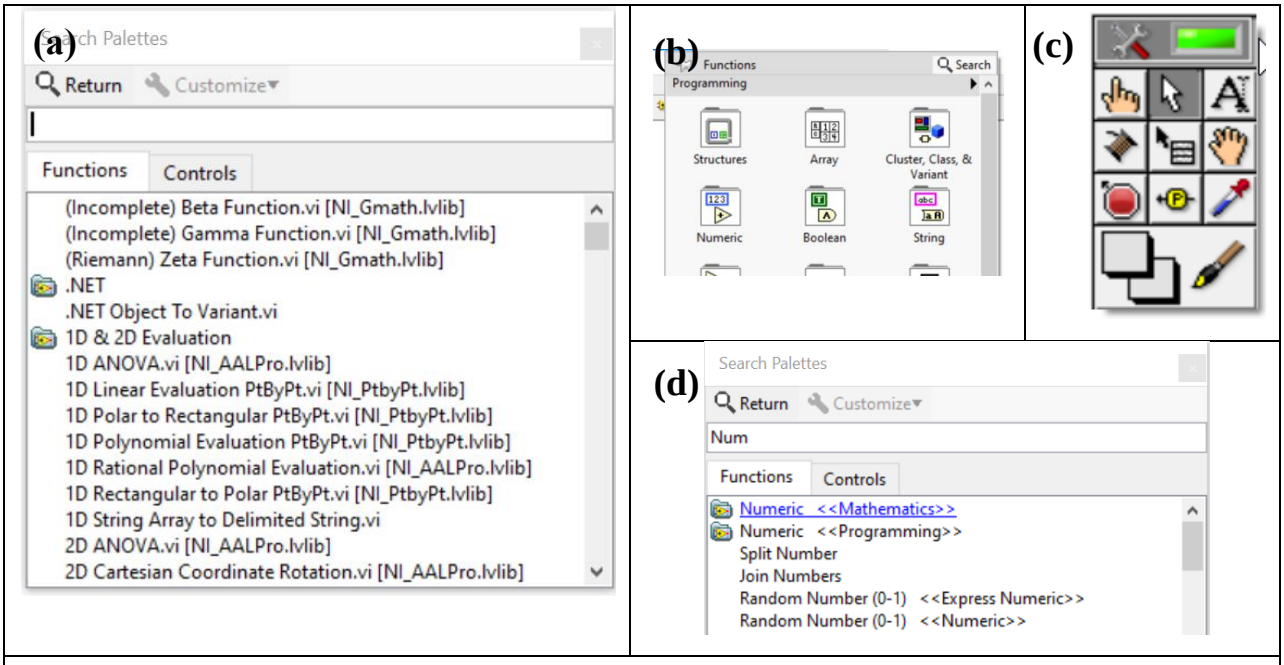


Рисунок 24. a) Стандартный **Category (Standart)** вид отображения палитры. b) Стандартный **Category (Icons and Text)** вид отображения палитры. c) Стандартный **Icons** вид отображения палитры.

Стоит отметить, что использование инструментов пипетка и кисть возможно только на лицевой панели. На блок-диаграмме цвет какого-либо элемента изменить не получится (можно менять только цвет фона).

Таблица 1. Инструменты из плавающей палитры ***Tools Palette*** и их краткое описание.

Инструмент	Описание
	Инструмент управление (Operate Value) используется для управления объектами (например, перемещение ползунка) и изменения значений элементов управления или ввода текста
	Инструмент перемещение (Position / Size / Select) позволяет перемещать объекты или измененять их размер (стрелка меняет свой внешний вид, когда доступна опция изменения размеров объекта).
	Инструмент редактирование текста (Edit Text) используется для ввода и редактирования текста, как в блок-диаграмме, так и на лицевой панели. Также этот инструмент позволяет создавать свободные метки. Чтобы создать метку в режиме автоматического выбора инструмента нужно дважды щёлкнуть левой кнопкой мыши.
	Инструмент соединение (Connect Wire) применяется для соединения объектов на блок-диаграмме.
	Инструмент контекстное меню объекта (Object Shortcut Menu) вызывает контекстное меню соответствующего выбранного ответа. Также открыть контекстное меню можно щёлкнув правой кнопкой мыши по требуемому элементу.
	Инструмент быстрая прокрутка окна (Scroll Window) используется для просмотра окна без обращения к полосам прокрутки (одновременная прокрутка всех объектов).
	Инструмент получить цвет (Get Color) позволяет скопировать цвет объекта, расположенного на лицевой

	панели. Чтобы скопировать цвет необходимо подвести указатель и нажать левую клавишу мыши.
	Инструмент контрольная точка (Breakpoint) позволяет устанавливать точки (меток) останова программы в процессе её отладки (что такое отладка будет рассмотрено дальше). С его помощью можно на время остановить выполнение программы, посмотреть, что происходит, и изменить в случае необходимости значения входных данных.
	Инструмент установка пробника (Probe Data) позволяет устанавливать пробник на элементах блок-диаграммы, который показывает значение данных, проходящих через установленный пробник, во время работы ВП.
	Инструмент установить цвет (Set Color) используется для изменения цвета элементов и фона, которые расположены на лицевой панели. изменения цвета объекта. Установить цвет можно щелкнув мышью по соответствующей цветовой области в палитре инструменты.
	При включенном автоматическом выборе инструмента (Automatic Tool Selection) наведение курсора на объект лицевой панели или блок-диаграммы LabVIEW приводит к автоматическому выбору соответствующего инструмента из палитры инструментов.

Если в место инструмента Automatic Tool Selection выбрать любого другой инструмент, то это приведёт к отключению Automatic Tool Selection. Когда выбран какой-либо конкретный инструмент, то его вид можно изменять циклически с помощью нажатия на клавишу <Tab>. Для переключения между инструментом Перемещение и Соединение на блок-

диаграмме или между инструментом Перемещение и Управление на лицевой панели достаточно нажать пробел.

Линейка инструментов.

В верхней части окон блок-диаграммы и передней панели расположены линейки инструментов, которые содержат служебные кнопки и индикаторы состояния, предназначенные для контроля виртуального прибора. Содержание элементов в линейке инструментов и вид интерфейсной панели зависит от того находитесь вы в окне блок-диаграммы или в окне лицевой панели.

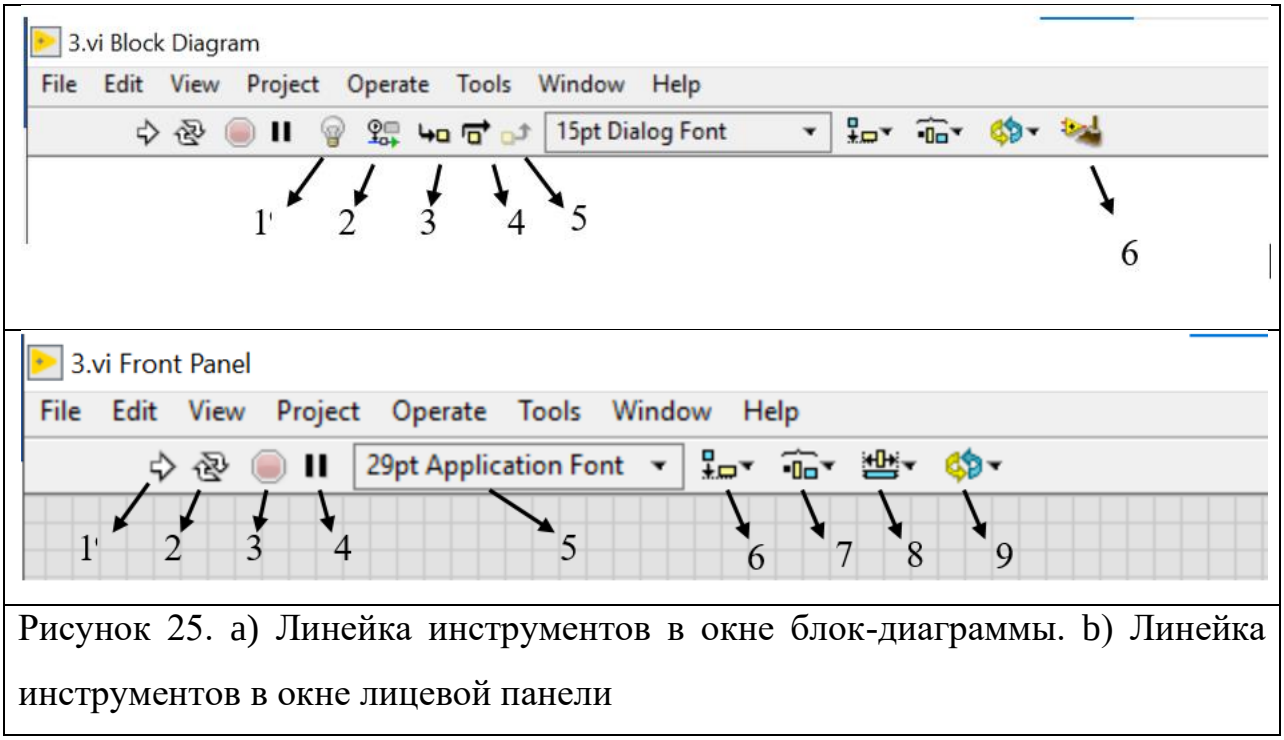


Рисунок 25. a) Линейка инструментов в окне блок-диаграммы. b) Линейка инструментов в окне лицевой панели

Линейка инструментов в окне лицевой панели (см.рис.25(b)):

1. Кнопка запуска программы на выполнение. Во время выполнения кода программы значок меняет свой внешний вид;
2. Кнопка запуска программы на выполнение в циклическом режиме. Приложение выполняется многократно без изменения вида значка до тех пор, пока выполнение программы не будет прервано вручную.

3. Кнопка остановки выполнения кода программы. Использование этой кнопки возможно только во время выполнения кода программы, когда код программы не выполняется, эта кнопка находится в неактивном состоянии.

4. Кнопка "ПАУЗА" позволяет приостановить исполнение программы до последующего нажатия этой кнопки;

5. Выпадающее меню редактирования свойств шрифта которое включает в себя: тип, размер, стиль и цвет;

6. Выпадающее меню состоящее их элементов, которые позволяют выравнивать и позиционировать объекты друг относительно друга;

7. Выпадающее меню, содержащее команды пространственного распределения графических объектов передней панели VI.

8. Выпадающее меню, содержащее элементы, которые позволяют изменять размеры большинства объектов на лицевой панели.

9. Если один объект установлен поверх другого объекта, то нижний объект может быть перекрыт и недоступен. Элементы, расположенные в выпадающем меню позволят размещать объекты поверх друг друга или под желаемым объектом.

3. Оформление программ LabVIEW

Также в верхней части окна лицевой панели имеются пункты главного меню (Alignment, Distribution и Reorder), которые позволяют осуществлять редактирование элементов, расположенных на лицевой панели. Содержимое меню для редактирования и их краткое описание представлено в таблице 2, таблице 3 и таблице 4.

Таблица 2. Инструменты выравнивания группы объектов, содержащиеся в меню Alignment Ring.

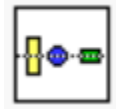
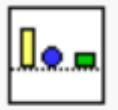
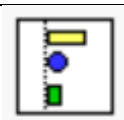
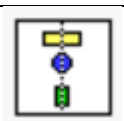
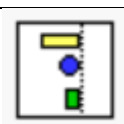
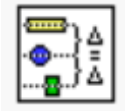
 **Alignment Ring** Инструменты выравнивания группы объектов. Выделите объекты, подлежащие выравниванию, затем выберете расположение оси выравнивания	
 **Top Edges**	Выравнивание выделенных объектов по верхнему краю самого верхнего объекта
 **Vertical Centers**	Выравнивание выделенных объектов по середине между самым верхним и самым нижним объектом
 **Bottom Edges**	Выравнивание выделенных объектов по нижнему краю самого нижнего объекта
 **Left Edges**	Выравнивание левых границ выделенных объектов по краю самого левого объекта
 **Horizontal Centers**	Выравнивание выделенных объектов по середине между самым левым и самым правым объектом
 **Right Edges**	Выравнивание правых границ выделенных объектов по краю самого правого объекта

Таблица 3. Инструменты управления распределением группы объектов в пространстве, содержащиеся в меню Distribution Ring.

 **Distribution Ring** Инструменты управления распределением группы объектов в пространстве. Выделите объекты, подлежащие пространственному переупорядочиванию, а затем выберете расположение оси выравнивания	
 **Top Edges**	Переупорядочивание выделенных объектов таким образом, чтобы их верхние границы были взаимно равномерно распределены
 **Vertical Centers**	Переупорядочивание выделенных объектов таким образом, чтобы их центры были взаимно равномерно распределены по вертикали
 **Bottom Edges**	Переупорядочивание выделенных объектов таким образом, чтобы их нижние границы были взаимно равномерно распределены
 **Vertical Gap**	Переупорядочивание выделенных объектов таким образом, чтобы вертикальные промежутки между ними были равны
 **Vertical Compress**	Сжатие выделенных объектов таким образом, чтобы не было промежутков между их верхними и нижними границами
 **Left Edges**	Переупорядочивание выделенных объектов таким образом, чтобы их левые границы были взаимно равномерно распределены
 **Horizontal Centers**	Переупорядочивание выделенных объектов таким образом, чтобы их центры были взаимно равномерно распределены по горизонтали

 **Right Edges**	Переупорядочивание выделенных объектов таким образом, чтобы их правые границы были взаимно равномерно распределены
 **Horizontal Gap**	Переупорядочивание выделенных объектов таким образом, чтобы горизонтальные промежутки между ними были равны
 **Horizontal Compress**	Сжатие выделенных объектов таким образом, чтобы не было промежутков между их правыми и левыми границами

Таблица 4. Инструменты управления распределением группы объектов в пространстве, содержащиеся в меню Reorder Ring.

 **Reorder Ring** Инструменты управления взаимным наложением, группировкой и фиксации объектов в поле редактора.	
Group	Группировка выделенных объектов в один
Ungroup	Разгруппировка выделенного объекта на составляющие
Lock	Фиксация пространственного положения объекта
Unlock	Снятие фиксации пространственного положения объекта
Move Forward	Перемещение выделенных объектов в группе на одну позицию вверх
Move Backward	Перемещение выделенных объектов в группе на одну позицию вниз
Move To Front	Перемещение выделенных объектов в группе на верхнюю позицию
Move To Back	Перемещение выделенных объектов в группе на нижнюю позицию

Линейка инструментов окна блок-диаграмм имеет все те же кнопки, что и на лицевой панели, только с дополнением кнопок, требуемых для отладки (debugging) программного кода (рис.25(a)):

1. Кнопка в виде лампочки позволяет запустить программу на выполнение в режиме отладки (режим отладки будет разобран ниже);
2. Нажатие на эту кнопку в процессе отладки позволяет "входить" в структуры, например, в циклы и подпрограммы;
3. Кнопка активизирует пошаговый режим отладки;
4. Кнопка позволяет сделать следующий шаг в пошаговом режиме отладки программы;
5. Кнопка позволяет выйти из процесса отладки (более подробно рассмотрим отладку ниже).

Отладка программы

Наиболее важной частью программирования является создание исправного кода программы. Если графический код виртуального прибора не исправен, то программу запустить не получится, стрелка запуска программы будет сломана и недоступна (рис.26(b)). Первое, что стоит проверить если стрелка запуска сломана – это наличие оборванных проводников, которые не редко являются причиной поломки. Если оборванные проводники данных отсутствуют, то для выяснения причины неисправности кода нужно щёлкнуть правой кнопкой мыши по сломанной стрелке запуска. Перед вами откроется диалоговое окно ***Error List***, которое состоит из ошибок, содержащихся графическом коде в виртуальном приборе (рис.26(c)). В верхней части окна расположено меню ***Items with error***, в котором указаны все открытые виртуальные приборы. В середине находится лист ошибок и предупреждений (***Error sand Warnings***), который соответствует выбранному ВП из меню Items with error. Если выбрать другой виртуальный прибор, то в диалоговом окне будут отображаться ошибки относящиеся к этому ВП. Чтобы увидеть в каком месте графического кода программы расположена

ошибка, нужно дважды кликнуть правой кнопки мыши по названию ошибки. Более детальное описание имеющихся ошибок и предупреждений можно увидеть в нижней части диалогового окна (окно Details).

Однако даже при работающей стрелке запуска программы графический код не всегда работает так, как хотелось бы разработчику программы. На этот случай разработчики средыLabVIEW создали инструменты, которые помогают программисту создать исправно работающий код.

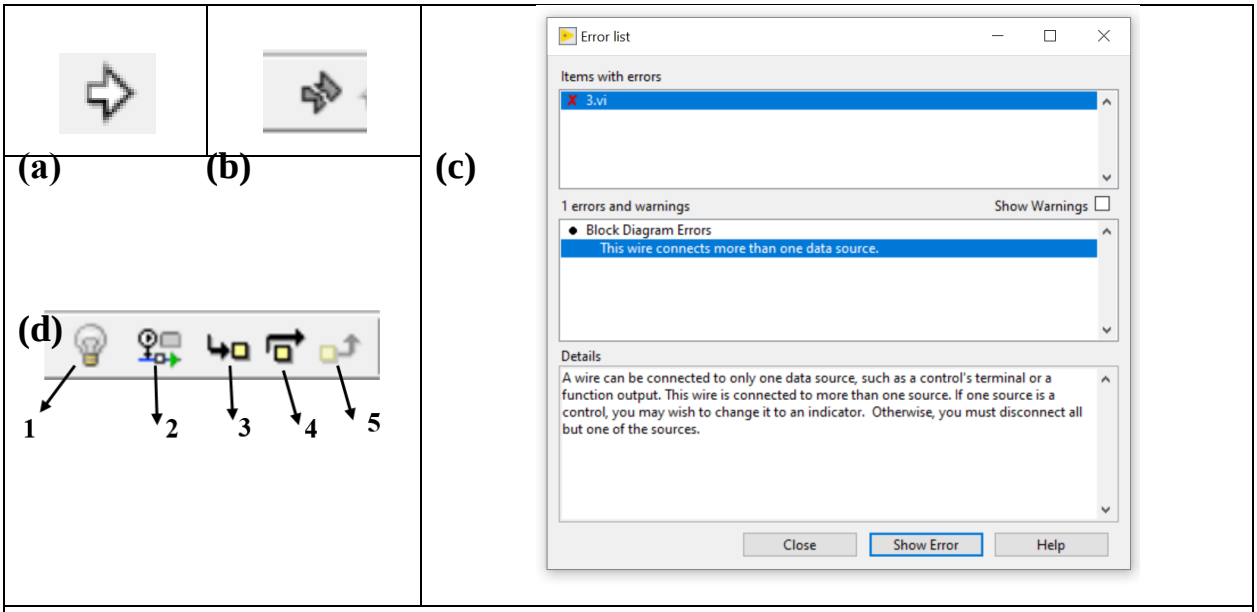


Рисунок 26. a) Кнопка запуска выполнения программного кода. b) Сломанная кнопка запуска выполнения программного кода. c) Вид диалогового окна Error List, состоящего из ошибок в графическом коде. d) Кнопки для редактирования графического кода в режиме отладки.

Процесс редактирования кода программы для его корректной работы называется отладка. Существует два возможных варианта редактирования кода программы:

- Подсветка при выполнении программы;
- Пошаговый режим отладки.

Подсветка при выполнении программы

Рассмотрим процесс отладки на примере написанной нами ранее программы. Первый вариант отладки программы (подсветка выполнения) представляет собой замедленное движение потока данных на блок-диаграмме от одного узла кода к другому с помощью пузырьков, которые перемещаются по проводам. Чтобы войти в режим отладки с подсветкой выполнения нужно нажать на лампочку в верхней части интерфейсной панели (рис.26(d)). После нажатия лампочка примет вид, представленный на рисунке 27(a). Теперь, когда включен режим подсветки выполнения, запустим программу с помощью стрелки и будем наблюдать за тем, как изменится вид графического кода на блок-диаграмме. В начальный момент после запуска программы мы увидим, что на терминалах будут отображаться значения чисел, которые мы ввели на лицевой панели. Далее это значения будут двигаться по проводникам данных до ближайшего узла программного кода. Значения чисел (данные) в режиме отладки обозначаются как пузырьки, которые перемещаются по проводникам данных. Проводники, через которые пузырьки с данными ещё не прошли обозначаются пунктирными линиями чёрного цвета, при этом проводники по которым данные уже проходили имеют вид сплошных линий. Когда данные достигнут первого узла программы на входных терминалах узла появятся пузырьки с этими данными (рис.27(c)). После выполнения этого программного узла данные выйдут из его выходного терминала и продолжат своё движение по проводникам к следующему узлу. При этом на выходном терминале узла, который покинули данные появится значение, которые было получено в результате выполнения этого программного узла (рис.27(d)). Далее данные будут также передвигаться от одного узла к другому до тех пор, пока программный код не будет полностью выполнен. При этом стоит учесть, что выполнение программного кода в процессе отладки происходит в замедленном режиме. При необходимости выполнение программного кода можно приостановить, а потом продолжить с того момента, на котором вы остановились. Для того,

чтобы приостановить выполнение программы нужно нажать на кнопку пауза (рис.27(b)). После нажатия кнопка станет красного цвета и остановит выполнение программного кода, при этом вид блок-диаграммы не изменится, вы всё также будете видеть значения данных на входных и выходных терминалах элементов и узлов программы, а также текущее положение данных в коде программы. Повторное нажатие на кнопку паузы запустит выполнение программного кода с того места, на котором было прервано выполнение программы.

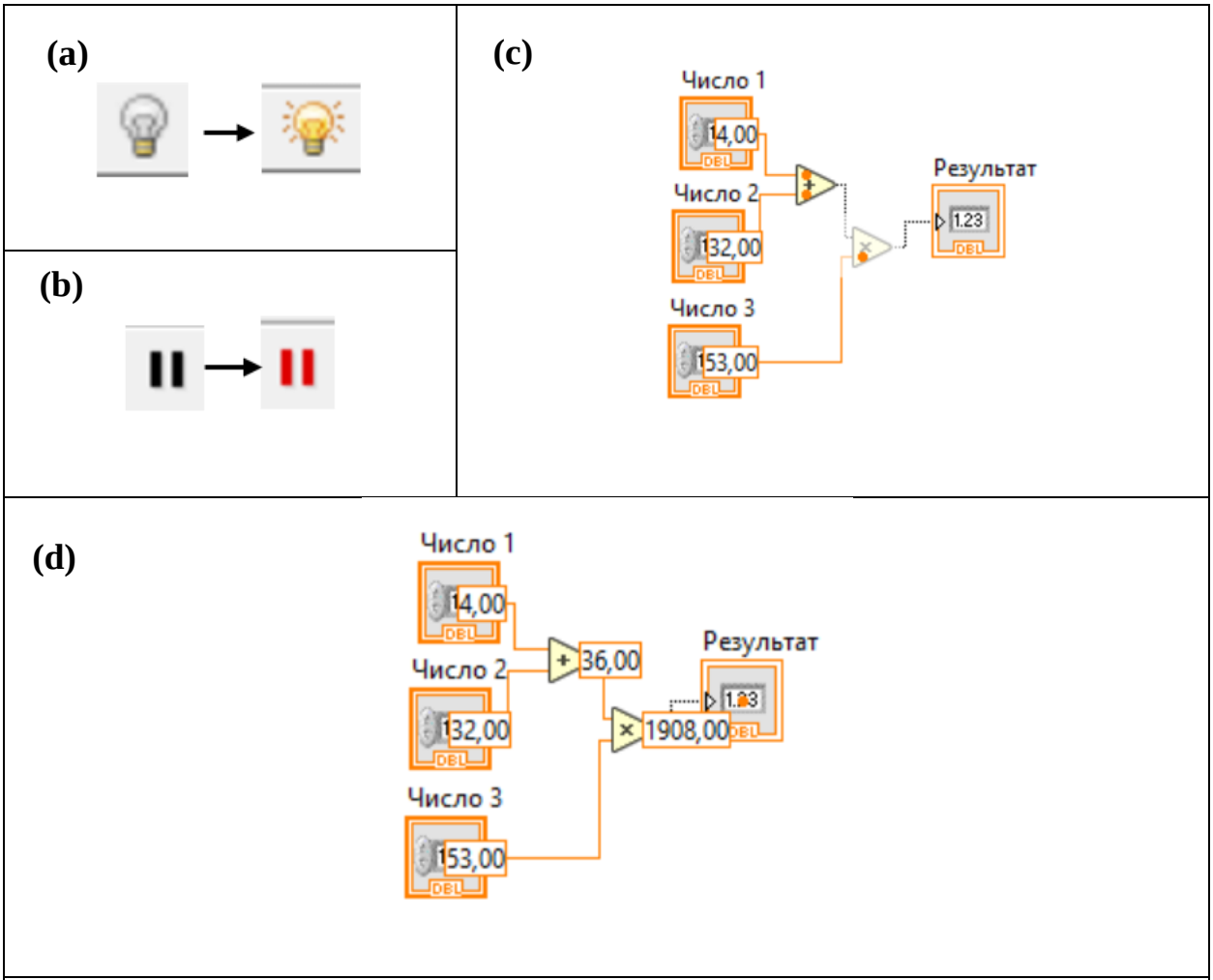


Рисунок 27. a) Изменение вида кнопки в виде лампочки при включенном режиме отладки. b) Вид кнопки пауза для приостановки работы программы в режиме отладки. c) Вид передаваемых по проводникам данных в режиме подсветки при выполнении программы. d). Отображение данных на терминале узла, полученных после прохождения данных через узел.

В процессе отладки в режиме подсветки значение движущихся по проводникам данных можно увидеть только на входных или выходных терминалах, что является не всегда удобным, особенно если данные на терминалах будут изменяться. На этот случай в среде LabVIEW есть инструмент пробник, который позволяет увидеть значения проходящих данных в любом месте программного кода, а не только на терминалах элементов и узлов. Разберёмся как же его использовать. Для начала откроем палитру инструментов и выберем среди них инструмент пробник, который имеет вид, представленный на рисунке 28(a), при этом курсор мыши должен поменять свой вид (рис.28(b)). Теперь мы можем выбрать любое место программного кода и установить туда пробник. Для примера поставим пробник после узла сложения чисел 1 и 2. Для этого нужно поднести пробник к проводнику, идущему из выходного терминала узла суммы. Если пробник поднесён правильно, тогда проводник данных должен начать мигать. Если же проводник не мигает, то следует поводить курсором вблизи проводника пока не будет найдено положение, при котором проводник станет мигать. Когда нужное положение курсора найдено, нажмём на левую кнопку мыши, тем самым установим пробник в выбранном месте (рис. 28(c)). После нажатия перед нами открывается диалоговое окно ***Probe Watch Window*** и место, в котором установлен пробник, отмечается цифрой 1 (рис. 28(d)). Диалоговое окно ***Probe Watch Window*** состоит из двух раздельных окон (рис. 28(e)). Первое окно (окно слева) представляет собой список всех установленных пробников в данном виртуальном приборе. Первая колонка – название пробник, вторая колонка – значения, проходящие через этот пробник. В качестве названия пробника используется числовая нумерация, которая начинается с 1. То есть первый установленный в коде пробник будет иметь название 1, второй установленный пробник – название 2, третий – 3 и так далее. На блок-диаграмме пробники обозначены также, как и в диалоговом окне, то есть пробнику с цифрой 1 на блок-диаграмме, соответствует название [1] Probe из диалогового окна ***Probe Watch Window***. Второе окно

(окно справа – ***Probe Display***) является отображением всех содержащихся значений в пробнике, который выбран в левой части диалогового окна (так как на данный момент у нас установлен только один пробник, то в окне ***Probe Display*** отображаются его значения). Так как после установки пробника на блок-диаграмме программа не была запущенна ни разу, то в окне ***Probe Display*** отсутствуют какие-либо значения, так как данные ещё не проходили через пробник. Запустим программу и понаблюдаем за диалоговым окном. Сразу после запуска выполнения графического кода в диалоговом окне пробника по-прежнему не будет никаких значений до тех пор, пока пузырик с данными не дойдёт до проводника, на котором мы установили пробник. Как только пузырик дойдёт до пробника в диалоговом окне сразу же появится строчка с данными, которые прошли через этот пробник (рис. 28(g)).

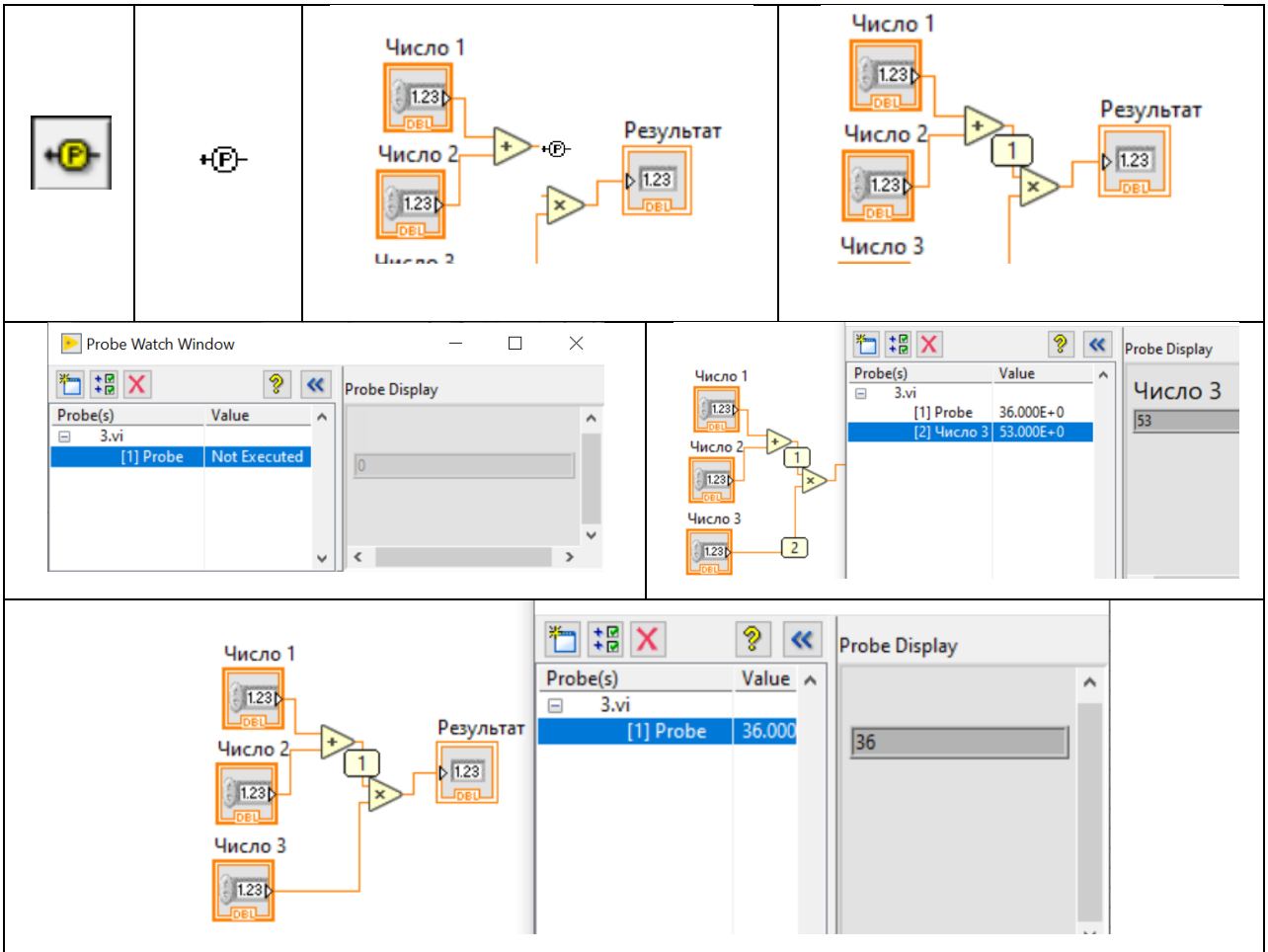


Рисунок 28. a) Вид кнопки пробника в палитре инструментов. b) Вид кнопки пробника в окне блок-диаграммы вместо курсора мыши. c). Установка

пробника на проводнике в программном коде. d) Вид установленного пробника на проводнике в программном коде. e) Вид диалогового окна Probe Watch Window, содержащего информацию о пробниках и данные, прошедшие через них. f) Значение данные проходящих через второй пробник [2] Probe, отображаемое в диалоговом окне Probe Watch Window. g) Значение данные проходящих через второй пробник [1] Probe, отображаемое в диалоговом окне Probe Watch Window

Так как у нас по проводнику проходит только одно значение, то и в диалоговом окне отобразится только одна строчка со значением. Однако по проводникам может проходить и несколько значений сразу (к примеру, массив, который будет рассмотрен ниже), при этом каждое прошедшее значение будет зафиксировано пробником и отображено в диалоговом окне отдельной строчкой. Количество пробников, которое можно установить на блок-диаграмме, не ограниченно. Каждый установленный пробник будет отображён в списке диалогового окна. Для примера установим ещё один пробник на проводнике, который выходит из терминала числа 3 ((рис. 28(f))). В этот же момент в списке диалогового окна появится строчка с названием – [2] Число 3. Чтобы посмотреть содержимое второго пробника нам нужно нажать на его название в левой части диалогового окна, тогда в окне ***Probe Display*** будут отображаться данные второго пробника, а не первого. Чтобы снова увидеть данные первого пробника в окне ***Probe Display***, нужно щёлкнуть по его названию в списке.

Когда в программном коде содержится большое количество данных использование отладки в режиме подсветки становится очень долгим, так как движение данных очень сильно замедленно и занимает много времени. В таком случае целесообразно использовать пробники с отключённым режимом подсветки данных. Иными словами, вы также устанавливаете пробники в программном коде, но не включаете режим подсветки, а просто запускаете программу в обычном режиме. Программный код выполнится, но

при этом в диалоговом окне вы будете видеть все значения данных, которые прошли через ваш пробник и сможете понять в каком месте кода обработка данных становится не корректной, и вы не получаете нужный вам результат. Чем больше пробников вы установите, тем точнее вы сможете определить область с неправильно написанным программным кодом. При этом не стоит устанавливать слишком большое количество пробников, так как попросту можно запутаться в таком большом количестве считываемых пробниками данных.

Пошаговый режим отладки

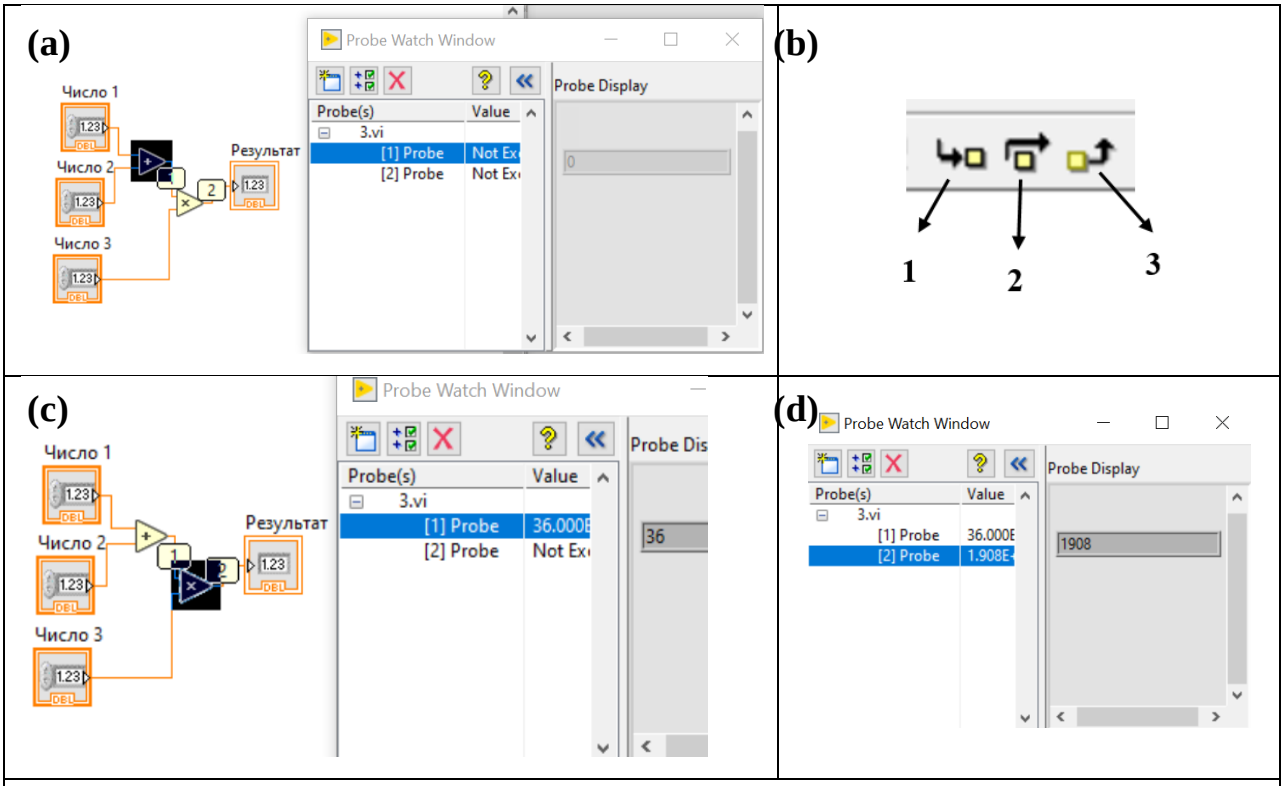


Рисунок 29. a) Вид программного узла в режиме пошаговой отладки. b) Набор кнопок в верхней части окна блок-диаграммы для работы в пошаговом режиме отладки. c). Значение данных в диалоговом окне для пробника [2] Probe после выполнения части программного кода. d) Значение данные проходящих через второй пробник [2] Probe, отображаемое в диалоговом окне Probe Watch Window.

Вторым вариантом отладки программного кода является пошаговый режим. Пошаговое выполнение ВП позволяет последовательно

просматривать результаты выполнения каждого узла программы. При этом стоит отметить, что проходящие по проводникам данные не отображаются на входных и выходных терминалах элементов или узлов, поэтому в пошаговом режиме отладки требуется использование пробников. Установим первый пробник после узла суммы, а второй пробник – после произведения. Чтобы войти в пошаговый режим отладки нужно нажать на кнопку Start Single Stepping (кнопка 1 на рисунке 29(b)), расположенную в верхней части интерфейсной панели. После нажатия мы увидим, что первый узел нашего программного кода (сумма) начал мигать (рис.29(a)). Часть кода, расположенная до первого узла, будет выполнена, при этом дальнейшее выполнение программного кода приостанавливается. На данном участке программного кода мы не устанавливали никаких пробников, поэтому в диалоговом окне ***Probe Watch Window*** не отображается никаких значений. Чтобы запустить выполнение следующей части программного кода нажмём на кнопку Step Over (кнопка 2 на рисунке 29(b)). После нажатия на кнопку программа выполнит этот узел и приостановит своё выполнение на следующем узле (рис.29(c)). Теперь в диалоговом окне мы увидим значение суммы, которое появится в установленном пробнике 1. Далее мы проделываем все те же действия до тех пор, пока мигающая рамка не окружит всю блок-схему (рис.29(d)). Такая мигающая рамка будет означать, что весь код программы выполнился полностью и можно завершать процесс отладки и выходить из пошагового режима. Для этого нужно нажать на кнопку Step Out (кнопка 3 на рисунке 29(b)), которая завершит выполнение программы и переведёт вас в режим редактирования программного кода. То есть отладка в пошаговом режиме заключается в проверки корректности данных, отображающихся в пробниках, при последовательном выполнении каждого узла программного кода.

Также в процессе программирования удобно использовать сочетания клавиш, которые позволяют совершать какие-либо действия без использования контекстного меню (таблица 5)

Таблица 5. Перечень "горячих" клавиш.

Работа с файлами	
Ctrl+N	Создание нового Виртуального Инструмента
Ctrl+O	Открыть файл
Ctrl+W	Закрыть файл
Ctrl+S	Сохранить программу
Ctrl+P	Печать проекта
Ctrl+I	Вывод на экран свойств программы
Ctrl+Q	Выход из LabVIEW
Работа с окнами	
Ctrl+E	Вывод передней панели или окна диаграмм
Ctrl+L	Вывод списка ошибок
Ctrl+T	Делит экран поровну между обоими окнами
Ctrl+/	Раскрывает активный экран до полного размера
Ctrl+H	Выдает на экран контекстнозависимую помощь
Команды редактирования	
Ctrl+V	Вставить объект
Ctrl+Sh+F	Вывести результаты поиска
Ctrl+B	Удалить разрушенные связи
Ctrl+C	Копировать объект
Ctrl+F	Найти терминал, переменную, узел
Ctrl+X	Вырезать объект
Ctrl+Z	Отмена последнего действия
Ctrl+Sh+Z	Повторить последнее действие

Команды выбора режимов	
Ctrl+R	Запустить программу
Ctrl+M	Сменить режим редактирование/выполнение
Ctrl+.	Прекратить выполнение программы
Ctrl+Y	Добавить в историю изменений

Составная арифметика

Не редко арифметические действия совершаются не на двумя, а над большим количеством чисел. Как мы уже увидели выше, самые простые арифметические функции в среде LabView имеют только два входных терминала, то есть позволяют одновременно производить действия только с двумя числами. К примеру, если бы мы захотели сложить три числа и вычесть из их суммы ещё одно число, то нам бы понадобилось написать большой графический код, представленный на рисунке 30(a). Такое большое количество функций для самых простых операций с числами весьма неудобно. Однако в палитре функций есть функция ***Compound Arithmetic*** (составная арифметика), которая позволяет производить одновременно простые арифметические операции с неограниченным количеством чисел. Для начала поместим это функцию в окно блок-диаграммы, чтобы её найти нужно перейти ***Function Pallete → Programming → Numeric→ Compound Arithmetic*** (рис.30(b)). Функция, помещённая на блок-диаграмму будет иметь вид, показанный на рисунке 31(a). Изначально эта функция выполняет операцию сложения и также, как и обычная арифметическая функция, имеет два входных терминала. Чтобы увеличить количество входных терминалов наведите курсор мыши на функцию так, чтобы у неё появились два синих квадратика. Установите курсор мыши на одном из этих квадратиков, курсор мыши должен превратиться в двустороннюю стрелочку (см.рис.31(e)). Зажмите левую кнопку мыши и растягивайте функцию вниз или вверх (зависит от того на какой из квадратиков вы навели курсор мыши) пока не

начнут появляться новые входные терминалы. Чем больше вы растянете функцию, тем больше будет появляться новых входных терминалов.

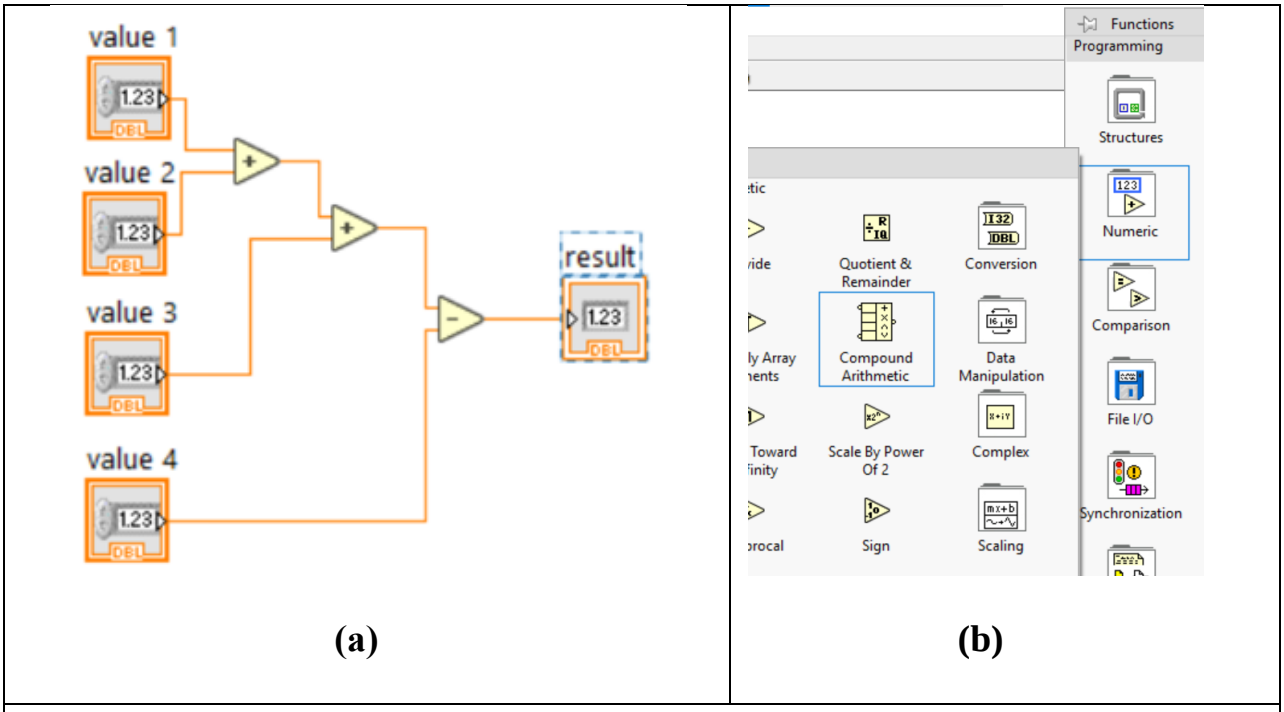


Рисунок 30. a) Вид графического кода для осуществления простых арифметический действий над числами. b) Расположение функции Compound Arithmetic в палитре Function Pallete.

Есть и другой способ увеличения количества входных терминалов, наведите курсор мыши на один из входных терминалов функции так, чтобы курсор превратился в катушку (см.рис.31(b)). В таком положении курсора щёлкните правой кнопкой мыши, перед вами откроется контекстное меню, в котором нужно найти строчку с названием ***Add Input*** (добавить вход). Если щёлкнуть по ней левой кнопкой мыши, то у функции добавится ещё один входной терминал. Чтобы добавить больше одного терминала нужно повторять те же самые действия столько раз, сколько требуется добавить новых входных терминалов. Теперь разберёмся как изменять арифметическое действие функции. Наведите курсор мыши на функцию так, чтобы он превратился в указательный палец, после чего щёлкните левой кнопкой мыши. Перед вами откроется контекстное меню, состоящее из двух пунктов: Invert и Change Mode. Если навести курсор мыши на пункт Change

Mode, то появится ещё одно контекстное меню, которое и позволяет изменять арифметическое действие данной функции. Заметим, что три последние строчки относятся не к арифметическим действиям, а к булевой логике. То есть функция ***Compound Arithmetic*** позволяет совершать над несколькими числами сразу не только арифметические операции, но и логические операции. Мы рассмотри только работу с арифметическими операциями, так как работа с логическими операциями происходит по тому же принципу. Как мы видим в контекстном меню содержится только два простейших арифметических действия: сумма (Add) и умножение (Multiply). Однако арифметического действия вычитания и деления в контекстном меню нет. В данной функции получение операции вычитания и деления происходит за счёт инвертирования операций суммы и умножения. То есть инвертированная сумма – это вычитание, а инвертированное умножение – это деление. Чтобы инвертировать функцию нужно навести курсор мыши на требуемый входной терминал (курсор должен стать катушкой) и щелчком правой кнопки мыши открыть контекстное меню, в котором нужно выбрать строчку с названием ***Invert*** (рис. 30(h)). Тогда на инвертированном входном терминале появится кружочек, который будет означать, что число, подключённое к данному терминалу будет вычитаться, а не прибавляться. Таким образом можно выборочно вычитать числа, подключенные к входным терминалам. Однако если требуется заменить на вычитание всю функцию полностью, то не обязательно инвертировать каждый входной терминал. На этот случай требуется открыть контекстное меню, представленное на рисунке 31(c), и нажать на первую строчку ***Invert*** (рис.31(d)). После нажатия на выходном терминале функции появится кружочек, который означает, что операция суммы полностью поменялась на операцию вычитания. Аналогично происходит получение деления с помощью инвертирования умножения. Тогда, если заменить все арифметические функции в начальном графическом коде, на функции ***Compound Arithmetic***, мы получим очень компактный и удобный графический код, представленный на рисунке 31(i).

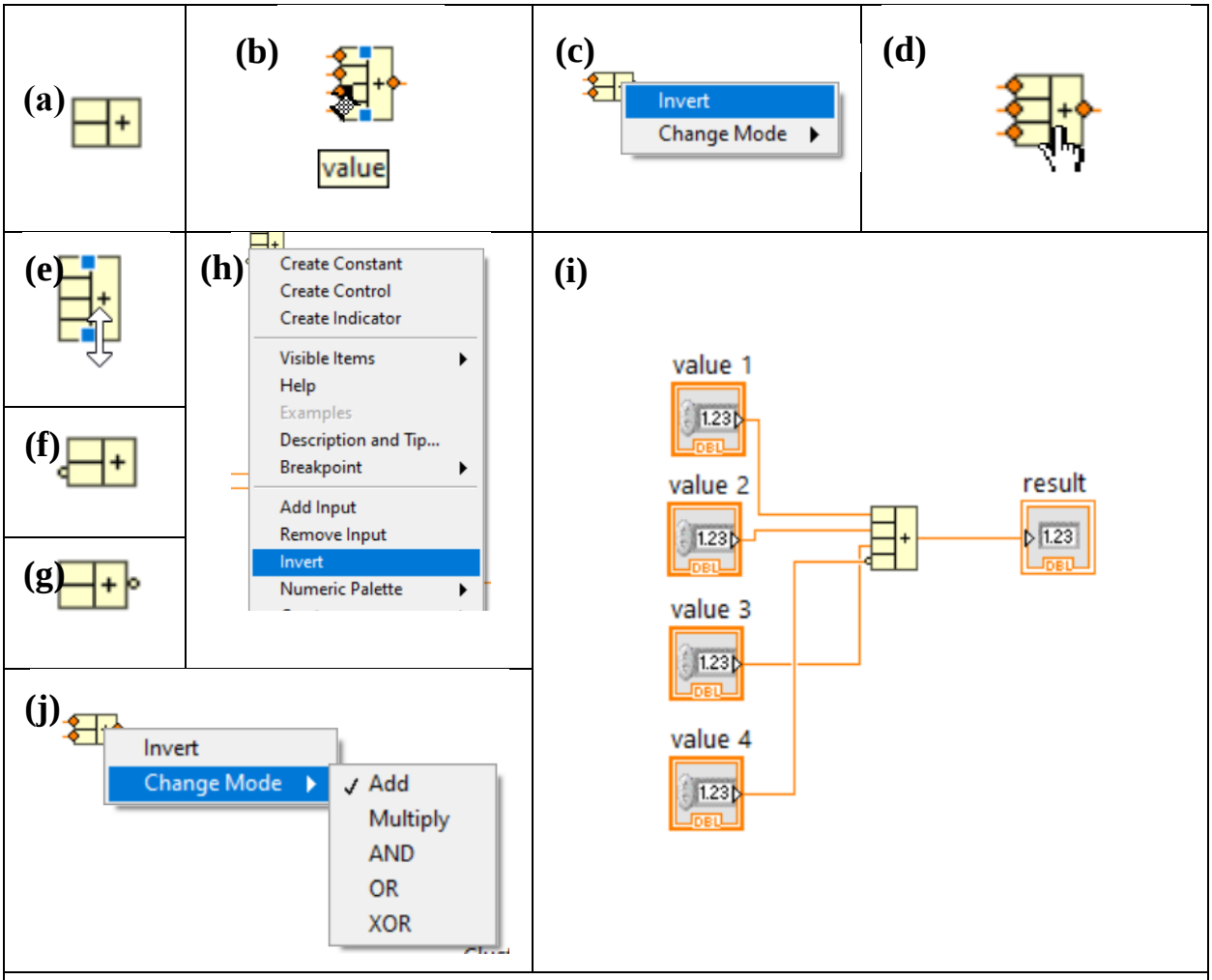
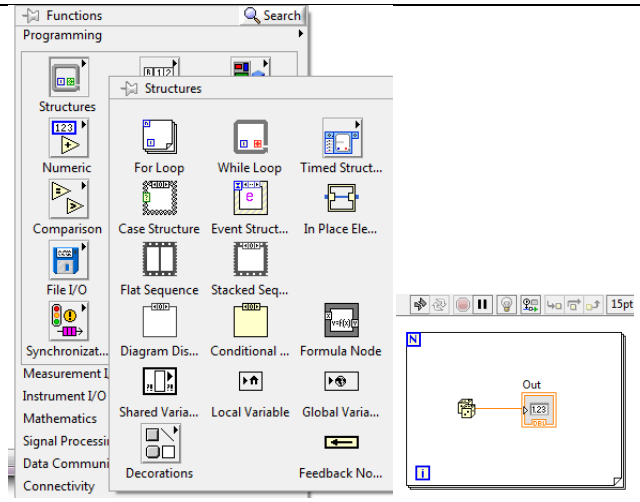


Рисунок 31. a) Вид терминала функции Compound Arithmetic. b) Вид входных и выходных терминалов функции Compound Arithmetic. c) Контекстное меню для инвертирования выходного терминала. d) Область открытия контекстного меню. e) Изменение количества входных терминалов. f) Инвертированный входной терминал. g) Инвертированный выходной терминал. h) Инвертирование входного терминала с помощью контекстного меню. i) Графический код для осуществления арифметических действий с помощью Compound Arithmetic. j) Контекстное меню для изменения арифметического действия функции Compound Arithmetic.

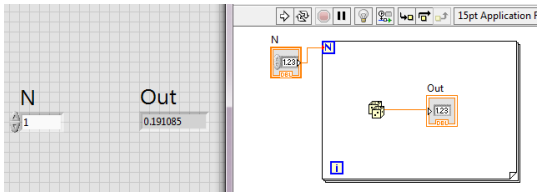
Циклы

Любые программы в любом языке программирования работают за счет выполнения повторяющихся действий, что организуется с использованием

циклов. Функции циклов находятся в пункте Structures основной палитры в кодовой панели (рисунок 32(a)). В пункте Structures находятся два основных типа циклов For Loop и While Loop. Поместим в кодовую панель генератор случайных чисел соединенный с числовым индикатором. Выберем в пункте Structures ркодовой палитры цикл For и обведем курсором генератор случайных чисел и индикатор (рисунок 32(b)). Обратим внимание, что стрелка на кнопке запуска программы Run отображается разорванной (рисунок 32(b)). Такое изображение стрелки появляется в случае, когда в кодовой панели есть несоединенные или соединенные неправильно элементы.



a) b)



c)

Рисунок 32. a) Пункт Structures раздела Programming палитры в кодовой панели. b) Цикл For без указания числа итераций цикла. b) Цикл For с заданием числа итераций с помощью Numeric Control.

Будем задавать число итераций цикла с использованием Numeric Control на передней панели. Добавим Numeric Control на переднюю панель и соединим в кодовой панели значок Numeric Control с входным терминалом N

цикла For. Стрелка на кнопке запуска программы Run станет целой и можно запустить программу. Если задать число итераций цикла равное, например, 10 и запустить программу, то 10 итераций выполнятся очень быстро и в окне Numeric Indicator увидим только последнее сгенерированное случайное число. Чтобы увидеть генерируемое на каждом шаге цикла случайное число введем задержку между итерациями цикла. В основной палитре в кодовой панели найдем пункт Timing и в нем выберем функцию Wait (ms). Поместим функцию Wait внутрь цикла For (рисунок 33). Функция Wait имеет входной терминал на которое нужно подать целочисленное число, определяющее величину задержки в миллисекундах. Значите задержки зададим числовой константой. Зададим задержку равную 500 миллисекунд. Теперь после задания числа шагов цикла и запуска программы можно наблюдать появление случайных чисел в окне Numeric Indicator.

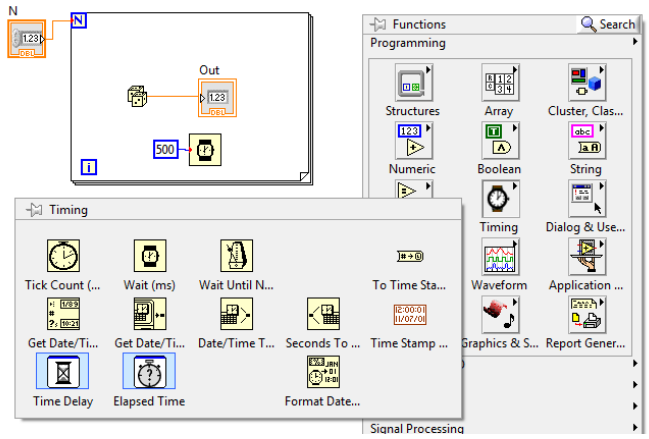


Рисунок 33. Вкладка Timing и функция Wait на палитре в кодовой панели и помещенная в цикл For.

Цикл For выполнится столько раз, сколько было задано итераций цикла во входном терминале N. Возможна организация выхода из цикла For до выполнения всех заданных итераций. Если навести курсор на рамку цикла и нажать правую кнопку мыши, то появится контекстное меню с пунктом Conditional Terminal (терминал условия, рисунок 34(а)). После выбора этого пункта внутри цикла появится красный кружок в зеленом квадрате. Этот элемент имеет входной терминал слева. Если навести курсор на Conditional

Terminal и нажать правую кнопку мыши, то появится контекстное меню с режимом работы этого элемента (рисунок 34(b)). По умолчанию он находится в режиме Stop if True т.е. по приходу во входной терминал элемента логической константы True цикл останавливается. Выберем в основной палитре на первой панели пункт Boolean (логические действия) и в этом пункте выберем кнопку Stop Button. Поместим эту кнопку на переднюю панель, а в кодовой панели поместим ее внутрь цикла For и соединим с терминалом элемента Conditional Terminal (рисунок 34(c)).

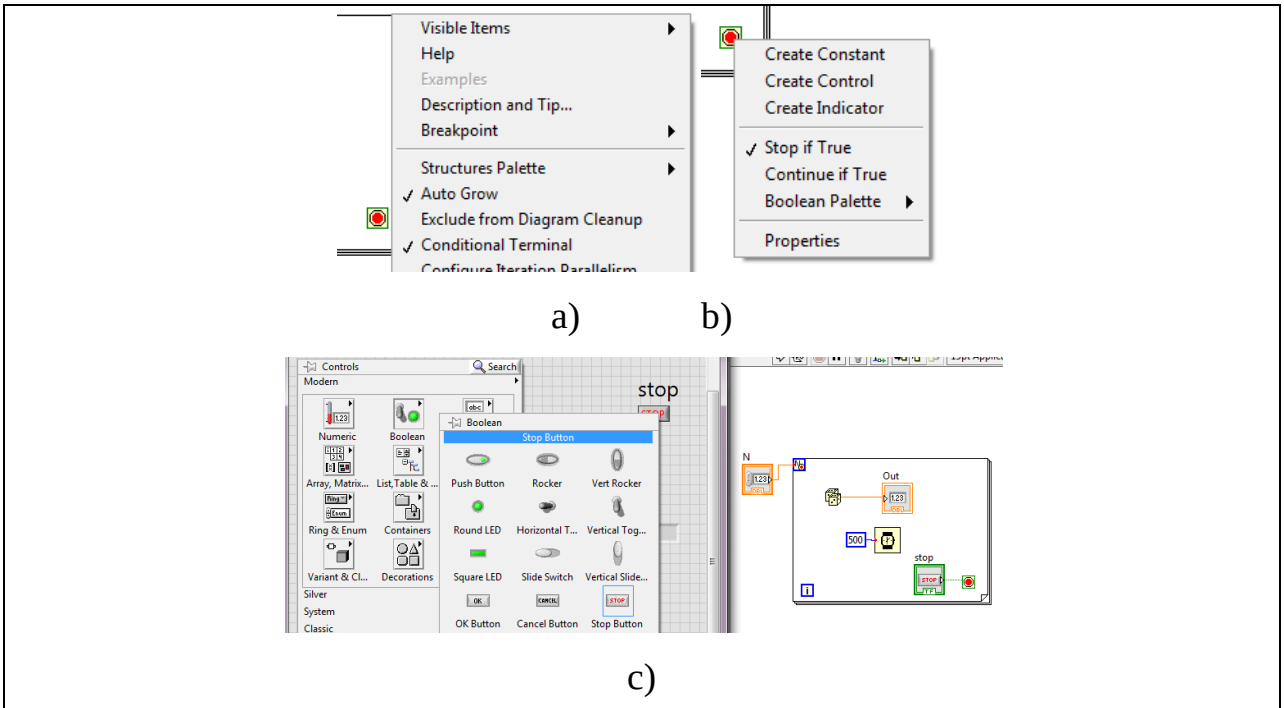
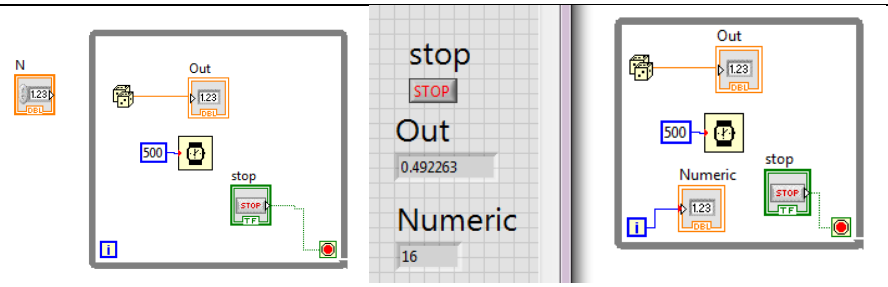


Рисунок 34. a) Контекстное меню цикла For. b) Контекстное меню элемента Conditional Terminal с выбором режима его работы. c) Элементы работы с данными логического типа на передней панели, а также кнопка Stop Button на передней панели и в цикле For.

Теперь после запуска программы цикл For может также быть остановлен по логическому условию (в частности по нажатию кнопки Stop). Обратим внимание, что элементы, работающие с логическим типом данных, имеют зеленый цвет. Удалим цикл For (например, выбрав пункт Remove For Loop в контекстном меню для цикла For), в пункте Structures палитры в кодовой панели (рисунок 32(a)) выберем цикл While Loop и обведем им

вокруг наших элементов. Цикл For сразу содержит элемент Conditional Terminal, поскольку по выполняется до достижения некоторого условия проверки логического значения True или False (рисунок 35(a)). По умолчанию цикл For выполняется пока на Conditional Terminal не придет логическое значение True. Соединим кнопку Stop с Conditional Terminal, а у элемента Numeric Control уберем неиспользуемый проводник (если показан разорванный проводник, то программа не запустится) или удалим элемент.



a) b)

Рисунок 35. a) Цикл For. b) Индикация текущей итерации цикла.

После запуска программы в цикле For будут генерироваться случайные числа до нажатия кнопки Stop. Циклы For и While внутри себя имеют целочисленный элемент i (Iteration terminal) с выходным терминалом. Этот терминал выдает номер текущей итерации цикла (на первой итерации выдает значение 0). На рисунке 35(b) представлена программа с выводом текущей итерации цикла в числовой индикатор. Поскольку цикл While останавливается, когда Conditional Terminal приходит определенная логическая переменная (по умолчанию True), то можно организовать выход из цикла не только по нажатию кнопки Stop, но и по выполнению какого-нибудь условия.

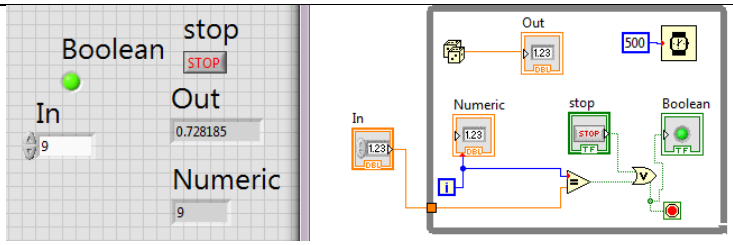


Рисунок 36. Цикл While с остановкой по нажатию кнопки Stop или по достижении заданного числа итераций.

Разместим на передней панели Numeric Control с помощью которого будем задавать максимальное число шагов цикла While (как в цикле For). В цикл For поместим элемент сравнения Equal?, находящийся в подразделе Comporation раздела Programming основной палитры в кодовой панели. Элемент Equal? имеет два входных терминала для числовых данных и один выходной для логических данных (рисунок 36). Если числа на входных терминалах элемента Equal? разные, то на выходном терминале присутствует логическое значение False, а если на оба входных терминала придут равные числа, то на выходном терминале появится значение True. Для того, чтобы сохранилась возможность остановить цикл While по нажатию кнопки Stop, нужно ввести логическую функцию ИЛИ. Эта функция Or (как и другие логические функции находится) в подразделе Boolean раздела Programming палитры в кодовой панели. Функция Or имеет два входных логических терминала и один выходной логический терминал. На выходном терминале присутствует значение False, если оба логических значения на входных терминалах являются False. Если на входных терминалах одно или оба значения True, то на выходном терминале появляется значение True. Соединим элементы как показано на рисунке 14. Зададим в Numeric Control на передней панели максимально число итераций цикла и запустим программу. Сейчас остановки цикла возможна как по нажатию кнопки Stop, которая передаст в функцию Or значение True, которое далее будет передано в Conditional Terminal, так и по достижению заданного числа итераций цикла. Происходит сравнение номера текущей итерации цикла (берется из Iteration

terminal) с заранее заданным числом (берется из Numeric Control). По достижению значения числа из Iteration terminal равного числу из Numeric Control, функция Equal? выдает значение True, которое через функцию Or передается в Conditional Terminal. На передней панели логические функции используются в кнопках, переключателях и индикаторах (лампочках). Разместим на передней панели индикатор-светодиод (Round Led в пункте Boolean в основной палитре на передней панели) и соединим его с выходом функции Or (рисунок 36). Светодиод загорится, когда на него придет значение True.

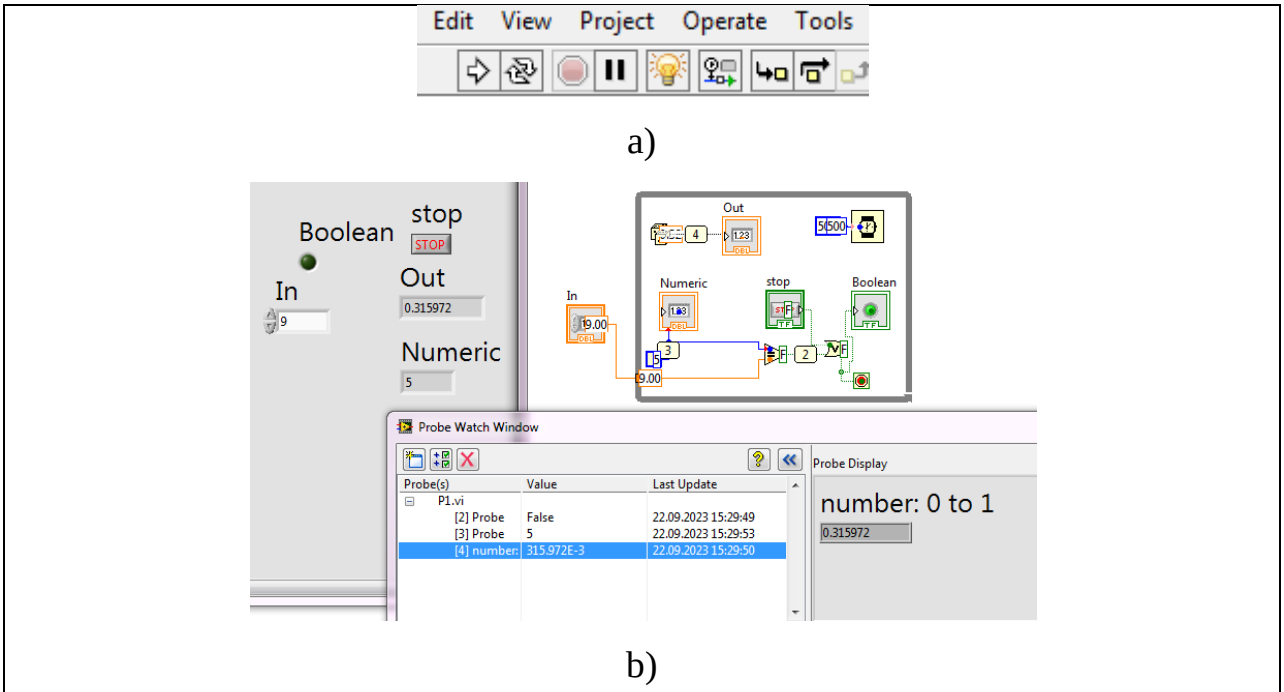


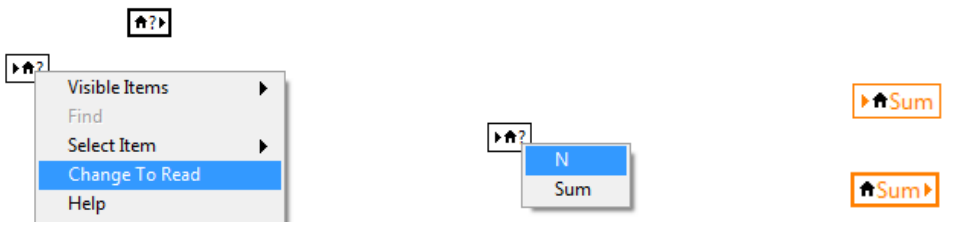
Рисунок 37. Включение режима отладки в кодовой панели (a) и использование пробника (b).

На основной (верхней) инструментальной панели кодового окна есть кнопка включения режима отладки программы, представленная в виде лампочки. По нажатию этой кнопки лампочка загорается. Теперь после запуска программы она будет выполняться медленно, что позволяет отследить значение передаваемых данных. Для того чтобы узнать значения данных в данном проводнике нужно нажать на проводник курсором в виде кружка с буквой P внутри (появляется при наведении курсора на проводник

во время работы программы в режиме отладки). После нажатия этим пробником на проводник появляется окно Probe Watch, в котором отображаются текущие данные. Можно нажать на несколько проводников и в окне Probe Watch отобразится список пробников и значение в выбранном пробнике (рисунок 37). Отображаются значения переменных любого типа (числовые, строковые, логические).

Локальные переменные

Введение локальной переменной (Local Variable) является аналогом объявления переменной в других языках программирования, но без указания типа данных (тип данных определиться автоматически). В предыдущих рассмотренных примерах передача данных осуществлялось соединением элементов в программном коде проводниками. Введение локальных переменных позволяет более гибко обращаться с данными и передавать их в разные участи программы без необходимости соединения функций проводниками. Заготовка для локальной переменной находятся в пункте Structures в палитре кодовой панели. По умолчанию заготовка локальной переменной выглядит в виде прямоугольника без цвета (т.к. тип данных пока не определен) с входным терминалом (рисунок 38(a)). Сделаем программу для получения суммы ряда натуральных чисел от 1 до вводимого числа N ($S = \sum_{i=1}^{N} i$). Разместим на передней панели элемент Numeric Control для ввода числа N (на передней панели переименуем элемент в N). Разместим элемент Numeric Indicator для отображения полученной суммы S (переименуем элемент в Sum). Разместим в кодовой панели локальную переменную и нажнем на нее мышкой (рисунок 38(b)). В появившемся меню выберем Sum, тем самым выполним привязку заготовки локальной переменной к введенной переменной Sum (рисунок 38(c)). Разместим в кодовой панели еще одну заготовку локальной переменной, переключим ее в режим считывания (рисунок 38(a)) и свяжем ее с переменной Sum.



a) b) c)

Рисунок 38. a) Заготовки локальной переменной Local Variable не связанные с какими-либо источниками данных, находящиеся в режиме записи (слева) и считывания (справа). b) Связь заготовки локальной переменной с введенной переменной. c) Локальная переменная в режиме записи значения и считывания текущего значения.

В кодовой панели сделаем программу с использованием цикла For, представленную на рисунке 39(a). Обратим внимание, что используются две локальные переменные в режиме записи, связанные с переменной Sum. Для создания дубликата локальной переменной нужно поместить новую заготовку Local Variable из пункта Structures и настроить ее нужным образом. Если выделить в кодовой панели имеющуюся локальную переменную, нажать Ctrl+C, а потом Ctrl+V, то создаться не дубликат локальной переменной, а вторая независимая локальная переменная со вторым связанным элементом на передней панели. Сначала запишем первоначальное значение суммы *S* в локальную переменную. Обычно первоначальное значение равно нулю. Если не ввести обнуление суммы, то при повторном запуске программы числа будут суммироваться с предыдущим полученным значением переменной Sum. После присвоения переменной Sum значения равного нулю, на первом шаге внутри цикла к этому значению будет прибавлено значение 1, которое берется из счетчика итераций цикла (используется элемент увеличения значения на единицу из пункта Numeric, т.к. счетчик итераций начинается с 0). После суммирования полученная сумма записывается в ту же локальную переменную Sum. На следующем шаге цикла будет использоваться предыдущее значение переменной Sum.

При этом на передней панели будет отображаться текущее значение переменной Sum.

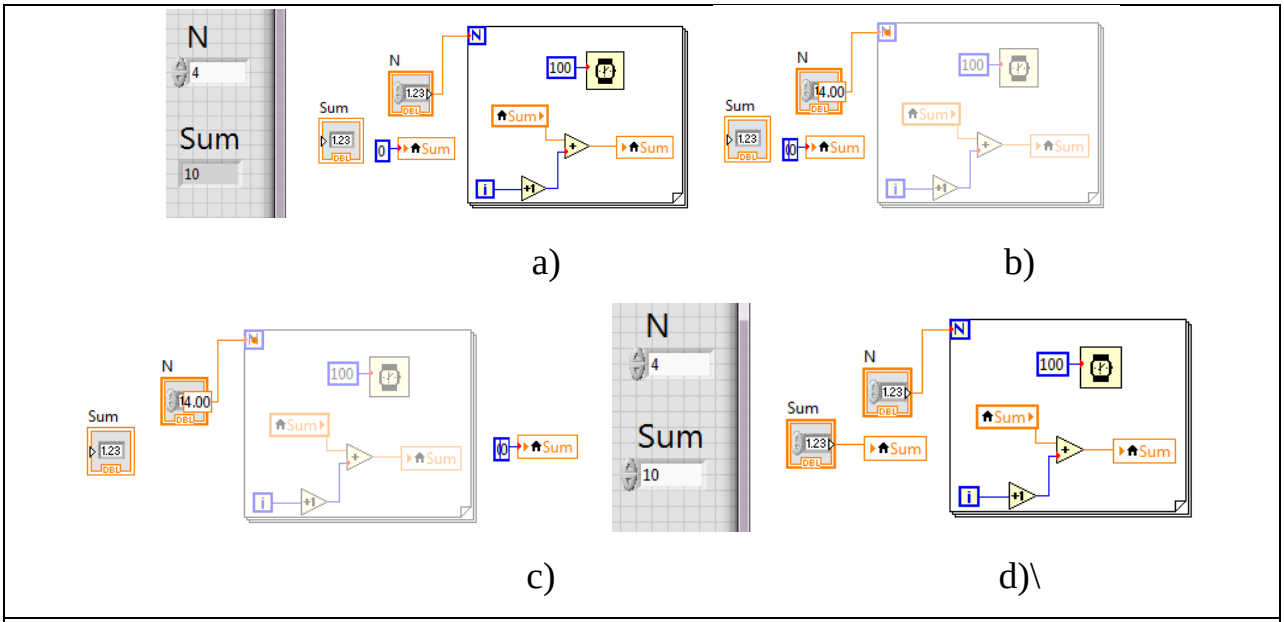


Рисунок 39. a) Программа для суммирования ряда натуральных чисел. b) Программа в режиме отладки при расположении элементов обнуления суммы до цикла. c) Программа в режиме отладки при расположении элементов обнуления суммы после цикла. Передача значений локальной переменной в Numeric Control.

В LabView поток данных идет слева направо, однако расположение элементов на кодовой панели не задает строгую последовательность их выполнения. Для однозначного задания последовательности выполнения элементов программы необходимо использовать структуры Sequence, о чем будет изложено позднее. В LabView все элементы программного кода, расположенные в кодовой панели, в общем случае выполняются параллельно. Запустим рассматриваемый пример по получению суммы в режиме отладки (лампочка на основной инструментальной панели в кодовом окне горит, Рисунок 37(a)). Посмотрим на поток данных. Где бы мы не расположили элементы по записи значения 0 в переменную Sum (до цикла или после цикла), это действие выполнится быстрее чем цикл сделает первую итерацию (рисунок 39(b) и (c)). Это связано с тем, что на запись числовой

константы 0 в переменную Sum требуется меньше вычислительных ресурсов чем на начало работы цикла. Если бы действия с циклом были бы быстрее, то могло случиться так, что на первой итерации цикла произошла запись в переменную Sum, а потом произошло бы ее обнуление. Это нарушило бы логику работы программы. Для предотвращения неоднозначностей в последовательности действий вводится последовательность Sequence. Также обратим внимание на то, что значение суммы в Numeric Indicator (названный нами Sum) обновляется на каждом шаге цикла, хотя явного подводящего проводника к Numeric Indicator нет. Обновленные значения записываются в локальную переменную Sum, которая связана с Numeric Indicator. Обновление значения локальной переменной может быть передано не только в Numeric Indicator, а также в Numeric Control. Заменим на передней панели элемент отображения Numeric Indicator (Sum), на элемент ввода Numeric Control (Sum). После запуска программы значение в окне элемента Numeric Control будет обновляться вслед за перезаписыванием значения в локальную переменную Sum. Таким образом возможна передача данных из локальной переменной в не только в элемент отображения, но и в элемент управления на передней панели.

Вложенные циклы

Во многих случаях требуется выполнение периодических действий внутри каждой итерации цикла. Применительно к взаимодействиям с приборами это может быть набор в массив определенного количества измеряемого параметра при задаваемом значении изменяемой величины. Например, при измерении вольт-амперной характеристики требуется задавать значение приложенного напряжения (с определенным шагом по величине напряжения и времени между значениями напряжения) и несколько раз измерять величину тока при текущем значении заданного напряжения (обычно для последующего усреднения измеренной величины). Таким образом, нужен внешний цикл, задающий шаг по напряжению, и вложенный

цикл, несколько раз считывающий измеренное значение тока. Для иллюстрации вложенных циклов рассмотрим задачу по суммированию сумм случайных чисел - сложим N раз сумму N случайных чисел. Введем Numeric Control задающий число N, Numeric Indicator (назовем Sum Random), отображающий сумму N случайных чисел и Numeric Indicator (назовем Sum Total), отображающий общую сумму. Сделаем программу, представленную на рисунке рисунок 40(a). Сначала присваиваем нулевое значение переменным Sum Random и Sum Total. Потом во вложенном цикле For суммируем N раз случайные числа и повторяем это N раз во внешнем цикле.

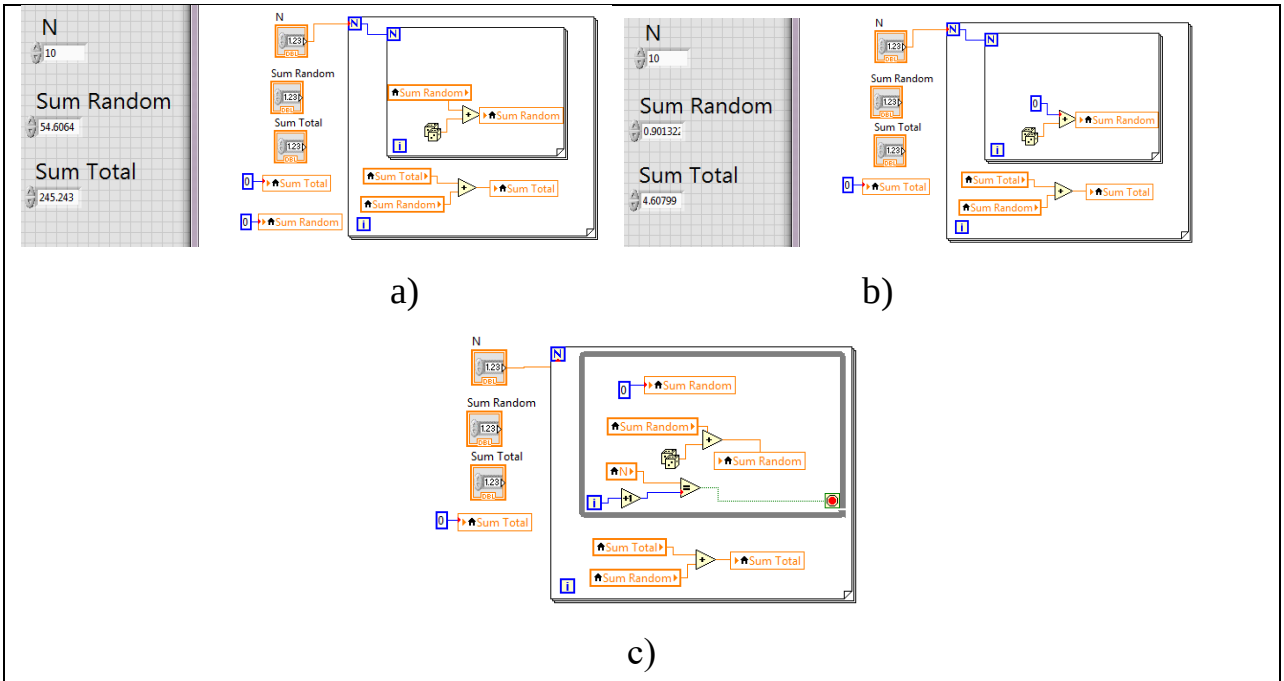
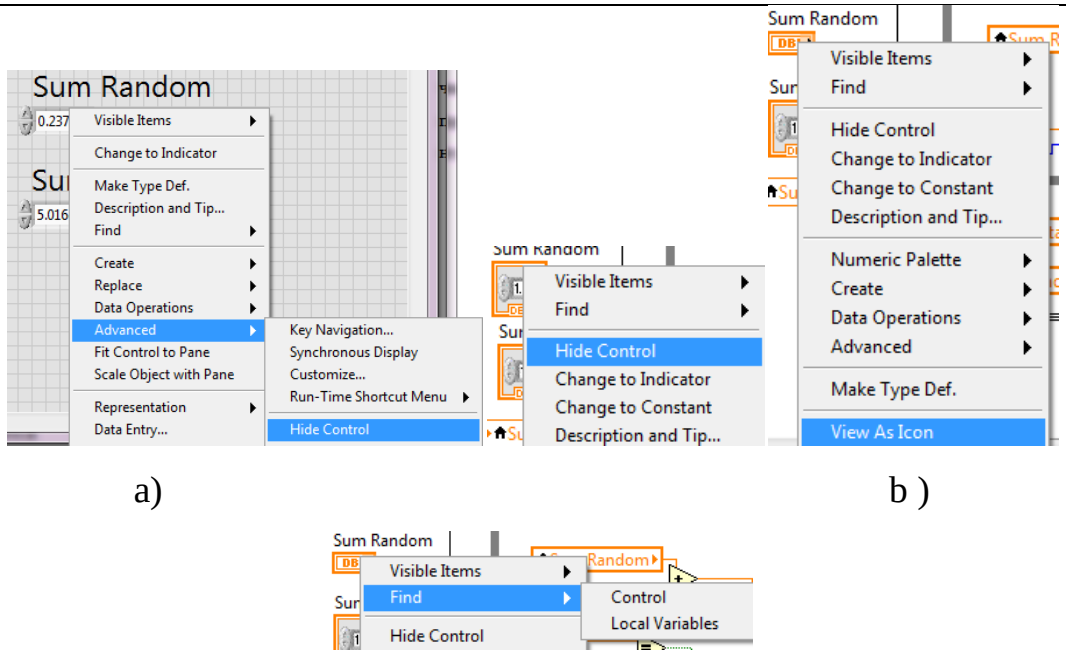


Рисунок 40. a) Неправильная программа по сложению N раз суммы N случайных чисел. b) Правильная программа. c) Использование вложенных циклов разного типа.

Вариант программы, представленный на рисунок 40(a), выдаст неправильный (завышенный результат). Ошибка связана с тем, что локальной переменной Sum Random мы присвоили нулевое значение до запуска циклов и после первой итерации внешнего цикла эта переменная стала равна сумме N случайных числе. На второй итерации цикла случайные числа добавляются к текущему (не нулевому) значению локальной

переменной Sum Random. Поэтому Переменную Sum Random надо обнулять при каждом запуске вложенного цикла или складывать с нулем случайные числа во внутреннем цикле (рисунок 40(b)). Очевидно, что можно делать вложенные циклы разного типа. На рисунке Рисунок 40(c) представлен вариант программы с циклом While, вложенным в цикл For.

Если отображение введенной переменной на передней панели в виде Numeric Indicator и Numeric Control не обязательно, то можно скрыть эти элементы на передней панели. Это можно сделать выбрав в контекстном меню нужного элемента пункт Advanced -> Hide Control на передней панели или Hide Control в кодовой панели (рисунок 41(a) и (b)). После этого с передней панели элемент пропадет. Для возврата отображения на передней панели нужно будет выбрать пункт Show Control в контекстном меню элемента в кодовой панели. В кодовой панели элементы Numeric Indicator и Numeric Control могут отображаться в компактном виде в виде небольших прямоугольников. Для этого нужно снять галочку в пункте View As Icon в контекстном меню элемента в кодовой панели. В контекстном меню элементов на передней панели и в кодовой панели, с которыми связаны локальные переменные, есть пункт Find в котором можно команду по подсвечиванию связанных элементов интерфейса (Indicator или Control) на передней панели, локальных переменных и элементов (Terminal) в кодовой панели.



c)

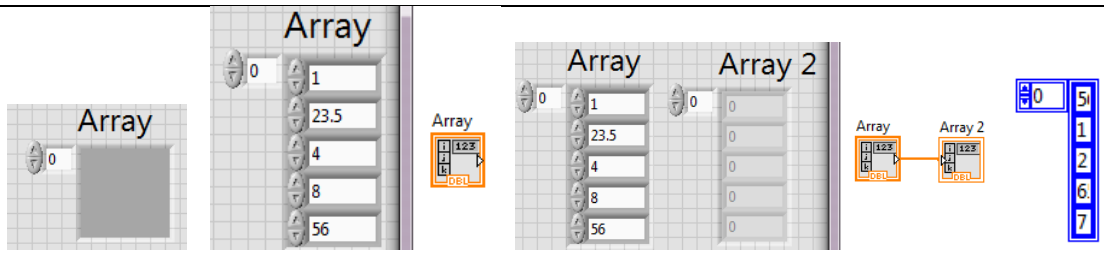
Рисунок 41. a) Скрытие и отображение элементов интерфейса на передней панели. b) Отображение в кодовой панели элементов интерфейса в компактном виде. b) Отображение связи переменной с локальными переменными в кодовой панели.

Массивы

В большинстве программ, в том числе программ по работе с приборами и установками, необходима обработка и сохранение массива данных. Например, в программе по регистрации ВАХ необходимо не только задавать приложенное напряжение и измерять ток, но и сохранять массив значений приложенного напряжения и соответствующего тока. Для примера с ВАХ нужно сохранять двумерный массив - матрицу из двух колонок (напряжение и ток) и N строк (или матрицу из N строк и двух колонок). Наиболее простым массивом является одномерный массив - набор данных одного типа (числового, строкового или логического).

Создадим одномерный массив данных числового типа. В пункте Array, Matrix & Cluster (в новых версиях LabView пункт Data Containers) палитры на лицевой панели выберем элемент Array и поместим на переднюю панель

(рисунок 42(a)). Это заготовка для массива любого типа и любой размерности. Сделаем на основе этой заготовки элемент управления по вводу числовых данных в массив. Для этого выберем на передней панели Numeric Control и поместим в заготовку массива, таким образом создадим элемент управления по вводу чисел в массив. В текущем виде массив одномерный, на что указывает одно квадратное окошко по отображению номера элемента массива. Если навести на правую часть элемента, то появятся восемь синих квадратов. Потянув за крайние квадраты вправо или вниз, можно увеличить число одновременно отображаемых элементов массива. Сделаем массив на пять элементов и введем любые числа в массив (рисунок 42(b)). В кодовой панели созданный массив отобразится в виде квадрата (который через пункт View As Icon можно свернуть до прямоугольника в компактном виде). Создадим на передней панели еще одну заготовку массива и поместим в не Numeric Indicator.



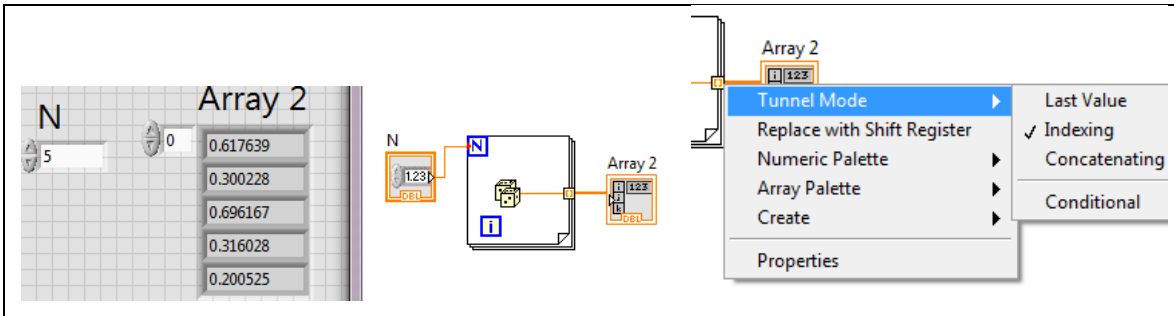
a) b) c) d)

Рисунок 42. a) Заготовка массива на передней панели. b) Элемент управления по вводу чисел в одномерный массив из пяти элементов и отображение этого элемента в кодовой панели. c) Передача массива из элемента ввода данных в массив в элемент отображения массива. d) Массив-константа в кодовой панели.

Тем самым создадим на передней панели элемент отображения массива. Соединим эти элементы в кодовой панели и запустим программу — данные из массива ввода появятся в массиве отображения (рисунок 42(c)). Массив можно создать в кодовой панели, если выбрать в пункте Array палитры в кодовой панели элемент Array Constant. Это будет массив-

константа. Поместив в этот элемент числовую константу создадим массив-константу числового типа (рисунок 42(d)).

Заполним массив с использованием цикла. Самый простой способ — это заполнение с использованием накопления (индексации) на границе массива. Поместим в цикл For генератор случайных чисел. Элемент отображения массива (массив-индикатор) разместим вне цикла и соединим генератор случайных чисел с массив-индикатором (рисунок 43(a)). Обратим внимание что тонкий проводник от генератора случайных чисел на правой границе цикла проходит через квадрат с желтым не заполненным квадратом внутри и вне цикла становится толстым (что соответствует проводнику данных массива). Квадрат на границе цикла называется тоннелем (Tunnel) и по умолчанию находится в режиме индексирования, т.е. накопления приходящих на границу массива данных. На границу массива на каждой итерации приходит число, а за все итерации накапливается массив. После запуска программы сформируется массив из N случайных чисел. Чтобы выключить индексацию на границе массива нужно навести курсор на туннель и нажать правую кнопку мыши. В контекстном меню выбрать Tunnel Mode -> Last Value. После этого из массива будет выходить число, сформированное на последней итерации цикла, а туннель примет вид закрашенного желтого квадрата.



a) b)

Рисунок 43. a) Создание массива случайных чисел путем индексации элементов массива на границе цикла. b) Выбор режима работы туннеля на границе цикла.

Заполнение массива можно проводить на каждой итерации цикла. Это в дальнейшем позволить визуализировать процесс изменения величины — построить график, отображающий изменения величин по ходу выполнения программы. Поместим внутрь цикла функцию Build Array из пункта Array в палитре кодовой панели. Изначально функция Build Array имеет один входной терминал и один выходной. Наведем курсор на функцию Build Array. Курсор изменит свой вид на вертикальную стрелку с двумя наконечниками. Нажмем левую кнопку мыши и потянем курсор вниз. По мере движения курсора вниз на функции Build Array будут добавляться входные терминалы. Нам достаточно двух входных терминалов (рисунок 44(а)). Создадим на передней панели пустой числовой массив (назовем Y) и растянем его на пять элементов. В кодовой панели создадим массив-константу с нулевым элементом и присоединим его к созданной локальной переменной в режиме записи для массива Y (это необходимо для того, чтобы при каждом запуске программы массив становился пустым). На верхний терминал функции Build Array подадим данные из массива Y с использованием локальной переменной в режиме считывания. На нижний терминал функции Build Array подадим случайное число, а выходной терминал Build Array соединим с массивом Y с использованием локальной переменной в режиме записи. При каждой итерации цикла в конец массива Y будет добавляться случайное число. Заполняемый массив будет отображаться на передней панели. Для вывода массива из цикла, например, для его использования в другом цикле, можно воспользоваться локальной переменной, связанной с массивом, или можно вывести проводник с данными массива через границу цикла. Если используется цикл For, то по умолчанию при прохождении проводника данных через границу цикла включается режим накопления данных на границе цикла (Indexing) в проходном терминале (туннеле, рисунок 43). Это значит, что на границе будет формироваться массив из приходящих данных. На первом шаге цикла сформируется массив из одного элемента {a1} и он сохранится на границе.

На второй итерации к массиву {a1} добавится второй элемент и сформируется массив {a1, a2}. А на границе индексируется массив из массивов {{a1}, {a1, a2}}. На третьей итерации на границе цикла сформируется массив {{a1}, {a1, a2}, {a1, a2, a3}} и т. д. Следовательно, выходной массив станет двумерным. Это можно увидеть, если на передней панели растянуть до двух окошки индексов массива (рисунок 22 b)). Обратим внимание, что вид проводника, по которому передается двумерный массив (широкая линия с белой серединой), отличается от вида проводника по которому передается двумерный массив. У проходного туннеля есть также режим последовательного соединения данных Concatenating (рисунок 44(b)). При выборе этого режима на границе цикла будет формироваться одномерный массив, каждый элемент которого будет являться массивом, сформированным на текущем шаге цикла, т.е.: {a1, a1, a2, a1, a2, a3}.

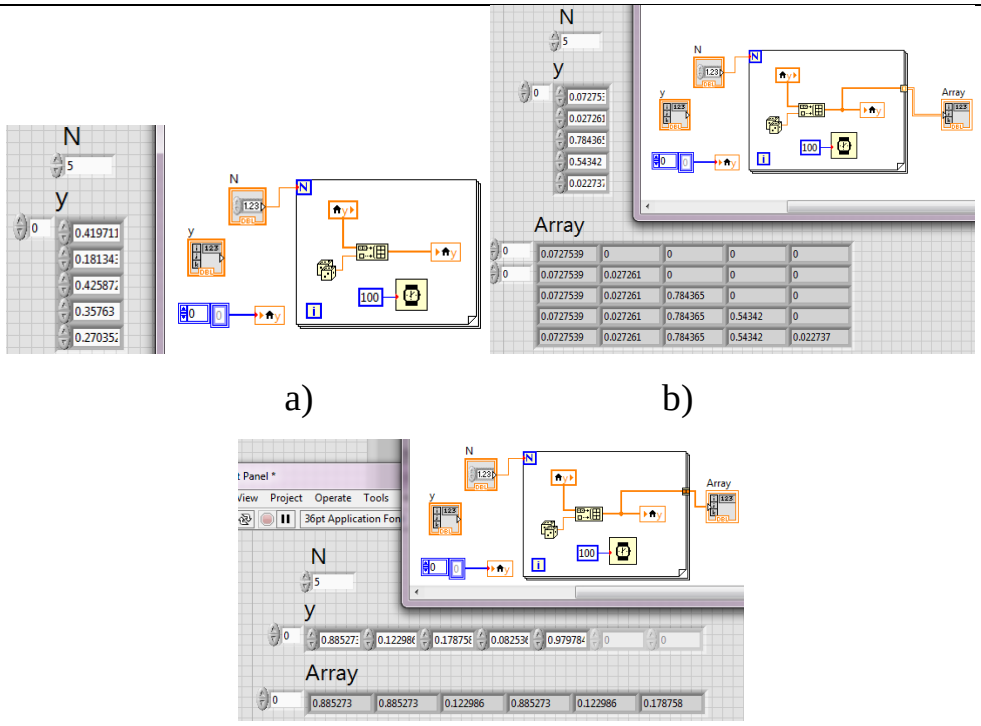
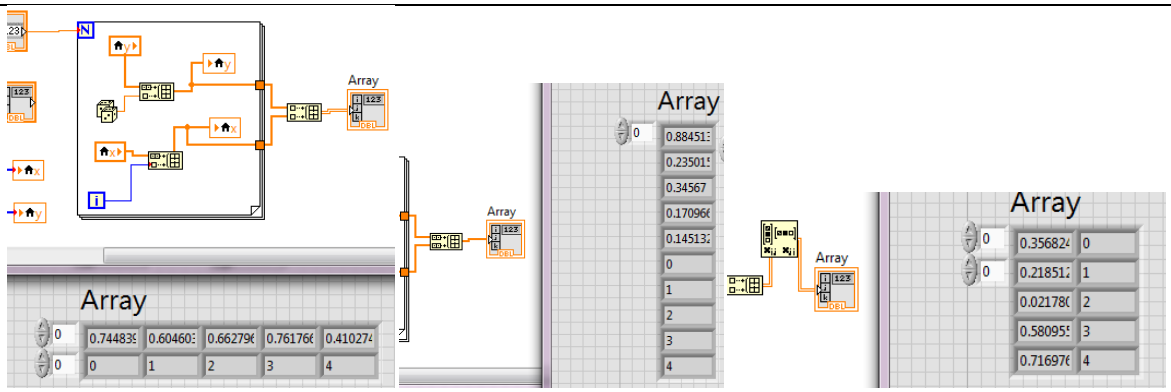


Рисунок 44. a) Заполнение массива числами на каждой итерации цикла. b) Вывод массива через границу цикла. Режиме накопления данных на границе в выходном туннеле - индексация (Indexing). c) Режиме накопления данных на границе в выходном туннеле - соединение (Concatenating).

Функция по добавлению элементов в массив Build Array может работать в двух режимах. Сгенерируем два массива — для примера массив X из номеров итераций цикла и массив Y из случайных чисел. Выведем эти массивы на границу цикла и поставим выходной туннель в режим последнего значения Last Value т.е. на последнем шаге цикла у нас на границе будет два одномерных массива X и Y (рисунок 45(a)). Вне цикла разместим функцию Build Array и подсоединим к входным терминалам сформированные массивы. По умолчанию функция Build Array работает в режиме объединения двух одномерных массивов ({a1, a2, a3} и {b1, b2, b3}) в двумерный ({{a1, a2, a3}, {b1, b2, b3}}, рисунок 45(a)). В контекстном меню функции Build Array есть пункт Concatenating Inputs. При выборе этого пункта данные из одного двумерного массива (подведенного к нижнему терминалу) будут добавлены к данным другого двумерного массива ({a1, a2, a3, b1, b2, b3}, рисунок 45(b)). Обратим внимание, что в режиме Concatenating Inputs у функции Build Array вид входных терминалов отличается (рисунок 45(b)).



a) b) c)

Рисунок 45. a) Генерация двух одномерных массивов и объединение их в двумерный. b) Объединение двух массивов в одномерный массив. c) Транспонирование двумерного массива.

Двумерный массив является матрицей. По умолчанию отображение двумерного массива, состоящего из двух одномерных, на передней панели происходит в виде матрицы с двумя строками. В пункте Array кодовой

панели есть подпункт Matrix в котором присутствует функция транспонирования матрицы Transpose Matrix. Использование транспонирования двумерного массива позволяет изменить вид его отображения на передней панели рисунок 45(c).

Кроме функции Build Array в пункте Array содержатся другие необходимы функции работы с массивами. Часто используется функция Array Size для определения числа элементов в одномерном массиве. При подаче на входной терминал функции Array Size одномерного массива на выходе появляется целочисленная числовая переменная с числом элементов в массиве (рисунок 46(a)). Если на вход функции Array Size подать двумерный массив или матрицу, то на выходе будет массив из двух целочисленных значений, указывающий число строк и столбцов матрицы или число одномерных подмассивов и число элементов в каждом из них (рисунок 46(b)). На рисунке 46(b) на передней панели представлен элемент интерфейса Real Matrix (его можно переключать в режим ввода или отображения).

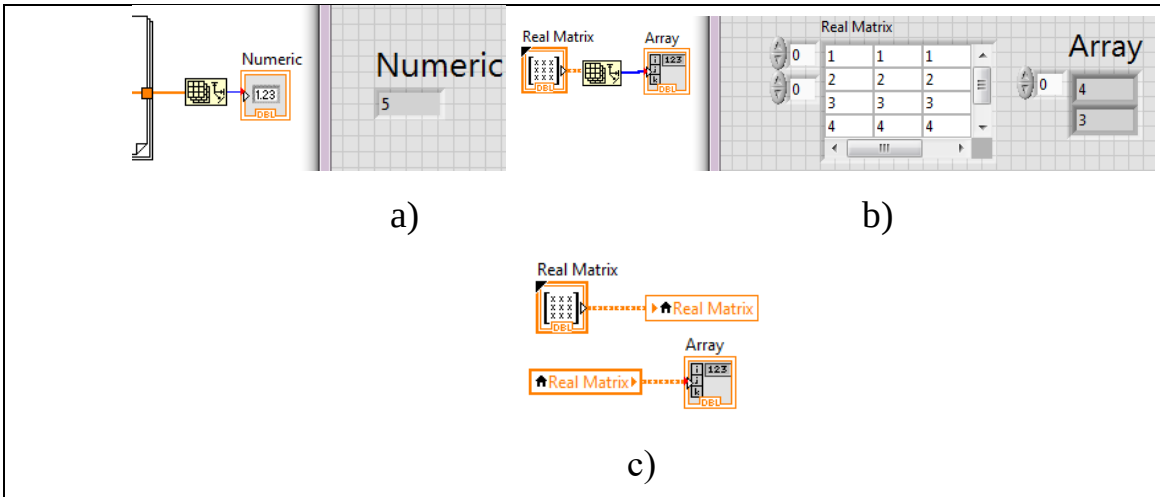
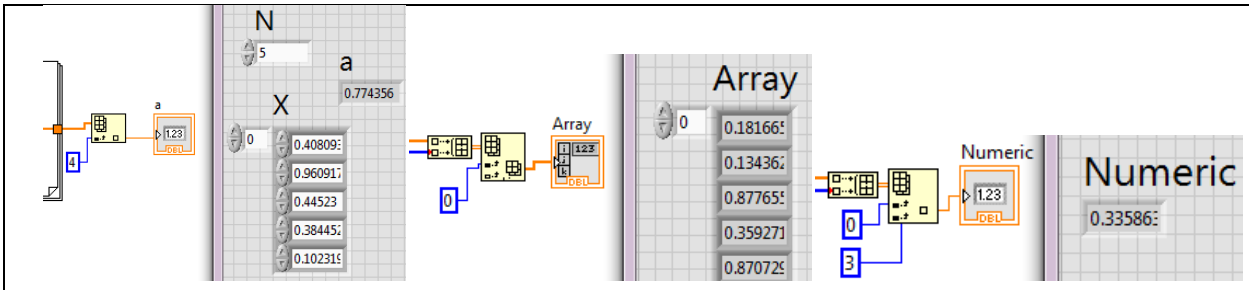


Рисунок 46. a) Определение числа элементов в массиве. b) Определение размера двумерного массива или матрицы. c) Использование локальной переменной для двумерного массива или матрицы.

Очевидно, что данные двумерного массива или матрицы (как и одномерного) можно передавать с использованием локальных переменных

(рисунок 46(c)). Обратим внимание, что вид проводника, по которому передается матрица, отличается от вида проводника по которому передается двумерный массив.

Для извлечения элемента из массива используется функция Index Array, находящаяся в пункте Array кодовой палитры. Функция имеет по умолчанию два входных терминала (верхний желтый для массива и нижний синий для целочисленного значения) и один выходной терминал. При подаче на верхний терминал одномерного массива и подаче на нижний терминал номер извлекаемого элемента на выходном терминале функции появляется значение элемента из массива с заданным номером (рисунок 47(a)). Обратим внимание, что элементы массива нумеруются от нулевого. Если растянуть значок функции Index Array вниз, то появятся дополнительные входные и выходные терминалы. С помощью их можно извлекать одновременно несколько элементов из массива. Если на вход функции Index Array подается двумерный массив, то возможно извлечение составляющих одномерных массивов. У функции Index Array появляются входные терминалы index (row) и index (col) (индекс ряда и колонки, соответственно). Подача целого числа от 0 позволяет извлечь соответствующий ряд или колонку из двумерного массива. На выходе из функции Index Array появляется соответствующий одномерный массив (рисунок 47(b)). Если одновременно подать на терминалы index (row) и index (col) целочисленные величины, то можно извлечь элемент с соответствующими индексами из двумерной матрицы (массива) (рисунок 47(c)). Отметим, что в подпункте Matrix пункта Array палитры в кодовой панели есть функция Get Matrix Elements которая работает точно так же, как Index Array для двумерного массива.



a) b) c)

Рисунок 47. a) Извлечение 5 элемента из массива. b) Извлечение одномерного массива из двумерного. c) Извлечение элемента из двумерного массива (из строки с номером 0 и колонки с номером 3).

В подпункте Matrix пункта Array есть функция Get Submatrix для извлечения матрицы меньшего размера (подматрицы) из исходной матрицы. Функция Get Submatrix имеет вход для исходной матрицы и четыре дополнительных входных терминала - два терминала для указания номера начальной строки (row 1) и конечной строки (row N), а также два терминала для указания номера начальной колонки (col 1) и конечной колонки (col N) (рисунок 48).

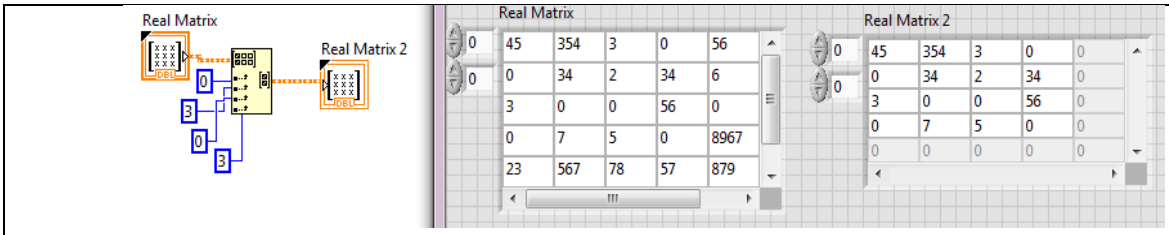


Рисунок 48. Извлечение матрицы меньшего размера из исходной матрицы.

Для добавления элемента в произвольное место используется функция Insert Into Array. Для добавления нового элемента в одномерный массив на верхний входной терминал функции Insert Into Array подается массив, на терминал index подается номер места в массиве куда нужно добавит элемент, а на терминал new element/subarray подается добавляемый в массив элемент или другой массив (рисунок 49(a)). Очевидно, что число элементов в массиве увеличивается. Для замены элементов в массиве используется функция

Replace Array Subset. На функцию подается исходный массив, на терминал index подается номер элемента в массиве какой нужно заменить и на терминал new element/subarray подается новый элемент или массив (рисунок 49(b)). При использовании функции Replace Array Subset число элементов выходного массива равно числу элементов входного. Если заменяется часть исходного массива другим массивом с числом элементов больше, чем остаток в исходном массиве (начиная с номера заменяемого элемента), то в результирующий массив добавится только часть элементов добавляемого массива. На рисунке 49(c) в исходный массив из 5 случайных чисел добавляется массив из 5 натуральных чисел (Array 2). В результирующем массиве (Array) получается 5 элементов, как в исходном.

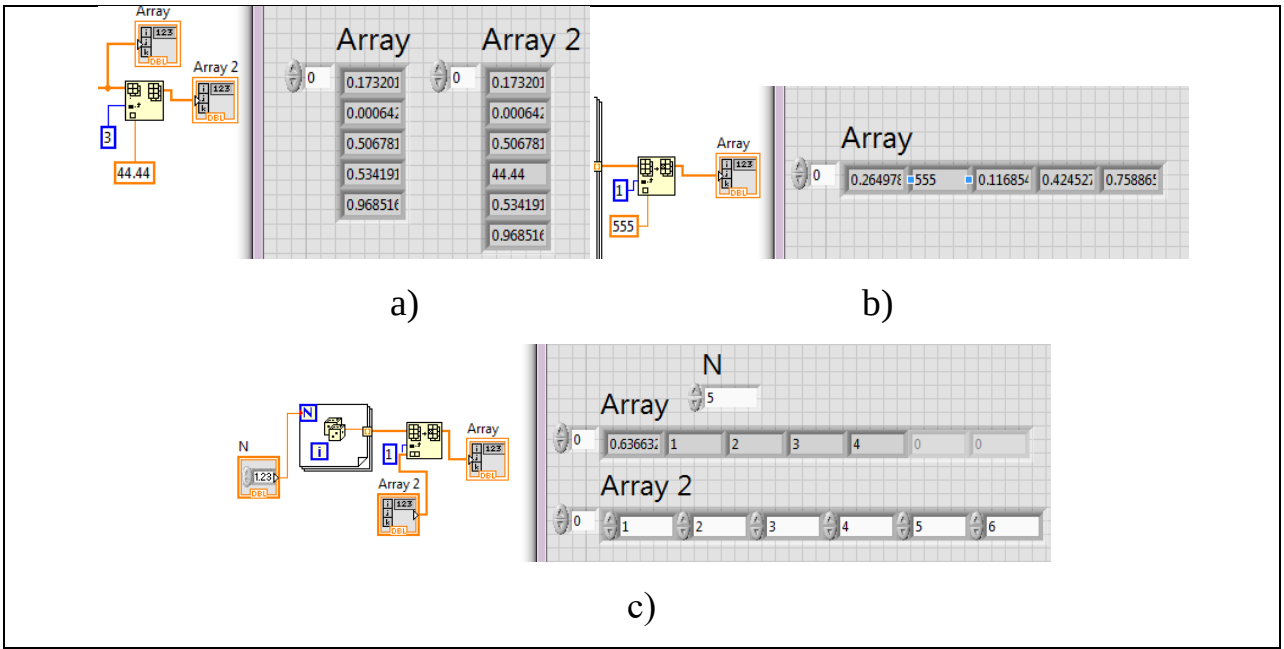
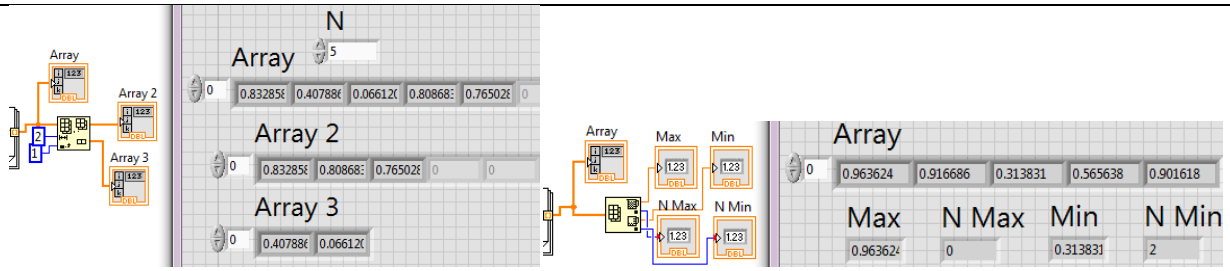


Рисунок 49. a) Вставка элемента в массив функцией Insert Into Array. b) Замена элемента в массиве функцией Replace Array Subset. c) Замена элементов в массиве элементами из другого массива.

Для удаления элементов из массива используется функция Delete From Array (рисунок 50(a)). На вход функции подается исходный массив, указывается номер элемента в массиве, который нужно удалить (или с которого нужно удалить), и указывается число удаляемых подряд элементов (length, если не указывать, то удаляется один элемент). На выходе получается

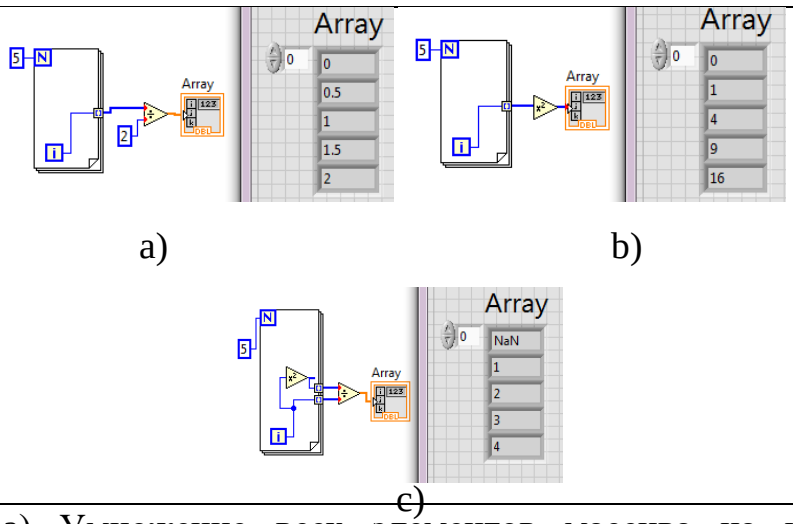
массив с удаленным элементом (Array 2 на рисунке 50(a)) и можно вывести удаленный элемент или удаленную часть массива (Array 3 на рисунке 50(a)). Полезной является функция Array Max & Min позволяющая найти максимальный и минимальный элемент в массиве, а также их номер. Если на вход функции Array Max & Min подается двумерный массив (матрица), то вместо номера максимального (минимального) элемента выдается их координата в массиве (номер столбца и строки).



a) b)

Рисунок 50. a) Удаление части массива функцией Delete From Array. b) Функция поиска максимального и минимального элемента в массиве.

Действия с элементами массива также можно выполнять путем использования математических функций из пункта Numeric палитры кодовой панели. Например, можно легко умножить все элементы массива на константу (рисунок 51(a)) или возвести в квадрат (рисунок 51(b)). Можно производить действия над парами элементов с соответствующим номером для двух массивов. На рисунке рисунок 51(c) представлено получение массива, каждый элемент которого это отношение элементов их двух массивов.



a) b) c)

Рисунок 51. a) Умножение всех элементов массива на константу. b) Возведение всех элементов массива в квадрат. c) Получение массива путем деления элементов из двух массивов.

Полиморфизм

Как мы знаем с элементами массива можно выполнять те же самые арифметические действия, что и с обычными числами. Мы можем выбрать элемент массива и прибавить к нему какое-то число или умножить его на 10. Однако не редко приходится иметь дело с задачами, в которых требуется произвести одно и тоже арифметическое действие с каждым элементом массива, например, к каждому элементу массива прибавить число 10. Тогда, казалось бы, чтобы произвести такое арифметическое действие нам бы понадобилось создавать цикл, который поочерёдно доставал каждый элемент массива, прибавлял к нему 10 и возвращал обратно в массив. Но в среде LabVIEWтак делать не придётся, ведь каждая функция из палитры, которая выполняет какое-либо арифметическое действие обладает свойством полиморфизма – свойство функции при котором входные данные функций могут иметь различные размерности и представления. К примеру, используя функцию сложение, вы можете сложить скалярную величину и массив или сложить два массива. Для наилучшего понимания рассмотрим это свойство на примере. Создадим цикл For с N=10 и поместим в него генератор случайных чисел (игральные кости). Создадим четыре пустых массива и

заполним два из них. В первый массив (Array 1) поместим десять чисел, созданных генератором случайных чисел, во второй массив (Array2) поместим десять значений итератора цикла i.Теперь попробуем сложить эти два массива, не доставая из них элементы. Для этого поместим на блок-диаграмму функцию сложения и подсоединим к её входным терминалам проводники с данными из Array 1 и Array2. Выходной терминал функции сложения соединим с одним из оставшихся пустых массивов (Array 1+Array2) для наглядного отображения результата сложения двух массивов. Теперь добавим случай, когда арифметическое действие происходит над массивом и единичным элементом (то есть над данными с разной размерностью). Сначала создадим единичный элемент поместив элемент управления на лицевую панель (либо можно поместить константу на блок-диаграмму).

В качестве арифметического действия возьмём функцию умножения и подсоединим к её входным терминалам проводник данных Array 1 и единичный элемент (Numeric). Выходной терминал подсоединим к оставшемуся пустому массиву. В результате у нас должен получится графический код, представленный на рисунке 52(a). Прежде чем запускать программу не забудьте ввести единичный элемент, для удобства лучше взять число 10. Теперь запустим программу и посмотрим полученный результат на лицевой панели (рис.52(b)). Массив Array 1 заполнился случайными числами, а Array2 заполнен числами от 0 до 9. После использования функции сложения двух массивов (результат отображён в массиве Array 1+Array2) мы видим, что к нулевому элементу массива Array 1 прибавился нулевой элемент из массива Array2, к первому элементу из Array 1 прибавился первый элемент из Array2 и так далее. То есть при сложении двух массивов складываются одинаковые по нумерации элементы массивов. Этот же принцип распространяется и на другие арифметические действия над массивами.

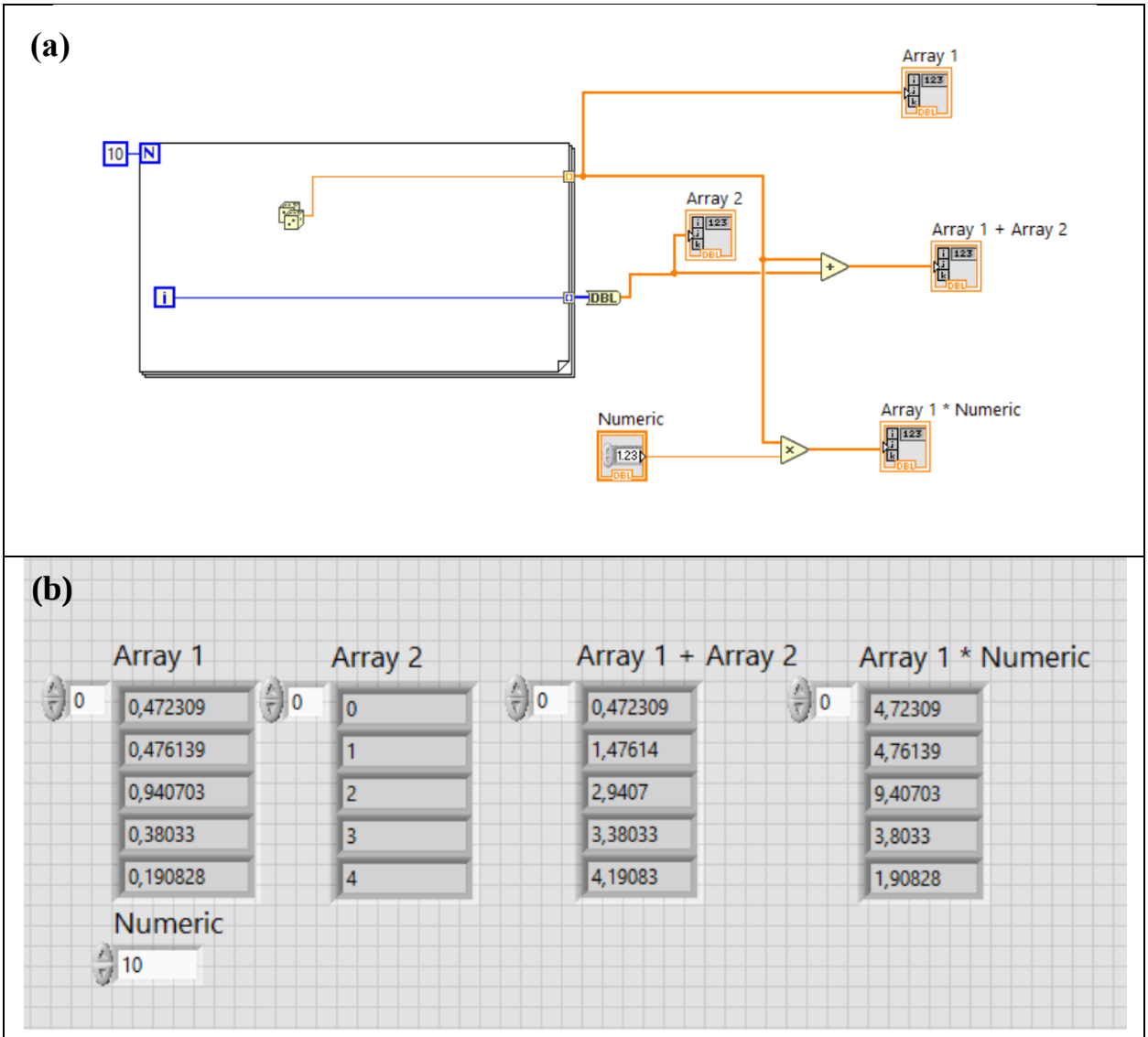


Рисунок 52. a) Графический код для изучения свойства функций – полиморфизм. b) Отображение выходных данных в массивах, полученных после запуска программы.

Теперь посмотрим какой результат получился при умножении массива на единичный элемент (число), который отображается в массиве Array 1*Numeric. Каждый элемент из массива Array 1 умножился на 10 (единичный элемент). То есть, при проведении какого-либо арифметического действия между единичным элементом и массивом, каждый элемент массива поочерёдно будет умножен (в нашем примере) на единичный элемент. Аналогичные арифметические действия можно производить не только с массивами, но и с кластерами.

Функции для работы со временем

Для создания исправного программного кода необходимо учитывать ещё один очень важный параметр – время. Время выполнения какой-либо части программного кода очень сильно влияет на исправную работу программы и своевременное выполнение какой-либо части кода. Наиболее важную роль время играет в программах, которые осуществляют коммуникацию с измерительными приборами. К примеру, считывание данных с прибора может производится слишком быстро, поступающие данные не будут успевать обрабатываться определённой частью программного кода, что приведёт к некорректному сбору данных с прибора и нарушению работы программы. Рассмотрим наиболее часто используемые функции для работы со временем: функция задержка (***Wait***), задержка до следующего кратного интервала (***Wait Until Next ms Multiple***), счётчик времени (***Tick Count***). Чтобы найти эти функции откройте ***Function Pallete*** → ***Programming*** → ***Timing***→…. Краткое описание каждой функции представлено в таблице 6.

Таблица 6. Функции работы со временем и их краткое описание.

Функция задержка (***Wait***) (мс) 	Ожидает указанное количество миллисекунд, прежде чем продолжить выполнение графического кода ВП.
Задержка до следующего кратного интервала (***Wait Until Next ms Multiple***) (мс) 	Ожидает, пока показания внутренних часов не сравняются или не превысят кратного количества миллисекунд, поданных на вход функции, прежде чем возобновить выполнение ВП.
Счётчик времени (***Tick Count***) (мс) 	Возвращает значение внутренних часов операционной системы в миллисекундах.

Счётчик времени (*Tick Count*) (мс)

Как мы знаем выполнение каждого программного кода занимает какое-то время, которое можно определить, используя функцию счётчик времени (***Tick Count***). Сама по себе функция ***Tick Count*** не показывает значение времени выполнения всего программного кода или части программного кода. Она показывает текущее значение времени, которое соответствует внутренним часам операционной системы. При этом стоит отметить, что значение времени, считанное в момент запуска программы, никогда не будет равно нулю, даже если функция счётчика времени будет установлена в самом начале программного кода. Поэтому для определения времени выполнения всей программы или части кода программы необходимо создать небольшой графический код, который позволит вычислять время выполнения ВП. Для примера, создадим цикл For, который создаст массив из миллиона случайных чисел, и посмотрим сколько времени программы выполняет создание такого массива. Для начала создадим Flat Sequence, которая будет состоять из трёх кадров. В первый и третий кадр поместим функцию времени ***Tick Count*** для того, чтобы фиксировать значение времени, в котором начинается и заканчивается выполнение цикла по созданию миллиона случайных чисел. Сам цикл по созданию чисел поместим в центральный кадр Flat Sequence. Для наглядности создадим два индикатора на лицевой панели, первый индикатор подсоединим к терминалу выхода функции ***Tick Count*** первом кадре, второй индикатор – к терминалу выхода функции ***Tick Count*** третьем кадре. Это позволит нам наглядно увидеть, как отображается время операционной системы. Теперь, когда мы знаем и можем увидеть время начало и конца выполнения программного кода, найдём время выполнения этого кода. Чтобы это сделать поместим в третий кадр функцию вычитания с двумя входными терминалами и подсоединим к ним счётчики времени так, как показано на рисунке 53(а). К верхнему терминалу подсоединили счётчик времени, считывающий время завершения выполнения цикла, к нижнему – счётчик времени, считывающий начало выполнения цикла. Полученное

значение времени выведем в индикатор Time, который создадим на лицевой панели. Теперь запустим программу и посмотрим какие получились результаты (рис. 53(b)). В индикаторе Current Timeнаходится время в момент начала выполнения цикла, в Current Time 2 – время после завершения цикла. Разница между этими временами, отображённая в индикаторе Time равна 27 мс, это и есть время, за которое выполняется наш цикл по созданию миллиона случайных чисел. Попробуем запустить программу ещё раз и увидим, что все числа изменяться и время выполнения цикла станет больше и равно 38 мс (вставка на рис. 53(b)). То есть время выполнения цикла при каждом запуске может изменяться при том как в большую сторону, так и в меньшую сторону (проверьте это самостоятельно многократно запуская программу и отслеживая полученный результат). Таким образом, устанавливая две функции счётчика времени в начале и в конце кода и находя их разницу, вы можете находить время выполнения этого кода.

В основном среду программирования LabVIEW используют для создания программ, которые осуществляют управление приборами и получают от них экспериментальные данные. Обычно измеряемые данные используют для построения каких-либо зависимостей двух величин и не редко одной из этих величин может быть время. Тогда возникает вопрос как же можно получить временную зависимость какой-либо величины, которая измеряется прибором. Использование временной функции Счётчик времени в программном коде позволит решить этот вопрос. Для наилучшего понимания разберём его на основе составленного примера. Представим, что числа, созданные генератором случайных чисел, это экспериментальные данные, которые мы получаем от прибора. Теперь нужно получить временную зависимость для этих чисел, то есть мы хотим знать через какое время от начала работы программного кода мы получили случайное число. К примеру, первое случайное число получили в 10 мс, второе случайное число – в 15 мс, третье случайное число – в 21 мс и так далее.

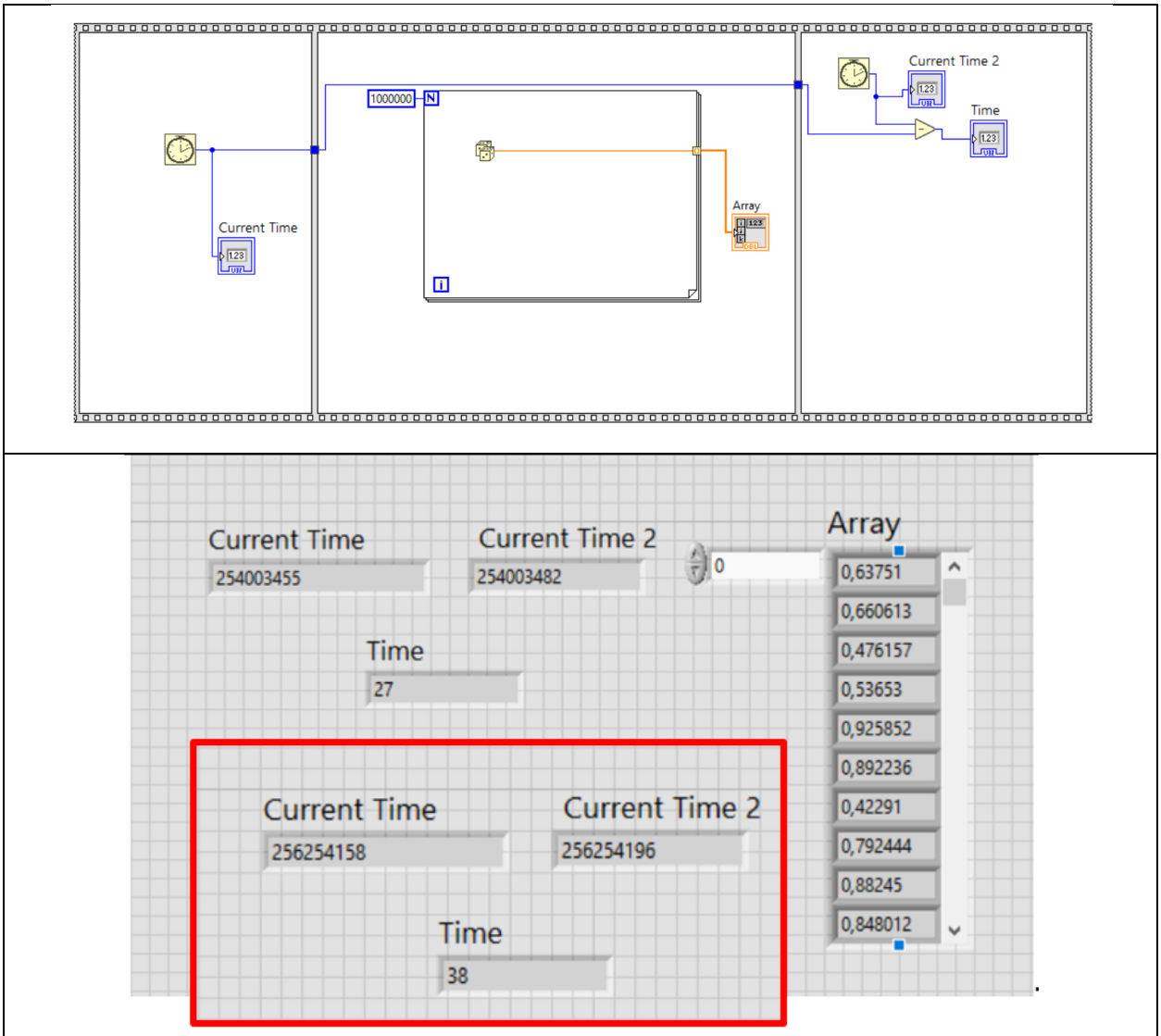


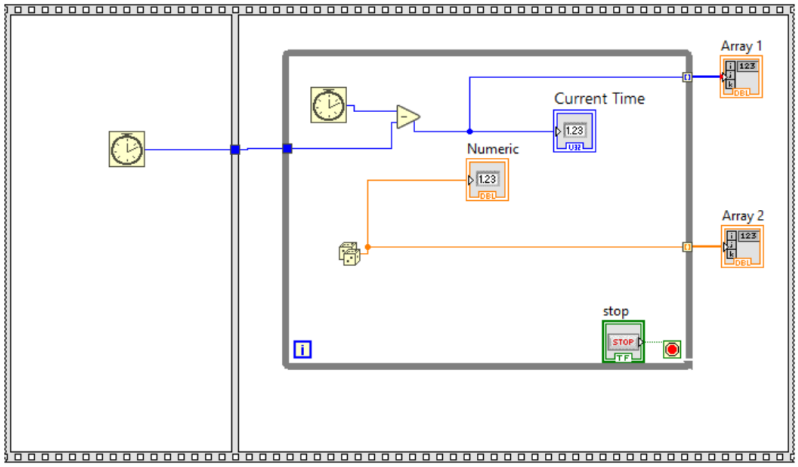
Рисунок 53. a) Графический код для изучения работы Счётчика времени. b) Отображение выходных данных в массивах, полученных после запуска программы

За начала отсчёта (за 0 мс) примем запуск программы, так как получение данных начинается только после запуска программы. Чтобы обозначить начало отсчёта в программном коде установим функцию Счётчик времени в структуру Flat Sequence в самом начале программного кода. Далее добавим ещё один кадр в структуре Flat Sequence и поместим в него генератор случайных чисел. В свою очередь генератор случайных чисел поместим в цикл While, чтобы создать непрерывный поток поступающих случайных чисел (аналогия считывания экспериментальных данных). Теперь в цикл

поместим функцию Счётчик времени и функцию разности. Так как функция Счётчик времени помещена внутри цикла, то она будет выдавать разные значения при выполнении каждой итерации цикла. При этом значение времени, которое выдаёт счётчик будет соответствовать времени получения (создания) нового случайного числа (новое значение поступающих данных), так как при запуске каждой итерации цикла эти функции выполняются одновременно. Таким образом помещении в цикл функции счётчика времени и генератора случайных чисел позволяет получать значение времени, в которое было создано новое случайное число (поступили новые данные). Но не стоит забывать, что это значение операционного времени. Чтобы получить реальное значение времени нам нужно из этого значения вычесть значение операционного времени, которое мы получаем из функции счёта времени, взятого за начала отсчёта времени. Записывая полученные данные в массивы и отображая текущие значения данных в индикаторы мы должны получить программный код, представленный на рисунке 54(a). Теперь перейдём на лицевую панель и запустим выполнение программного кода. Спустя некоторое время остановим программный код нажав на кнопку Stop, которая останавливает выполнение цикла, и посмотрим на полученные в массивах результаты (рис. 54(b)).

Так как значений в массивах очень много увидеть все числа растягивая массив не получиться. Чтобы их увидеть включите полосу прокрутки для массива. Щёлкните левой кнопкой мыши по массиву, чтобы он стал выделенным, затем после щелчка правой кнопкой мыши в контекстном меню выберите пункт ***Visible Items***, а затем ***Vertical Scrollbar*** (смотреть вставку на рис. 54(b)). Теперь мы видим, что каждому созданному случайному числу из массива соответствует значение времени, в которое оно было создано. Скорость выполнения программы очень высокая, поэтому в течении 1 мс(разрешение меньше 1 мс в среде LabView получить нельзя) создаётся большое количество случайных чисел. Чтобы увидеть разные значения времени прокрутите числа в массиве с помощью полосы прокрутки.

(a)



(b)

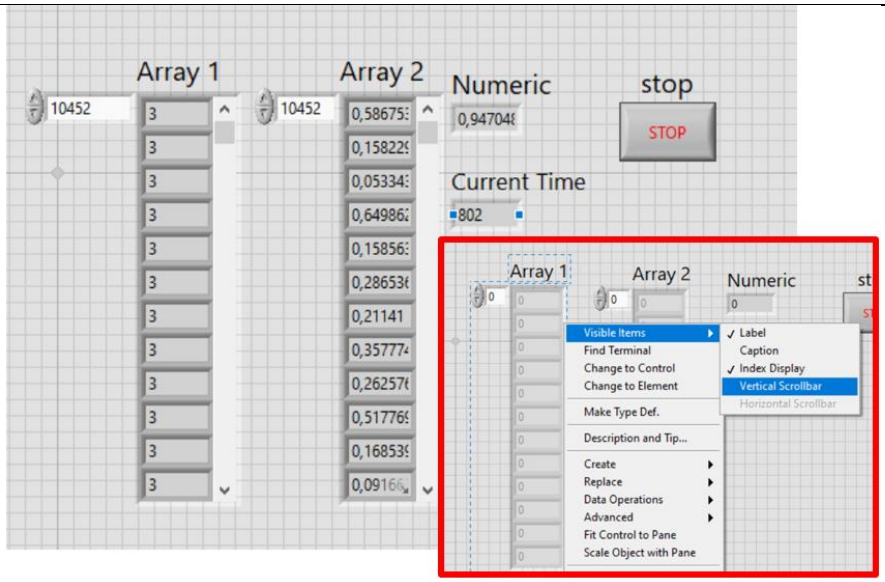


Рисунок 54. a) Графический код для изучения работы Счётчика времени. b) Отображение выходных данных в массивах, полученных после запуска программы

Иногда в экспериментальных задачах требуется получать данные с прибора какое-то определённое количество времени, например, 10 минут. Засекать время и ждать десять минут, когда нужно будет завершить выполнение программы неудобно и легко можно пропустить момент выключения программы. На этот случай можно использовать функцию Счётчик времени для того, чтобы написать программный код, который позволит задавать время работы программы и сам будет прекращать её выполнение по истечении этого времени. Рассмотрим, как это можно

сделать, используя предыдущий пример. Чтобы остановить работу программы нам требуется нажать на механическую кнопку Stop, которая останавливает выполнение цикла, соответственно, чтобы программа останавливала своё выполнение самостоятельно нужно ввести какое-то условие, которое будет подавать сигнал Truена кнопку и останавливать её. То есть задача сводится к тому, что нам нужно создать определённое условие, при котором встроенная кнопка остановки цикла будет нажиматься. При этом стоит оставить механическую кнопку Stop, размещённую на лицевой панели, чтобы иметь возможность преждевременного прерывания выполнения программного кода. Итак, чтобы оставить возможность прерывать программу с помощью механической кнопки и с помощью выполнения условия, используем логическую функцию ИЛИ. К одному из терминалов мы присоединим условие, которое напишем позже, а к другому присоединим механическую кнопку Stop. Когда на входные терминалы функции ИЛИ придёт хотя бы один сигнал, тогда выходной терминал функции ИЛИ отправит сигнал выключения цикла. Теперь остаётся написать условие, при выполнение которого будет подаваться сигнал на выключение цикла.

Нам требуется, чтобы программа прекращала своё выполнение после какого-то определённого времени работы. Для примера возьмём 100 мс. То есть когда время работы будет равно 100 мс или превысит его, программа должна завершить свою работу. Логическая функция больше или равно как раз выполняет это условие (рис. 55(a)). К верхнему терминалу функции подсоединим значение времени работы программы, а к нижнему подсоединим константу со значением 100. Теперь соединим всё проводниками и попробуем запустить программу (рис. 55(b)). После запуска программы мы видим, что время выполнения программы в индикаторе Current Time 2 равно 100 мс, это значит, что программа закончила своё выполнение на через 100 мс. При этом, если прокрутить содержимое массива со значением времени выполнения, мы также увидим, что заполнение

массива оборвалось на 100 мс. Самостоятельно вы можете проверить прекращается ли выполнение программы при нажатии на механическую кнопку Stop или нет.

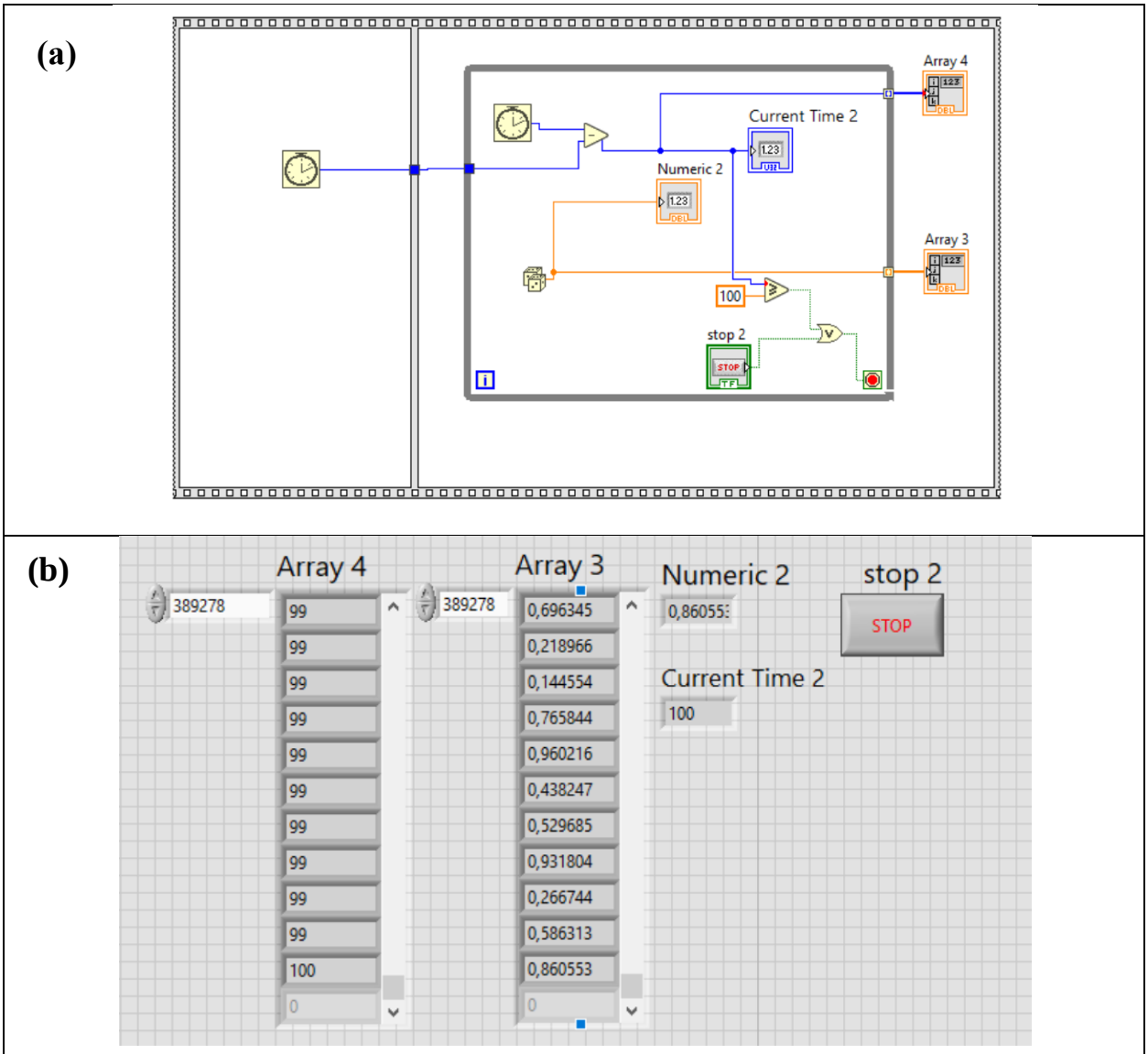


Рисунок 55. a) Графический код для изучения работы Счётчика времени. b) Отображение выходных данных в массивах, полученных после запуска программы

Функция задержка (*Wait*) (мс)

Теперь рассмотрим функцию работы со временем – ***Wait***, которая прерывает работу последующей части программного кода ВП на определённое время, установленное пользователем. Поместим функцию ***Wait*** на блок-диаграмму и рассмотрим её поподробнее. Для того, чтобы найти эту

функцию нужно выбрать ***Function Pallete → Programming → Timing → Wait.*** Эта функция имеет один входной и один выходной терминалы. Входной терминал функции позволяет задавать значение времени (в миллисекундах) в течении которого графический код ВП не будет выполняться. Выходной терминал выдаёт текущее операционное значение времени, по сути выполняет ту же самую функцию, что и Счётчик времени, который мы разобрали выше. Использование функции ***Wait*** очень важно значение для создания корректно работающей программы, которая осуществляет коммуникацию с измерительными приборами. Если программа может считывать данные с прибора, то количество данных, поступающих в единицу времени, будет настолько большим, что программный код не будет успевать их обрабатывать и ВП будет работать не корректно. Чтобы предотвратить некорректную работу графического кода ставят функцию задержки для считывания данных с прибора. В результате данные поступают не как один большой непрерывный поток, а как единичные значения, разделённые промежутком времени. Приведём пример для наглядного рассмотрения принципа работы функции. Создадим цикла While, который будет представлять собой непрерывное поступление экспериментальных данных. Внутрь цикла в качестве самих экспериментальных данных добавим цикл For, который будет создавать, к примеру, 100 случайных чисел. Выводить эти числа в какой-либо массив мы не будем, так как это не даст никакой информации нужной для понимания работы функции ***Wait***. Чтобы знать, как измениться время работы программного кода после того, как мы поставим функцию задержки, нам нужно знать изначальное время работы, которое занимает выполнение одной итерации в цикле While. Для этого установим счётчики времени так, как представлено на рисунке 56(a). Структуру Flat Sequence добавили для того, чтобы счётчик времени выдавал значение только после того как цикл For выполнит всю свою работу. Запустим программу и посмотрим сколько по времени будет выполняться одна итерация цикла While. Как мы видим из массива Array 3 (рис. 56(b)), время

работы одной итерации цикла составляет меньше, чем 1 мс. За 1 мс цикл успевает проработать очень большое количества раз. Теперь добавим в первый кадр структуры Flat Sequence функцию ***Wait*** и подадим на её входной терминал задержку по времени равную, к примеру, 100 мс (рис. 56(a)).

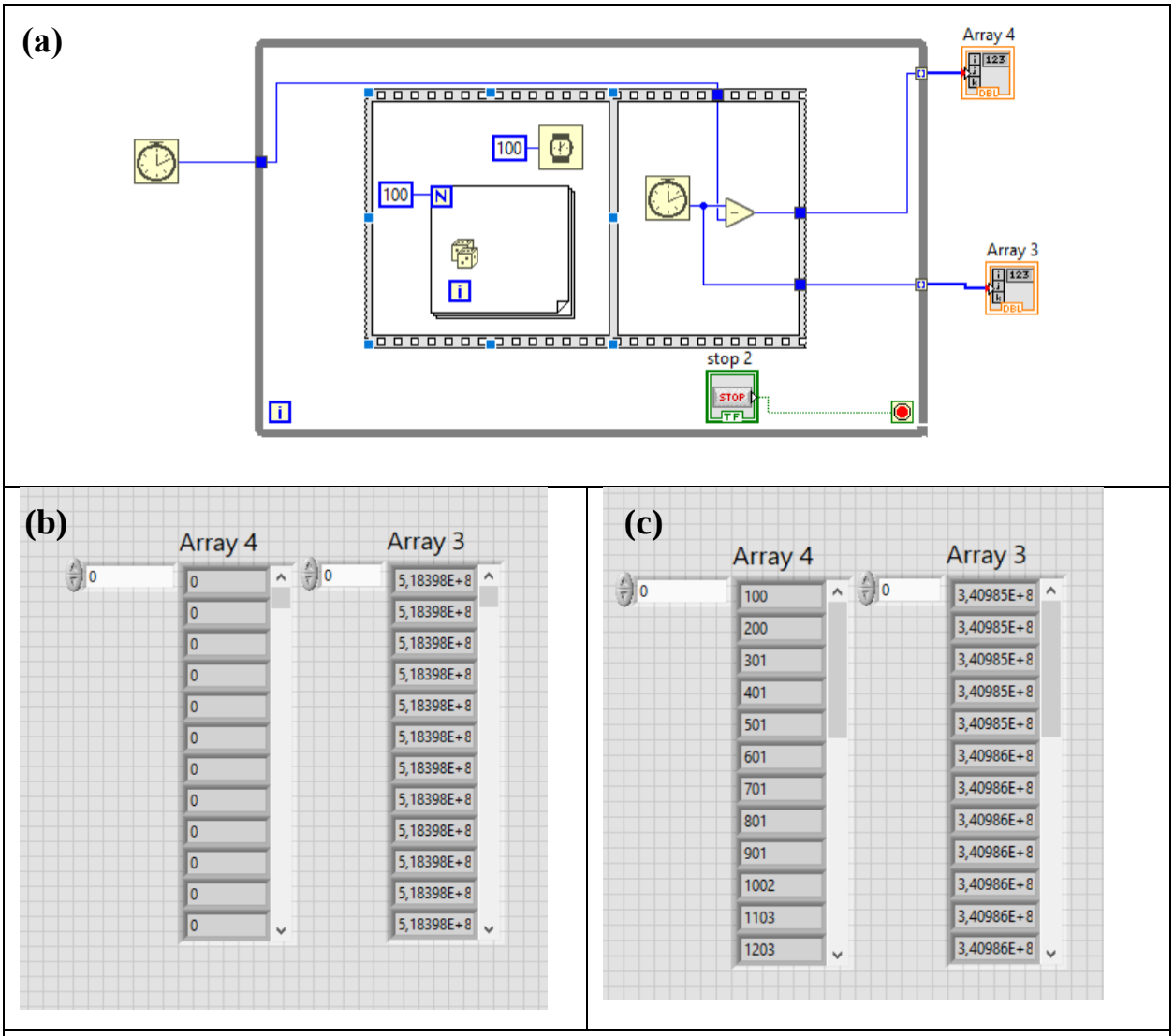


Рисунок 56. a) Графический код для изучения работы функции задержки времени Wait. b) Отображение выходных данных в массивах, полученных после запуска программы

Запустим программу и посмотрим, как изменилось время одной итерации цикла. Данные в массиве изменились и теперь, одна итерация цикла происходит за 100 мс, то есть происходит за время задержки, которое мы указали в функции ***Wait*** (рис. 56(c)). После того, как мы добавили функцию время, выполнения одной итерации увеличилось. Следовательно, эта

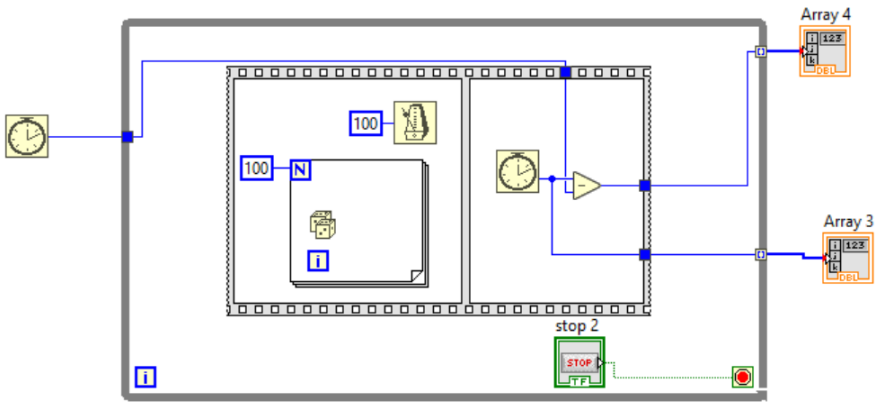
функция не завершает выполнение одной итерации до тех пор, пока не истечет указанное время. Стоит отметить, что функция задержки начинает отсчитывать время сразу же, как начинается новая итерация цикла, функция ***Wait*** не ждёт, когда цикл, создающий случайные числа, закончит свою работу, она отсчитывает время параллельно с работой цикла. Это очень важно понимать, потому что установленная задержка времени не сработает, если время задержки будет меньше, чем время работы цикла (или части программного кода, к которому применили задержку). Если время задержки будет меньше времени работы итерации цикла – задержки не произойдёт, так как функция ***Wait*** успеет закончить свою работу до того, как выполнение итерации закончиться. То есть задержка ни на что не повлияет. Будьте внимательны при выборе времени задержки. Попробуйте указать другое время и посмотрите, как изменятся данные в массиве.

Задержка до следующего кратного интервала (*Wait Until Next ms Multiple*)

Временная функция ***Wait Until Next ms Multiple*** останавливает выполнение последующей части программного кода до тех пор, пока значение операционного времени не сравняется или не превысит кратного количества миллисекунд, указанных в временной задержке. В целом эта функция похожа на функцию ***Wait***, однако у неё есть и отличия. Разберём эти отличия на основе предыдущего примера, только в место функции ***Wait*** поставим функцию ***Wait Until Next ms Multiple***, которую можно найти в ***Function Pallete → Programming → Timing → Until Next ms Multiple***. Время задержки оставим равным 100 мс такое же, как и в предыдущем примере, для более наглядного сравнения функций. В результате у вас должен получиться графический код, представленный на рисунке 57(a). Запустим программу и посмотрим на получившиеся результаты, представленные в массиве Array 3 (рис. 57(b)). Мы видим, что выполнение первой итерации цикла длилось всего 84 мс, в то время как выполнение первой итерации с функцией ***Wait*** завершилось через 100 мс. Время выполнения занимает меньше желаемого

времени, так как значение операционного времени стало кратно указанному времени задержки раньше, чем через 100 мс от начала выполнения программного кода. Но каждая последующая итерация цикла заканчивается миллисекундным таймером, кратным желаемому времени ожидания (то есть через 100 мс). Если у вас в графическом коде присутствует несколько циклов и требуется, чтобы они работали одновременно, можно использовать эту функцию для их синхронизации их действий. Эта функция ожидания гарантирует, что цикл завершится, когда время в мс станет кратным входному значению, поэтому несколько выполняющихся циклов будут заканчивать свою работу в одно и то же время, то есть будут работать синхронно.

(a)



(b)

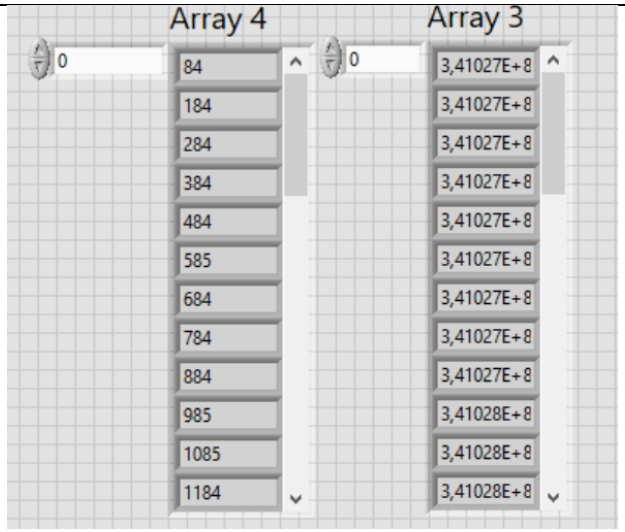
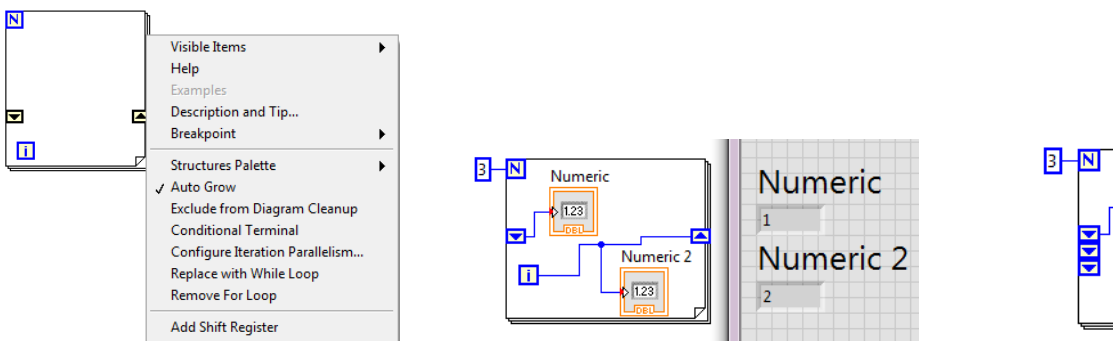


Рисунок 57. a) Графический код для изучения работы функции задержки времени до следующего кратного интервала. b) Отображение выходных данных в массивах, полученных после запуска программы.

Сдвиговый регистр

В среде LabView существует специфическая функция для передачи данных с предыдущей итерации цикла в текущую - сдвиговый регистр (Shift Register). Ранее это выполнялось с использованием локальной переменной, в частности при суммировании ряда чисел (рисунок 39) и при заполнении массива (рисунок 44). Для переноса данных между итерациями цикла можно также использовать сдвиговый регистр. Для создания сдвигового регистра нужно нажать правой кнопкой мыши на левую или правую границу цикла (как For, так и While) и выбрать пункт Add Shift Register (рисунок 58(a)). Прямоугольник сдвигового регистра на левой границе цикла соответствует данным на предыдущей итерации цикла, а на правой границе соответствует данным на текущей итерации цикла. Изначально символы сдвигового регистра имеют черный цвет, т.к. тип данных в сдвиговом регистре не определен. Д0ля примера подсоединим счетчик итераций цикла к входному терминалу сдвигового регистра на правой границе цикла. Символы сдвигового регистра станут синими, что соответствует целочисленным данным. В сдвиговый регистр будет передаваться значение номера итерации цикла. Разместим на передней панели два числовых индикатора и поместим их в цикл в кодовой панели. Соединим один числовой индикатор с линией номера итерации цикла, а второй с выходным терминалом сдвигового регистра на левой стороне цикла (рисунок 58(b)). Сделаем три итерации цикла и после увидим, что один числовой индикатор покажет 2 т.е. номер последней итерации (номер итерации начинается с 0), а второй покажет 1 т.е. номер предыдущей итерации. Если растянуть значок сдвигового регистра на левой границе цикла, то появятся терминалы на которых будут присутствовать данные с более предыдущих итераций цикла чем предпоследняя итерация (рисунок 58(c)).

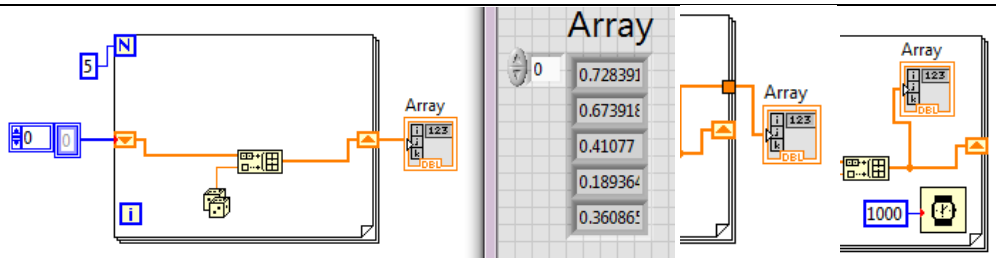


a) b) c)

Рисунок 58. a) Создание сдвигового регистра. b) Передача данных в сдвиговый регистр и использование данных с предыдущей итерации. c) Терминалы для доступа к данным на нескольких предыдущих итерациях цикла.

Сдвиговый регистр удобно использовать для заполнения цикла. Сделаем массив из 5 случайных чисел. Разместим в цикле генератор случайных числе, а также функцию Build Array. Растянем значок функции функцию Build Array на два входных терминала. Выходной терминал Build Array соединим с входным терминалом сдвигового регистра на правой границе цикла - туда будем передавать массив на текущей итерации цикла. Верхний входной терминал Build Array соединим с выходным терминалом сдвигового регистра на левой границе цикла - оттуда будем брать массив с предыдущей итерации цикла, а к нижнему входному терминалу Build Array подсоединим генератор случайных чисел (рисунок 59(a)). При текущей итерации цикла в массив будет добавляться случайное число, массив уходит в сдвиговый регистр, а на следующем шаге в массив с предыдущего шага будет добавляться новое случайное число. Сдвиговый регистр на правой границе цикла имеет выходной терминал вне цикла, это позволяет передать сформированные в цикле данные для дальнейшего использования. Обычно нужно на входной терминал сдвигового регистра на левой границе цикла подавать пустой массив. Если этого не сделать, что при последовательных запусках программы новые данные будут добавляться к данным,

сформированным при предыдущем запуске программы (данные сохраняются в оперативной памяти). Отметим, что вывод сформированного массива из цикла может быть выполнен не только из выходного терминала сдвигового регистра на правой границе цикла, но и через туннель в цикле в режиме Last Value (рисунок 59(b)). Очевидно, что можно получить доступ к массиву и внутри цикла. Например, размести индикатор массива числовых данных внутри цикла и соединим его с потоком данных, выходящих из функции Build Array. Поставим задержку Wait и сможем наблюдать в индикаторе массива заполнение массива в процессе выполнения цикла (рисунок 59(c)).



a) b) c)

Рисунок 59. a) Заполнение массива случайными числами с использованием сдвигового регистра и вывод массива из выходного терминала сдвигового регистра. b) Вывод массива через туннель в границе цикла. c) Использование данных из сдвигового регистра внутри цикла.

Для одного цикла можно использовать несколько сдвиговых регистров. Для этого надо несколько раз выбрать пункт Add Shift Register в контекстном меню на границе цикла. На рисунке 60 представлена программа по формированию трех одномерных массивов с использованием трех сдвиговых регистров (массив из номера итерации цикла, массив случайных чисел и массив из произведения номера итерации на случайное число) и объединение их в двумерный массив. Объединение массивов производится функцией Build Array которая растянута до трех входных терминалов на которые подаются три одномерных массива (далее массив транспонируется). В результате функция Build Array формирует двумерный массив каждый столбец которой являлся одномерным массивом.

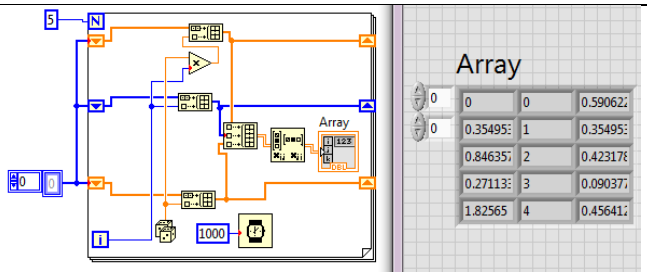
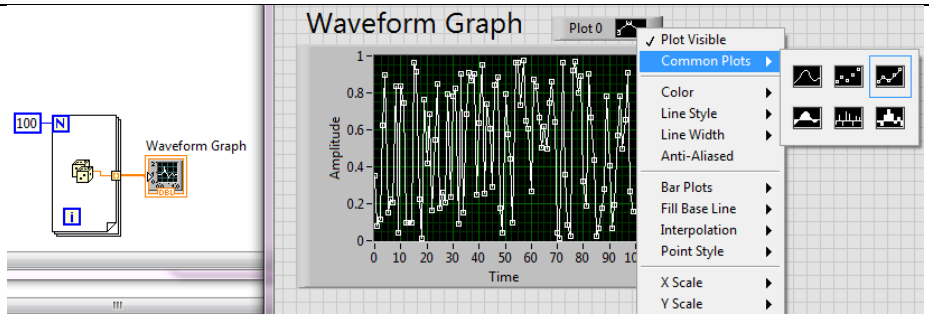
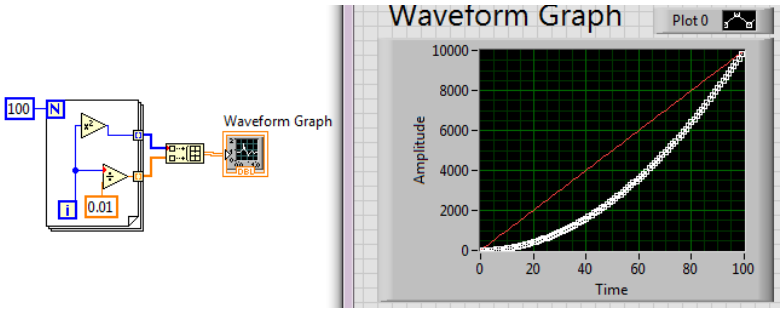


Рисунок 60. Использование трех сдвиговых регистров в одном цикле для формирования двумерной матрицы из трех столбцов.

Представим в графическом виде формируемые массивы. Для отображение данных в LabView есть ряд функций для передней панели. Простейшей функцией визуализации данных является функция Waveform Graph, которая находится в пункте Graph палитры передней панели. При выборе этой функции на передней панели появляется окно графика, а в кодовой панели значок с одним входным терминалом. Подадим на этот терминал одномерный массив из случайных чисел (рисунок 61(а)). После формирования массива в окне Waveform Graph отобразится график зависимости значения случайного числа от номера этого числа в массиве. По умолчанию ось абсцисс имеет название Time, но по этой оси отображается не время. Название осей можно изменить. Если навести курсор на правую верхнюю часть окна Waveform Graph и нажать левую кнопку мыши, то появится меню настройки параметров отображения, в котором есть пункт Common Plots, содержащий варианты представления графика (рисунок 61(а)). Обычно требуется построение графика функции от задаваемого или измеряемого аргумента, а не просто построение графика значения элемента массива от номера этого элемента. Сформируем два одномерных массива одинаковой длины и попробуем использовать один массив в качестве аргумента x, а другой массив в качестве функции y.



a)

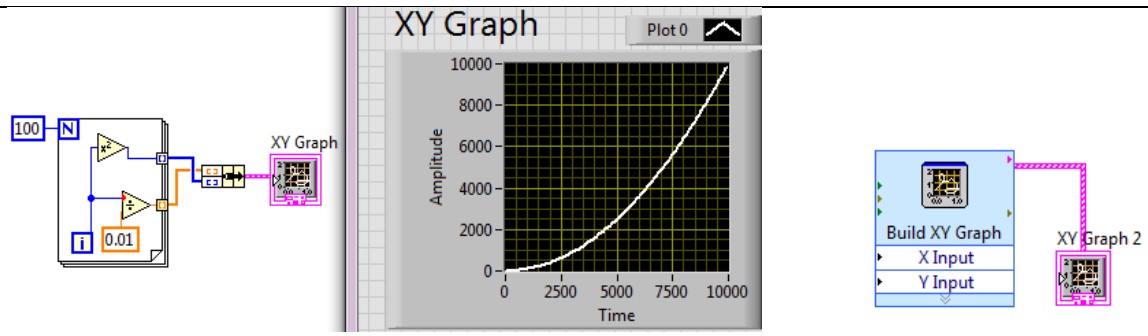


b)

Рисунок 61. a) Использование функции построения графика Waveform Graph. b) Функция Waveform Graph не позволяет построить зависимость x(y).

Например, создадим массив x из элементов равных номеру итерации цикла умноженному на 0.01 и массив y из квадрата номера итерации цикла. Очевидно, что зависимость y(x) является квадратичной, т.е. графиком функции y(x) является парабола. После цикла объединим эти два массива функцией Build Array и подадим результирующий массив на функцию Waveform Graph. После запуска программы мы увидим не один график функции y(x), а два графика, каждый график — это график значения элементов из соответствующего массива от номера этого элемента (рис.61(c)). Для построения нужной функции y(x) нужно использовать функцию отображения XY Graph также из пункта Graph палитры передней панели. Однако подать сформированный с помощью функции Build Array нельзя подать на вход функции XY Graph - отобразиться разорванный проводник данных. Это связано с тем, что на вход XY Graph подается не двумерный массив, а кластер, в данном случае являющийся объединением в одном потоке данных двух независимых одномерных массивов (в общем

случае кластер может объединять в один поток набор данных любого типа и любой размерности). Для формирования кластера используется функция Bundle (объединения данных), находящаяся в пункте Cluster, Class & Variant палитры кодовой панели. По умолчанию функция Bundle имеет два входных терминала, на которые нужно подать два одномерных массива. Массив аргумента x подадим на верхний входной терминал, а массив функции y на нижний. На выходе функции Bundle сформируется кластер. Отметим, что проводник для кластера обычно имеет коричневого толстого проводника с белой серединой, но при подключении к функции XY Graph проводник кластера имеет красный цвет. После запуска программы получим нужный график y(x) (рисунок 62(a)). В пункте Graph палитры передней панели есть функция Express XY Graph. Эта функция отличается от функции XY Graph тем, что в кодовой панели имеет виртуальный подприбор Build XY Graph (подпрограмму) соединенный с XY Graph у которого есть входные терминалы X Input и Y Input на которые подаются массивы функции и аргумента (рисунок 62(b)).

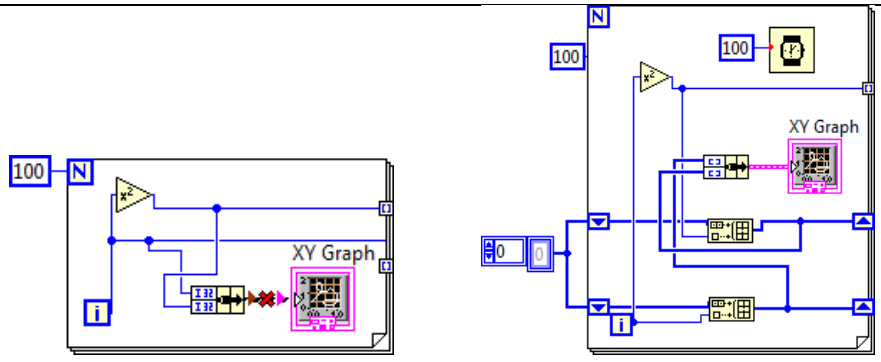


a) b)

Рисунок 62. a) Построение графика зависимости x(y) с использованием функции XY Graph. b) Дополнительная функция построения графиков Express XY Graph.

Функция Express XY Graph более простая в использовании чем XY Graph. Если в контекстном меню подприбора Build XY Graph выбрать пункт View As Icon, то подприбор будет отображаться в более компактном виде.

В ходе проведение эксперимента часто необходимо наблюдать построение экспериментальной зависимости по мере получения измеряемых величин. В примерах, рассмотренных на рисунках 62 и 63, данные передавались в функции отображения графиков Waveform Graph и XY Graph после формирования данных в массиве. Для отображения данных на графике по мере формирования массивов нужно графические функции разместить внутри цикла. Если поместить функцию графика XY Graph с функцией объединения данных Bundle внутрь массива и подсоединить поток данных аргумента и функции к входным терминалам функции Bundle, то проводник между выходным терминалом Bundle и XY Graph получится разорванным (рисунок 63(a)). Это связно с тем, что на выходе функции Bundle сформируется кластер из двух чисел, а не кластер из двух массивов, что необходимо для построения графика. Для построения графика нужно формирование внутри цикла массива, обновляемого при каждой итерации цикла. Оптимальным способом организации таких массивов является использование двух сдвиговых регистров в цикле. В одном сдвиговом регистре формируется массив данных аргумента x, а в другому массив функции y. Сформированные массивы подаются на входные терминалы функции Bundle, а затем сформированный кластер из двух массивов подается на XY Graph (рисунок 63(b)).



a) b)

Рисунок 63. a) Ошибка с типом данных при попытке построения графика функцией XY Graph внутри цикла. b) Формирование двух массивов внутри цикла для функции XY Graph.

Сохранение данных

В ходе проведения реального эксперимента необходимо не только визуализировать получаемые данные в виде графиков, но и сохранять полученные массивы с данными. Наиболее просто сохранение в файл полученных данных можно реализовать с использованием функции Write To Spreadsheet File.vi, находящейся в пункте File I/O раздела Programming кодовой палитры. Название vi у функции указывает, что это не просто функция, а подпрограмма. При двойном нажатии на такую функцию открывается новая передняя панель LabView и можно перейти в соответствующую ей кодовую панель (рисунок 64). Функция Write To Spreadsheet File основана на функции Write Spreadsheet String которая в свою очередь также является подпрограммой (можно в нее зайти по двойному нажатию), основанной на функциях более низкого уровня, тиках как Open/Create/Replace, Set File Size и других.

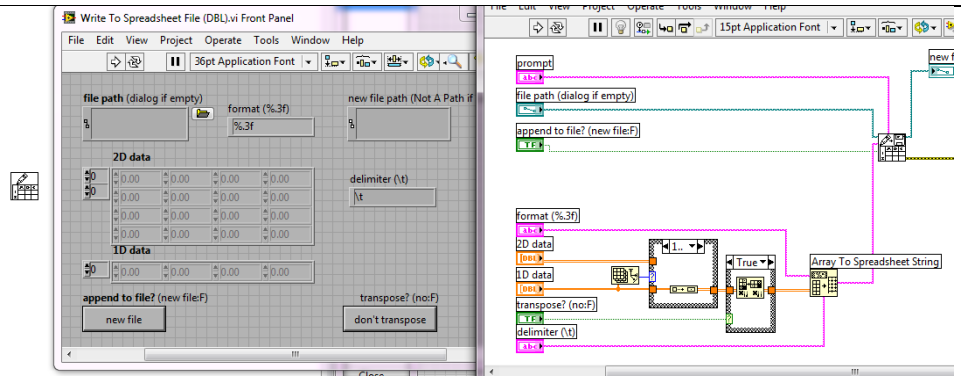
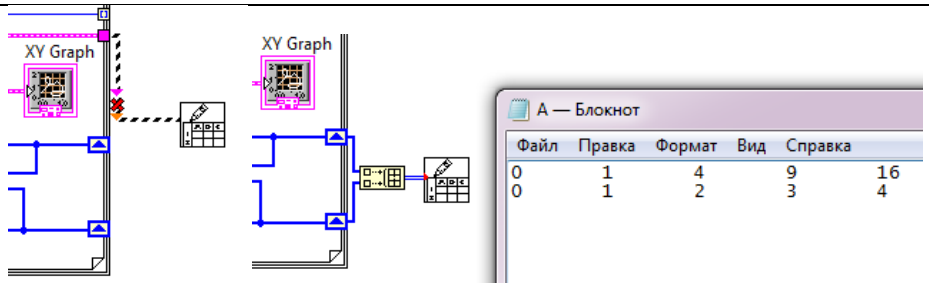


Рисунок 64. Функция-подпрограмма Write To Spreadsheet File в кодовой панели основной программы, ее лицевая и кодовая панели.

Можно создавать собственные подпрограммы на основе участков кода в кодовой панели, сохранять их и в дальнейшем использовать. Это позволяет уменьшить объем графического кода в кодовой панели.

Функция Write To Spreadsheet File имеет несколько входных терминалов. Основными входными терминалами являются терминалы 2D data и 1D data, на которые подаются двумерный и одномерный массивы, соответственно. В примере, представленном на рисунке 63(b), внутри цикла

For формировали два одномерных массива, которые объединяли функцией Bundle для отображения в графике XY Graph. Функция Bundle формирует кластер, и поток данных направляемый на функцию XY Graph нельзя направить на входной терминал 2D data функции Write To Spreadsheet File даже с учетом отключения индексации данных на границе цикла (рисунок 65(a)). Нужно использовать функцию Build Array для формирования двумерного массива из двух одномерных и двумерный массив соединить с терминалом 2D data функции Write To Spreadsheet File (рисунок 65(b)). Следует обращать внимание, когда одномерные массивы объединяются в кластер функцией Bundle, а когда в двумерный массив функцией Build Array. Число входных терминалов для функции Bundle и Build Array может быть увеличено растягиванием значка этих функций. После завершения участка кода программы (например, выполнения циклом For заданного числа итераций) до функции Write To Spreadsheet File (после того как на входные терминалы функции 2D data или 1D data придет массив данных) появится диалоговое окно Choose file to write в котором нужно будет ввести название сохраняемого файла и выбрать место сохранения. После открытия сохраненного файла в текстовом редакторе (например, Блокнот) можно увидеть сохраненные данные, строки в которых формируются подаваемыми одномерными массивами (если сформирован двумерный массив из двух одномерных) (рисунок 65(b).



a) b)

Рисунок 65. a) Подача на терминал 2D data функции Write To Spreadsheet File кластера их двух одномерных массивов. b) Подача на терминал 2D data двумерного массива из двух одномерных и результат в сохраненном файле.

Очевидно, что двумерный массив может состоять не только из двух одномерных массивов, а из любого их количества (рисунок 66(a)). Если одновременно подключить проводник с данными одномерного массива к терминалу 1D data и проводник с данными двумерного массива к терминалу 2D data, то в файл запишутся только данные из одномерного массива, т.к. на функцию Write To Spreadsheet File эти данные придут раньше, чем данные двумерного массива. Функция Write To Spreadsheet File имеет входной терминал transpose? логического типа. При подаче логического значения True на этот терминал, в выходной файл записывается транспонированная двумерная матрица (рисунок 66(b)).

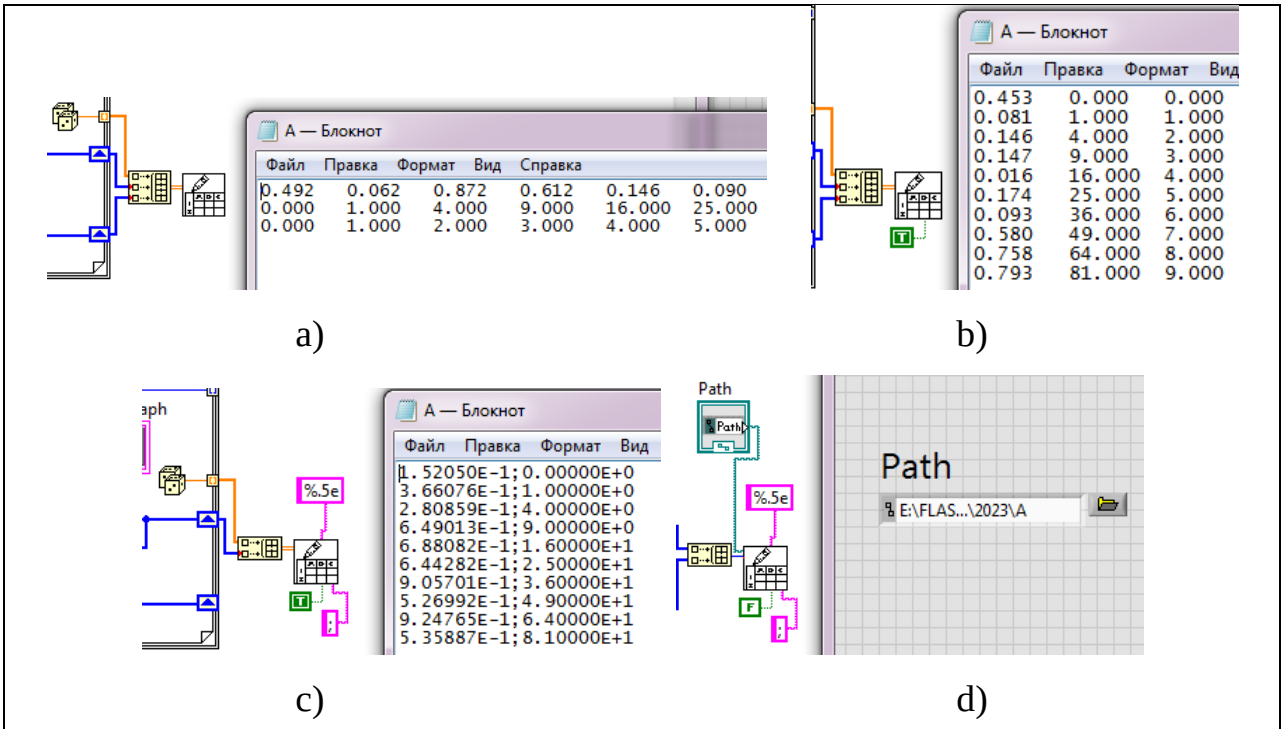


Рисунок 66. a) Сохранение в файл двумерного массива из трех одномерных. b) Транспонирование двумерной матрицы при записи в файл. c) Задание формата сохраняемых данных и разделителя между столбцами. d) Использование элемента интерфейса Path для задания пути к сохраняемому файлу и его имени.

Логическая константа True располагается в пункте Boolean кодовой палитры. Другим важным входным терминалом функции Write To

Spreadsheet File является терминал format для данных строкового типа. Данные строкового типа отмечаются в LabView красным цветом. Подача на терминал format строковой константы определенного вида будет задавать вид сохраняемых в файл чисел. Про строковый тип данных подробнее будет рассказано ниже. По умолчанию формат данных определяется командой %.3f. Детально про задание формата представления данных рассказано в справке среды LabView. Наиболее просто в этот раздел справки попасть по гиперссылке format string syntax в Help для функции Write To Spreadsheet File. Символ % означает начало команды по заданию формата данных. Символ f задает формат данных с плавающей точкой. Символом .3 задается три значащих цифры после десятичного разделителя. Если нужно увеличить число знаков после запятой, то нужно задать больше соответствующее число в этом параметре. Например, если подать на терминал format строковую константу %.5e, то получим сохраненные данные в формате чисел с плавающей точкой в экспоненциальном представлении, с 5 знаками после десятичного разделителя в мантиссе и использованием буквы E в качестве основания степени (рисунок 66(c)). Функция Write To Spreadsheet File имеет еще один входной терминал для данных строкового типа — терминал delimiter. На него подается строковая константа с символом, который будет выступать в сохраненной таблице с данными в качестве разделителя между столбцами или между элементами строк. На рисунке 66(c) в качестве разделителя между столбцами задан символ точка с запятой.

Путь к сохраняемому файлу и его имя можно задавать с лицевой панели до запуска функции Write To Spreadsheet File. В разделе String&Path палитры основной панели есть элемент интерфейса Path у которого есть стандартный значок для выбора пути к файлу. Значок элемента Path в кодовой панели имеет выходной терминал, который подключается к входному терминалу file path функции Write To Spreadsheet File (рисунок 66(d)). Элемент Path позволяет задать путь для сохранения файла и имя файла до запуска программы. Это может быть удобно когда запись в файл

производится в ходе выполнения программы несколько раз — т.е. когда происходит добавление данных в существующий файл по ходу выполнения программы.

Структура последовательности

Как указано выше, весь поток данных идет слева направо, но расположение элементов на кодовой панели не задает строгую последовательность их выполнения. Рассмотрим программу на рисунке рисунок 67(a). В цикле For заполняется массив X случайными числами, а правее цикла располагается функция Index Array которая будет извлекать из массива пятый элемент (нумерация элементов начинается с нулевого). Хоть выполнение программы идет слева на право, но после выполнения программы функция Index Array выдаст нулевое значение пятого элемента массива (хотя в данном примере пятый элемент заполненного массива не равен нулю, что видно в отображаемом на передней панели массиве).

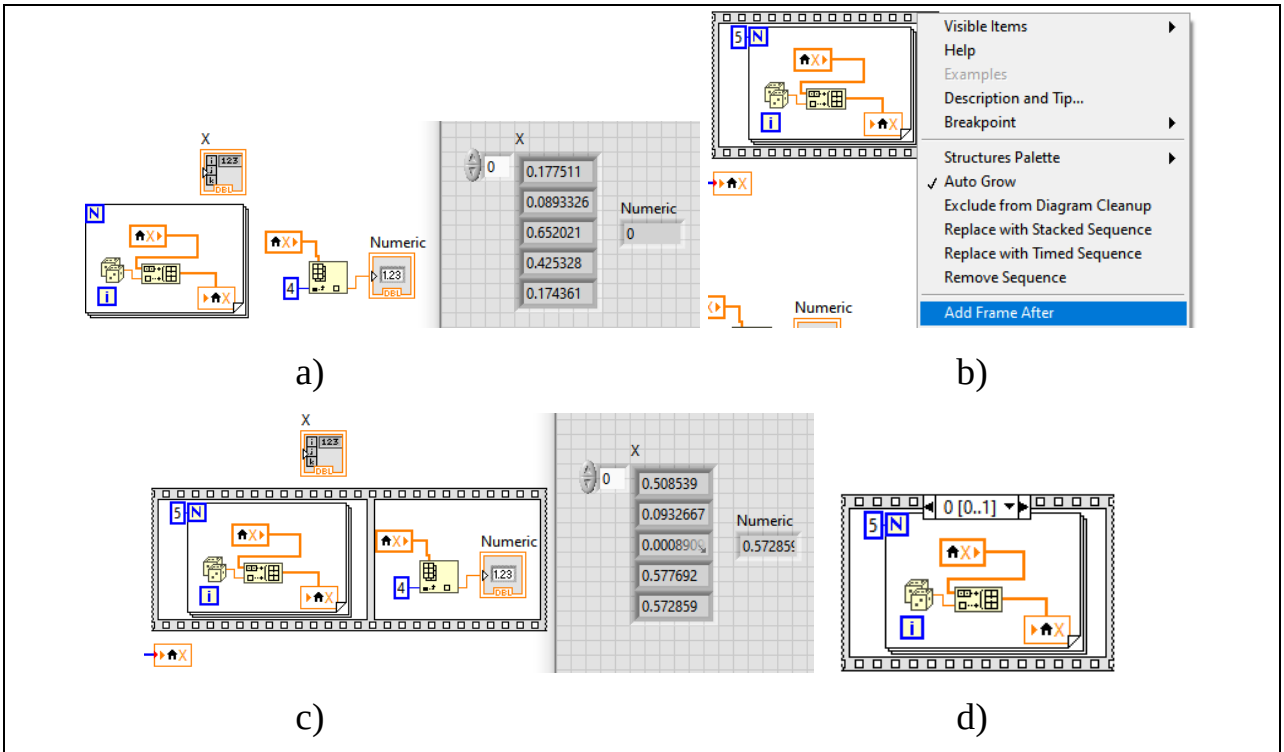
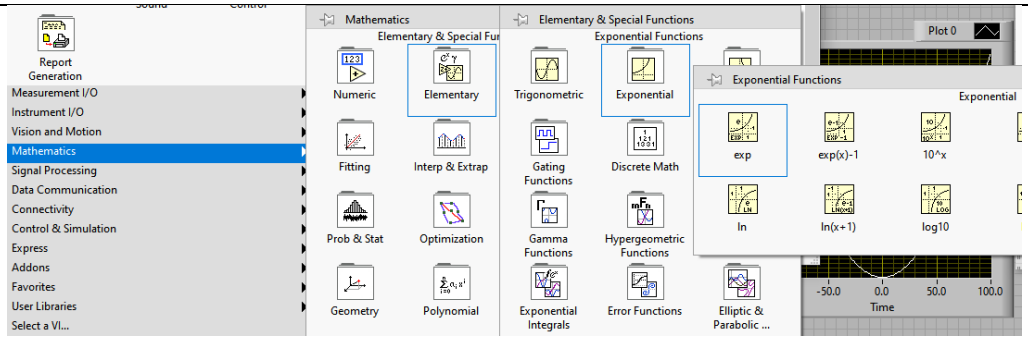


Рисунок 67. a) Неправильная работа программы по извлечению 5 элемента из массива. b) Использование последовательности Flat Sequence. c) Определение последовательности работы программы путем размещение кода в различных кадрах. d) Сворачивание Flat Sequence в Staked Sequence.

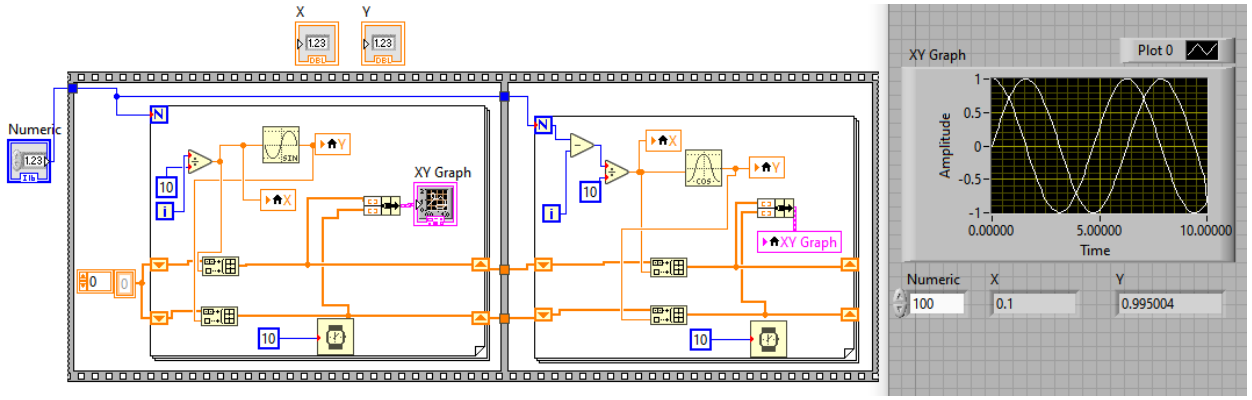
Неправильная работа программы связана с тем, что заполнение массива и извлечение элемента функцией Index Array будут выполняться одновременно, и в момент запуска функции Index Array пятый элемент в массив X еще добавлен не будет. Это наглядно видно при выполнении программы в режиме отладки (горит лампочка на основной инструментальной панели). Для задания однозначной последовательности выполнения действий в программе необходимо использовать структуру последовательности Flat Sequence. Эта функция находится в пункте Structures и имеет вид в виде кадров фотопленки. Выберем эту функцию и обведем курсором участок, который нужно добавит в первый кадр. В рассматриваемом примере обведем цикл, в котором происходит генерация массива. После нужно навести курсор на границу кадра, и нажать правую кнопку мыши. В появившимся контекстном меню нужно выбрать пункт Add Frame After, который добавляет новый кадр после существующего (рисунок 67(b)). В зависимости от области границы функции Flat Sequence, где вызвали контекстное меню можно добавит кадр до (Before) или после существующего (After), а также можно вставить новый кадр между существующими (Insert Frame) и объединить существующие кадры (Merge Frames). После появления нового пустого кадра нужно перенести в него участок кода, который должен выполнится только после выполнения кода в предыдущем кадре (рисунок 67(c)). Теперь после запуска программы функция Index Array запустится после завершения заполнения массива X всеми элементами и сможет правильно извлечь пятый элемент массива. В контекстном меню функции Flat Sequence есть пункт Replace With Staked Sequence. При выборе этого пункта все кадры последовательности будут свернуты в один кадр, а наверху получившегося кадра появится окошко с отображением номера текущего кадра и возможностью выбора кадра последовательности (рисунок 67(d)).

Использование структуры последовательности необходима в реальном эксперименте для задания последовательности взаимодействия с прибором.

В частности, при измерении вольт-амперной характеристики требуется задавать значение приложенного напряжения и делать развертку этого напряжения по величине (в том числе для разных полярностей напряжения). Например, нужно задать минимальное и максимальное напряжение, а также шаг по напряжению и направление изменения напряжения. Рассмотрим простую программу, которая будет изменять задаваемую величину от минимального до максимального значения и обратно. Пусть программа будет строить синус от целого числа, меняющегося от 0 до N, а затем строить косинус от целого числа, меняющегося от N до 0. Функции синуса и косинуса находятся в подпункте Trigonometric пункта Elementary раздела Mathematics контекстного меню кодовой панели (рисунок 68(a)).



a)

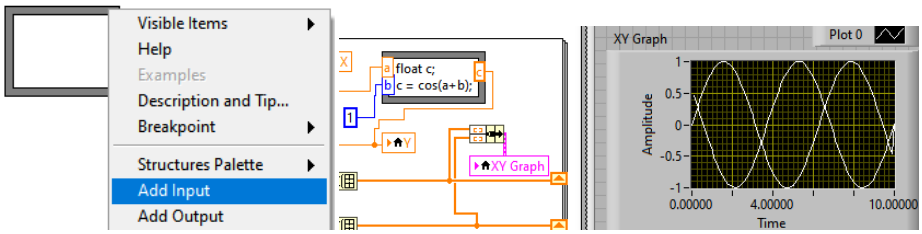


b)

Рисунок 68. a) Раздел Mathematics со встроенными математическими функциями. b) Пример использования плоской последовательности для последовательно увеличения и уменьшения аргумента.

Раздел Mathematics содержит большое количество встроенных функций из различных разделов математики (число функций может быть различным для разных версий LabView). На рисунке 68(b) представлена программа по построению синуса, увеличивающегося от 0 до N аргумента и косинуса, уменьшающегося от N до 0 аргумента. В качестве аргумента используется номер итерации цикла, разделенный на 10 (для уменьшения шага аргумента тригонометрической функции). Обратим внимание, что для одновременного отображения на одном графике построенного синуса и косинуса нужно данные добавлять в один массив, для этого выходные терминалы сдвиговых регистров цикла в первом кадре соединены с входными терминалами сдвиговых регистров цикла во втором кадре. Как упоминалось выше, в LabView можно использовать математические функции, представленные не только графическими значками (функциям), а также имеющие текстовый синтаксис, как обычных языках программирования. В частности, для задания функции текстовым образом можно использовать графическую функцию Expression Node (рисунки 17(c)) и 16). Для задания функции нескольких переменных в текстовом виде или введения части кода в текстовом виде используется функция Formula Node (находится в пункте Structures раздела Programming палитры кодовой панели). При помещении в кодовую панель функция Expression Node имеет вид прямоугольника без терминалов. Для добавления терминалов нужно в контекстном меню (появляется при нажатии правой кнопки мыши на границе функции) выбрать Add Input (входной терминал) и Add Output (выходной терминал). Можно добавлять несколько входных и выходных терминалов. Заменим в предыдущем примере функцию косинуса, представленную ранее в виде графического значка Cos, на функцию косинуса, задаваемую текстовым образом в функции Expression Node. Разместим во втором цикле (рисунок 69(a)) функцию Expression Node, добавим два входных терминала и один выходной. В терминалы необходимо ввести названия переменных. Назовем входные терминалы a и b, а выходной c. Подведем к верхнему входному

терминалу проводник, который раньше подходил к функции Cos, а к нижнему подведем любую числовую константу. Тип переменных, которые соответствуют входным терминалом, определяется данными, подаваемыми на эти терминалы. В примере на рисунке 69(b) на верхний терминал подается поток численных данных с плавающей точкой, поэтому верхний входной терминал желтый, а на нижней терминал подается целочисленная константа, поэтому нижний терминал синий. Особенностью функци Expression Node является то, что тип выходных данных нужно объявлять текстовым образом, как в обычных языках программирования. Справка по синтаксису может быть найден в Help для Expression Node (по гиперссылке Formula Node Syntax). Объявим, что переменная *c* является числовой с плавающей точкой (float). Запишем далее присвоение значение переменной *c*=cos(*a*+*b*). После запуска программы функция Expression Node будет производить вычисление косинуса задаваемого аргумента.



a) b)

Рисунок 69. a) Заготовок функции Expression Node. b) Вычисление косинуса в Expression Node.

Рассмотрим еще пример использования вложенных циклов и дополнительных элементов интерфейса на передней панели. Поместим внутри цикла While цикл For, который будет формировать массив значений косинуса для линейно возрастающего ряда чисел (задаваемого номером итерации цикла For деленным на 10). Для вычисления косинуса используем Expression Node из предыдущего примера. Переменная *b* будет фактически задавать сдвиг по фазе для вычисляемого косинуса. Разместим на передней панели элемент управления Horizontal Pointer Slide из пункта Numeric в

контекстном меню передней панели, кнопку Stop (пункт Boolean) и элемент отображения Waveform Graph (раздел Graph). Элемент управления задает значение числовой переменной в пределах, указанных на подписи к ползунку. Меняя пределы на подписи, задаем пределы изменения при крайних положения ползунка. Соединим значок функции Horizontal Pointer Slide в кодовой панели с входным терминалом *b* функции Expression Node (рисунок 70. Отметим, что значок функции Horizontal Pointer Slide нужно разместить в пределах цикла While. После запуска программы цикл While будет работать пока не нажмем кнопку Stop. На каждой итерации цикла будет заданное число раз запускаться вложенный цикл For и формировать массив значений косинуса. Данные из массива будут отображаться в Waveform Graph, находящемся в цикле For и обновляться при каждой его итерации. Меняя положения ползунка в Horizontal Pointer Slide, мы новое значение переменной *b* отправляем во вложенный цикл For, где строится массив данных функции косинуса со сдвигом по фазе относительно предыдущего значения переменной *b*. Это позволяет при работе программы наблюдать смещение графика косинусоиды по фазе.

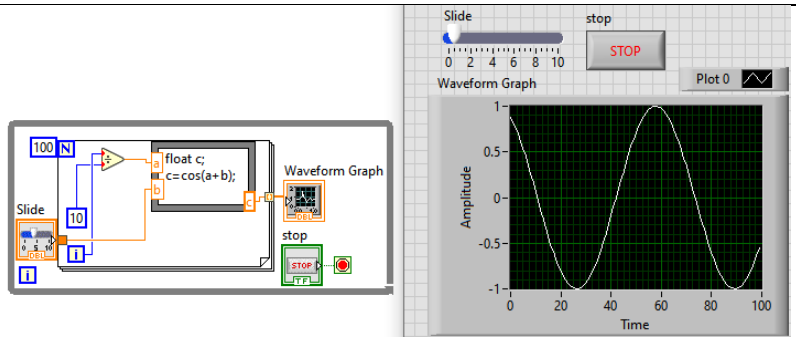


Рисунок 70. Смещение по фазе графика косинусоиды в реальном времени.

Структура выбора. Ветвление

Одним из основных и часто используемых элементов в любом языке программирования являются операторы ветвления, определяющие вариант работы участка программы после выполнения определенного условия. В большинстве языков программирования такими операторами являются

условный оператор If и оператор множественного ветвления Case. В LabView оператором ветвления является функция Case Structure (расположена в пункте Structures в кодовой палитре). Определенным дополнением к нему является функция Select (расположена в пункте Comparison в кодовой палитре).

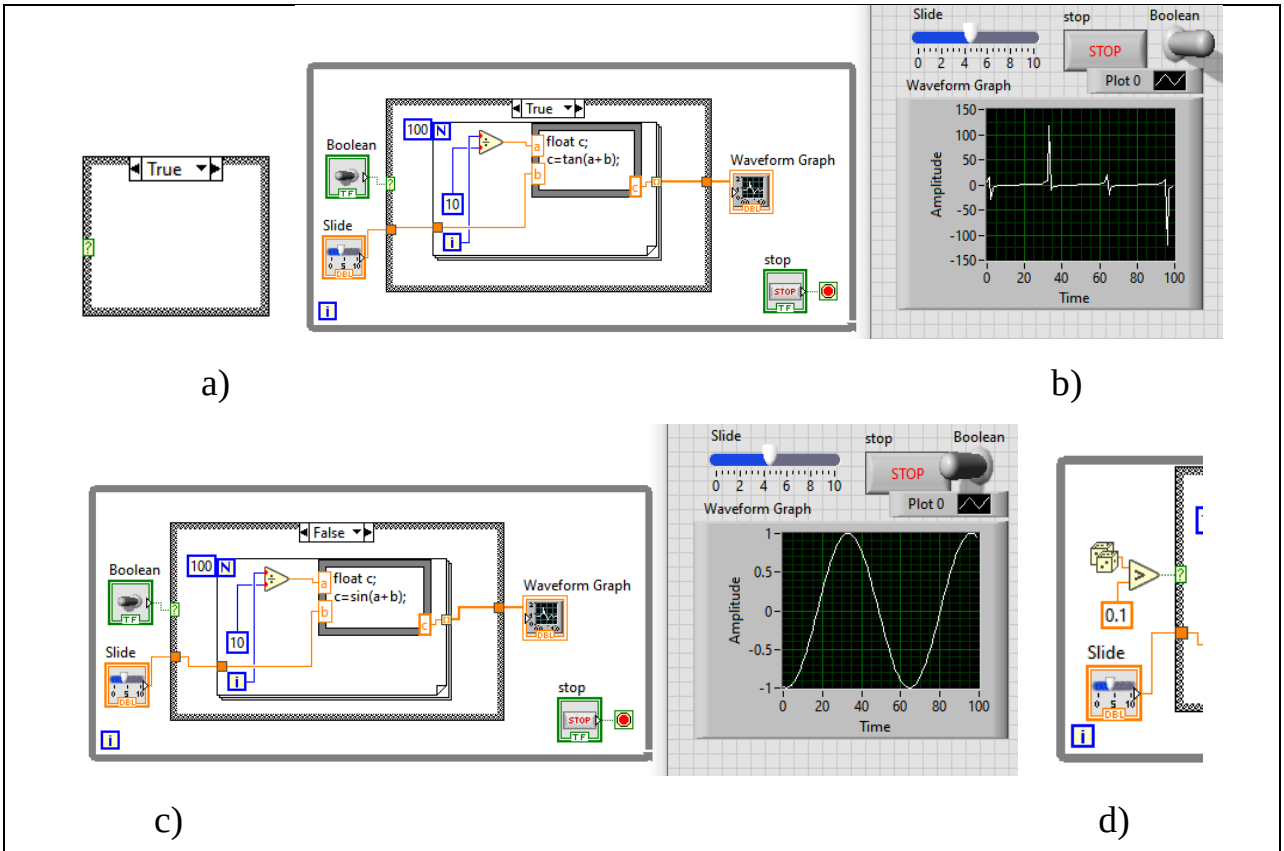


Рисунок 71. a) Пустая функцию Case Structure. b) Построение тангенса при реализации варианта True. c) Построение синуса при реализации варианта False. d) Действие сравнения, определяющее направление ветвления.

Выберем функцию Case Structure и разместим ее на передней панели. По умолчанию функция имеет входной терминал для логических данных и две вкладки – True и False (похожие на вкладки в структуре последовательности Staked Sequence, рисунок 119(d)). В таком виде функция выполняет роль условного оператора If. При подаче на входной терминал логической переменной True выполняется участок кода, расположенный во вкладке True, а при подаче переменной False выполняется вкладка False. Возьмем пример, рассмотренный на рисунке 70. Поместим внутри цикла

While функцию Case Structure (рисунок 71(a)). Во вкладке True разместим цикл For с заполнением массива функцией тангенса, а во вкладке False аналогичный цикл For, но с заполнением массива функцией синуса (очевидно, что для тригонометрических функций можно использовать не обязательно Expression Node, а функции из раздела Mathematics).Обратим внимание, что элемент управления Horizontal Pointer Slide и отображения Waveform Graph находятся вне Case Structure, а проводники, связанные с ними, не должны обрываться в каком-либо варианте Case Structure. Добавим на переднюю панель элемент управления логического типа Horizontal Toggle Switch (в виде тумблера) из раздела Boolean палитры передней панели. Этот элемент, как и кнопка Stop, дает на выходном терминала логическую переменную True или False (рисунок 71(b), 71(c)). При запуске программы можно переключать положение тумблера Horizontal Toggle Switch и тем самым переходить от построения косинуса к построению синуса. Очевидно, что логическая переменная, подающаяся на вход Case Structure, может генерироваться в результате действия сравнения, полностью аналогично условному оператору If в обычных языках программирования. На рисунке 71(d) показано, что происходит сравнение случайного числа с 0.1 и это действие определяет направление ветвления в Case Structure. Рассмотрим работу структуры выбора в режиме множественного выбора (ветвления). Удалим участок кода по формированию логической переменной, подаваемой на входной терминал Case Structure (рисунок 72(a)). Разместим на передней панели элемент управления Ring из пункта Ring&Enum палитры передней панели. В кодовой панели разместим значок функции выбора варианта Ring внутри цикла While и соединим выходной терминал функции с входным терминалом Case Structure. Входной терминал станет синим, что соответствует целочисленным данным, а вкладки Case Structure получат названия 0, Default и 1 вместо True и False (рисунок 72(b)). Выберем вкладку 1, нажмем на границе Case Structure правую кнопку мыши и в контекстном меню выберем пункт Add Case After, который добавит новый случай с

именем 2. Разместим в этом случае участок, аналогичный случаям 0 и 1, но с построением, например, функции экспоненты. Теперь функция Case Structure имеет три случая т.е. три варианта ветвления программы.

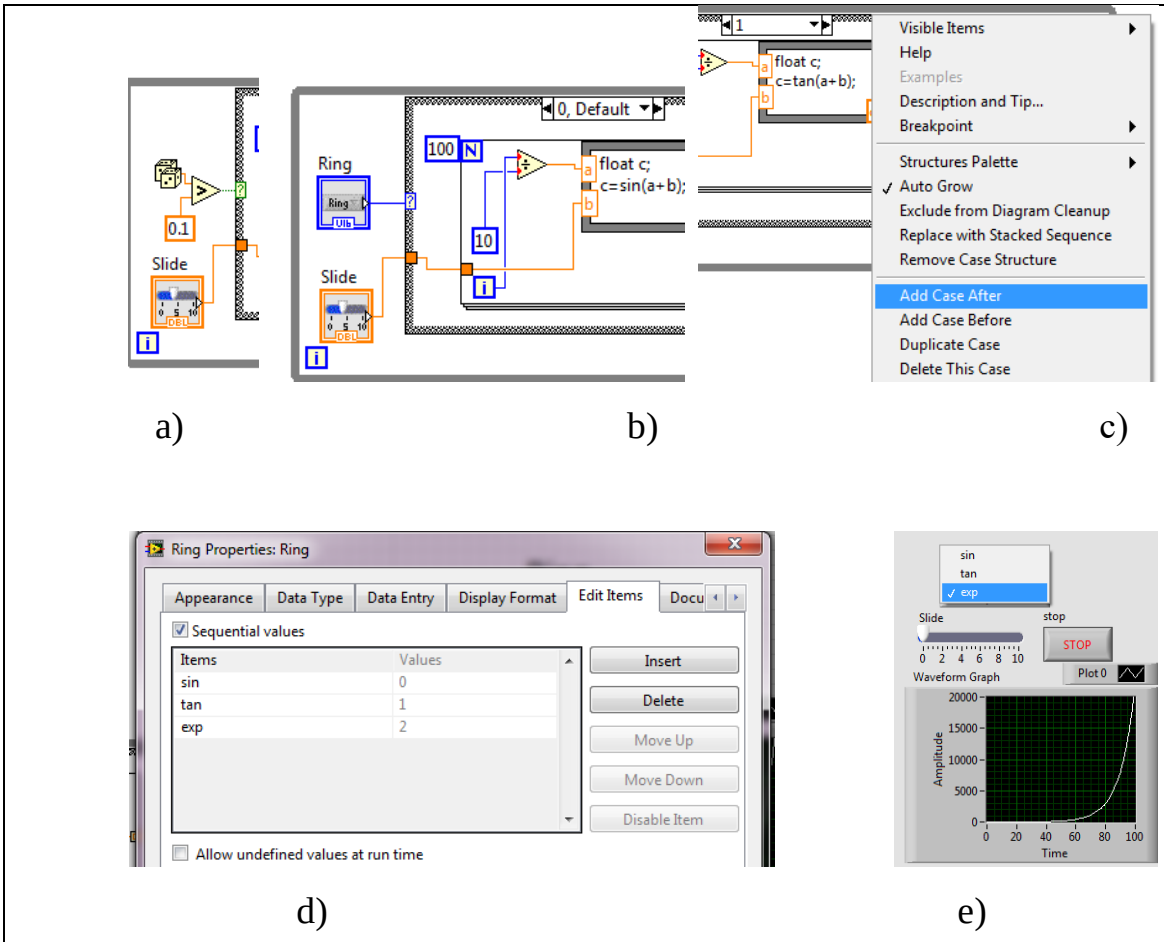
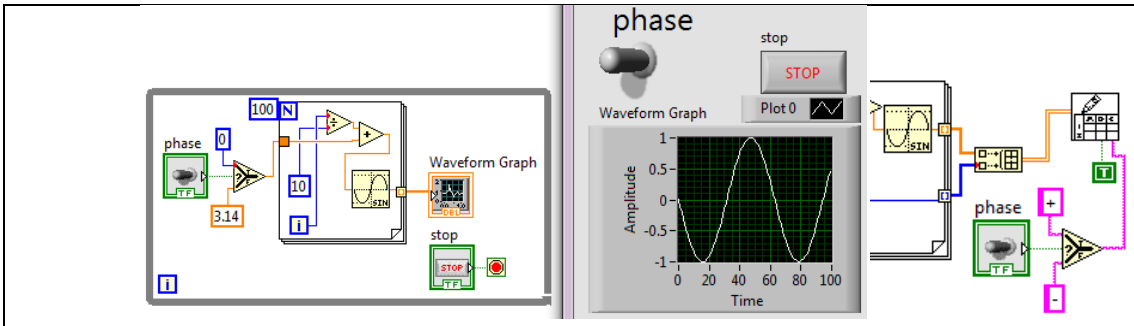


Рисунок 72. a) Проверка условия для генерации логической переменной, подаваемой на вход Case Structure. b) Подача целочисленного значения на входной терминал Case Structure. c) Добавление нового варианта события в Case Structure. d) Задание названия случая и его номер в функции выбора варианта Ring. e) Отображение вариантов в Ring.

На передней панели в контекстном меню для функции Ring выберем пункт Properties и в открывшемся окне выберем пункт Edit Item (рисунок 72(c)). В этом пункте можно задать число выдаваемых функцией Ring целочисленных значений (начиная от 0) и их названия. Введем в столбце Items название sin для выдаваемой константы Value равной 0 и нажнем клавишу Enter. Появится вторая строка для Value=1, которую назовем tan, и

далее создадим случай с названием exp для Value=2 (рисунок 72(d)). Теперь при запуске программы можно выбрать в меню Ring название нужной функции и произойдет ее построение т.е. произойдет выбор нужного варианта в Case Structure (рисунок 72(e)). Очевидно, что в Case Structure можно создавать вариантов больше, чем три. Один из вариантов обязательно должен в названии иметь слово Default через запятую после номера варианта. В ряде случаев ветвление в программе может быть просто организовано функцией Select (расположена в пункте Comparison в кодовой палитре). В кодовой панели значок функции имеет вид треугольника с тремя входными терминалами и одним выходным. В центральный входной терминал подается логическая переменная. Если подается True, что на выходном терминале появляется переменная с верхнего входного терминала, если подается False, то на выход подается переменная с нижнего терминала (рисунок 73(a)). Создадим программу по построению синуса с использованием цикла For внутри цикла While (аналогично как в рассмотренных выше примерах). Будем в реальном времени менять фазу синусоиды на π/2. Разместим на передней панели логический тумблер Horizontal Toggle Switch и соединим его с центральным терминалом функции Select. На верхний терминал Select подадим константу 0 радиан, а на нижний 3.14 радиан (аргумент тригонометрических функций задается в радианах), а выход из Select будем прибавлять к аргументу функции синуса (рисунок 73(a)). После запуска программы переключением тумблера фаза синусоиды в окне Waveform Graph будет меняться на π/2. В данном примере функция Select позволяет обойтись без применения структуры выбора Case Structure. Функция Select работает с любым типом данных. На рисунке 73(b) функция Select используется для передачи строковой константы в терминал delimiter функции Write To Spreadsheet File для задания разделителя между столбцами в сохраняемом файле.



a) b)

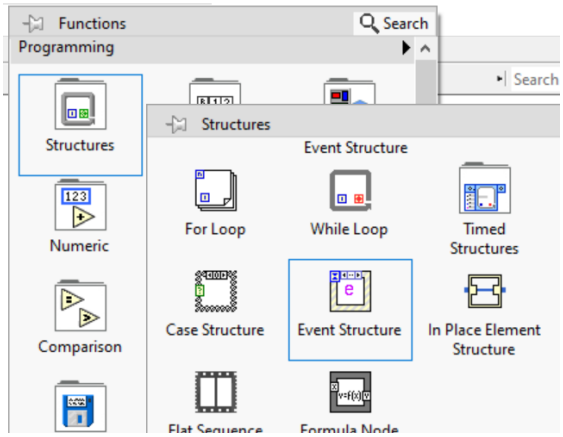
Рисунок 73. Использование функции Select для выбора в программа варианта числовой константы (a)) и строковой константы (b)).

Структура события

В разделе Structures подпалитры Programming, содержится ещё одна полезная для создания графического кода структура, которая называется ***Event Structure*** (рис.74(a)). Стоит отметить, что эта структура всегда используется с циклом While, независимое её использование невозможно, так как это никак не отразится на работе программы. ***Event Structure*** – это программная конструкция, которая ожидает наступления какого-то события на лицевой панели (например, нажатие кнопки), чтобы выполнить программный код, который выполняет соответствующий запрос для обработки этого события. Когда никаких событий не происходит, программа находится в режиме ожидания события, при этом операционная система дает возможность работать другим программам. Реакция программы на наступление какого-либо события значительно быстрее, чем в случае постоянного опроса всех элементов лицевой панели. Такой вид программирования называется программирование типа "событие-запуск" (Event Driving).

Event Structure также, как и ***Case Structure*** содержит множество имеет одну или несколько поддиаграмм, или случаев событий, которые создаются непосредственно самим пользователем. При этом во время наступления события выполняется ровно одна из этих поддиаграмм. Вы можете настроить любое количество структур событий для реагирования на одно и то же

событие. Структура ***Event Structure*** также похожа на функцию ***Wait***, так как во время исполнения LabVIEWструктуры ***Event Structure*** код программы не исполняется до тех пор, пока не наступит нужное событие.



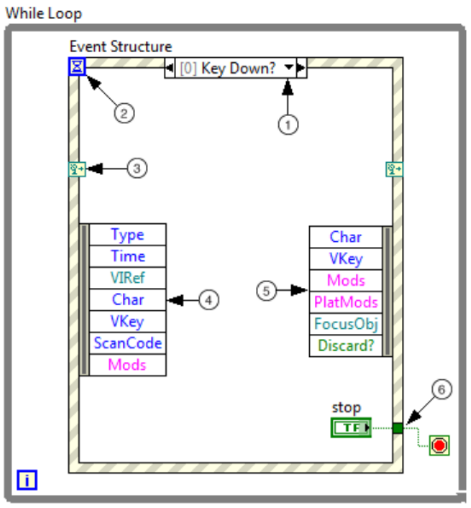


Рисунок 74. a) Расположение функции Event Structure в палитре функций Function Palette. b) Основные части структуры событий Event Structure и их вид на блок-диаграмме.

Теперь опишем основные части ***Event Structure*** и их функциональное предназначение (рис. 74(b)):

1. Метка выбора событий указывает, какие события вызывают выполнение отображаемого в данный момент обращения. Чтобы просмотреть другие случаи событий, щелкните стрелку вниз рядом с названием случая.

2. Терминал тайм-аута определяет количество миллисекунд, в течение которых необходимо дождаться события перед истечением времени ожидания. Если вы передаете значение на терминал тайм-аута, вы должны указать случай события тайм-аута, чтобы избежать ошибки.

3. Терминалы динамических событий, принимают регистрационный номер события или группу регистрационных номеров событий для динамической регистрации событий. Если вы подключите внутреннюю правую клемму, эта клемма больше не будет передавать те же данные, что и

левая клемма. Вы можете подключить регистрационный номер события или кластер регистрационных номеров событий к внутреннему правому терминалу через функцию Регистрации событий и динамически изменять событие. В зависимости от палитры, из которой вы выбираете структуру событий, терминалы динамических событий могут не отображаться по умолчанию. Чтобы отобразить эти терминалы, щелкните правой кнопкой мыши структуру событий и выберите в контекстном меню пункт – Показать динамические терминалы событий (***Show Dynamic Event Terminals***).

4. Узел данных события (***Event Data Node***) идентифицирует данные, которые LabVIEW возвращает при возникновении события. Вы можете изменить размер узла по вертикали (растягивать узел) и выбрать нужные вам элементы. Используйте узел ***Event Data Node*** для доступа к элементам данных события, таким как тип и время, которые являются общими для всех событий. Вы можете показать или скрыть узел данных события для случая, когда все элементы данных не используются. Щелкните правой кнопкой мыши на границе структуры и выберите ***Visible Items → Event Data Node for this Case***. Вы не можете скрыть узел, когда он подключен к сети или находится внутри регистра событий.

5. Узел Фильтра событий (***Event Filter Node***) определяет данные о событии, которые вы можете изменить, прежде чем эти данные будут обработаны программой. Этот узел отображается в случаях структуры событий, которые обрабатывают события фильтра (более подробно рассмотрим ниже). Если вы хотите изменить данные о событии, вы можете подключить и изменить элементы данных из узла Данных о событии к узлу Фильтра событий. Вы также можете изменить данные события, подключив новые значения к терминалам узла. Чтобы полностью отменить событие, передайте в ***Discard*** значение ***TRUE?*** терминал. Если вы не привязываете значение к элементу данных узла Фильтра событий, этот элемент данных остается неизменным.

6. Как и в других структурах, в ***Event Structure*** поддерживает туннели (прохождение данных через границу структуры). Однако в этой структуре вам не обязательно подключать выходные туннели структуры событий в каждом случае, при этом программа не будет выдавать ошибку. Все неподключенные туннели используют значение по умолчанию для типа данных туннеля. Щелкните правой кнопкой мыши по туннелю и в контекстном меню снимите флажок со строчки “Использовать по умолчанию“ (***Use Default If Unwire***), если он не подключен в контекстном меню, чтобы вернуться к поведению структуры по умолчанию, при котором туннели должны быть подключены во всех случаях.

В целом для создания базовых программ не требуется использование всех частей структуры ***Event Structure***. Достаточно использовать только метку выбора событий и, при необходимости, терминал тайм-аута. Зачастую использование узлов ***Event Data Node*** и ***Event Filter Node***не требуется для реализации не сложных программных кодов. Для наилучшего понимания рассмотрим принцип работы структуры на основе примера. Сначала создадим базовую конструкцию из ***Event Structure*** и цикла While, которая представлена на рисунке 74(b) (***Event Structure*** находится внутри цикла While). После создания, в структуре ***Event Structure*** уже будет находится одно событие с названием Timeout (расположено в метке событий). Это событие всегда будет автоматически создаваться при помещении структуры ***Event Structure*** на блок-диаграмму. В зависимости от задачи, которую вам необходимо запрограммировать, вы можете как использовать это событиеTimeout, так и удалить его. Для чего же нужно это событие? Событие ***Timeout*** – это событие, которое выполняется в момент, когда не происходит никаких других событий, которые вызывают выполнение части программного кода, относящийся к ним. Иными словами, весь программный код, который вы поместите в событие ***Timeout,*** будет всё время выполняться, если на лицевой панели не происходит никаких событий, вызывающих выполнение определённого случая события. Стоит отметить, что выполнение

события Timeout происходит не сразу же после того, как закончили выполнятся другие события, а только после того, как закончится время ожидания уведомления о событии, которое мы разбирали в пункте 2 (терминал Timeout). Время ожидание события, которое установлено по умолчанию равное -1, которое означает, что время ожидания события бесконечно (или время ожидания отсутствует). Иными словами, если значение Timeout равно -1, значит, что программный код, относящийся к событию Timeout никогда не будет исполнен, так как время ожидания событий, происходящих на лицевой панели, никогда не закончится. Однако значения времени ожидания можно указывать и самостоятельно. Для этого отправьте значение времени (в миллисекундах) на терминал Timeoutв левом верхнем углу структуры событий, в течение которых структура событий ожидает события. Для примера можно написать значение 600 мс (рис. 75(a)). Теперь разберёмся как удалить событие Timeout, если оно вам не требуется. Правым щелчком мыши на панели метка события откройте контекстное меню и выберите в нём строчку ***Edit Events Handled by This Case*** (изменить события, обрабатываемые в данном случае) (рис. 75(a)). Перед вами откроется диалоговое ***окно Edit Events***, в котором находится три отдельных окна. Рассмотрим, как устроен интерфейс этого диалогового окна, для понимания работы по созданию и удалению событий (рис. 75(c)). В верхней части диалогового окна расположена строчка с названием случая для которого отображается информация в окне редактирования. Нажатием левой кнопкой мышки на случай вы можете развернуть эту строчку и увидеть другие случаи, которые вы создали в вашей ***Event Structure***. Если вы нажмёте на другой случай, то в окне редактирования будет отображаться информация, относящаяся к выбранному случаю. Теперь рассмотрим содержимое трёх окон. Первое окно с названием ***Event Specifiers*** – это окно, в котором содержится информация о событиях, вызывающих выполнение этого случая. В первом столбце Event Source отображаются источники, с которыми должны произойти какие-то события, для того чтобы выполнился

программный код, относящийся к данному случаю. Во втором столбце указано конкретное событие (действие), которое должно произойти с конкретным источником для того, чтобы запустилось выполнение программного кода, относящегося к этому случаю. Сейчас у нас в этом столбике расположено только событие Timeout. Под этим окошком расположено две кнопки: **Add Event**, **Remove**. Кнопка **Add Event** позволяет создать новый источник и событие, которое будет запускать выполнение этого случая. То есть для того, чтобы произошло выполнение какого-то определённого случая, можно создать несколько источников и событий, которые будут вызывать выполнение этого случая. Не обязательно должно происходить только одно событие для запуска выполнения какого-то одного случая. Вторая кнопка **Remove** позволяет удалять лишние или случайно созданные события, которые запускают выполнение этого случая. Для этого нужно выбрать щелчком левой кнопкой мышки нужное событие и нажать на кнопку **Remove.** Именно так и происходит удаление ненужного вам события Timeout, вместо которого вы можете создать любое другое событие, которое вам потребуется.

Теперь рассмотрим второе окно под названием ***Event Sources***, в котором содержится список допустимых источников, которые можно выбрать для создания какого-то события. В этом окне есть источники, которые установлены по умолчанию: Application (приложения), This VI (этот ВП), Pane (панель). Эти источники событий вы можете выбрать всегда независимо от того, что расположено у вас на лицевой панели. Далее в источниках событий находится раздел с названием Control (Элементы управления), который включает в себя список элементов управления, которые вы расположили на лицевой панели. Если на лицевой панели у вас нет никаких элементов управления, то раздел Control в окне будет пустым. Элементы управления, помещённые на лицевую панель, автоматически будут появляться в разделе Control и вы сможете выбрать их в качестве источника для создания события.

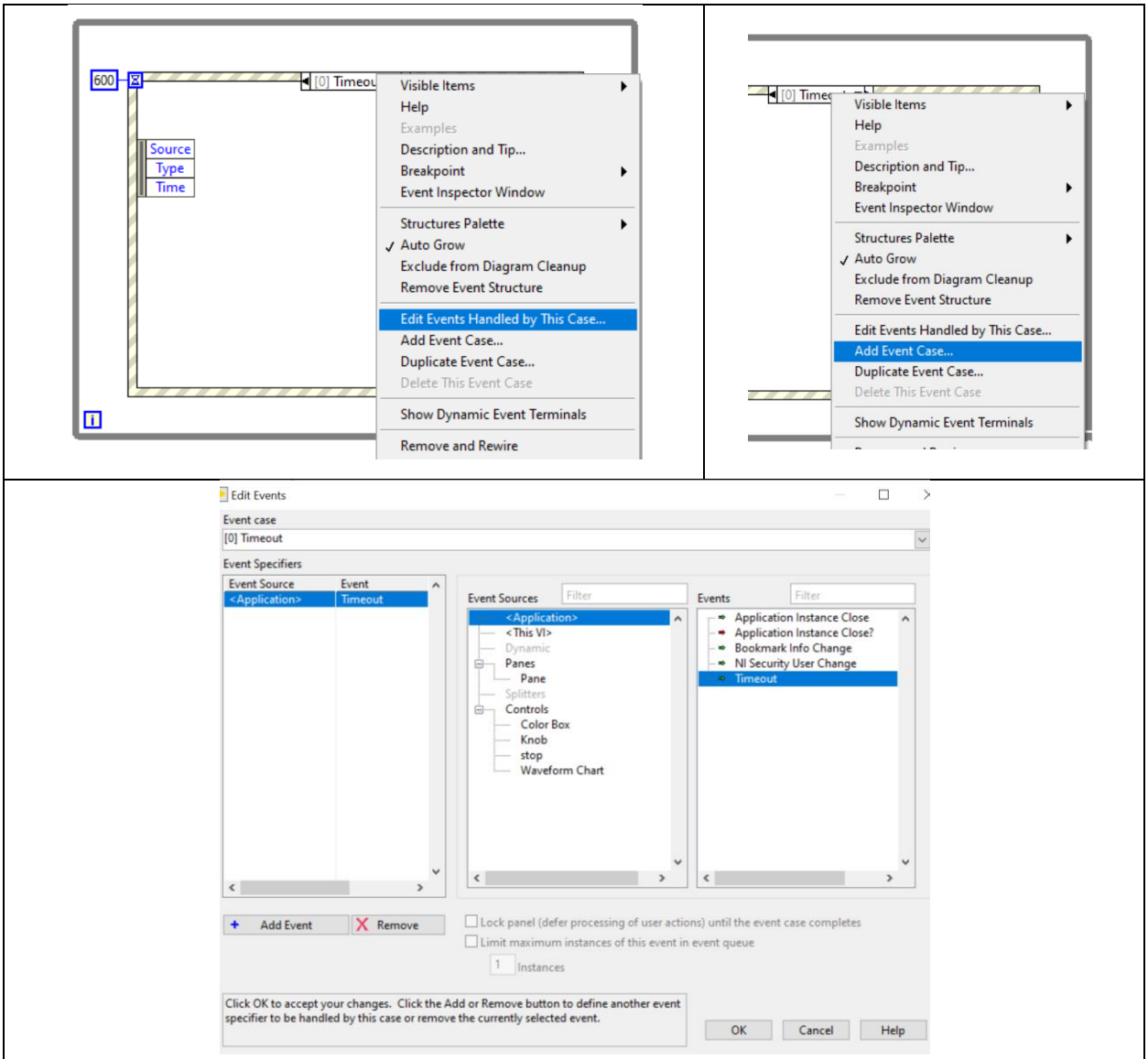


Рисунок 75 a) Кнопка редактирования событий в плавающем контекстном меню. b) Кнопка добавления нового события в плавающем контекстном меню. c) Диалоговое окно для редактирования и создания событий для Event Structure.

Третье окно под названием ***Events*** (события) содержит все варианты событий, которые можно выбрать по отношению к определённому источнику

событий. То есть вы выбираете какой-то источник во втором окне, и все возможные события для этого источника отображаются в третьем окне. Как вы можете заметить у каждого события (действия) слева от его названия находится стрелочка либо красного, либо зелёного цвета. Не случайно цвета стрелок имеют два разных цвета, определённый цвет события зависит от того, к какому типу события оно относится. LabView классифицирует события пользовательского интерфейса на два разных типа событий: уведомлять (***Notify Events***) и фильтровать (***Filter Events***). ***Notify Events***–это события уведомления, которые информируют вас о том, что произошло действие на лицевой панели, например, была нажата кнопка. В простейшем случае написания программ достаточно использовать только события уведомления, так как они позволяют закрыть большинство поставленных задач, которые требуется решать в случае создания программного кода для проведения эксперимента. Эти события обозначены зелёной стрелочкой, поэтому для создания событий, которые будут происходить за счёт изменения пользователем значения элемента управления на лицевой панели, используйте события ***Notify Events***.Когда вы выбираете события уведомления на левой границе ***Event Structure*** появляется дополнительный элемент ***Event Data Node***, описание которого мы рассмотрели выше. ***Filter Events*** – это события, которые позволяют вам проверять или изменять данные события до того, как LabView выполнит действие, связанное с этим событием. Вы также можете полностью отменить событие, чтобы изменение не повлияло на VI. События фильтра обозначаются красными стрелочками и заканчиваются вопросительным знаком, чтобы помочь вам отличить их от событий уведомления. Более подробно описывать на примере этот тип событий мы не будем, так как он используется только в очень сложных программных кодах. В случае выбора события ***Filter Events*** в структуре ***Event Structure*** помимо узла ***Event Data Node*** добавляется узел ***Event Filter Node***, который расположен на правой границе структуры.

Теперь попробуем создать программный код и для начала добавим кнопку на лицевую панель, чтобы создать для неё случай и событие. Далее открываем блок-диаграмму, щёлкаем правой кнопкой по метке событий, и в контекстном меню выбираем строку ***Add Event Case***, который позволяет создать новый случай в структуре (рис. 75(b)). После нажатия перед вами откроется диалоговое окно редактирования, в котором мы будем создавать событие для нашей кнопки. Теперь во втором окне выбираем в качестве источника кнопку stopи в третьем окне для примера выберем событие Value Change (изменение значения), в результате у вас должен получится выбор, который представлен на рисунке 76(a). После чего нажимаем кнопку ОК в окне редактирования, это окно закрывается, а на блок диаграмме появляется новый случай, в названии которого указан источник и событие. Теперь поместим терминал кнопки на блок-диаграммы внутрь нашего созданного случая и соединим выходной терминал кнопки с кнопкой остановки работы цикла (рис. 76(b)). Это кнопка stop будет служить кнопкой остановки программы (работы цикла While). То есть мы создали событие – нажатие на кнопку stop, которое запускает программный код, который останавливает работу программы. При желании можете запустить программу и проверить её работу. Также для корректности работы программы измените режим работы кнопки на Latch When Released, расположенную в контекстном меню кнопки (рис. 76(c)).

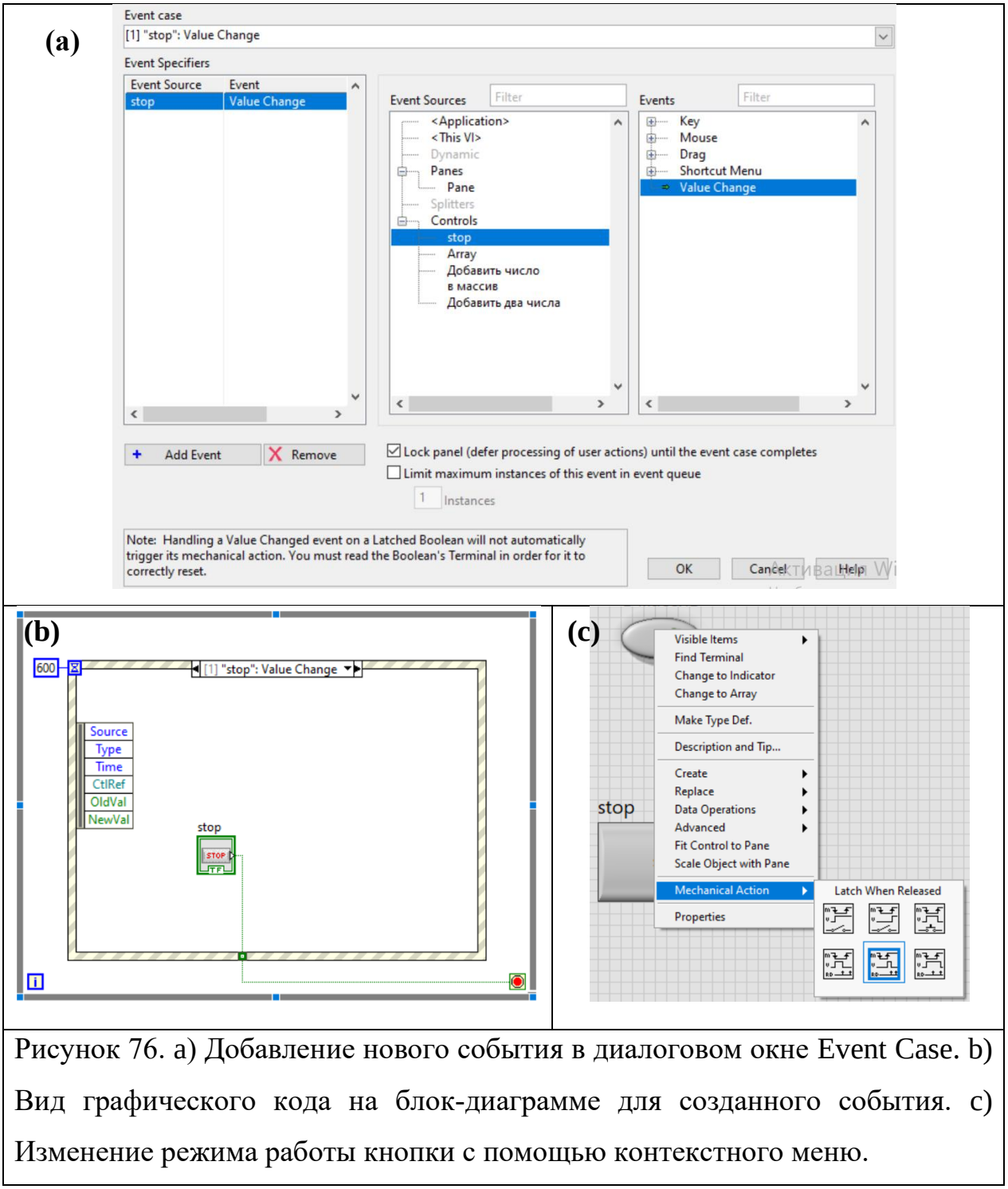


Рисунок 76. a) Добавление нового события в диалоговом окне Event Case. b) Вид графического кода на блок-диаграмме для созданного события. c) Изменение режима работы кнопки с помощью контекстного меню.

Теперь создадим ещё одну кнопку на лицевой панели и один пустой массив. Допустим мы хотим, чтобы при нажатии на кнопку в массив добавлялось одно число. Поэтому переименуем нашу кнопку на “Добавить число в массив”. Теперь опять выполним все те же действия, что и для кнопки stop, и создадим новый случай для созданной кнопки. Только вместо источника stop, выбираем нашу кнопку “Добавить число в массив”, которая

автоматически появляется в списке источников в разделе Controls. После того, как мы создали новый случай, открываем его на блок-диаграмме и создаём внутри этого случая графический код, который представлен на рисунке 77(a). В качестве чисел, которые мы будем добавлять в массив выберем числа, которые будет создавать генератор случайных чисел. Сам терминал кнопки “Добавить число в массив” на блок-диаграмме также помещаем внутри программного кода этого случая. При этом выходной терминал кнопки не надо ни к чему присоединять, кнопка будет реагировать на нажатия и выполнять программный код без какого-либо подсоединения, главное не забывать помещать терминал кнопки в соответствующий случай. Теперь запустите программный код и попробуйте нажимать на кнопку, после каждого нажатия на кнопку в массиве будет появляться одно новое случайное число (рис. 77(b)). В массиве будет находиться ровно столько чисел, сколько раз вы нажмёте на кнопку. Если вы нажмёте на кнопку stop, то программный код прекратит свою работу.

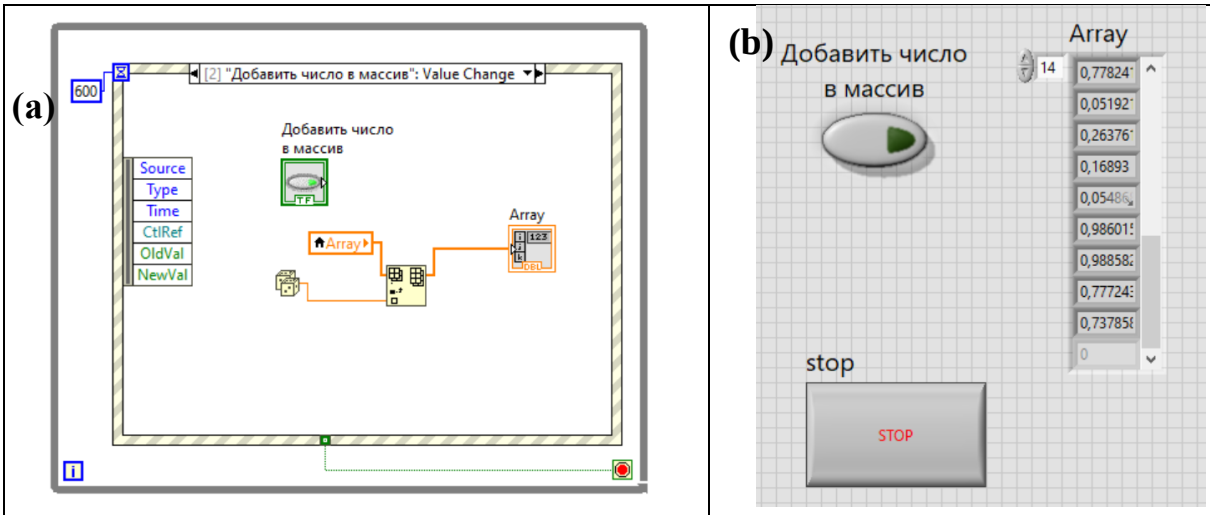


Рисунок 77. a) Вид графического кода на блок-диаграмме для события “Добавить число в массив”. b) Отображение результата выполнения события “Добавить число в массив” на лицевой панели.

Теперь для наглядности добавим ещё одну кнопку, которая будет добавлять в уже созданный массив два числа, а не одно, как предыдущая кнопка. Переименуем её как “Добавить два числа”. Также, как и до этого,

создаём новый случай, в который помещаем терминал нашей кнопки. Не забываем, что при создании нового случая в качестве источника выбираем новую созданную кнопку “Добавить два числа”. Графический код создадим такой же, как и в предыдущем случае, только весь код (кроме терминала кнопки) поместим в цикл For, который будет выполняться два раза, чтобы помещать в массив два числа (рис. 78(a)). Теперь запустим наш программный код и попробуем нажимать на кнопки. Если мы нажмём на кнопку “Добавить два числа”, то увидим, что в массиве появилось два новых числа. Если нажмём на кнопку “Добавить число в массив”, то в массив добавляется одно число. То есть когда мы выбираем на какую кнопку хотим нажать, мы выбираем какое-то событие (рис. 78(b)). После того, как мы нажали на выбранную кнопку (выбрали событие) происходит выполнение случая, который привязан к этому событию. То есть мы выбираем, какую часть программного кода нам нужно выполнить и в соответствии с этим случаем создаём наше событие (нажатие на кнопку).

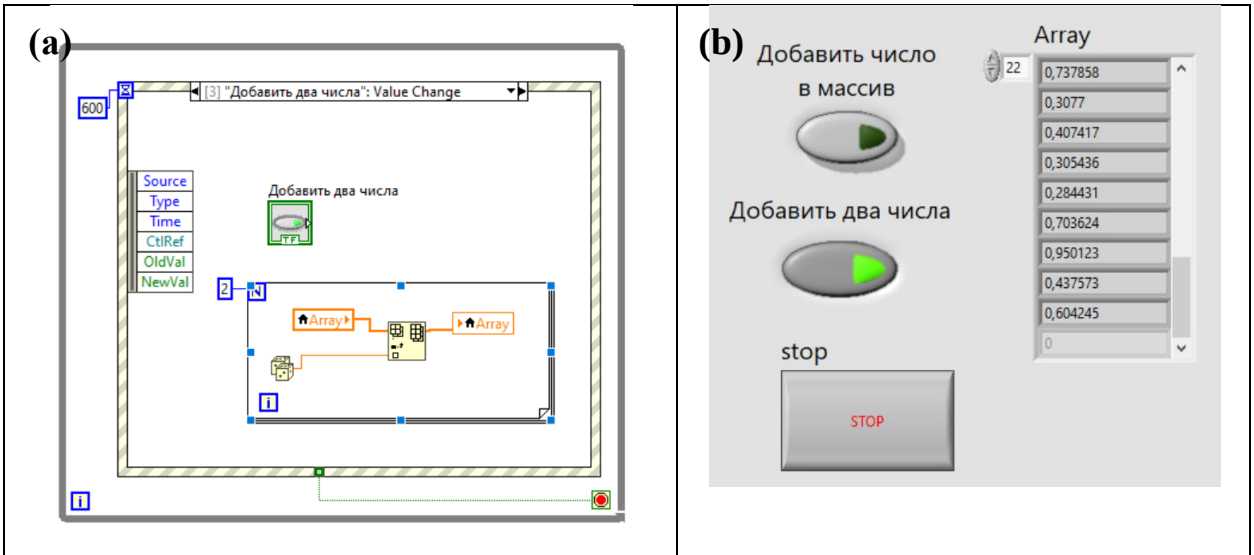


Рисунок 78. a) Вид графического кода на блок-диаграмме для события “Добавить два числа”. b) Отображение результата выполнения события “Добавить два числа” на лицевой панели.

Таким образом мы осуществляем выбор событий, который отвечает за дальнейший ход программы и позволяет выбирать нужный нам программный

код. В этом и заключается суть использования структуры событий ***Event Structure***.

4. Графики и кластеры (расширенные функции)

Среда LabView позволяет создавать пользователям программы, которые бы осуществляет сбор данных и управление измерительным прибором, который используется в ходе эксперимента. Однако не редко требуется не только сбор данных, но и их наглядное графическое отображение, которое поможет определить правильность и корректность проведения эксперимента, а также сделать некоторое первостепенные выводы и рассуждения об изучаемом объекте. Поэтому графическое отображение данных является очень важно и неотъемлемой частью большого количества экспериментов. Для отображения данных в среде LabView используются различные графики, которые вы можете поместить на лицевую панель вашего подприбора. Всего существует 9 вариантов графиков, которые вы можете найти в палитре ***Controls Pallete → Modern → Graph***. Рассмотрим каждый вид графика, их интерфейс и случаи использования более подробно.

Осциллограммы

Первый вид графика в подпалитре это ***Waveform Chart*** (развёртка осциллограммы). Также, как и у обычных графиков, график ***Waveform Chart*** имеет две оси: горизонтальную (ось X), вертикальную (ось Y). По оси Y откладываются данные, которые вы измеряет в ходе эксперимента, а по оси X откладывается время. То есть график развёртки осциллограмм позволяет отображать временные зависимости измеряемых величин в самом процессе измерения. Возникает вопрос откуда же берутся значения времени для построения графика. Для начала представим процесс сбора данных. Когда мы запускаем нашу программу, измерительный прибор начинает посылать на наш компьютер измеренные данные, сначала одно значение, потом второе, третье и так далее. То есть данные поступают постепенно, одно значение за другим. Если мы напишем программный код без цикла, то в компьютер

поступит только одно измеренное значение, которое мы запросили у прибора. Чтобы осуществить непрерывный поток поступающий данных, нам нужно создать цикл, который будет запрашивать измеренные прибором значения. То есть мы запускаем программу и в первой итерации цикла запрашиваем у прибора измеренное значение, прибор присылает нам его. Потом наступает вторая итерация цикла, мы снова запрашиваем значение и получаем его, так будет продолжаться с каждой итерацией цикла. То есть за одну итерацию цикла мы будем получать одно измеренное значение. Поэтому в качестве времени в графике развёртки осциллограмм используется номер итерации цикла. Для наглядного примера напишем графический код, где в качестве поступающих с прибора данных используем генератор случайных чисел. Поместим этот генератор случайных чисел в цикл While и установим кнопку остановки этого цикла. Добавим график ***Waveform Chart*** в окно лицевой панели и перейдем на блок-диаграмму, чтобы поместить терминал графика ***Waveform Chart*** внутрь цикла. Также добавим функцию задержки ***Wait*** с временем задержки 100 мс, чтобы случайные числа не создавались очень быстро (рис. 79(a)).

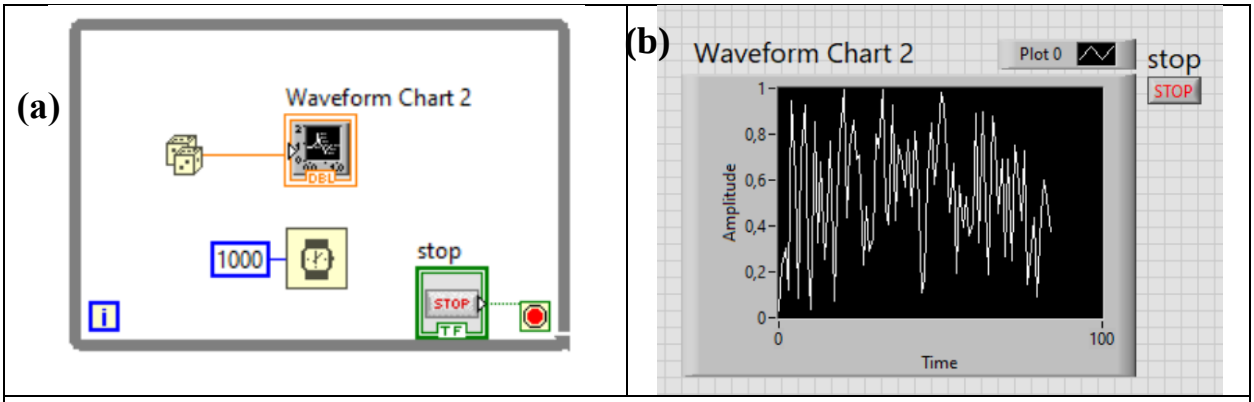


Рисунок 79. a) Вид графического кода на блок-диаграмме для отображения данных в виде осциллограммы. b) Отображение данных, полученный при выполнение программного кода, в виде осциллограммы на лицевой панели.

Запустим программу и посмотрим, как строится график развёртки осциллограмм (рис. 79(b). Если вы хотите узнать время, в которое было получено числовое значение, а не знать номер итерации, можно

воспользоваться счётчиком времени (который мы разбирали выше), с помощью которого рассчитать время работы всего цикла While и разделить на число совершённых итерации. Теперь если запустить программу ещё раз, то мы увидим, что график будет строиться из той же точки, на которой остановилось построение графика в предыдущем запуске программы, а не из нуля. Это происходит из-за того, что график сохраняет предыдущие значения до тех пор, пока не достигнет своего предельного значения 1024. Для того, чтобы при каждом запуске программы график строился из нуля нужно очищать сохранённые в истории графика данные. Для этого переходим на лицевую панель, выделяем график и щелчком правой кнопки мыши открываем контекстное меню, в котором выбираем строчки ***Create→ Property Node →History Data*** (рис. 80(a)).

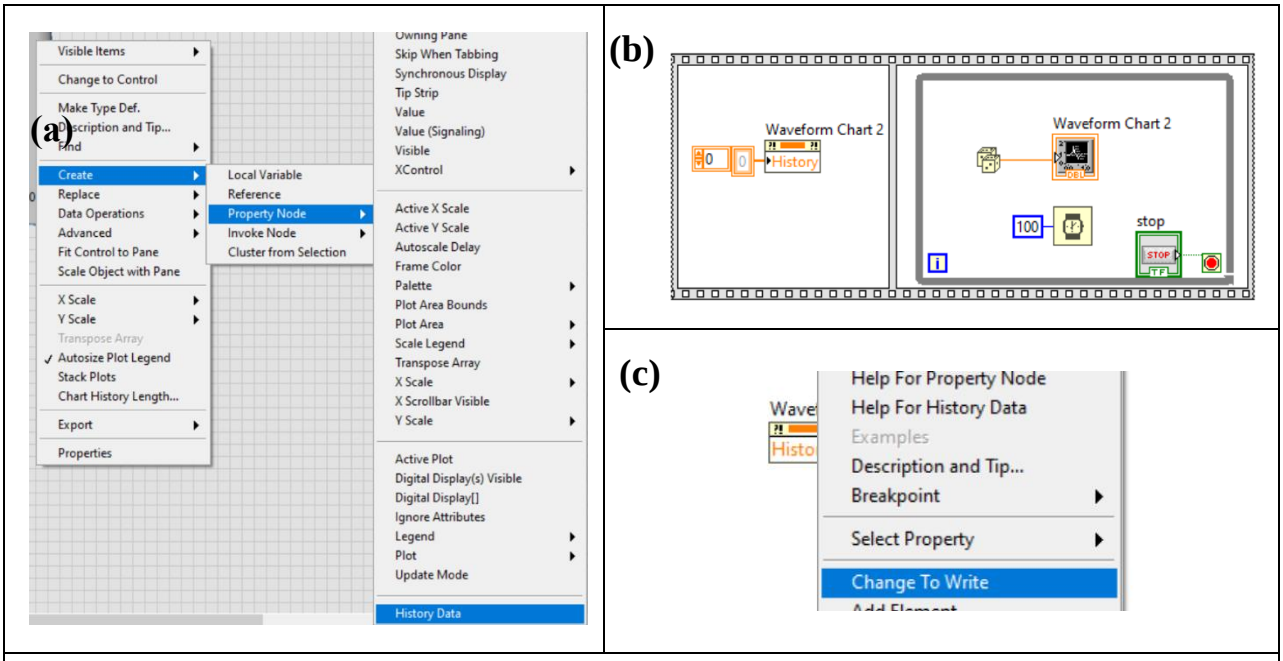
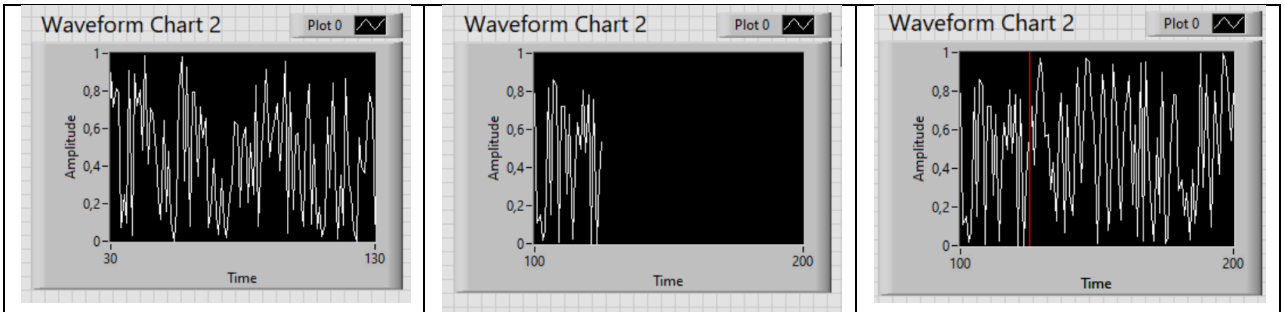


Рисунок 80. a) Создание узла из контекстного меню графика для очистки истории графика. b) Графический код для построения осциллограммы с очищением историй построения. c) Кнопка изменения вида терминала в контекстном меню элемента.

После этого на блок-диаграмму вы увидите появившийся узел, который позволяет стирать историю графика, но сначала нам нужно переключить узел на запись, для этого правым щелчком мыши по узлу открываем контекстное

меню и щелкаем на строчку Change To Write (рис. 80(c)). Теперь нам надо подключить нулевую константу на входной терминал, для этого снова открываем контекстное меню и выбираем строчку Create Control. Теперь, для того чтобы история графика очищалась до построения нового графика, поместим узел для очистки графика в первый кадр Flat Sequence, а созданный цикл во второй кадр. В результате должен получится графический код, представленный на рисунке 80(b). Теперь запустим программу и посмотрим, как будет очищаться график. В ***Waveform Chart*** есть три режима отображения развёртки осциллограмм: панорамная развёртка (*strip chart mode*), временная развёртка (*scope chart mode*), временная развёртка с маркером (*sweep chart mode*). По умолчанию в среде LabVIEW используется режим панорамной развёртки. Чтобы поменять режим отображения графика откройте контекстное меню графика сначала выберите раздел **Advanced**, затем строчку **Update Mode** и выберите нужный режим (рис. 81)



(a) **(b)** **(c)**

Рисунок 81. a) Вид осциллограммы в режиме панорамной развёртки графика. b) Вид осциллограммы в режиме временной развёртки графика. c) Вид осциллограммы в режиме временной развёртки графика с маркером.

У всех графиков есть дополнительные функции, которые позволяют ими управлять, но для того, чтобы их использовать, нужно включить их отображение. Данные, которые отображаются на графике, постоянно обновляются, поэтому часть из них перестаёт убираться в окне построения графика и становится невидимой. Для того, чтобы увидеть невидимые (спрятанные) данные, находящиеся вне видимой части графика, используют

полосу прокрутки – ***X Scroll Bar***, которую можно добавить, выбрав в контекстном меню графика ***Visible Items X Scroll Bar***. Теперь двигая ползунок появившейся полосы прокрутки, вы можете просматривать все построенные данные (рис. 82(a)). В правом верхнем углу графика расположено окошечко с названием Plot, которое позволяет изменять вид графика: цвет, вид линии, толщину и так далее. Щёлкните по нему левой кнопкой мыши и перед вами откроется контекстное меню, которое содержит все возможные функции редактирования (рис. 82(a)). При это полосу прокрутки и окно редактирования графика можно выделить и перемещать по всему полю лицевой панели. По умолчанию на графике может отображаться только 1024 точки, но это значение можно изменить. Откройте контекстное меню графика и выберите строчку **Chart History Length.** Перед вами откроется диалоговое окно, в котором вы можете изменить количество точек на любое значение до 100000 точек (рис. 82(b)). Очищать данные, сохраненные на графике от предыдущего запуска программы, можно не только создавая графический код, но и вручную.

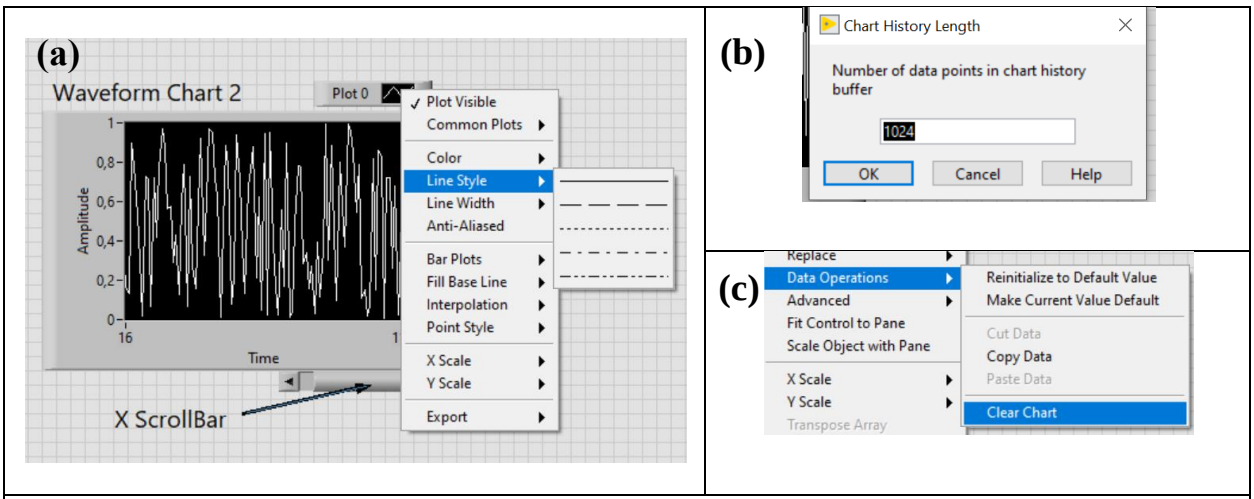


Рисунок 82. a) Контекстное меню редактирования основных параметров графика. b) Диалоговое окно для изменения размера буферных данных графика. c) Кнопка очищения данных на графике в контекстном меню.

Для этого в контекстном меню графика выберите раздел **Data Operation** и нажмите **Clear Chart** (рис. 82(c)). Сразу после нажатия все данные на графике будут удалены.

На графиках развёрток осциллограмм помимо одного графика также можно строить несколько графиков. Так как у графика только один входной терминал то мы не можем к этому терминалу подключить много проводников, соответственно нам нужен элемент, который позволит объединить несколько проводников данных в один. Такую функцию можно найти в разделе структур – Кластеры. Для понимания кратко разберём что это такое. Кластеры – это структура (похожая на массивы), которая позволяет группировать элементы данных смешанных типов (массивы позволяют группировать элементы только одного типа). Кластер аналогичен записи или структуре в текстовых языках программирования. Для того, чтобы объединить в группу несколько проводников данных для построения нескольких графиков осциллограмм, воспользуемся функцией Объединения (Bundle), которую можно найти в палитре функций ***Show Function Palette → Programming → Cluster, Class, Variant → Bundle***. Помещённая на блок-диаграмму функция Объединения имеет изначально 4 терминала (рис. 83(а)):

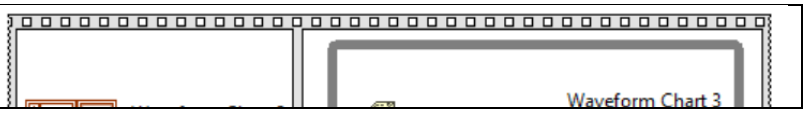
- два входных терминала на левой границе для двух проводников данных, которые требуется объединить в кластер;
- один входной терминал сверху, который позволяет добавлять в уже существующий кластер новые элементы, которые подключены к входным терминалам данных;
- выходной терминал, из которого выходит либо новый кластер, либо старый кластер к которому добавили новые элементы.

Также количество входных данных (входных терминалах) можно изменять с помощью растягивания элемента за синие квадратики на верхней и нижней грани элемента (по аналогии с растягиванием структуры **Build Array**). Для примера растянем элемент, чтобы было доступно три входных терминал, которые позволят нам построить три графика. Пример будем составлять на основе уже написанного кода.

(a)



(c)



(b)

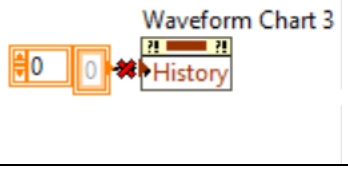
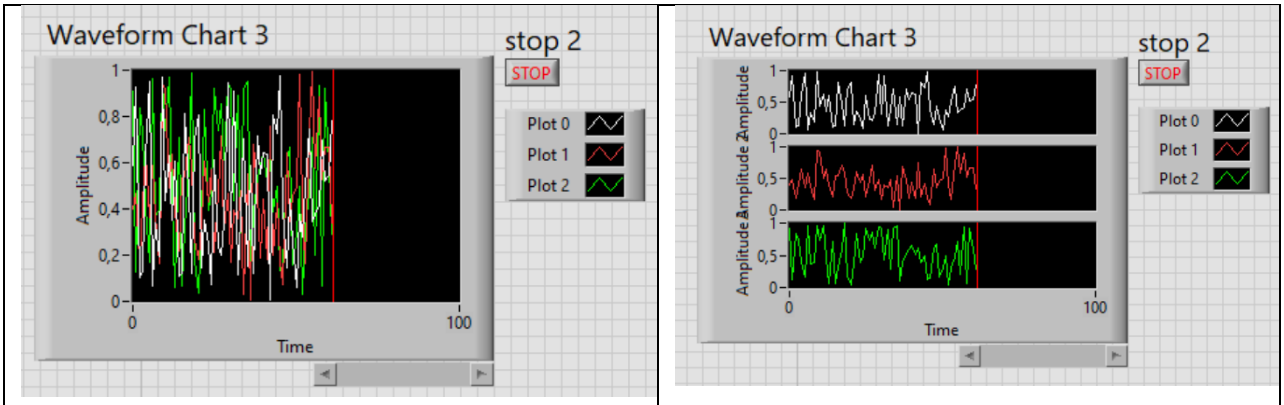


Рисунок 83. a) Вид терминала функции Bundle на блок-диаграмме. b) Отображение ошибки подключения нулевой константы для очищения графика. c) Графический код для построения нескольких осциллограмм на одном графике с предварительной очисткой данных на графике.

Сначала подсоедините к входному терминалу графика выходной терминал структуры Объединения, вместо генератора случайных чисел. В качестве данных для построения также будем использовать генератор случайных чисел, поэтому требуется добавить ещё два генератора случайных чисел. Подсоедините три генератора к трём входным терминалам к структуре Объединения, в результате должен получится графический код, представленный на рис. 83(c). Если у вас в код добавлена очистка данных на графике, то требуется изменить подаваемую на него нулевую константу, чтобы устранить ошибку, которая возникла из-за того, что мы подключили к терминалу графика другой тип данных (рис. 82(b)).

Запустим программу и посмотрим, как выглядит график (рис. 84(a)). На графике отображаются все три кривые, причём каждая кривая рисуется своим цветом, который можно изменить, если растянуть окно редактирования графиков (растягивается также, как и структура **Build Array**).



(a) **(b)**

Рисунок 84. a) Отображение нескольких осциллограмм на одном графике в режиме Stack Plots. b) Отображение нескольких осциллограмм на одном графике в режиме Overlay Plots.

Когда все три графика строятся в одном окне это может быть не удобно, так как они накладываются друг на друга и это перекрытие не позволяет разглядеть их. На этот случай в среде LabVIEW есть режим **Stack Plots**, который позволяет отображать каждую кривую в своём окне (рис. 84(b)). Чтобы перейти в этот режим откройте контекстное меню графика и нажмите на строчку **Stack Plots,** структура графика перестроится. Если вы хотите вернуть режим построения графиков в одном окне (**Overlay Plots**) также откройте контекстное меню и нажмите на **Overlay Plots**.

Помимо возможности редактирования графиков, в среде LabVIEW также можно редактировать оси графиков. Функции редактирования осей графика можно найти в контекстном меню, которое появляется после щелчка левой кнопкой мыши по выделенной оси. Свойства для каждой оси устанавливаются отдельно друг от друга, если вы открыли контекстное меню щёлкнув по оси Y, то вы редактируете только ось Y, если щёлкнули по оси X, то происходит редактирование только оси X. Стоит отметить, что контекстное меню для осей графика универсально и содержит все виды функций редактирования, которые могут быть использованы и для других типов графиков, поэтому использование некоторых функций к данному типу

графика (Развёртка осциллограмм) может быть невозможно. Перечислим все эти функции редактирования осей и опишем их (рис. 85(b)).

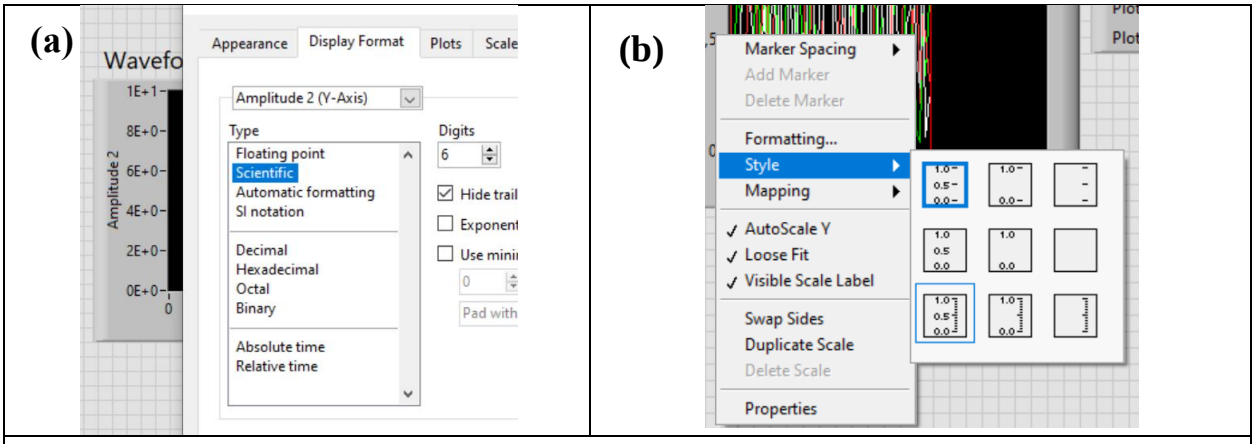


Рисунок 85. a) Диалоговое окно, позволяющее изменить тип отображаемых на графике данных. b) Вид контекстного меню, содержащего основные возможности редактирования шкал графика.

Первые три функции в контекстном меню редактирования следует использовать для шкалы на графике Intensity Chart (рассмотрим его ниже). Использование функций на графике ***Waveform Chart*** либо недоступно, либо не даёт никакого значительного результата.

- Marker Spacing

Marker Spacing– это функция редактирования осей (шкал), которая позволяет регулировать расстояния между делениями шкалы. В ней всего есть два варианта: Uniform, Arbitrary. Вариант Uniformрасставляет деления шкалы на одном и том же расстоянии друг от друга, то есть равномерно. Вариант Arbitraryпозволяет устанавливать произвольное положение делений на шкале, которое требуется пользователю.

- Add Marker

Add Marker– это функция позволяющая добавить новое деление в произвольном месте шкалы, которое выбирает пользователь. Учтите, что использование этой функции возможно только если у вас выбран вариант Arbitrary для редактирования осей.

- Delete Marker

Delete Marker– это функция, которая позволяет удалять созданные вами деления на шкале.

- Formatting

Нажатие на строчку Formatting открывает перед вами диалоговое окно Display Format (рис. 85(a)), в котором вы можете изменять вид отображения значений у каждого деления шкалы. Для этого в окне Туренужный вам формат данных, по умолчанию изначально там стоит автоматический подбор формата (Automatic Formatting). Также в окошечке с названием Digits вы можете изменять точность, с которой будут отображаться данные на графике (то есть изменять количество цифр после запятой).

- Style

Нажатие на строчку Style открывает меню, в котором содержаться 9 возможных стилей отображения шкалы оси. У каждого стиля есть своя иконка показывающая вид шкалы. Стиль шкалы с белым пустым квадратиком полностью убирает шкалу с графика.

- Mapping

Строка Mapping позволяет изменять масштаб шкалы с линейного на логарифмический и обратно. Щёлкните по строке и выберите Logarithmic, чтобы перейти к логарифмическому масштабу.

- Auto Scale Y

Когда мы строим график, наши данные могут выходить за пределы граничных значений, указанных на шкале графика. В результате чего часть графика не будет видна, и нам придётся вручную изменять граничные значения шкал, чтобы для полного отображения графика. Чтобы не изменять граничные значения шкалы вручную в среде LabVIEW есть функция Auto Scale Y, которая автоматически будет изменять граничные значения. Когда текущие значения графика будут выше (или ниже) граничного значения шкалы и часть графина начнёт строится за пределами окна построения, функция Auto Scale Y сама увеличит (или уменьшит) граничное значение так, чтобы было видно отображение всего графика.

- Loose Fit

Функция Loose Fit округляет граничные значения шкалы до целых чисел. То есть если, к примеру, граничное значение шкалы равно 0,9998, то эта функция округлит это значение до 1 и именно его будет отображать на графике, а не значение 0,9998.

- Visible Scale Label

У каждой оси есть подписи (наименование), которое можно изменить, если два раза щёлкнуть мышкой по названию, стереть его и ввести новое. Однако не всегда требуется, чтобы оси имели название. Если вы щёлкните на строчку Visible Scale Label, то название оси на графике пропадёт, чтобы вернуть название обратно снова щёлкните на Visible Scale Label.

- Swap Side

Иногда удобнее, чтобы вертикальная шкала находилась не с левой стороны графика, а справой. Для смещения шкалы на правую сторону графика нажмите на строчку Swap Side, расположенную в контекстном меню.

- Duplicate Scale

Duplicate Scale создать дополнительную (вторую) шкалу по оси Y, которую удобно использовать для построения нескольких графиков, если значения на нескольких графиках очень сильно отличаются друг от друга (ниже рассмотрим пример).

- Delete Scale

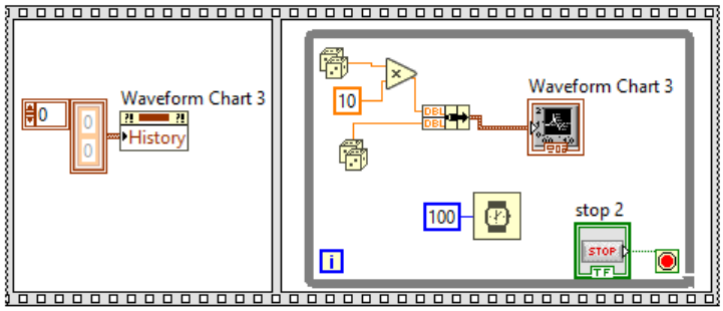
Delete Scale удаляет созданную вами ранее дополнительную шкалу. Если на графике всего одна шкала, то эта функция будет не доступна и убрать отображение шкалы на графике возможно только с помощью функции Style.

- Properties

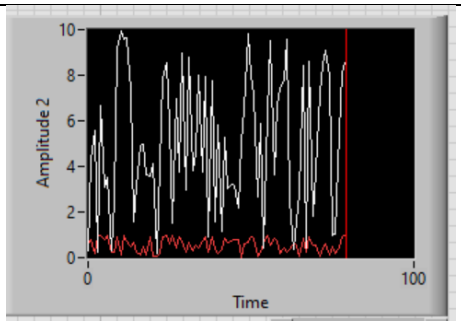
Нажатием на строчку Propertiesвы можете открыть диалоговое окно с большим количеством вкладок, которые содержат функции редактирования любой части графика. В контекстном меню редактирования находится минимальный набор функций, а в диалоговом окне Properties содержится значительно больше функций редактирования, которые позволяют изменять

и добавлять очень большое количество параметров на график. К примеру, во вкладке Scalevы можете добавить или убрать вертикальные и горизонтальные линии для создания сетки на графике (Grid Style and Color), а также выбрать требуемый вам цвет.

Предположим, что мы хотим построить две зависимости на одном графике чтобы сравнить их, однако, значения одной зависимости в несколько раз отличаются от значений другой зависимости, что не позволяет сравнить эти две зависимости (или сопоставить их значения). Разберёмся как же можно решить эту задачу. Возьмём программный код, написанный в предыдущем примере, но вместо трёх графиков будем строить только два. Умножим одно из создаваемых генератором случайных чисел, к примеру, на число 10, чтобы значения одной зависимости были значительно больше другой. В результате должен получится графический код, представленный на рисунке 86(a). Запустим программу и увидим, что одна зависимость действительно стала значительно меньше другой, то есть является сжатой по оси Yотносительно зависимости с большими значениями (рис. 86(b)).



(a)



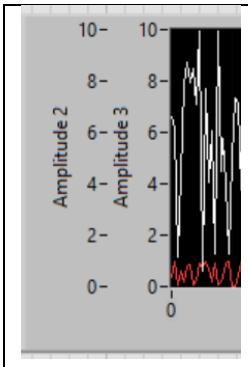
(b)

Рисунок 86. a) Графический код для построения двух разных осциллограмм на одном графике. b) Отображение данных, полученных после работы программы, в графическом виде.

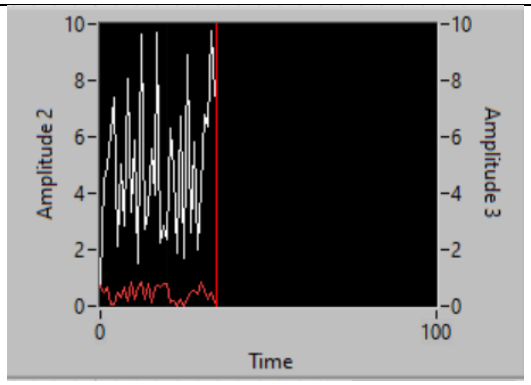
Чтобы исправить это потребуется создать дополнительную шкалу по оси Y, чтобы зависимость с меньшими значениями случайных чисел откладывалась по своей отдельной шкале. Для того, чтобы это сделать выделим шкалу по оси Y, щелчком правой кнопки мыши откроем

контекстное меню и создадим дубликат шкалы нажав на строчку Duplicate Scale. После чего вы увидите, что с левой стороны графика находится две одинаковые шкалы, расположенные одна за другой (рис. 87(a)). Такой расположение шкал не удобно, поэтому одну шкалу нужно разместить с другой стороны графика. Для этого ткните по шкале, которую вы хотите перенести на другую сторону, откройте контекстное меню и выберите строчку Swap Side (рис. 87(b)). Теперь попробуем запустить программу и посмотреть, что будет с графиком. Однако ничего не изменилось и графики продолжают строится точно также. Это связано с тем, что при создании шкалы ни один из графиков автоматически не начнёт использовать новую шкалу для своего отображения, то есть созданная нами новая шкала не привязана ни к какому графику. Поэтому для одного из графиков надо вручную выбрать ось Y, относительно которой и будет строится один из графиков. Сделать это можно двумя способами. Первый способ, это воспользоваться окном Plot, которое отображает настройки всех существующих графиков и позволяет их редактировать. Допустим мы хотим сменить шкалу для красного графика, тогда в окне Plot мы находим красную линию с названием Plot 1, нажимаем левой кнопкой мыши по маленькой иконке рядом с названием, которая отображает вид графика. Перед нами открывается контекстное меню, в котором мы должны найти строчку Y Scale. В строчке Y Scale выпадает ещё одно контекстное меню, которое содержит названия двух наших шкал (Amplitude 2, Amplitude 3) (рис. 87(c)). Сейчас у нас оба графика строятся относительно шкалы Amplitude 2, поэтому шкалу по оси Y для выбранного графика нам надо поменять на Amplitude 3, выбрав его в выпавшем контекстном меню.

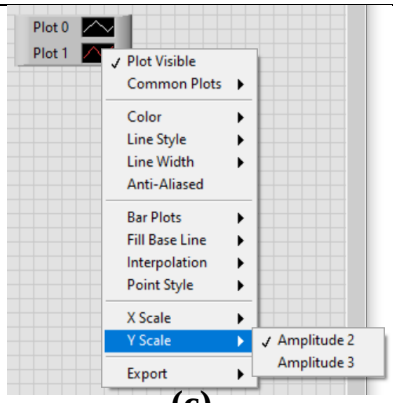
Теперь если запустить программу вы увидите, что каждый график строится относительно своей шкалы. Если значения шкал не изменились проверьте стоит ли у шкалы режим Auto Scale Y, если этот режим не установлен, то граничные значения шкалы сами не будут подстраиваться под значения графика!



(a)



(b)



(c)

Рисунок 87. a) Создание дополнительной вертикальной шкалы на графике. b) Перемещение дополнительной вертикальной шкалы на противоположную сторону графика. c) Контекстное меню графика позволяющее выбрать вертикальную шкалу относительно которой будет построен график.

Рассмотрим второй способ изменения оси Y у одного из графиков. Для начала нам надо открыть диалоговое окно Properties (рис. 88(a)), стоит отметить, что это строчка Properties есть в самом низу любого контекстного меню, относящегося к элементам графика, поэтому не обязательно открывать контекстное меню именно для осей. Когда диалоговое окно открыто находим вкладку Plots и переходим в неё (рис. 88(b)). В верхней части вкладки есть разворачивающийся список, который содержит названия двух построенных графиков (если графиков больше, то и названий больше). Информация, отображаемая для редактирования во вкладке Plots, относится к графику, название которого выбрано в разворачивающемся списке (отмечен галочкой) (рис. 88(c)). Если вы хотите редактировать другой график, то выберите его название в списке (рис. 88(b)). В нижней части вкладки находится список с названием Y-scale, который при развороте показывает названия осей Y. Выберите для графика другую ось Y, чтобы два графика строились по двум разным шкалам (рис. 88). После чего нажмите OK, диалоговое окно закроется, запустите программу и увидите, что получили точно такой же результат.

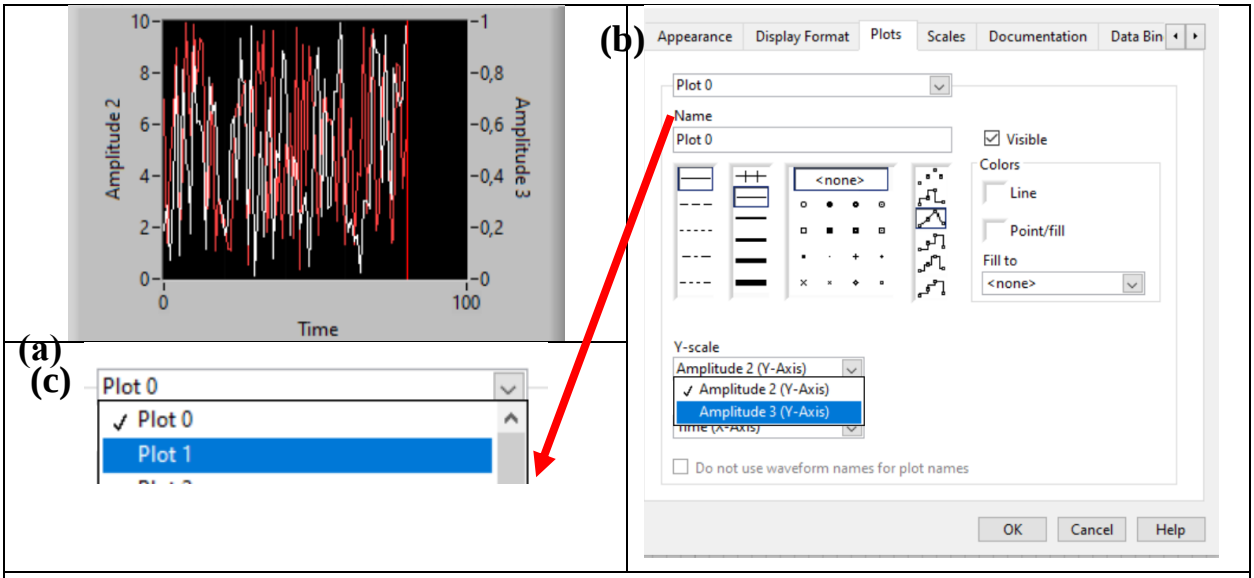
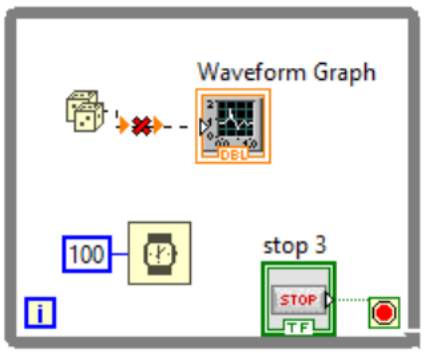


Рисунок 88. a) Графическое отображение данных двух графиков, построенных относительно двух разных вертикальных шкал. b) Диалоговое окно для редактирования осциллограмм, построенных на графике. c) Разворачивающийся список, содержащий названия всех построенных графиков.

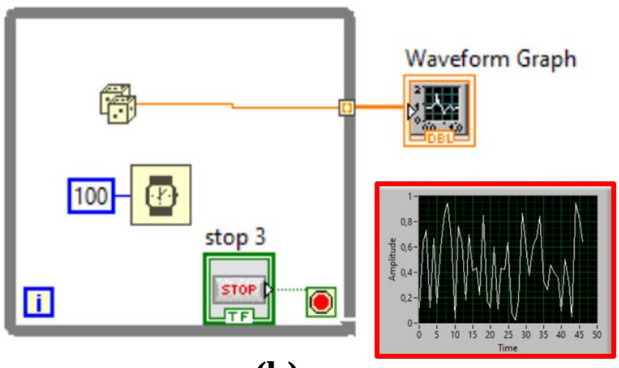
Waveform Graph

График ***Waveform Graph*** (график осциллограмм), также, как и ***Waveform Chart***, позволяет строить графики осциллограмм. Однако в ***Waveform Graph*** невозможно динамическое отображение данных, так как визуализация данных на графике происходит только после того как все значения будут собраны в массив (в процессе сбора данных нет визуального отображения). То есть ***Waveform Graph*** не отображает каждое новое сгенерированное число, а отображает только конечный результат, который содержит все значения, полученные в процессе выполнения программы. Разберём это на примере предыдущего программного кода (скопируем программный код, находящийся в цикле While), оставим только один генератор случайных чисел и заменим график ***Waveform Chart*** на ***Waveform Graph***. Соединим генератор случайных чисел с терминалом графика и увидим, что образуется оборванный проводник данных (рис. 89(a)), то есть подключение одного элемента к терминалу графика невозможно и требуется

подключать массив данных. Вынесем терминал графика за пределы цикла и снова подсоединим к его входному терминалу генератор случайных чисел (если проводник оборван, в контекстном меню выберите режим Index в Tunel Mode) (рис. 89(b)).



(a)



(b)

Рисунок 89. a) Отображение ошибки присоединения данных для построения графика. b) Графический код для построения графика с использованием Waveform Graph (вставка – отображение данных на графике, полученных в результате выполнения программы).

Теперь, когда в графику подключены входные данные, которые представляют собой одномерный массив, запустим программу и посмотрим, как они будут отображаться на графике. Программа запущена, но на графике не появляется ни одного значения, если нажмём на кнопку остановки работы цикла (кнопка Stop), то выполнение цикла и программы закончится, но в ***Waveform Graph***, появится график из набора случайных чисел, которые были сгенерированы в цикле. То есть на графиках осциллограмм возможна визуализация только конечного результата (сформированный набора данных), а не показывают изменение данных на каждом этапе (на каждой итерации) их формирования. Стоит отметить, что значение времени, откладываемое по оси X, всегда начинается из нуля, а приращение (изменение) значения по оси Xвсегда равно 1. Эти настройки для графика осциллограмм заданы по умолчанию, однако можно задавать и свои пользовательские настройки (рассмотрим ниже).

Waveform Graph также позволяет строить несколько осциллограмм в одном окне, однако, в отличие от ***Waveform Char***, отображение двух графиков в разных окнах невозможно. Для добавления второго графика вместо кластера **Bundle** используется обычная функция работы с массивами – **Build Array**. Создадим ещё один генератор случайных чисел, чтобы построить второй график, и удалим проводник данных, подключённый к входному терминалу графика. Поместим функцию **Build Array** на блок-диаграмму за пределами цикла и увеличим до 2 количество входных терминалов. К каждому терминалу подключим одномерный массив, полученный накоплением определённого количества случайных чисел на границе цикла. Двумерный массив, выходящий из **Build Array**, подсоединим к терминалу графика (рис. 90(a)) и запустим программу. После остановки цикла в окне появятся две кривые разного цвета, которые отображают два разных массива из случайных чисел (рис. 90(b)).

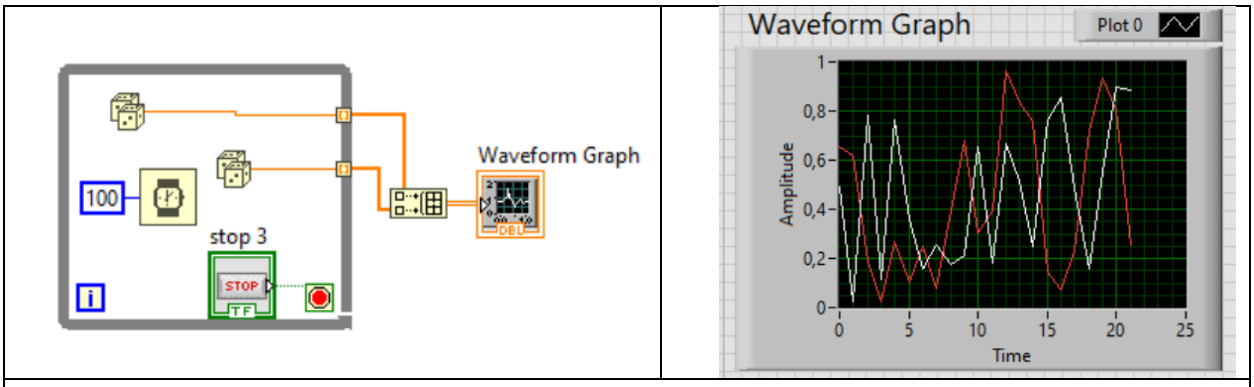


Рисунок 90. a) Графический код для построения двух графиков в одном окне с использованием Waveform Graph. b) Отображение построения двух графиков в одном окне с использованием Waveform Graph.

Также, как и в ***Waveform Char***, вы можете изменять вид и цвет графиков, создавать дополнительные шкалы и перемещать их с одной стороны на другую. Теперь рассмотрим, как осуществляется настройка времени, откладываемого по оси X. Для начала разберём как образуются значения по оси X. Предположим, что мы запустили какое-то измерение и начали записывать данные, к примеру, каждые 10 секунд. Допустим, первое

измеренное значение поступило в компьютер через 5 секунд после начала проведения измерений. Тогда 5 секунд – это начальный момент времени, а 10 секунд – это интервал времени между получениями данных. В среде LabVIEWначальный момент времени обозначают как X_0, а временной интервал – ΔX. По умолчанию начальный момент времени X_0 равен 0, а временной интервал ΔX равен 1. Для того, чтобы изменить значения времени, используемого для построения графика, нужно использовать функцию создания кластера – **Bundle**. Всего требуется объединить в один кластер три значения входных данных: начальный момент времени X_0, временной интервал ΔX и массив с данными для построения Поместим функцию **Bundle** на блок-диаграмму, развернём её так, чтобы было три доступно три входных терминала. Каждый входной терминал служит для подключения определённого проводника с данными, поэтому присоединение задаваемых данных для построения графика должно быть произведено в строго установленном порядке (вставка на рис. 91(b)). К верхнему входному терминалу подсоединяется начальный момент времени X_0, к среднему – временной интервал ΔX, к нижнему входному терминалу – массив с данными. Теперь подключим проводниками данные к функции **Bundle**, в качестве X_0 и ΔX можно использовать целочисленные константы или константы числа с плавающей запятой. Для примера возьмём X_0 равную 20, а шаг по времени ΔX равным 10, в результате должен получится код, представленный на рисунке 91(a). Теперь запустим программу, наберём какое-то количество данных, остановим её нажатием на кнопку Stop и посмотрим, как изменился вид графика (рис. 91(b)). Первая точка, которая построилась на графике начинается со значения 20 по оси X, то есть установленное нами значение X_0 сдвинуло график со значения 0 на 20.

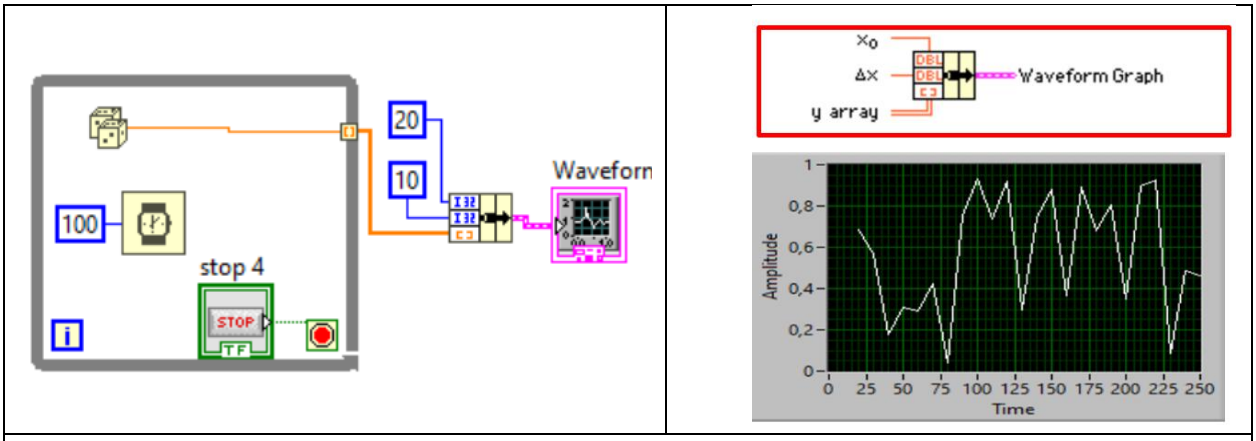


Рисунок 91. a) Графический код для построения графика с пользовательскими временными параметрами с использованием Waveform Graph. b) Отображение построения графика с пользовательскими временными параметрами в окне Waveform Graph (вставка – схема подключения временных параметров к функции Bundle).

Расстояние между двумя соседними точками также изменилось с 1 на 10 (значение, заданное в ΔX). Попробуйте поставить другие значения и посмотрите, как будет изменяться график. Таким образом вы можете самостоятельно устанавливать временные параметры, которые будут соответствовать вашему эксперименту.

Теперь разберём как создать графический код для настройки временных параметров, если мы хотим построить две осциллограммы в одном окне. Всего есть два варианта написания кода для изменения временных параметров графика. Выбор варианта зависит от поставленной задачи, которую требуется решить. Сначала разберём более простой пример, в котором требуется, чтобы оба графика, к примеру, начинались не с 0, а с 10, и значение временного интервала было равно 5. Как мы уже знаем, для задания параметров требуется функция **Bundle** с тремя входными терминалами, поместим её на блок-диаграмму и скопируем весь графический код, который выполняет построение двух осциллограмм на одном графике (код из рис. 91). Мы знаем, что на нижний входной терминал функции для создания кластера нужно подавать массив данных, которые мы хотим

отобразить на графике. При этом стоит отметить, что размерность подаваемого массива с данными не имеет значения, можно подключать как одномерные массивы (как при построении одной осциллограммы), так и двумерные массивы. Поэтому ничего не мешает нам присоединить двумерный массив, выходящий из функции **Build Array,** к функции **Bundle**. На первый и второй входные терминалы подадим значения X_0 и ΔX равные 10 и 5, соответственно, выходной терминал **Bundle** подсоединим к графику. Должен получится код, представленный на рисунке 92(a). Если мы запустим программу и посмотрим на графики, то увидим, что расстояние между двумя соседними точками каждого графика стало равно 5, при этом оба построенных графика начинаются не из нуля, а со значения 10 по оси X (рис.92(b)). Таким образом вы можете изменять временные параметры сразу двух (или нескольких) разных графиков, которые требуется построить в одном окне.

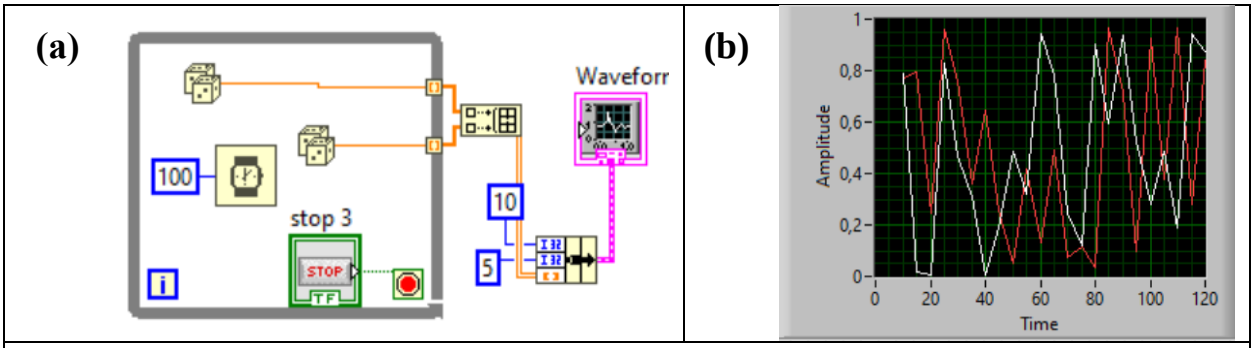


Рисунок 92. a) Графический код для построения двух графиков с одинаковыми пользовательскими временными параметрами с использованием Waveform Graph. b) Отображение построения двух графиков с одинаковыми пользовательскими временными параметрами в окне Waveform Graph.

Однако, для того, чтобы осуществить построение двух разных графиков с разными временными параметрами, потребуется написать более сложный графический код. В качестве данных, на основе которых будут построены два графика осциллограммы, также, как и в предыдущем примере, будем использовать два генератора случайных чисел. У каждой

осциллограммы должны быть свои временные параметры, следовательно, использовать одну функцию **Bundle** для обоих графиков мы не можем, поэтому создадим для каждого из них отдельную функцию для создания кластера с тремя входными терминалами. Присоединим к каждому терминалу данные точно также, как мы это делали для одного графика. Пусть у первого графика значения X_0 и ΔX останутся по умолчанию (X_0=0, ΔX=1), а на втором графике поставим значения X_0=10 и ΔX=5. Теперь, когда мы создали два кластера с разными временными параметрами для двух разных осциллограмм, нужно построить их в одном окне. Так как входной терминал у графика только один, а кластеров с данными у нас два, то потребуется объединить их для корректной работы программного кода. Это можно сделать уже ранее использованной нами функцией **Build Array.** Отметим, что функция **Build Array** позволяет объединять не только массивы и элементы, но и кластеры. Объединение двух кластеров с помощью **Build Array** создаст массив, который будет состоять из кластеров, а не из чисел (как мы это видели ранее). Подсоединим полученным массив к терминалу графика (рис.93(a)) и запустим программу.

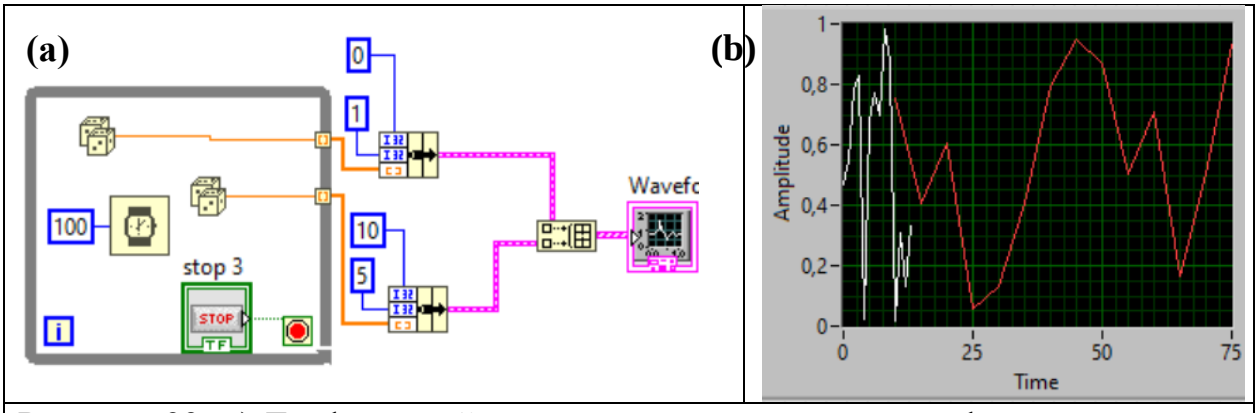


Рисунок 93. a) Графический код для построения двух графиков с разными пользовательскими временными параметрами с использованием Waveform Graph. b) Отображение построения двух графиков с разными пользовательскими временными параметрами в окне Waveform Graph.

После того, как программа завершит свою работу, в окне отобразятся две построенные осциллограммы (рис.93(b)). Первая (белая линия)

осциллограмма начинает строится из нуля и имеет временной шаг 1, вторая осциллограмма (красная линия) начинается с заданного значения 10 и ставит каждую точку на графике с интервалом равным 5. То есть у нас получилось построить два графика в одном окне с различными временными параметрами. Попробуйте самостоятельно поварьировать временные значения и количество отображаемых графиков. Такой вид графического кода можно использовать и для построения двух разных графиков с одинаковыми временными параметрами. Для этого на оба кластера нужно подавать одинаковые значения X_0 и ΔX. Если вы заранее знаете, что временные параметры одинаковые, то стоит воспользоваться первым вариантом графического кода.

Кластеры

Кластеры, как мы уже убедились, являются неотъемлемой частью в создании программного кода, служащего для отображения полученных экспериментальных данных. Однако в примерах мы не рассматривали подробное представления о том, что такое кластеры, как выглядит содержимое каждого кластера, а также какие функции позволяют работать с кластерами. Для того, чтобы в дальнейшем создавать корректно работающие программы с кластерами (кластеры могут использоваться не только для графиков), нужно понимать, как создаются кластеры, как упорядочивать элементы в кластерах и почему это нужно, а также знать, как правильно формировать содержимое кластера и не совершать ошибок.

Для начала на основе выше созданных примеров для визуализации данных на графиках разберём как можно отображать кластеры на лицевой панели и как выглядит содержимое внутри кластера. Возьмём программный код из рисунка 92(а) и удалим проводник, соединяющий выходной терминал кластера с терминалом графика. Далее кликая правой кнопкой мыши на выходной терминал кластера открываем контекстное меню и нажимаем на строчку с названием Create Indicator (создать индикатор) (рис.94(а)). На блок-

диаграмме появляется автоматически подключённый к **Bundle** терминал с названием output cluster. На лицевой панели также появился элемент с названием output cluster, который представляет собой серое окно, которое содержит два индикатора и один двумерный массив (рис.94(b)). Серое окно – это сам кластер, а точнее его графическое отображение на лицевой панели. Всё, что находится внутри серого окна, является содержимым кластера, которое мы в него поместили. Если нажать на серое окно левой кнопкой мыши, то вокруг него появится выделение с синими квадратиками, потянув за которые можно изменять размер кластера на лицевой панели. Размеры элементов, находящихся внутри кластера, изменяются точно также, как если бы они находились вне кластера на лицевой панели.

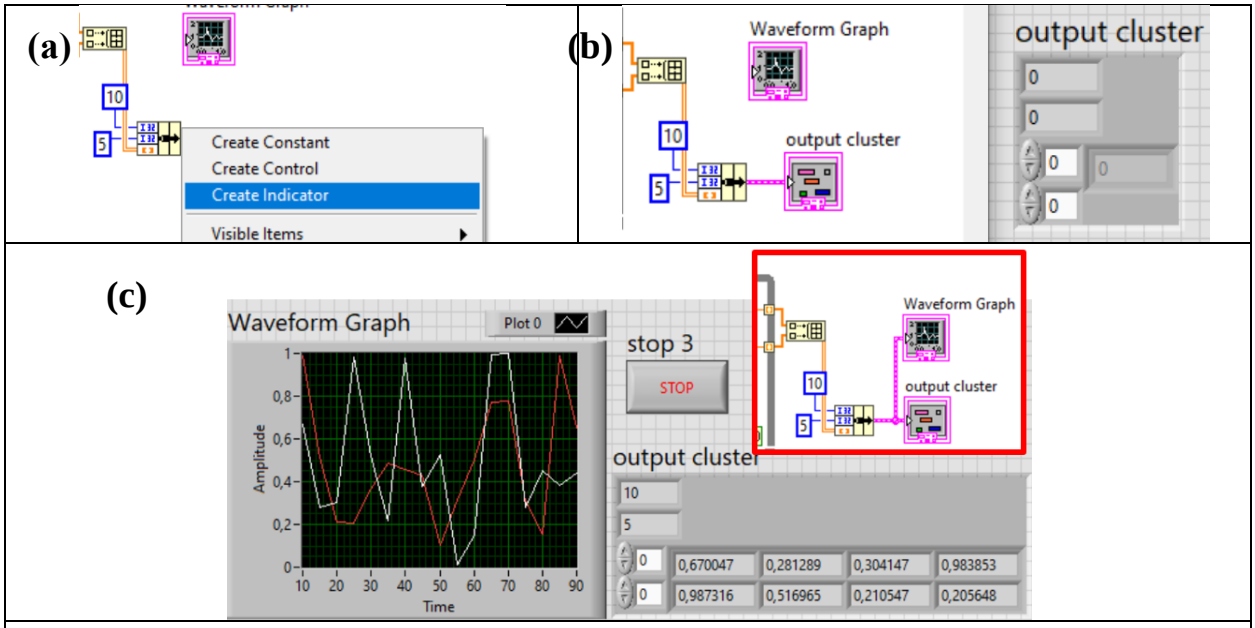


Рисунок 94. a) Создание нового выходного кластера с помощью контекстного меню функции Bundle. b) Вид созданного выходного кластера на блок-диаграмме и лицевой панели. c) Отображение полученных данных в окне лицевой панели (вставка – графический код для построения графиков).

Изначально кластер был пустым (было просто пустое серое окно), но в процессе создания программного кода мы поместили туда два временных параметра X_0 и ΔX, а также массив с данными. Поэтому в серое окно

кластера добавилось два индикатора, в которых будут отображаться значения временных параметров и двумерный массив, в котором будут отображаться данные для построения двух осциллограмм. Присоединим проводник от кластера обратно к терминалу графика не разрывая проводник с терминалом output cluster (вставка на рисунке 94(c)) и запустим программу (рис.94(c)). Посмотрим на кластер, расположенный на лицевой панели (рис.94(c)). В верхнем индикаторе кластера появилось число 10, которое мы добавили в верхний терминал функции **Bundle**, во втором индикаторе появилось число 5 – число, поданное на средний терминал функции **Bundle**. Третьим элементом (смотрим сверху вниз) в кластере является двумерный массив, заполненный входными данными, поданными на нижний терминал функции **Bundle**. То есть все данные, которые мы собрали в кластер имеют строго определённый порядок, который сохраняется на протяжении всего пути передачи данных.

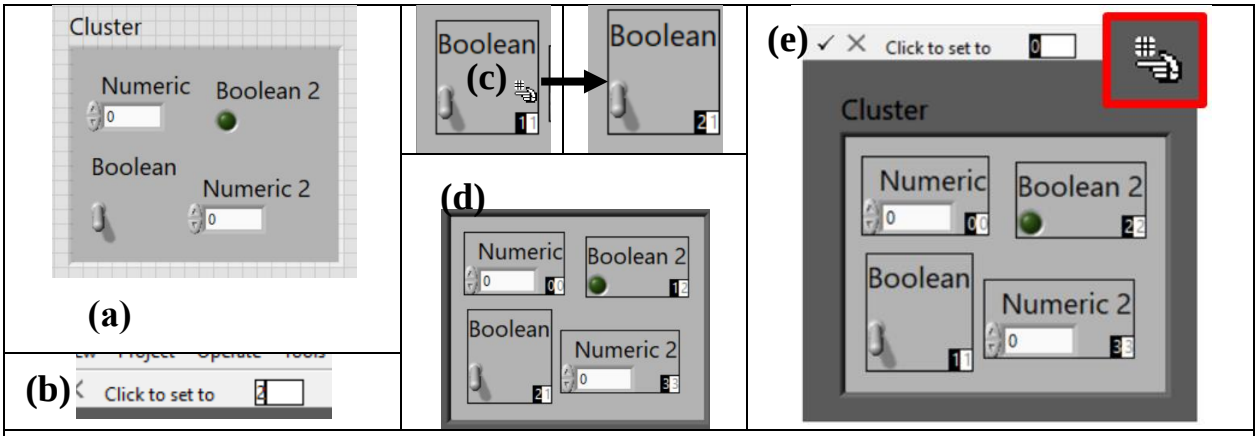


Рисунок 95. a) Вид кластера на лицевой панели с о всем содержимым. b) Окно установки порядкового номера элементов в кластере. c) Изменение порядкового номера элемента управления. d) Вид кластера в режиме редактирования порядковых номеров элементов. e) Установка порядковых номеров элементов кластера.

Важно! Порядок данных в функции **Bundle** устанавливается строго сверху вниз, при этом нумерация начинается с 0. Верхний входной терминал считается нулевым, терминал лежащий ниже – первый, терминал ниже первого – второй и так далее. Строго в соответствии с этим порядком будут

расположены все элементы в кластере на лицевой панели! Теперь рассмотрим, как можно создавать кластеры на лицевой панели. Откроем палитру управления и найдём в ней элемент ***Cluster*** нажимая ***Controls Pallete→Modern→Data Container→Cluster***. В окне лицевой панели должно появится пустое серое окно с названием ***Cluster***. Теперь разберём как поместить элементы управления и индикаторы внутрь кластера.

Допустим мы хотим поместить в кластер числовой элемент управления, для этого откроем палитру управления, выберем в разделе Numericэлемент управления Control. Теперь, когда элемент управления Control привязан к курсору мыши и следует за ней, щёлкните левой кнопкой мыши внутри серого поля кластера, в результате элемент должен оказаться внутри него. Так происходит заполнение кластера элементами. Для примера, поместите в кластер ещё три элемента, которые представлены на рисунке 95(а). Если места на кластере не хватает, выделите его и увеличьте до требуемого размера. Количество элементов, которые можно поместить в кластер, не ограничено, при этом элементы могут иметь абсолютно разные типы данных.

Так как в кластерах очень важно сохранение строгого порядка элементов, то возникает вопрос о том каким образом определяется порядок элементов, которые были добавлены в кластер, расположенный на лицевой панели. Когда мы помещаем элементы в кластер, то располагаем их внутри него произвольным образом, однако местоположение элементов в кластере никак не повлияет на их порядок. Порядок определяется очерёдностью помещения элементов. Изначально кластер пуст, когда мы поместили в него элемент Control, то ему автоматически присвоился порядковый номер ноль. Теперь в кластере есть один элемент с порядковым номером 0. Далее мы помещаем в кластер второй элемент, к примеру, Indicator, который автоматически получает следующий порядковый номер – 1. Третий помещённый элемент приобретёт порядковый номер 2 и так будет продолжаться далее. Допустим мы захотим удалить один из элементов,

который, к примеру, имел порядковый номер 1. Тогда порядковые номера всех других элементов изменятся, элемент с порядковым номером 2 поменяет свой порядковый номер на 1, третий станет 2 и так далее. Элементы, имеющие порядковый номер меньше, чем порядковый номер удалённого элемента, не изменят свой номер. Однако пользователю может потребоваться настроить совершенно другой порядок элементов. Для того чтобы это сделать откройте контекстное меню кластера, щёлкнув по нему правой кнопкой мыши, и нажмите на строчку ***Reorder Controls in Cluster*** (Порядок элементов в кластере). Окно на лицевой панели переходит в режим редактирования и принимает вид, представленный на рисунке 95(d). Каждый элемент, который расположен в кластере обводится прямоугольным контуром, который обозначает область пространства, занимаемую конкретным элементом. В правом нижнем углу обводки у каждого элемента есть два числа: число на чёрном фоне (первое число), число на белом фоне (второе число). Число на белом фоне обозначает порядковый номер элемента, который был присвоен ему при помещении в кластер. На чёрном фоне указан порядковый номер элемента, который вы хотите задать этому элементу в окне редактирования. Изначально эти числа одинаковые, так как изменения порядка элементов ещё не было произведено. При переходе в режим редактирования кластера в верхней части лицевой панели появляется дополнительная полоса, содержащая элементы для изменения порядкового номера. Допустим мы хотим присвоить какому-то элементу порядковый номер 2. Введём значение 2 в окошечке рядом с названием **Click to set to** (Кликните для установки), которое расположено на дополнительной полосе (рис. 95(b)). Теперь кликнем по элементу в кластере порядковый номер которого мы хотим изменить на номер 2. Для примера возьмём элемент переключатель с текущем порядковым номером 1. Заметим, что курсор мыши в режиме редактирования приобретает вид указательного пальца, который представлен на вставке с красной обводкой на рисунке 95(e). После того как мы нажали на элемент, черное число, находящееся в правом нижнем

углу обводки элемента, изменилось на число 2. Это значит, что теперь порядковый номер элемента стал равен не 1, а 2. При этом порядковый номер элемента, который изначально был равен двум (это логический элемент лампочки, расположенный в правом верхнем углу кластера) поменялся на порядковый номер элемента, которому мы присвоили порядковый номер 2. То есть порядковый номер логической лампочки стал равен 1 (рис. 95(c)), так как в кластере не может быть двух элементов с одинаковыми порядковыми номерами. По сути произошёл обмен порядковыми номерами между двумя элементами. Если вам потребуется изменить порядок всех элементов, то можно в окошечке рядом с названием **Click to set to** ввести значение 0 и нажимать на элементы кластера в той очередности, в которой вы хотите расположить их по порядковому номеру. После каждого клика по элементу число в окошке автоматически будет изменяться на следующую (был 0, стало 1 и т.д.) и вам не потребуется каждый раз вводить значение порядкового номера. То есть при нажатии на элемент числа в окошке будут циклично изменяться от меньшего значения, которое вы установите, до большего 95(e). Также в дополнительной полосе есть две кнопки: кнопка в виде галочки, кнопка в виде крестика. Если очерёдность элементов изменять больше не требуется, то вам нужно сохранить установленные порядковые номера нажав на кнопку в виде галочки. Если сохранение изменённых значений не требуется, и вы хотите выйти из режима редактирования – нажмите на крестик. При повторной смене порядковых номеров элементов, в белых окошечках каждого элемента будут отображаться не порядковые номера, присвоенные при помещении элемента в кластер, а номера, которые были установлены в предыдущем редактировании кластера.

Кластеры также, как и другие типы данных, имеют свой набор функций для создания и манипулирования ими, а также преобразования данных из одного формата в другой. Все функции работы с кластерами расположены в подпалитре ***Cluster, Class & Variant VIs***, рассмотрим наиболее часто используемые из них (рис. 96(b)). Кластер – это набор сгруппированных

данных различного типа, поэтому основными функциями работы с ними являются функции Объединения (**Bundle**) и Разделения (**Unbundle**) элементов. Как осуществляется использование функции **Bundle** мы разобрали на основе графиков, поэтому сразу перейдём к рассмотрению функции **Unbundle** (рис. 96(a)). С левой стороны функции расположен один входной терминал, к которому подключается кластер, с правой стороны расположены два выходных терминала, каждый из которых содержит информацию об одном из элементов кластера. Количество выходных терминалов изменяется с помощью растягивания функции. Задачей, для которой используется функция **Unbundle**, является разделение кластера на каждый из его отдельных элементов (разгруппировка данных кластера). Чтобы разобрать принцип работы функции, подключим созданный нами кластер к её входному терминалу. При подключении кластера к этой функции размер функции (количество выходных терминалов) автоматически изменяется для отображения выходных данных для каждого элемента в подключенном кластере. То есть мы подключили кластер из 4 элементов и получили 4 выходных терминала (рис. 96(d)). У каждого выходного терминала есть соединительная панель, на которой отображается тип данных, выходящих из этого терминала. К примеру, первый терминал содержит числовые данные с плавающей запятой, поэтому на соединительной панели написано оранжевыми буквами DBL, второй терминал – это логический терминал с надписью TF. Функция **Unbundle** также, как и функция **Bundle** выдает выходные данные в строго определённом порядке, в котором они отображаются в кластере.

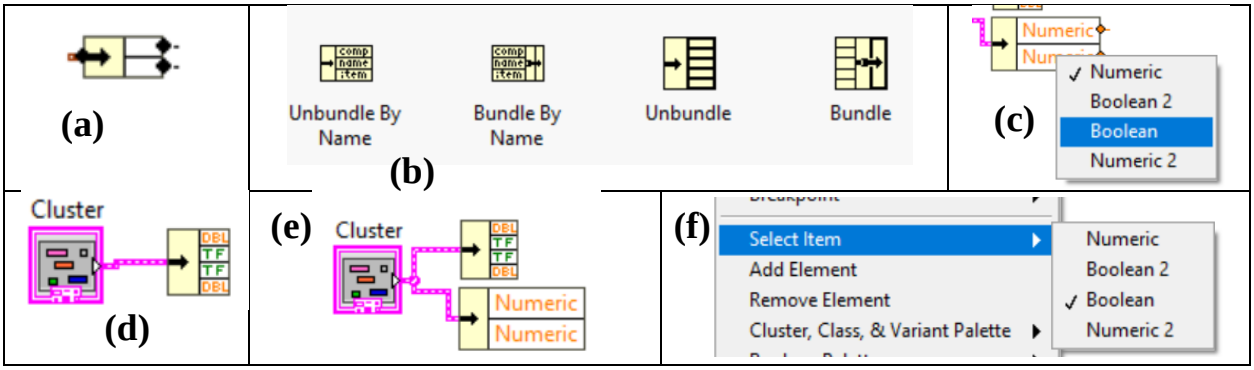


Рисунок 96. a) Вид терминала Unbundle. b) Основные функции работы с кластерами. c) Контекстное меню выбора выходных данных из функции Unbundle by Name. d) Разделение кластера на отдельные элементы. e) Разделение кластера на отдельные элементы по имени. f) Выбор выходных данных из кластера через контекстное меню.

Если у вас есть два или более элемента одного типа, следите за их порядком в кластере. Если вы по ошибке обращаетесь к неправильному элементу, LabView не сообщает об этом как об ошибке. То есть в созданном нами кластере есть два элемента (2 и 3 элементы), имеющих логический тип данных. Эти элементы разные, но их выходные терминалы одинаковые, поэтому не зная порядкового номера каждого элемента мы не сможем отличить их друг от друга. Помимо функций Объединения (**Bundle**) и Разделения (**Unbundle**) элементов для работы с кластерами также не редко используются функции Объединить по имени (**Bundle by Name**) и Разделить по имени (**Unbundle by Name**). Преимущества этих функций заключается в том, что при их использовании вы можете достать один или несколько элементов из всего кластера, а не разделять весь кластер на элементы, как это делают функции **Bundle** и **Unbundle**. Сначала рассмотрим принцип работы функции **Unbundle by Name**. Поместим функцию **Unbundle by Name** в окно блок-диаграммы и присоединим к её входному терминалу созданный кластер, который мы ранее подключили в функции **Unbundle** (рис. 96(e)). Кластер содержит в себе 4 элемента, однако после подключения кластера к функции **Unbundle by Name** количество выходных не стало равно 4, а осталось равным 2, точно таким же каким и было до подключения кластера. Это связано с тем, что эта функция может выделять только один или несколько элементов из кластера, поэтому она не требует, чтобы количество элементов соответствовало количеству выходных терминалов в кластере. В отличии от функции **Unbundle**, функция **Unbundle by Name** выделяет элемент из кластера не по порядковому номеру элемента, а по имени

элемента. Разберёмся как выбирать имя элемента, который мы хотим достать. Когда мы подключили кластер к функции, то в обоих выходных терминалах автоматически появилось имя одного из элементов кластера –Numeric. То есть оба выходных терминала выводят один и тот же элемент. Для примера поменяем имя второго выходного терминала на Boolean (выведем данные логического элемента Boolean). Это можно сделать двумя способами. Первый способ, щёлкните левой кнопкой мыши по имени элемента второго выходного терминала, перед вами появится список со всеми возможными именами элементов, которые содержаться в кластере (рис. 96(c)). Щёлкните левой кнопкой мыши по имени элемента, который вы хотите выбрать, выбранный элемент будет отмечен галочкой. Второй способ, щёлкните правой кнопкой мыши по выходному терминалу элемента и в контекстном меню наведите курсор мыши на строчку ***Select Items*** (Выбрать элемент) и вы выпадающем контекстном меню выберите имя нужного вам элемента (рис. 96(f)). Количество выходных терминалов, которые вы можете создать для одного и того же элемента в кластере не ограниченно.

Теперь рассмотрим принцип работы функции **Bundle by Name**. Функция **Bundle by Name** также, как и функция **Unbundle by Name**, обращается к каждому элементу по имени, а не по порядковому номеру. Однако, в отличии от функции **Bundle**, она не может быть использована для создания новых кластеров. Основной задачей этой функции является замена элементов в уже существующем кластере. Всего есть два варианта, с помощью которых можно осуществить замену элемента. Сначала рассмотрим вариант, в котором используется функция **Bundle**. Чтобы изменить один элемент в кластере, нам нужно подключить исходный (ещё не изменённый) кластер к функции (рис. 97(a)). Для этого подсоедините уже существующий кластер к входному терминалу, расположенному на верхней грани функции. Количество входных терминалов функции должно совпадать с количеством элементов в кластере (в нашем примере должно быть 4 терминала). Если после подключения количество терминалов автоматически

не стало равно количеству элементов, растяните функцию вручную, увеличивая количество терминалов до нужного значения. Далее с помощью контекстного меню выходного терминала создайте выходной кластер нажав на строчку **Create Indicator** (рис. 97(b)). На лицевой панели должен появится второй кластер, который будет отображать данные после замены элемента.

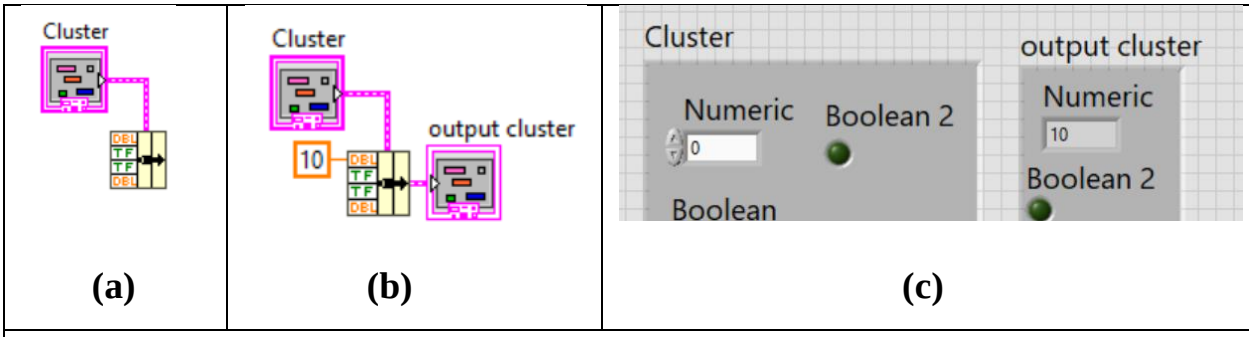


Рисунок 97. a) Схема подключения имеющегося кластера к функции Bundle. b) Графический код для замены значения в одном из элементов кластера. c) Отображение результата замены значения в элементе управления кластера.

Допустим мы хотим в элементе Numeric изменить значение на число 10. Элемент Numeric имеет порядковый номер 0, соответственно ему принадлежит самый верхний терминал функции. Создадим константу числа с плавающей запятой и поставим в неё значение 10. Подсоединим эту константу к верхнему терминалу функции **Bundle** и запустим выполнение программного кода. В результате в элементе Numeric, расположенном в выходном кластере (output cluster) на лицевой панели, значение станет равным числу 10, на которое мы произвели замену (рис. 97(c)). Аналогично происходит изменение значений других элементов, которые находятся в кластере. Главное не забывать следить за порядковым номером каждого элемента, чтобы произвести замену в нужном вам элементе. Стоит отметить, что не все элементы в кластере могут быть изменены.

Теперь рассмотри второй вариант замены элемента в кластере, который осуществляется с помощью функции **Bundle by Name**. Также, как и в функции **Bundle**, нам требуется подключить созданный кластер к входному терминалу на верхней грани функции (рис. 98(a)). Количество входных

терминалов при подключении кластера не изменится, так как для этой функции не важен порядок элементов. Если вам требуется заменить больше одного элемента в кластере, то количество входных терминалов требуется изменять вручную. Сразу после того, как мы подключили входной кластер, у входного терминала данных появляется самого первого элемента в кластере. Пусть мы хотим, чтобы светилась лампочка Boolean 2. Для этого нам нужно в элемент Boolean 2 подать логическую константу TRUE (рис. 98(b)).

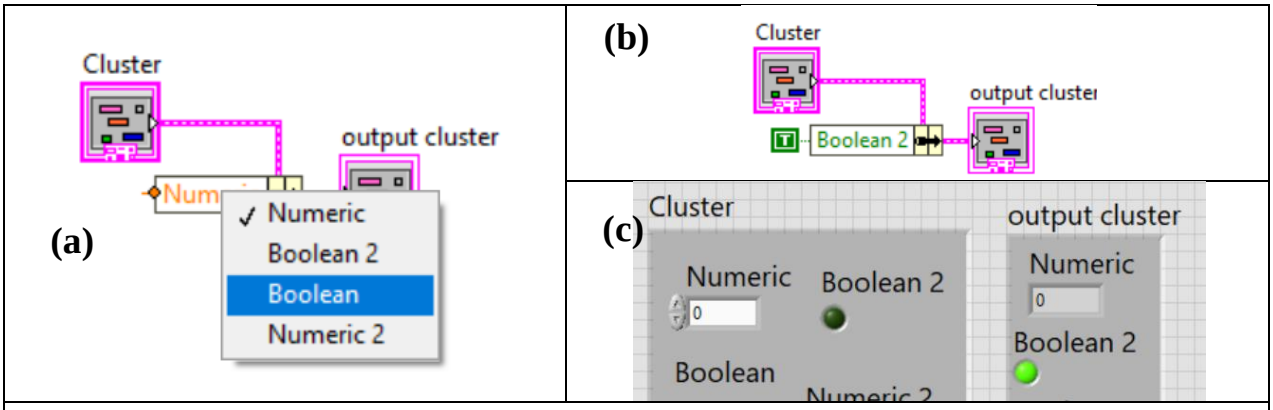


Рисунок 98. a) Схема подключения имеющегося кластера к функции Bundle. b) Графический код для замены значения в одном из элементов кластера по имени элемента. c) Отображение результата замены значения в элементе управления кластера.

Нажмём левой кнопкой мыши по имени терминала и выберем имя Boolean 2 (рис. 98(a)). Добавим логическую константу TRUE на окно блок-диаграммы (находится в подпалитре Boolean) и присоединим её к входному терминалу и запустим программу. В результате после запуска программы на лицевой панели мы увидим, что лампочка в кластере, отображающем выходные данные, стала светится (рис. 98(c)). Аналогично происходит изменение значения других элементов кластера. Использование функции **Bundle by Name** для изменения значений элементов удобнее, чем использование функции **Bundle**, так как нужно просто выбрать имя элемента, а не искать его среди большого количества других элементов и отслеживать его порядковый номер, чтобы не совершить ошибки.

Когда мы строили графики то использовали функцию **Build Array** для объединения двух кластеров. Разберёмся как эта функция работает с кластерами и в том, какой результат мы можем получить после её использования. Для примера возьмём уже написанный код для построения графиков, представленный на рисунке 93(a). Удалим проводники данных, соединяющие выходной терминал функции **Bundle** с входным терминалом **Build Array** (рис. 99(a)). Для каждого выходного терминала функции **Bundle** через контекстное меню создадим кластер, который будет отображать вид данных до объединения двух кластеров. После чего подключим снова каждый кластер к функции **Build Array** (рис. 99(b)). Теперь создадим код, который позволит увидеть, как выглядит содержимое кластеров после их объединения функцией **Build Array**. Для этого удалим проводник, который соединяет выходной терминал функции с терминалом графика. Щелчком правой кнопкой мыши по выходному терминалу функции **Build Array** откроем контекстное меню и нажмём на строчку **Create Indicator**, в окне блок-диаграммы появится терминал индикатора. Подсоединим проводник с выходными данными к графику и запустим программу (рис. 99(c)). Теперь перейдём на лицевую панели и посмотрим получившийся результат. Изначально кластеры, которые содержат данные для построения графика, выглядели как два раздельных кластера в каждом из которых содержится два элемента индикатора и одномерный массив с данными (рис. 99(d)). После объединения кластеров выходные данные будут отображаться в элементе с названием **appended array**. После запуска программы в элементе **appended array** будет отображаться только часть элемента, которая будет полностью повторять кластер с именем **output cluster** (рис. 99(f)).

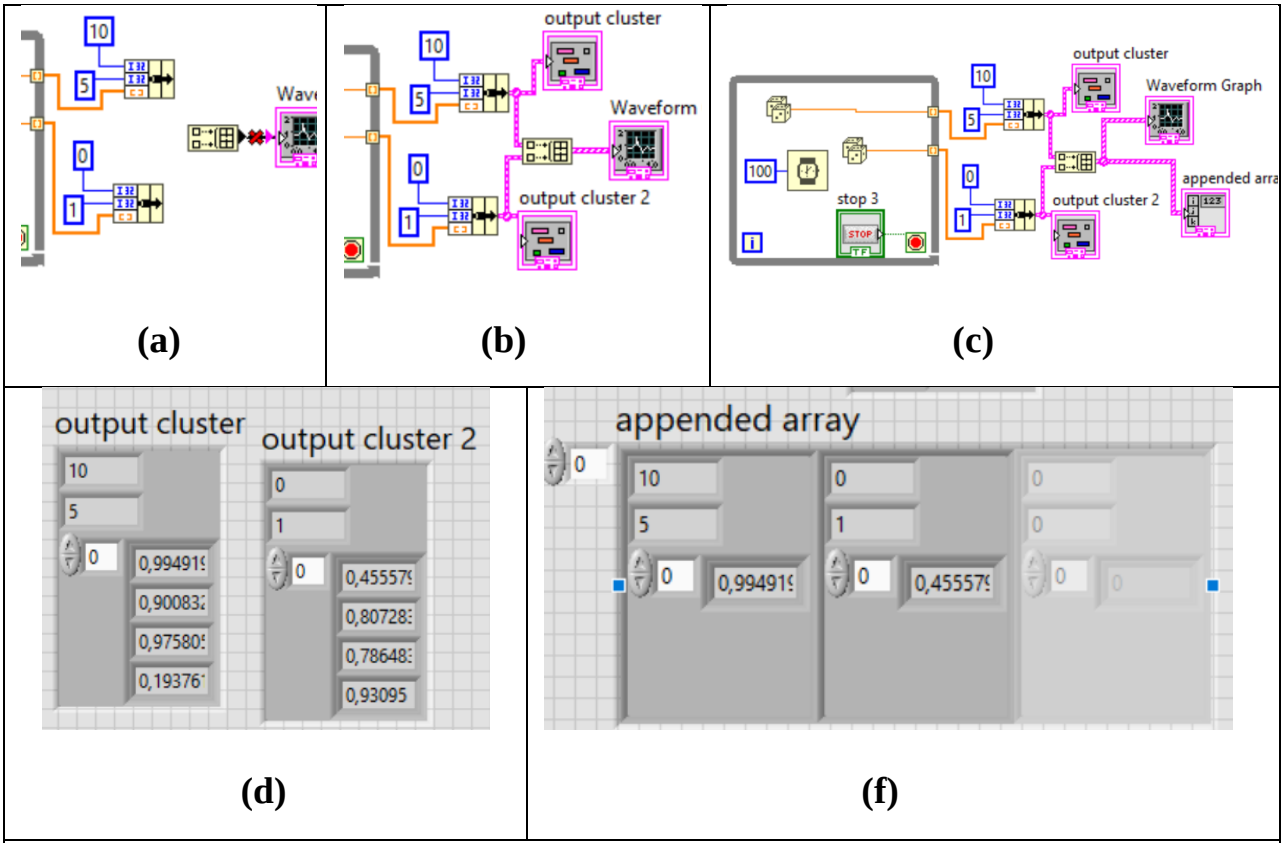
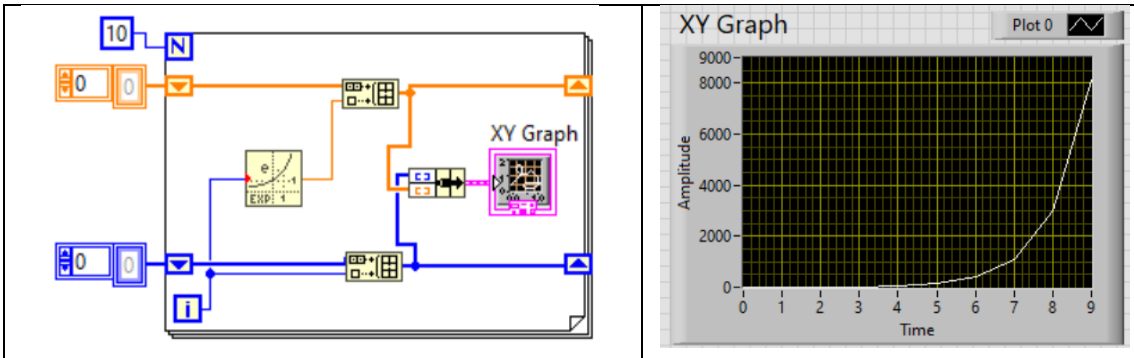


Рисунок 99. a) Вид графического кода после удаления проводников данных. b) Соединительная схема для построения нескольких графиков с различными временными параметрами. c) Графический код для построения двух графиков с разными пользовательскими временными параметрами с использованием Waveform Graph. d) Отображение построения двух графиков с разными пользовательскими временными параметрами в окне Waveform Graph. e) Вид массива состоящего из кластеров.

Для того, чтобы увидеть всё содержимое элемента выделите его и растяните (рис. 99(f)). Теперь мы видим, что после объединения двух кластеров с помощью функции **Build Array**, мы получили массив, который состоит из кластеров, которые полностью идентичны двум отдельным кластерам. То есть, если мы присоединяем кластеры к функции **Build Array**, то в результате получим массив, который будет состоять из кластеров, которые мы объединили. Следовательно, массив может состоять не только из обычных числовых значений, но и из группы данных различного типа. Логические элементы также могут быть использованы как элементы массива.

Двухкоординатные графики

Следующий вид графика, который можно использовать в среде LabVIEW, это график ***XY Graph***. ***XY Graph*** – это двухкоординатный график, в котором положение каждой точки описывается двумя координатами: координатой X, координатой Y. Соответственно, для того, чтобы построить двухкоординатный график, требуется подавать на входной терминал графика два массива, один массив, содержащий значения X, а второй массив, содержащий значения Y. В разделе про массивы данный тип графика уже был подробно разобран, поэтому в этой части пособия будет разобраны дополнительные (не базовые) возможности графика, которые не редко используются для создания программ осуществляющих обработку данных. Но для начала рассмотрим возможности интерфейса графика, которые не были разобраны в предыдущих видах графиков. Создадим программный код на основе которого будем разбирать все возможности графиков. Для примера возьмём математическую функцию, которая строит экспоненту и напишем код, представленный на рисунке 434. Запустите программу и проверьте правильно ли она написана, на графике, расположенном на лицевой панели, должна появится экспоненциальная линия (рис. 100(a)).

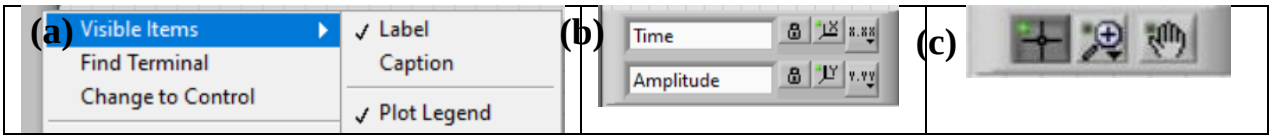


a) b)

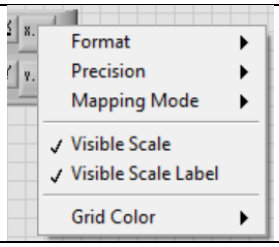
Рисунок 100. a) Графический код для построения функции экспоненты. b) График экспоненты, построенный с помощью XY Graph.

У каждого графика есть набор дополнительных палитр, которые расширяют возможности редактирования графиков. Чтобы увидеть весь список палитр

щёлкните правой кнопкой мыши по графику, чтобы открыть контекстное меню, после чего наведите курсор мыши на первую строчку с названием **Visible Items**, в контекстном меню которой содержится список с палитрами редактирования (рис. 101(a)): Scale Legend, Graph Palette, Cursor Legend. Включим отображение палитры Scale Legend и разберём её функционал. На лицевой панели появится окно, которое состоит из двух полос, каждая из которых позволяет осуществлять редактирование шкалой X (первая полоса) или шкалой Y (вторая полоса) (рис. 101(b)). В начале каждой полосы расположено белое окно, в котором указано название шкалы. Нажав на это окно левой кнопкой мыши, вы можете переименовать выбранную шкалу удалив текущее название после чего написать новое. Изменение названия в окне графика вдоль осей X или Y произойдёт автоматически. Справа от окна с именами шкал расположена кнопка в виде замка – кнопка блокировки. Нажатие на эту кнопку позволяет переходить между ручным и автоматическим режимом настройки шкалы. В ручном режиме настройки шкалы на кнопке изображён значок открытого замка, в автоматическом режиме – значок закрытого замка. Иными словами, нажатие на кнопку блокировки позволяет включать или выключать режим автоматического масштабирования шкалы – AutoScale X (или AutoScale Y). Справа от кнопки блокировки расположен индикатор, который отображает включен режим автомасштабирования или нет.



(d)



(e)

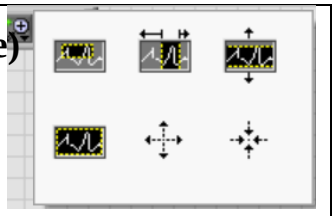


Рисунок 101. a) Вид контекстного меню, содержащего все палитры для редактирования графика. b) Вид палитры редактирования шкал Scale Legend. c) Вид палитры элементов управления графиком Graph Palette. d) Контекстное меню крайней кнопки из палитры шкал Scale Legend. e) Контекстное меню с вариантами прокрутки графика.

Если огонёк в левом верхнем углу кнопки горит зелёным, значит шкала находится в режиме AutoScale, если огонёк серого цвета, то шкала находится в режиме ручного масштабирования. Далее в палитре редактирования находится кнопка, нажатие на которую открывает контекстное меню (рис. 101(d)). В контекстном меню содержатся функции для настройки формата (Format), точности (Precision) и режима отображения шкал (Mapping Mode). Вы можете убрать название шкалы графика нажатием на строчку Visible Scale Label и при необходимости полностью убрать отображение всей шкалы нажав на Visible Scale. Нажатие на строчку Grid Color позволяет изменить цвет сетки, на фоне которой осуществляется построение графика.

Следующая палитра, позволяющая осуществлять редактирование графика – это палитра элементов управления графиком (Graph Palette). Включим её отображение поставив галочку в контекстном меню **Visible Items** в строчке **Graph Palette**. В левом нижнем углу графика должна появится небольшая палитра, которая состоит из трёх кнопок, позволяющих управлять графиком, плавно прокручивать изображение графика, а также фокусироваться на определенной области его области (рис. 101(c)). Нажатие на одну из кнопок включает определённый режим работы с графиком. Лампочка, расположенная в левом верхнем углу каждой кнопки, позволяет определить какой режим работы графика выбран на данный момент. Зелёная

лампочка означает, что выбран режим работы графика, соответствующий нажатой кнопке, серая лампочка – режим соответствующей данной кнопке не активен. Рассмотрим подробнее функцию каждой кнопки этой палитры.

- Первая кнопка палитры позволяет выбрать стандартный режим работы графика (по умолчанию изначально график находится в этом режиме). В стандартном режиме отображения графика вы можете выбирать установленные вами курсоры на графике (как устанавливать курсоры на графике будет рассмотрено ниже), а также осуществлять их перемещение по всему пространству графика.
- Вторая кнопка скрывает внутри себя большое контекстное меню, которое содержит множество функций для различных вариантов прокрутки графика с помощью курсора мыши (рис. 101(e)).

1. Первый способ изменения вида графика – это увеличение с помощью прямоугольной области (Zoom by Rectangle). Чтобы включить этот режим увеличения нажмите на первый элемент первой строки контекстного меню. После нажатия на элемент увеличения курсор мыши превратится в лупу со знаком плюс внутри. Чтобы увеличить нужную вам область графика, зажмите левую кнопку мыши и выделите прямоугольником требуемый участок графика, не отпуская левую кнопку мыши. Когда область для увеличения выделена, отпустите кнопку мыши, после чего оси автоматически изменят масштаб так, чтобы показать лишь выделенную прямоугольником область графика.

2. Второй способ изменения вида графика – это увеличение размеров графика вдоль оси X (Zoom by Rectangle in X). Чтобы включить этот режим увеличения нажмите на второй элемент первой строки контекстного меню. После нажатия на элемент увеличения курсор мыши превратится в лупу со знаком плюс внутри и стрелочкой вправо-влево в нижней части лупы, что будет указывать на возможность увеличения графика только по оси X. Для выделения области зажимаем левую кнопку мыши и выделяем двумя пунктирными вертикальными линиями область,

которую хотим увеличит. Область, заключенная между двумя линиями, будет увеличена только по оси X, при этом размер графика по оси Y остаётся неизменным и равным размеру всего графика.

3. Третий способ изменения вида графика – это увеличение размеров графика вдоль оси Y (Zoom by Rectangle in Y). Для включения этого режима нажмите на третий элемент первой строки контекстного меню. Курсор мыши превратится в лупу со знаком плюс внутри и стрелочкой вверх-вниз, расположенной слева от лупы, что будет указывать на возможность увеличения графика только по оси Y. Для выделения области, которую надо увеличить, зажмите левую кнопку мыши и ограничьте её двумя пунктирными горизонтальными линиями. После отпустите кнопку мыши и график автоматически увеличится под размер выделенной области по оси Y, при этом размер по оси X останется неизменным.

4. Первая кнопка во втором ряду возвращает график к исходному состоянию. То есть, если вы изменяли масштаб графика и теперь хотите отобразить его в полном размере, нажмите на эту кнопку, чтобы вернуть прежний масштаб.

5. Второй элемент, расположенный во втором ряду контекстного меню, позволяет увеличивать размер графика около выбранной вами точки (Zoom in about a Point). Выберите точку, в которой вы хотите увеличить размер графика и установите в неё курсор мыши. Далее зажмите левую кнопку мыши для того, чтобы увеличить график. График будет увеличиваться до тех пор, пока вы держите левую кнопку мыши зажатой, когда вы отпустите кнопку, увеличение графика в выбранной точке прекратится. При этом вы можете изменять точку, в которой хотите увеличить график, не отжимая кнопку мыши, просто передвигайте курсор по полю графика, область увеличения будет следовать за курсором.

6. Третий элемент, расположенный во втором ряду контекстного меню, уменьшает размер графика около выбранной вами точки (Zoom out about a Point). Механизм изменения масштаба графика аналогичен механизму

в предыдущем пункте. Вы выбираете точку на графике, в которой вы хотите уменьшить размер графика и устанавливаете в неё курсор мыши. После зажатия левой кнопки мыши график будет уменьшаться всё время, пока левая кнопка мыши остаётся зажатой. Когда кнопка будет отпущена, уменьшение графика в выбранной точке прекратится. При передвижении курсора по полю графика, область также, как и в предыдущем случае, будет следовать за курсором.

- Третья кнопка палитры представляет собой ладонь, с помощью которой вы можете перетаскивать график по всему окну. Для этого зажмите левую кнопку мыши в любой точке графика и передвигайте курсор мыши по окну. Когда кнопка мыши зажата курсор в виде ладони принимает вид сжатой ладони.

Для работы и редактирования графиков в среде LabVIEW есть ещё одна спрятанная палитра с названием Cursor Legend, которая позволяет создавать и редактировать курсоры, созданные для графика. Чтобы сделать эту палитру видимой откройте контекстное меню графика и выберите ***Visible Items→ Cursor Legend***. Перед вами появится большое пустое окно с тремя колонками: Cursor, X, Y, а также небольшое окно с набором кнопок в виде ромба (рис. 102(a)). Разберёмся как создавать курсоры для графика. Щёлкнем правой кнопкой мыши внутри окна с тремя колонками, чтобы открыть контекстное меню палитры. Чтобы создать курсор на графике выберите строчку Create Cursor, нажатие на которую открывает ещё одно контекстное меню, в котором нужно выбрать тип курсора. Можно выбрать один из трёх вариантов курсора, разберём подробнее каждый из них (рис. 102(b)).

- Первый вариант курсора – это свободный курсор (Free). Данный тип курсора не привязывается ни к какому графику, поэтому его можно свободно перемещать в пределах области построения, независимо от положений построения.

- Второй вариант курсора – это одиночный график (Single Plot). Данный тип позволяет размещать курсор только на том графике, с которым

связан данный курсор. Вы можете перемещать курсор вдоль всего связанного графика.

- Третий вариант курсора – это многоплоскостной курсор (Multi-plot). Данный тип позволяет размещать курсор только на определенной точке данных в области построения. Вы можете расположить курсор на любом графике в области построения. Этот режим курсора возможно использовать только для нескольких графиков, построенных в одном окне.

Обратите внимание, что вы не можете изменить режим курсора (тип курсора) после его создания. Необходимо удалить курсор и создать другой курсор.

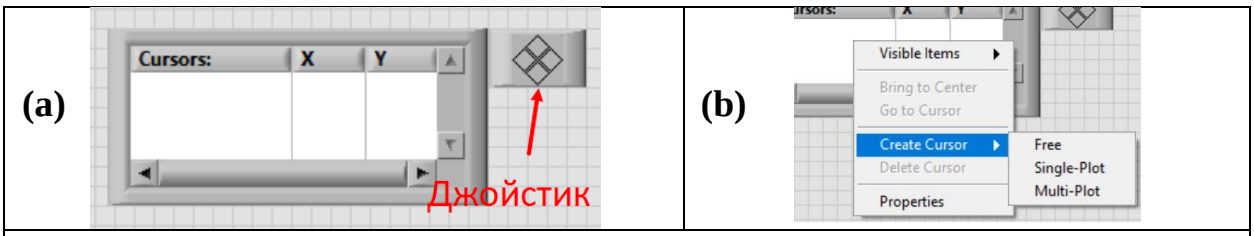


Рисунок 102. a) Диалоговое окно для редактирования курсоров графика. b) Контекстное меню с выбором типа создаваемого курсора.

Количество курсоров, которое вы можете создать для одного графика, не ограничено. Создадим, для примера, курсор типа Single Plot, в окне отображения курсора должна появится информация о нём (рис. 103(a)), в окне графика также должен появится созданный курсор. Первая колонка с названием **Cursors:** отображает информацию о самом курсоре. В чёрном окошечке представлен уменьшенный вид курсора при его отображении в окне графика. То есть, если курсор на графике экспоненты обозначается в виде жёлтого крестика с пересекающимися жёлтыми линиями, то и на уменьшенной копии в палитре редактирования курсор будет иметь точно такой же вид. Справа от миниатюрного изображения курсора находится его имя (в нашем случае Cursor 0), которое можно изменить на любое другое. Для это двойным щелчком левой кнопки мыши кликнете по названию, удалите его и введите новое. Под названием курсора расположено название

графика, к которому привязан созданный курсор – Plot 0. Название графика можно скрыть, нажав на кнопку с минусом, расположенную слева от изображения миниатюры курсора. Вторая и третья колонка в панели отображают координату курсора X и Y, соответственно. После создания нового курсора он появляется в начале координат графика, поэтому его координаты имеют значения X=0 и Y=0 (рис. 103(a)). Попробуем подвигать курсор и посмотреть, как будут изменяться значения его координат в палитре. Установим курсор мыши на пересечение двух жёлтых линий Cursor 0 так, чтобы стрелка мыши изменила свой вид на крестик. Зажмём левую кнопку мыши и начнём передвигать курсор по полю графика. Вместе с изменением положения курсора его координаты, отображаемые в палитре, также будет изменяться (рис. 103(c)).

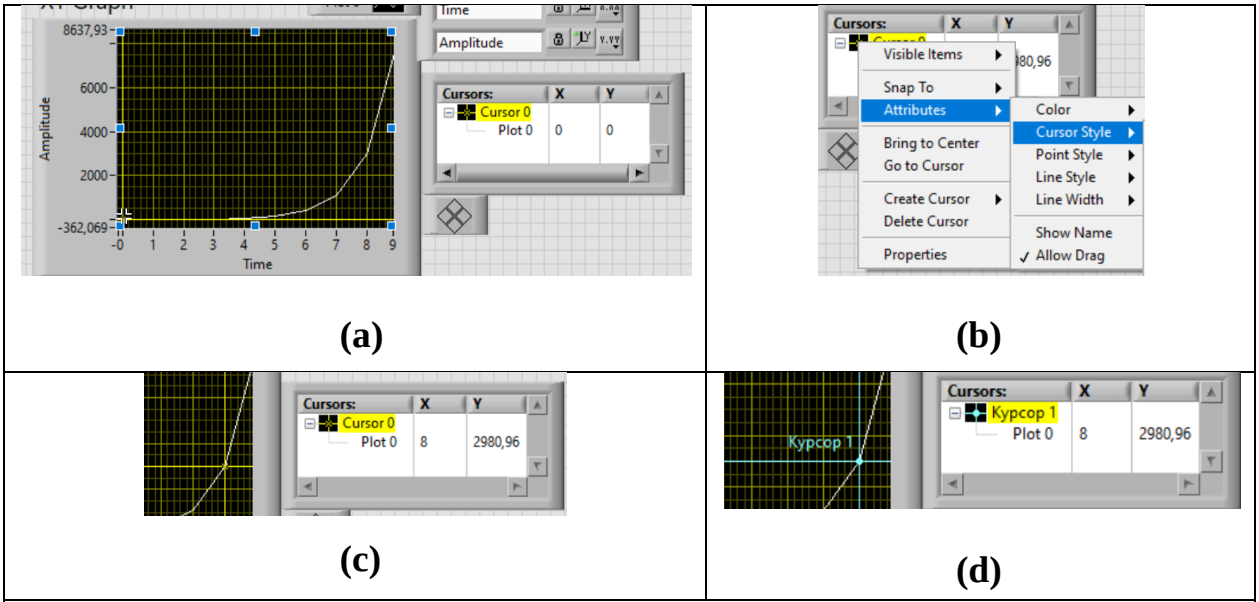


Рисунок 103. a) Вид курсора в окне графика. b) Контекстное меню с элементами редактирования графика. c) Отображение координат курсора в палитре редактирования курсора. d) Изменённый вид курсора графика.

Перемещать курсор по полю графика можно с помощью джойстика, который имеет вид ромба, состоящего из четырёх отдельных ромбиков. Каждый ромбик позволяет передвигать курсор в определённом направлении. Кнопка в виде ромбика становится тёмно-серого цвета, когда она зажата и курсор осуществляет движение. У каждого курсора есть своё контекстное

меню, которое содержит функции для его редактирования. Чтобы его открыть нажмите правой кнопкой мыши по названию курсора (рис. 103(b)). Рассмотрим функции, содержащиеся в этом контекстном меню.

- ***Visible Items*** содержит в себе контекстное подменю, в котором находятся функции, позволяющие включать отображение вертикальной (Vertical Scrollbar) и горизонтальной полосы (Horizontal Scrollbar) прокрутки в палитре, а также функция отображения названия каждой колонки в палитре (Column Headers).
- ***Snap To*** (привязка к) — фиксирует курсор на графике. Когда режим курсора - одиночный график, вы можете привязать курсор к одному или ко всем графикам. Когда режим курсора - многоплоскостной, привязка влияет только на перемещение курсора вдоль оси X.

Следующие две функции появляются в контекстном меню курсора только в случае свободного типа курсора (то есть курсор не привязан ни к одному графику).

- ***X Scale*** — устанавливает масштаб курсора по оси X.
- ***Y Scale*** — Устанавливает масштаб курсора по оси Y.
- ***Watch*** (Просмотр). Функция ***Watch*** есть в контекстном меню только в режиме курсора Multi-plot. Эта функция связывает курсором один или несколько указанных графиков или все графики. Многоплоскостной курсор сообщает значения с указанным значением x для всех графиков, с которыми связан курсор.
- Контекстное подменю Attributes состоит из параметров, которые позволяют настроить внешний вид каждого курсора. Опишем каждый из параметров (рис.213):

1. Color (Цвет) — отображает средство выбора цвета, чтобы вы могли выбрать цвет курсора.
2. Cursor Style (Стиль курсора) — предоставляет различные стили курсора.

3. Point Style (Стиль точки) — предоставляет различные стили точек для обозначения точки пересечения горизонтальной и вертикальной линий курсора.

4. Line Style (Стиль линии) — предоставляет различные стили сплошных и пунктирных линий.

5. Line Width (Ширина линии) — обеспечивает различную ширину линии.

6. Show Name (Показать название) — отображает название курсора на графике. Зажимая название курсора на поле графика левой кнопкой мыши, вы можете перемещать это название относительно курсора.

7. Allow Drag (Разрешить перетаскивание) — позволяет мыши перетаскивать курсор по колю графика.

- ***Bring to Center*** (Привести к центру) — Центрирует курсор на графике без изменения масштабов X и Y. Когда режим курсора Single Plot или Multi-plot, этот параметр центрирует курсор на графике, на котором курсор находится в данный момент, и обновляет координаты курсора в условных обозначениях курсора. Когда режим курсора свободен, эта опция центрирует курсор в области построения и обновляет координаты курсора в условных обозначениях курсора.

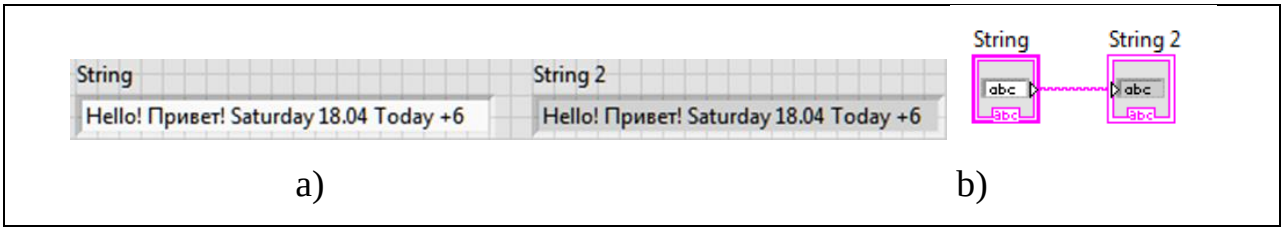
- ***Go to Cursor*** (Перейдите к курсору) — нажатие на строчку помещает курсор в центр окна построения графика, при этом сам график также перемещается вместе с курсором. После перемещения курсора в центр часть графика может быть обрезана, поэтому может потребоваться изменение масштаба осей графика.

- ***Delete Cursor*** – удаляет курсор, для которого было открыто это контекстное меню.

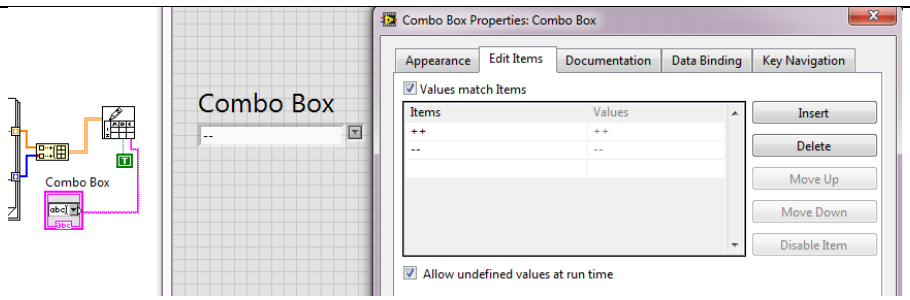
Попробуйте изменить вид курсора так, чтобы он более явно выделялся на графике (к примеру, как на рис. 103(d)).

Строковый тип данных

С практической точки зрения знания о типе данных String нужны для взаимодействия с реальными приборами в ходе эксперимента. Приборы, работающие с использованием разных физических интерфейсов (COM, USB, GPIB, LAN и др.) взаимодействуют с LabView путем получения и передачи данных строкового типа в кодировке ASCII. Рассмотрим основы работы с данными такого типа. На передней панели необходимые элементы находятся в пункте String and Path, а в кодовой панели в пункте String. Простейшее действие, как и с числовыми данными – создание элемента управления String Control и элемента отображения String Indicator. Как упоминалось выше, элементы управления/отображения и проводники сданных строкового типа в кодовой панели будут выделены красным цветом. Разместим на передней панели String Control, String Indicator и соединим их в кодовой панели. Если написать что-нибудь в String Control и запустить программу, что написанное будет передано в String Indicator и отображено на передней панели (рисунок 104 (a, b)). В пункте String and Path палитры кодовой панели есть еще элемент ввода строковых данных Combo Box. Он работает так же, как элемент выбора Ring для числовых данных – можно записать несколько выбираемых вариантов с разными названиями (Items) и будут выводиться разные записанные строки (Values), соответствующие этим названиям. По умолчанию название пункта и выводимая строковая константа совпадают (поставлена галочка в пункте Value much Items, рисунок 104(c)).



a) b)



c)

Рисунок 104. Элементы интерфейса String Control и String Indicator (a)) и соответствующие им значки в кодовой панели (b)). c) Элемент интерфейса Combo Box и задание с помощью его типа разделителя в сохраняемом файле.

Возьмем пример, рассмотренный выше (рисунок 73(b)). Разместим на передней панели Combo Box и создадим два варианта Items - один с двумя символами "+", второй с двумя символами "-" и подключим Combo Box к терминалу delimiter функции Write To Spreadsheet File вместо функции Select (рисунок 104(c)). Теперь в выпадающем меню функции Combo Box можно задать тип разделителя ("++" или "--") между колонками в сохраняемом файле.

При работе с реальным прибором, который выдает запрашиваемые данные в строковом виде, нужно уметь работать с получаемыми строковыми данными для выделения нужных данных и их дальнейшей обработке в программе. По своей сути строка является одномерным массивом символов и работа со строковыми данными похожа на работу с массивами. Нужные функции находятся в разделе String палитры кодовой панели. Функция String Length определяет длину строки, т.е. сколько символов (включая пробелы), так же, как команда для определения длины числового массива. Функция Concatenate Strings объединяет строки путем добавления одной к концу другой (рисунок 105(a)). Важный момент - в контекстном меню, появляющемся при нажатии правой кнопки мыши на элементы интерфейса на передней панели или строковую константу в кодовой панели, есть пункт выбора вида отображения строковых данных (рисунок 105(b)). По

умолчанию включен вид отображения Normal Display. Часто полезно переключить вид отображения в Code Display, в этом случае отображаются служебные символы, в частности пробел в виде символов \s. Важным символом также является символ Подача на строку (\n) задающий перемещение в начало следующей строки (вводится комбинацией клавиш Shift+Enter). Например, в окне String Indicator на рисунке 105(c) после символа * есть пробел, а после пробела есть перевод строки. В обычном виде эти символы не видны. В режиме Code Display видно, что пробелу соответствует символ \s и виден символ \n (рисунок 105(d)). Также в кодовом виде кириллица имеет свою специфику, но при взаимодействии с приборами она не используется. Кодовой вид необходим при формировании команд, отправляемых в прибор, в частности, хорошо видено присутствие пробела (символ \s). В обычном отображении пробел заметно плохо и это может привести к отправке в прибор неверно сформированной команды.

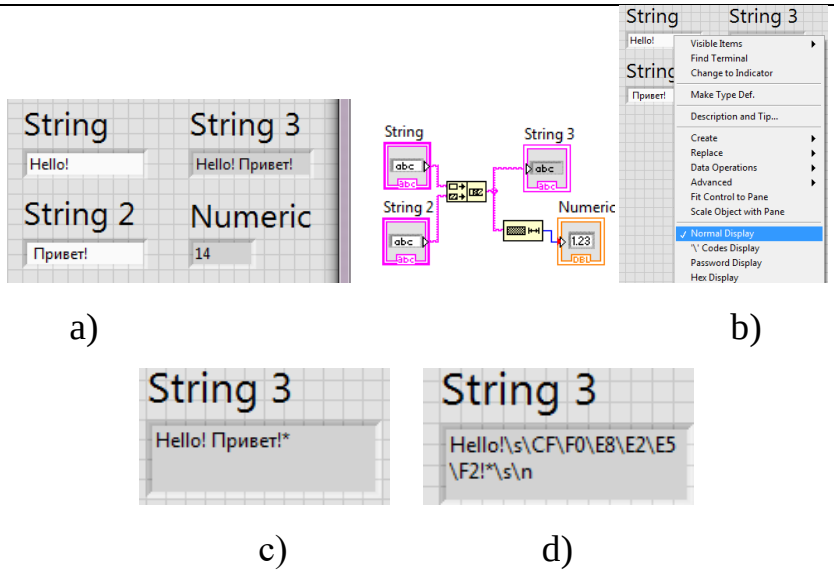


Рисунок 105. a) Объединение двух строк функцией Concatenate Strings и определение числа символов в результирующей строке функцией String Length. b) Вид отображения данных строкового типа.

Из данных типа String можно создавать массивы. На передней панели добавим пустой массив из пункта Array и поместим туда пустой String Control, тем самым задав тип массива. Пусть будет это будет матрица 3 на 3

строки. Заполним словами строки созданного массива (рисунок 106). Для работы с матрицами данных строкового типа можно использовать функцию Index Array из пункта Array палитры кодовой панели. При подсоединении матрицы строковых данных к функции Index Array у не появится два входных терминала row и col, на которые нужно подать числовые константы, определяющие номер колонки и строки матрицы из которых извлекается нужное слово (строка) (рисунок 106).

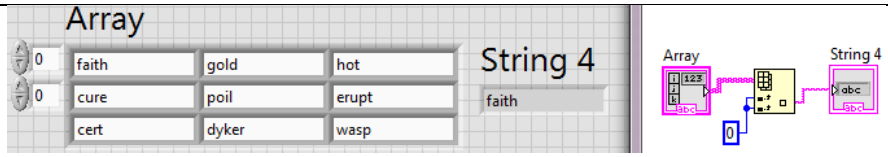


Рисунок 106. Извлечение слова из матрицы строковых данных.

С большим классом приборов удаленное (с компьютера) взаимодействие происходит с использованием системы команд SCPI (Standard Commands for Programmable Instruments), в виде данных строкового типа в формате ASCII. Синтаксис SCPI имеет иерархическую структуру с относительно интуитивно понятными командами. Разделим приборы на два уловных класса - измерители и источники. Есть также комбинированные источники-измерители. Измерители должны выдавать в компьютер значения измеренной электрической величины. Такие приборы – АЦП, мультиметры, осциллографы и др. Источники — это генераторы напряжения или тока всевозможных видов. Начнем с измерителей, например, вольтметры. Нам требуется фиксация и протоколирование в программе величины измеряемого вольтметром напряжения. Чтобы прибор начал передавать данные в аппаратный интерфейс (например, в COM порт) требуется отправка в прибор соответствующей команды. Например типичные в системе SCPI команды :READ?, SENS? или :MEAS? Команды могут быть расширенные, например, :MEAS:CURR? (измерение тока), SENS:FREQ:DATA? (запрос на измерение частоты). В ответ на команду прибор будет непрерывно выдавать в интерфейс строковые данные различной структуры, которые нужно считать и

обработать в LabView. За один такт выдачи данных прибор может выдавать, как простую строковую структуру (например, просто напряжение в вольтах с десятичным разделителем) или сложную строку, которая содержит несколько параметров (напряжение с нескольких каналов или разные данные – частоту, напряжение, ток, сопротивление и т.д.). Часто нужно из полученной строки выделить данные, находящиеся в определенной позиции. Данные в строке могут иметь разделитель, например, tab, пробел, запятая, точка с запятой и др. Смоделируем простой пример. Пусть прибор выдает строку из трех различных значений в виде одного разряда до десятичного разделителя (точка) и трех разрядов после десятичной точки. Значения разделены пробелом, в конце строки стоит команда Подача на строку (рисунок 107(a)). В пункте String кодовой палитры есть специальные функции для работы со стоками. Простейшая функция String Subset - извлечение части строки. Функция имеет входной терминал для ввода обрабатываемой строки, а также входные терминалы offset и length для задания номера первоначально извлекаемого символа и числа извлекаемых символов, соответственно. В выходном терминале будет появляться вырезанная часть строки. Извлечем в рассматриваемом примере с использованием этой функции первое число (первые 5 символов) (рисунок 107(a)). Для выделения второго числа нужно использовать еще одну функцию String Subset. При этом offset будет 6 символов (5 символов первого числа и пробел), а length также 5, как в первом случае (рисунок 107(b)). Для работы с извлеченными числами нужно их преобразовать в числовой тип данных. Для этого используется функцию Fract/Exp String To Number Function из подпункта Number/String Conversion пункта String палитры кодовой панели. Функция распознает знак «-», а знак «+» игнорируется. Не надо забывать, что эти знаки считаются отдельным символом. Выделим первое и второе число в строке и получим их отношение (рисунок 107(b)). Следует обратить внимание, что функцию Fract/Exp String To Number Function не нужно путать с функцией Decimal String To Number

Function. Функция Decimal String To Number Function выдаст целочисленное число при конверсии из строкового типа данных в числовой.

Однако при работе с данными, получаемыми с реального прибора, функция String Subset обычно не подходит. Проблема состоит в том, что длина (в символах) выдаваемого прибором значения может меняться. Например, вольтметр вместо величины 1.545 В выдал 10.545 В, и обработка строки путем отсчета количества символов от ее начала даст сбой.

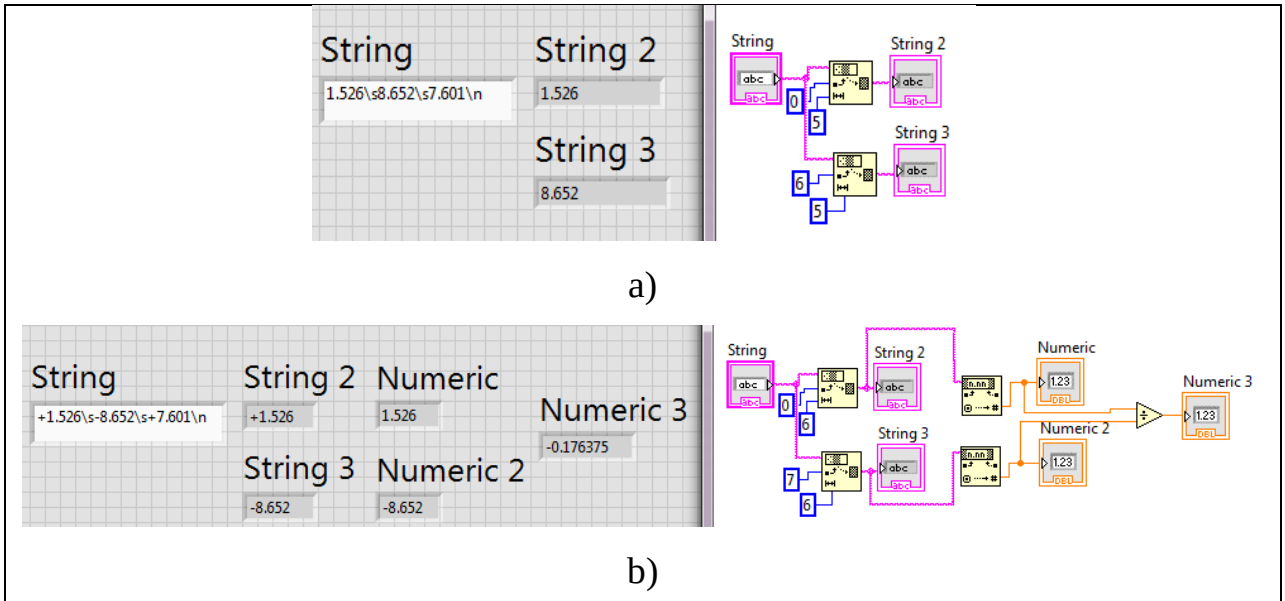


Рисунок 107. Модель данных с прибора в виде строки, содержащей три значения, разделенные пробелом. Извлечение с помощью функции String Subset первого числа (первых 5 символов из строки) и второго числа.

Нужно использовать более гибкие функции. Например, функция Match Pattern, расположенная в пункте String. Функция имеет входной терминал string для исходной строки и входной терминал regular expression для задания строковой константой (как правило) участка строки (символы), которые нужно найти в строке. Выходные строковые терминалы: before substring (часть строки до заданного символа или символов), match substring (найденные символы) и after substring (часть строки после найденных символов). Обратим внимание, что функция срабатывает для первых попавшихся совпадающих символов. В нашем примере три числа разделены

пробелом. Будет найден первый пробел, он отделяет первое число от двух остальных. На выходном терминале before substring появится нужное число в строковом формате (рисунок 108(a)). Для разделения двух остальных нужно снова использовать эту функцию для остатка строки (выходит из терминала after substring). После использования этой функции длина чисел может становиться любой, главное, чтобы разделитель (в рассматриваемом примере пробел) сохранялся. Очевидно, что разделитель может быть любым – символ, слово, группа слов и символов. Еще есть полезная функция Scan From String. Она позволяет легко получать из строки данные непосредственно в числовом формате с плавающей точкой. Основной входной терминал функции – строковый терминал input string. Если данные в исходной строке разделены пробелом, что функция Scan From String автоматически выделит участи строки между пробелами и на выходе функции (если потянуть вниз значок функции) будет три выходных числовых терминала с данными из строки, преобразованные в числовые данные. Однако, если разделитель в строке не пробел, то требуется указать разделитель. Для этого нужно использовать входной терминал format string. К этому терминалу нужно подвести строковую константу, содержащую тип разделителя и тип искомых в строке данных. Например, если во входной строке есть три группы чисел, разделенные точкой с запятой, то в этом случае нужно на терминал format string подать строковую константу %f;%f;%f (после последней f никаких символов нет) (рисунок 108(b)). Символ % указывает на начало определения участка формата строки, символ f означает, что нужно выделить данные с плавающей точкой, а символ «;» задает, что в строке данные с плавающей запятой разделены точкой с запятой. Использование символа f три раза означает, что нужно найти три числа с плавающей точкой. Нужно обратить внимание, что символ «;» используется два раза, т.к. после последнего числа этого разделителя нет. Также следует обратить внимание, что по умолчанию индикатор числовых данных Numeric Indicator отображает значащих цифр. В рассматриваемом примере второе число состоит из 10 цифр и будет

отображено не полностью. Чтобы полностью отображать числа нужно в свойствах Numeric Indicator найти раздел Display Format и поставить нужно число отображаемых цифр (рисунок 108(b)).

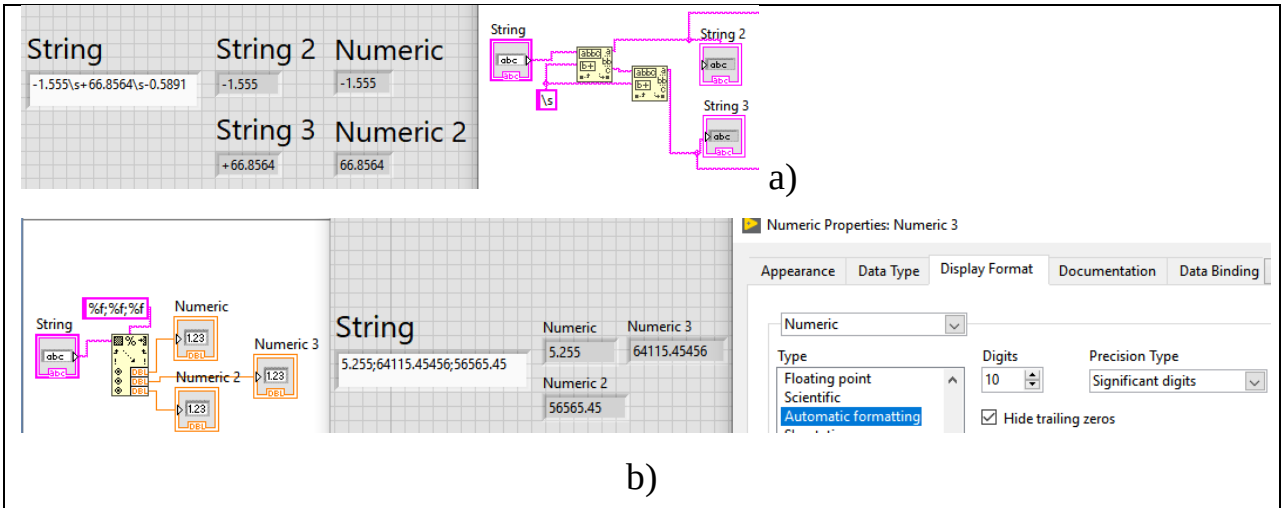


Рисунок 108. a) Использование функции Match Pattern для разделения строки до и после заданного символа. b) Использование функции Scan From String для поиска в строке чисел и преобразования в числовой формат.

Для управления приборами часто требуется также обратная задача - формирование командной строки по определенному шаблону с меняющейся числовой частью. Например, для выдачи источником питания напряжения заданной величины он должен получить команду вида :SET:VOLT:1.5, где часть строки «1.5» задает нужное напряжение и будет меняться в ходе выполнения программы. После получения команды источник выставит свое напряжение выхода 1.5 В. Пусть нужно проводить развертку напряжения на источнике питания, тогда нужно в прибор отправлять команду :SET:VOLT: с меняющейся величиной напряжения. Изменение величины напряжения реализуется циклом, меняющим этот числовой параметр. Далее числовой параметр нужно добавить к строке :SET:VOLT:. Сначала нужно преобразовать генерируемое в цикле число в строку. Для этого используем функцию Number To Fractional (из пункта String в кодовой палитре). На терминал number функции Number To Fractional подается нужное число, а на терминал precision подается числовая константа, определяющая число знаков

после запятой в выходной строке. Далее функцией Concatenate Strings объединим получившуюся строку со строкой :SET:VOLT: и тем самым получим нужную команду для прибора (рисунок 49).

Для решения этой задачи также можно использовать функцию Format Into String (из пункта String в кодовой палитре). Функция имеет строковый терминал для исходной строки и числовой терминал для добавляемого числа. Следует обратить внимание, что может быть необходимо указать число знаков после десятичной запятой. Если этого не сделать, что после запятой кроме значащих чисел могут быть поставлено несколько нулей, что может быть плохо воспринято прибором. Для задания формата числа в выходной строке нужно использовать входной терминал format string. Подав, например, строковую константу «%.2f» укажем, что нужно поставить только два символа после десятичной запятой (рисунок 109(b)).

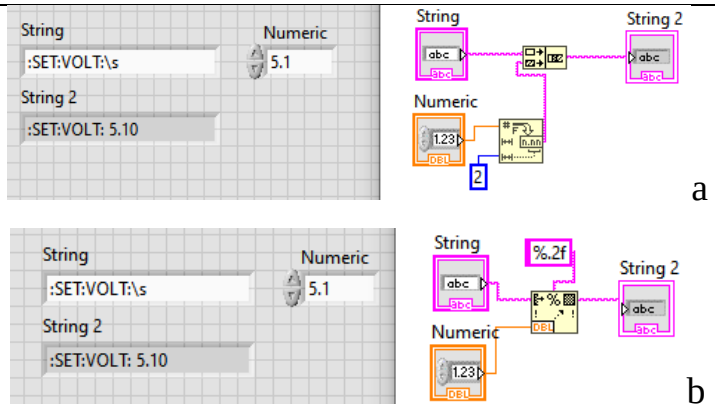


Рисунок 109. Добавление числа к строке комбинацией функций Number To Fractional и Concatenate Strings (a) и с использованием функции Format Into String (b).

5. Взаимодействие среды LabView с приборами

Использование звуковой карты

Примером взаимодействия LabView с аппаратными средствами может служить работа со звуковой картой. Звуковая карта является довольно качественным цифро-аналоговым (ЦАП) и аналогово-цифровым (АЦП) преобразователем для сигналов в диапазоне частот порядка 20 Гц- 20 кГц. В

каталоге со встроенными в LabView примерами в папке Graphics and Sound (C:\Program Files\National Instruments\LabVIEW 20XX\examples\Graphics and Sound) можно найти примеры работы со звуковой картой. Многоканальный АЦП звуковой карты может найти практическое применение в экспериментальной установке при необходимости регистрации сигнала звуковой частоты. В частности, это может быть спектроскопическая установка, работающая с аналоговым усилителем сигнала, например селективным вольтметром-усилителем. Аналоговый сигнал с измерителя-усилителя может быть оцифрован и обработан в LabView с использованием линейного входа звуковой карты. Необходимые функции для начала работы со звуковой картой находятся в кодовой панели в разделе Programming, пункте Graphics/Sound → Sound. В подпункте Imput (Вход) находятся функции для работы с АЦП звуковой карты. Сначала нужно использовать функцию Sound Input Configure VI. Функция имеет четыре основных входных терминала и один выходной (также сквозную линию ошибок). Входной числовой терминал number of samples задает число сэмплов (значений), захватываемых с АЦП за один такт обращения к АЦП, данные записываются в буфер. Входной числовой терминал (со значениями 0 или 1) sample mode определяет тип работы АЦП – одиночный или непрерывный. Для постоянного захвата сигнала с линейного входа нужен непрерывный тип работы (Continuous Samples, подаваемая константа 1). Device ID фактически определяет номер звуковой карты, обычно нужно подать константу 0. Входной терминал sound format является входом для типа данных кластер и содержит информацию о sample rate (S/s) – задаваемое число сэмплов в секунду, number of channels – число обрабатываемых каналов линейного входа, bits per sample – фактически разрядность АЦП. Кластер в LabView в общем случае объединение переменных различного типа в один поток. В данном случае в кластере все данные числового типа. Для создания кластера в пункте Array, Matrix and Cluster палитры передней панели нужно выбрать Cluster и поместить на переднюю панель. Далее последовательно взять три

числовых элемента управления Numeric Control и поместить в прямоугольную область кластера. Кластер готов, можно соединить его выходной терминал с соответствующим входом функции Sound Input Configure (рисунок 110(a). Первый элемент управления Numeric Control будет задавать sample rate (поставим 10000), второй number of channels (поставим 2), третий bits per sample (поставим 16).

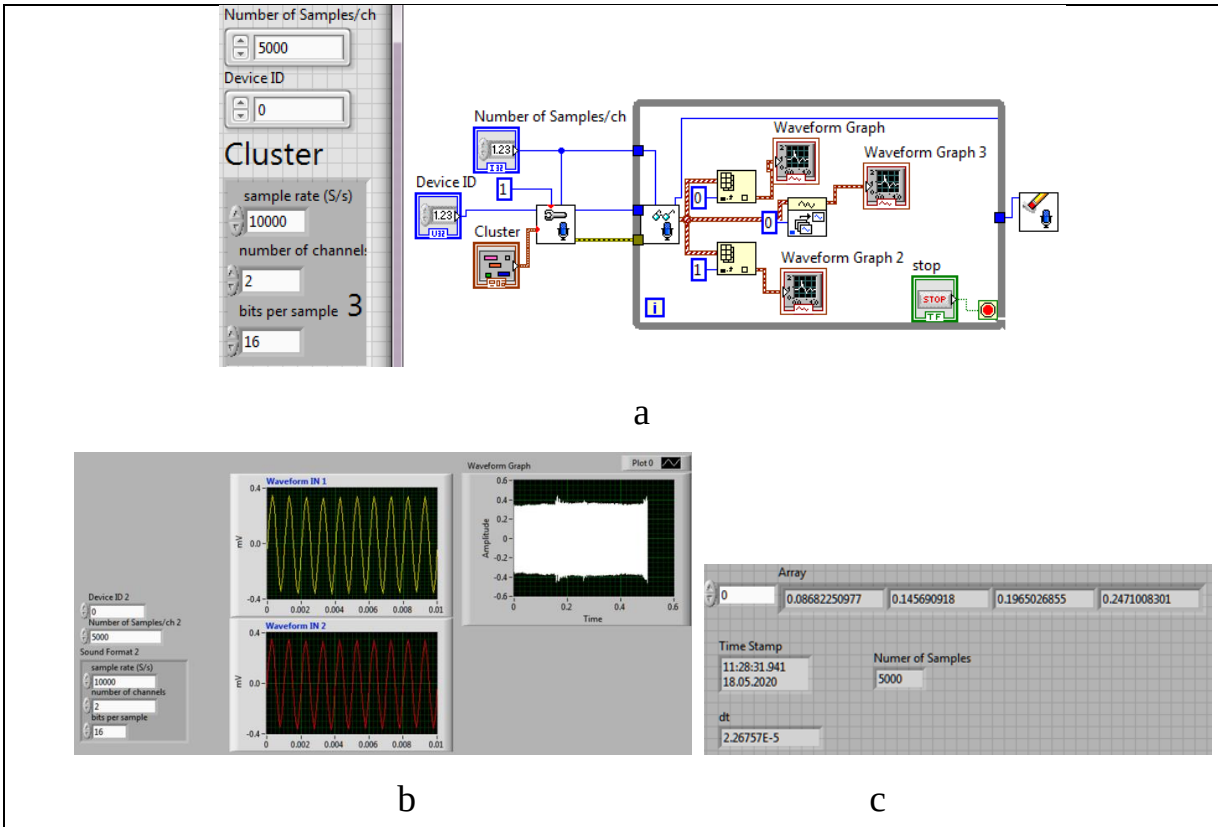


Рисунок 110. a) Конфигурация функции Sound Input Configure для считывания данных со звуковой карты, считывание данных функцией Sound Input Read, разделение кластера с данными со звуковой карты и их построение. b) Индикаторы Waveform Graph, отображающие массив данных со звуковой карты. c) Данные, содержащиеся в кластере Waveform с одного входного канала звуковой карты.

Такой же кластер-константу можно сделать в кодовой панели (если нет необходимости менять параметры работы АЦП звуковой карты). Основным выходным терминалом функции Sound Input Configure VI является числовой терминал task ID, содержаний числовой параметр сконфигурированного

АЦП звуковой карты. Функция получает данные с АЦП звуковой карты. Функция имеет три основных входных терминала: task ID – соединяется с соответствующим выходным терминалом функции Sound Input Configure, number of channels (нужно подать то же значение, что и для Sound Input Configure VI) и timeout (sec) – время ожидания данных на АЦП (можно подать значение -1 для постоянного ожидания). Основные выходные терминалы снова task ID и терминал data содержащий данные с АЦП звуковой карты. Выходной терминал data является кластером, содержащим данные со всех каналов линейного входа звуковой карты (рисунок 110(a). Информация с каждого канала также является кластером (рассмотрим структуру данных с отдельного канала позже). Первое, что нужно сделать при обработке этих данных, разделить кластер выходных данных data на каналы. Структурно этот кластер является массивом кластеров данных с каждого канала. Поэтому для разделения на каналы можно использовать функцию Index Array Function для выделения элементов массива. Для индексации элемента массива (номера канала) нужно на терминал index этой функции подать номер канал (начиная с 0). Для работы с Waveform данными (звуковыми данными или в целом данными, представляющими совокупность гармонических сигналов) в разделе Programming палитры кодовой панели имеется пункт Waveform. В нем, в частности. имеется специальная функция Index Waveform Array для разделения исходного кластера на каналы. Эта функция отличается от Index Array Function наличием линии ошибок (рисунок 110(a). Полученные данные с отдельного канала являются кластером и для визуализации могут быть сразу поданы на элемент отображения Waveform Graph (на лицевой панели). На рисунке 110(b) представлена лицевая панель с двумя индикаторами Waveform Graph, отображающими данные с двух каналов после функции Index Array Function. И отдельный индикатор Waveform Graph отображающий данные с одного канала после функции Index Waveform Array. Данные на индикаторе Waveform IN 1 и Waveform Graph одинаковые, но в различных режимах

отображения. Подадим на микрофон синусоидальный сигнал частотой 1000 Гц.

В рассматриваемом примере задано получить 5000 сэмплов (number of samples, значений с АЦП). Также задано число сэмплов в секунду (sample rate) равное 10000. Значит за один цикл работы АЦП можно получить данные с временного отрезка равного отношению number of samples[число значений]/sample rate[значений в секунду] секунд, в данном случае это отношение равно 0.5 секунды. На индикаторе Waveform Graph стоит настройка автомасштаба шкалы абсцисс (AutoScale X, параметр в контекстном меню в пункте Scale X) – временной шкалы. Поэтому на Waveform Graph отображаются данные до 0.5 – до половины секунды – собранного массива данных за один такт АЦП. На индикаторе Waveform IN параметр AutoScale X отменен и поставлено максимальное значение шалы абсцисс равное 0.01 секунды. При частоте 1000 Гц в этом отрезке укладывается 10 периодов колебаний.

Структуру Waveform кластера можно увидеть (и разделить) используя функцию Get Waveform Components. На вход функции подается Waveform кластер после выделения одного канала. Далее значок функции нужно растянуть вверх и появятся выходные терминалы Y, dt и t0. Терминал Y выдает массив числовых данных, содержащий сэмплы (значения с АЦП). Число элементов в массиве Y равно заданному параметру number of samples. Терминал dt выдает значение шага по времени между сэмплами в секундах. Терминал t0 выдает специфический кластер Time Stamp с информацией о времени получения массива сэмплов (рисунок 50 с). Отобразить значения в Time Stamp Indicator можно в соответствующем индикаторе на передней панели (в пункте Numeric палитры передней панели).

Использование COM порта

Для взаимодействия среды LabView с приборами в большинстве случаев используется программный интерфейс VISA (Virtual Instrument Software Architecture). Интерфейс VISA реализуется в LabView в виде линии

последовательной отправки команд в прибор и получения ответа с прибора. Линия VISA не является единственным вариантом взаимодействия LabView с приборами. В частности, можно использовать библиотеки DLL, поставляемые с приборами, посредством функции Call Library Function Node. Программный интерфейс VISA удобен тем, что не требует знаний об особенностях программирования конкретного аппаратного интерфейса (COM, LAN, GPIB, USB). В принципе, структура программы, использующая линию VISA для любого аппаратного интерфейса, будет одинакова, конечно, особенности и возможности интерфейса (в частности скорость передачи данных) в той или иной степени должны учитываться. Для использования программного интерфейса VISA с приборами, должен быть установлен отдельно от среды LabView набор драйверов NI-VISA. Для выполнения исполняемого файла на компьютере без среды LabView также нужно устанавливать драйверы NI-VISA (помимо исполняемого файла программы, созданной в LabView). Много научных приборов имеют COM порт для взаимодействия с компьютером. Рассмотрим основы работы с COM портом через линию VISA. В каталоге LabView есть папка с примерами, в которой есть раздел Instrument IO, содержащий раздел Serial и файл полезной программы-примера Simple Serial. На основе этого примера можно создать собственные программы для работы с COM портом.

Для взаимодействия с прибором через COM порт постпредством программного интерфейса VISA нужно сначала сконфигурировать (задать нужные параметры) линю VISA под эту задачу. Сначала поместим на передней панели элемент управления VISA Resource name, находится в пункте I/O палитры передней панели. Элемент используется для всех VISA-совместимых аппаратных интерфейсов. Для COM порта ключевым элементом является функция VISA Configure Serial Port, расположенная в палитре кодовой панели в разделе Instrument I/O в пункте Serial. Эта функция имеет входной терминал для проводника от функции VISA Resource name и числовые терминалы конфигурации COM порта, такие как скорость порта

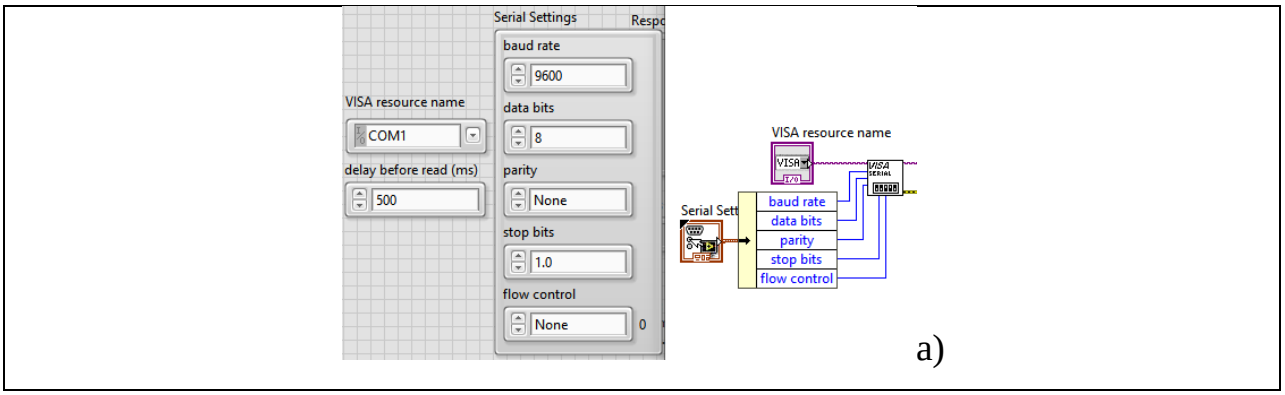
(baud rate) и другие. Эти параметры должны быть совпадать с параметрами, используемыми прибором (настраиваются отдельно в приборе или описаны в инструкции к нему). Эти параметры могут находиться внутри элемента обобщения переменных – кластере Serial Settings (рисунок 51 а, аналогично в примере Simple Serial). Кластер может содержать любое количество переменных любого типа и передавать их совместно по одной линии. В данном случае он содержит пять числовых констант, которые передаются в функцию работы с кластерами Unbundle By Name (пункт Cluster, Class, & Variant VIs палитры кодовой панели), разделяются в этой функции и передаются в функцию VISA Configure Serial Port. Можно структуру кластера не использовать, а задать параметры для VISA Configure Serial Port просто числовыми константами в кодовой панели. Функция VISA Configure Serial Port имеет два выхода – выход VISA Resource name и выход линии сообщения об ошибках. Линия VISA Resource name должна проходить через все функции работы с конкретным интерфейсом в рамках программы (линия ошибок не обязательна). Если в прибор нужно отправлять команды (в большинстве случаев это так), то за функцией VISA Configure Serial Port нужно ставить следует функцию передачи ASCII строковых данных в COM порт VISA Write. Пример работы функции VISA Write будет рассмотрен ниже (работа с прибором через USB интерфейс). После получения нужной команды прибор передаст в буфер COM порта запрошенные данные в строковом формате (прибор может передавать данные в COM порт и без команды передачи данных). COM порт медленный интерфейс и часто требуется введение в программу задержки межу моментом отправки команды и началом приема данных из буфера порта. Обычно для этого используется таймер Wait в одном кадре плоской последовательности на линии VISA. Время задержки зависит от скорости порта и объема ожидаемых данных. При работе с реальным прибором требует подбора оптимального значения задержки.

Моделью прибора с интерфейсом COM может служить микроконтроллер типа Arduino. Контроллер имеет USB интерфейс, работающий в режиме эмуляции COM порта. Взаимодействия с контроллером происходит как с прибором с COM интерфейсом. Оборудование типа Arduino относительно широко распространено и доступно и может успешно использоваться в образовательных (и не только) целях при отработке взаимодействия среды LabView с аппаратными средствами. Каждый микроконтроллер Arduino содержит АЦП и TTL входы, может содержать ЦАП. Для примера будем передавать в LabView данные с трех аналоговых (АЦП) входов микроконтроллера. Программа для микроконтроллера содержит следующие элементы: объявление переменных, открытие и конфигурирование COM интерфейса, опрос аналоговых входов, отправка данных в порт. Для опроса входов используется команда analogRead (команда среды программирования Arduino IDE, не среды LabView!) – опрос аналоговых входов A1, A2, A3. Для отправки данных в порт в строковой форме используется команда Serial.print. Команда Serial.print(" ")
используется для задания разделителя между данными с аналоговых входов в строке, отправляемой в COM порт. Команда Serial.println() вводит перенос строки, чтобы данные нового цикла выводились с новой строки.

```
int ARead1=A1;
int ARead2=A2;
int ARead3=A3;
void setup() {
Serial.begin(9600);
}
void loop() {
  int V1 = analogRead(ARead1);
  int V2 = analogRead(ARead2);
  int V3 = analogRead(ARead3);
  Serial.print(V1);
```

```
  Serial.print(",");
  Serial.print(V2);
  Serial.print(",");
  Serial.print(V3);
  Serial.println();
  delay(100);
}
```

После записи программы в контролер посредством Arduino IDE можно открыть встроенный в Arduino IDE монитор COM порта. Данные будут постоянно выводится в COM порт (заданный временной интервал 100 мс) и отображаться в виде срок с тремя группами данных (с каждого аналогового входа), разделенных Tab (следует проверить правильность значения параметра baud rate для монитора COM порта). Данные с АЦП входов будут выводиться в виде величины отсчетов АЦП. Для используемого 10 битного АЦП это будут числа в диапазоне 0-1023. На аналоговые входы можно подать напряжение в диапазоне 0-5 В. Например, используем модуль 3-х осевого гироскопа с аналоговыми выходами. Питание Vcc подсоединим к контакту 5V контроллера, соединим контакты GRD и подсоединим три выхода акселерометра к аналоговым входам A1, A2, A3 контроллера. После запуска монитора можно потрясти плату акселерометра и наблюдать, что отображаемые цифры величины отсчетов изменяются.



а)

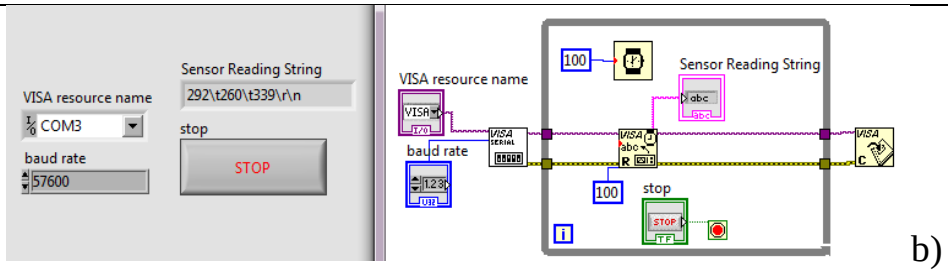


Рисунок 111. a) Конфигурирование параметров функции VISA Configure Serial Port для взаимодействия с прибором через COM порт.

Сделаем в LabView простую программу для считывания данных (рисунок 111(b)). Используем элементы конфигурации COM порта, как рассматривалось выше. Функция записи в COM порт VISA Write не используется, т.к. считываем постоянно передающиеся в буфер COM порта данные. Используем только функцию VISA Read. К функции VISA Read к терминалу byte count подведем числовую константу, которая будет определять объем считываемых данных в битах (можно задать с запасом). После функции VISA Read следует поставить функцию VISA Close закрывающую сессию работы с прибором по линии VISA. Из функции VISA Read данные поступают в виде строки. Далее нужно выделить данные с каждого канала ЦАП и перевести в числовой формат для дальнейшей обработки. Обратим внимание, что при работе акселерометра величина данных меняется и данные могут представляться не только в трех группах по три цифры – число цифр в группе может меняться. В связи с этим разделять строку путем отсчета числа символов с начала строки может не получиться. Нужно проводить поиск символа-разделителя и проводит разделение до и после символа. Примеры разделения строки рассмотрены на предыдущих занятиях.

Использование USB порта

Рассмотрим взаимодействие с лабораторным мультиметром АКИП В7-78. Мультиметр АКИП В7-78 является копией мультиметра Agilent 34401

и является типичным представителем класса лабораторных мультиметров и в целом лабораторных приборов, поддерживающих систему команд SCPI. Мультиметр АКИП В7-78 соединяется с компьютером посредством интерфейса USB. Приборы с интерфейсом USB для корректной работы с компьютером могут потребовать установки драйверов от производителя прибора. Приборы с интерфейсом COM и GPIB не требуют установки на компьютер специфичных драйверов. При использовании USB интерфейса нужно указать линии VISA какой прибор, подключенный к USB будет использоваться, по аналогии с необходимостью выбора COM номера порта. Для этого используем функцию VISA Open, расположенную в разделе Instrument I/O кодовой палитры (Instrument I/O →VISA→Advanced→ VISA Open). Разместим на передней панели элемент управления VISA Resource name, находящийся в пункте I/O палитры передней панели, и соединим выходной терминал Resource name с одноименным входным терминалом функции VISA Open. После подключения прибора к компьютеру в выпадающем меню элемента управления VISA Resource name появится адрес подключенного прибора (например, USB0::0x164E::0x0DB7::TW00016753::INSTR). Теперь с прибором можно взаимодействовать по линии VISA. Для взаимодействия необходимо отправлять команды в прибор. Для этого далее к линии VISA подсоединим функцию отправки команд VISA Write. Будем отправлять команды с передней панели с использованием функции ввода данных строкового типа String Control. Для считывания приходящих с прибора данных используем функцию VISA Read. Между VISA Write и VISA Read поставим кадр Flat Sequence с функцией задержки для того, чтобы прибор успел передать данные в буфер обмена. USB является быстрым интерфейсом и время задержки может быть значительно меньше, чем при использовании COM порта.

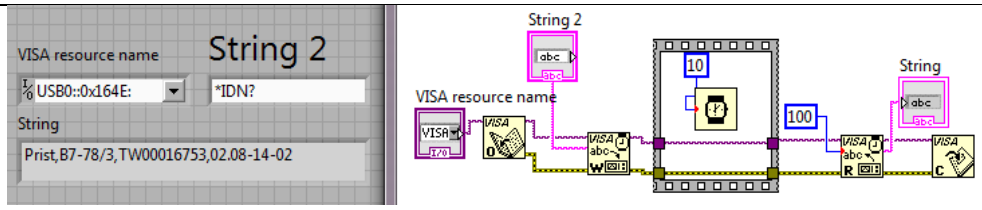


Рисунок 112. Программа по однократному считыванию данных с прибора.

Проверим связь прибора с компьютером. Введем в окне ввода String Control команду *IDN?, являющейся стандартной для приборов с поддержкой SCPI и означающая выдать в буфер обмена данные о модели прибора. После запуска программы в окне String Indicator увидим название прибора, его серийный номер, версию прошивки и другие данные в зависимости от прибора (рисунок 112). Введем команду READ? и запустим программу. Команда READ? делает запрос прибору на выдачу в буфер обмена данные однократного измерения в текущем режиме прибора. Например, если после включения мультиметр находится в режиме вольтметра постоянного тока, то отправка команды READ? выдаст текущее измеряемое напряжение. Будем непрерывно считывать с прибора измеряемую величину. Для этого функции подачи команды в прибор и считывания данных из прибора поместим в цикл While. Теперь пока работает цикл, данные из прибора будут отображаться в окре String Indicator (рисунок 113(а)). Обратим внимание на то, что данные, которые выдает функция VISA Read, имеют строковый формат. Для математических действий с этим данными нужно их преобразовывать в числовой формат функцией Fract/Exp String To Number. Сделаем возможность усреднения измеряемой величины по N измерениям. Для этого внутри цикла While поместим цикл For с числом итераций N, в который поместим функции VISA Write, VISA Read и Fract/Exp String To Number. После преобразования в числовой тип данных сделаем накопление измеренной величины на границе цикла (режим выходного туннеля в цикле For - Indexing), а далее полученный массив данных усредним функцией Mean, находящейся в пункте

Probability&Statistics раздела Mathematics палитры кодовой панели (рисунок 113(b)). Обратим внимание, что для проводников линии VISA на выходе из цикла For нужно поставить режим туннеля Last Value.

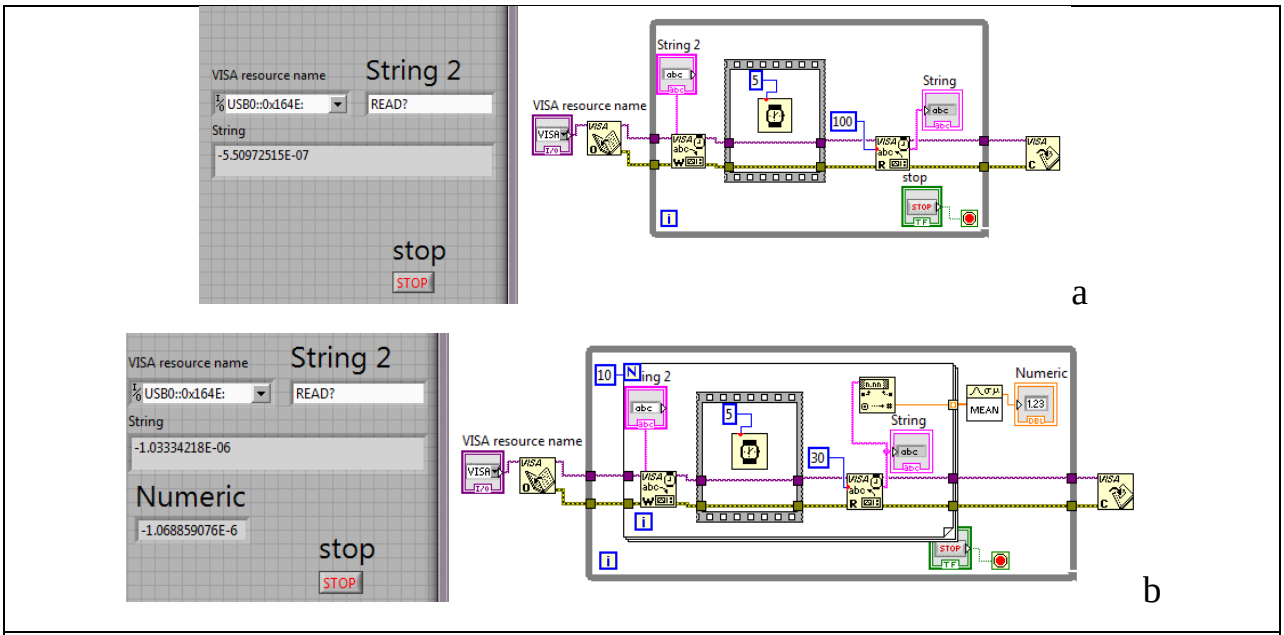


Рисунок 113. a) Непрерывное считывание данных из прибора. b) Непрерывное считывание и усреднение данных.

Литература

1. LabVIEW для всех : монография / Трэвис Дж.; Кринг Дж. - Москва : ДМК-пресс, 2023. - 905 с.

2. LabVIEW 8.20. Справочник по функциям : справочник / Суранов А.Я. - Москва : ДМК-пресс, 2023. - 537 с.

3. LabVIEW: стиль программирования : монография / Блюм П. - Москва : ДМК-пресс, 2023. - 401 с.